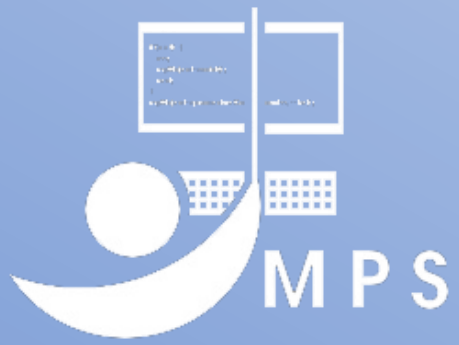
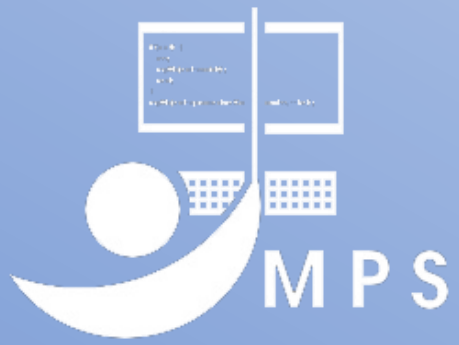
Managementul Proiectelor Software



Curs #0

Introducere



The conventional definition of management is getting work done through people, but real management is developing people through work.

Obiective

- Activitatea unui **Manager de Proiect** în domeniul dezvoltării de software
- Vedere de ansamblu asupra rolului, metodelor de management și a responsabilităților
- Nu este necesară cunoașterea prealabilă a tehnicilor de management
- Dedicat studenților care doresc să își dezvolte aptitudini, stiluri și abordări specifice în această arie
- Necesită cunoștințe de bază referitoare la analiza programelor și algoritmilor

Componente

- Curs
  - Noțiuni teoretice
  - Elemente de bază de management de proiect
- Laborator
  - Noțiuni aplicate de management de proiect
  - Lucru în echipă
  - Doua proiecte pe semestru în echipă

Resurse

- Wiki-ul cursului
  - http://ocw.cs.pub.ro/courses/mps
- Portalul http://cs.curs.pub.ro
- Sala de laborator EG-405A

Echipa MPS

Nume	E-mail
Costin BOIANGIU	**costin.anton.boiangiu@gmail.com**
Răzvan DEACONESCU	**razvand@gmail.com**
Giorgiana VLĂSCEANU	giorgiana.vlasceanu@gmail.com
Iulia STĂNICĂ	iulia.stanica@gmail.com
Costin SPECIAC	costin.speciac@gmail.com
Laura ANTONACHE	lauraantonache@gmail.com
Mihai SOARE	soare.mihai1992@gmail.com
Alexandru BREBU	alex.brebu13@gmail.com
Marius BÎNĂ	bina.marius@gmail.com
George OPREA	georgeo4567@gmail.com
Diana NIȚĂ	diana.nita.luiza@gmail.com

Evaluare

- Pentru absolvirea materiei trebuie să obțineți cel puțin 50% din punctajul de curs (lucrări de curs și examen final) și cel puțin 50% din punctajul de laborator (laborator și teme).
- Notare
  - Curs – 4 puncte
  - Activitate parcurs (laborator + proiect) – 6 puncte

Evaluare

- Curs (4 puncte)
  - **3.5p**: examen final (se poate lua 10 fără bonusuri, prin rotunjire superioară, doar dacă a fost obținut 10 la fiecare activitate în parte)
- Alternative/Extra/Bonusuri:
  - Articol de cercetare referitor la abordări de proiecte din domeniul MPS
  - Prezentarea (pe baza unei programări) a unui articol științific relevant preluat de pe internet
  - Prezentarea (pe baza unei programări) a unui curs din cadrul materiei
  - Participarea la o dezbatere pe o temă dată
  - Diverse intervenții, răspunsuri la întrebări, interacțiuni în cadrul “cafenelelor publice” sau discuții la filme si prezentările colegilor.
  - Prezenta la curs

Detalii Alternative/Extra/Bonusuri:

A. **4p**: printr-un articol de cercetare referitor la abordări de proiecte din domeniul MPS (se pot înscrie echipe de 2-3 studenți, se oferă material suport și detalii la cerere), se prezintă articolele în ultimele 2 săptămâni (cu **echivalarea** examenului final)

B. **1.5p**: Prezentarea (pe baza unei programări) a unui articol științific relevant preluat de pe internet (se pot înscrie studenți individual, sunt oferite câteva exemple de articole, orice subiect din domeniu este însă valid, încurajăm împărtășirea experiențelor personale)

C. **1p**: Prezentarea (pe baza unei programări) a unui curs din cadrul materiei (se pot înscrie echipe de 2 studenți, se oferă prezentări PPT ce pot fi ajustate pentru o interactivitate cât mai ridicată)

D. **0.75p:** Participarea la o dezbatere pe o temă dată: (4-6 studenți împărțiti în 2 tabere)

E. **0.5p**: intervenții, răspunsuri la întrebări, interacțiuni în cadrul “cafenelelor publice” sau discuții la prezentările colegilor.

F. **1p:** prezenta la curs

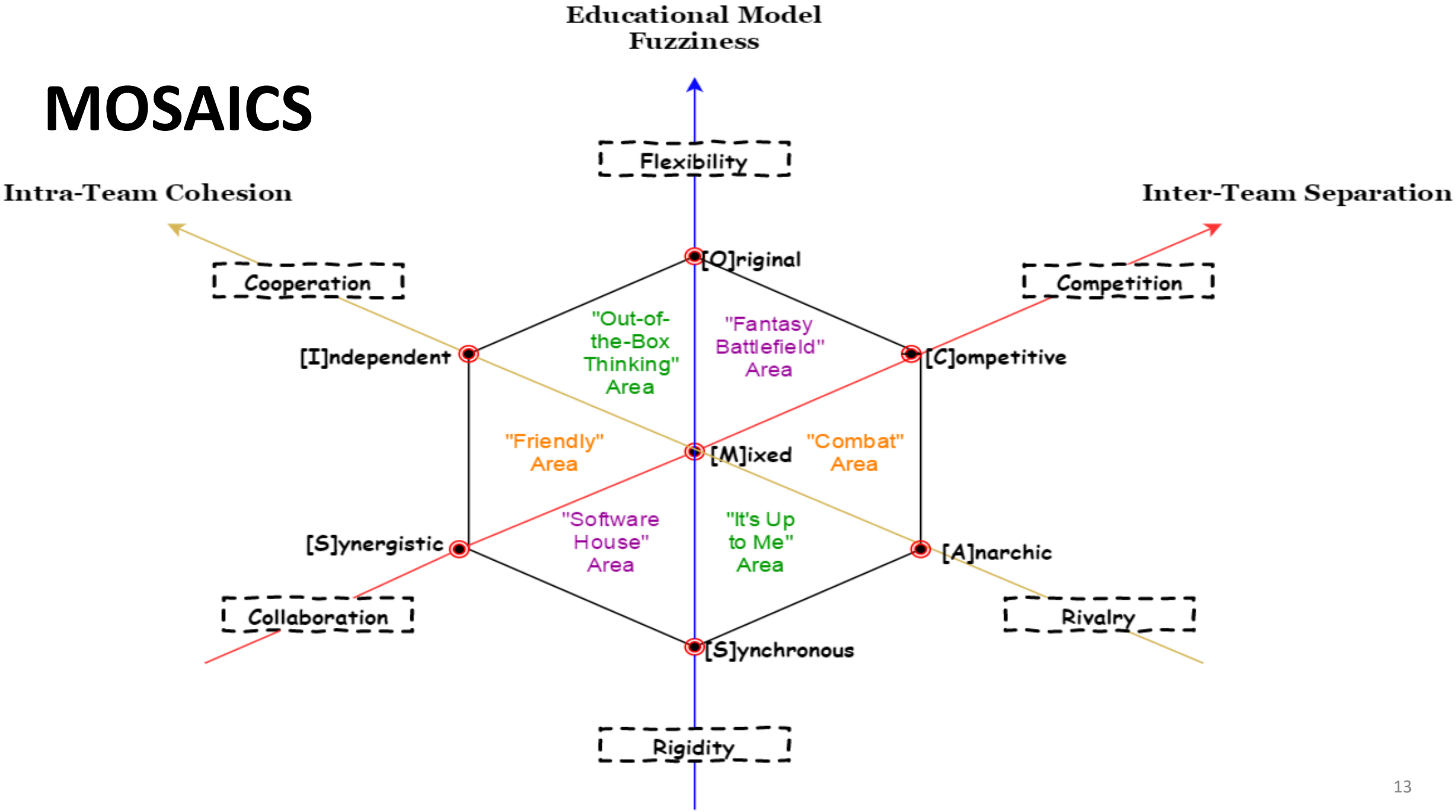
Evaluare

- Laborator + Proiect (6 puncte)
  - **3p** Proiect 1
    - **0.5p** activitate individuală în cadrul proiectului
    - **1p** respectarea deadline-urilor individual
    - **1p** rezultat final (respectarea cerințelor și specificațiilor, funcționalitate minimală a proiectului, notă per echipă)
    - **0.5p** impresia finală, nota colegilor și prezentarea.
  - **3p** Proiect 2 (repartizate identic)
  - **0.5p** bonus pentru intervenții și activitate remarcabilă în cadrul laboratoarelor.
- În total ar fi 6.5 puncte pe laborator care se trunchiază la 6p
- Se evaluează atât activitatea inidividuală cât și cea în echipă.
- Detalii pot fi găsite la: http://ocw.cs.pub.ro/courses/mps/reguli

Abordari educationale variate

- Curs
  - Studii de caz / Probleme de tip examen
  - Prezentarea părții teoretice a cursului
  - Prezentări articole de cercetare
  - Redactarea și prezentarea unui articol de cercetare
  - Dezbateri
  - Cafenele publice
  - Filme comentate
- Proiect
  - Încercăm să găsim cea mai bună modalitate de a învăța studentii aspecte practice legate de lucrul în echipă și management de proiect software

MOSAICS


Educational Model
Fuzziness
Flexibility
Intra-Team Cohesion
Inter-Team Separation
Cooperation
Competition
[O]riginal
[I]ndependent
[C]ompetitive
"Out-of-the-Box Thinking" Area
"Fantasy Battlefield" Area
"Friendly" Area
[M]ixed
"Combat" Area
[S]ynergistic
[A]narchic
"Software House" Area
"It's Up to Me" Area
Collaboration
Rivalry
[S]ynchronous
Rigidity

Abordarea Independentă (“Independent”)

Propuneri proiect

- Mortar Mayhem
- Fastest and Brightest
- Battleship
- E-commerce Lite
- SimTraf
- Stonemason
- TTSched
- Analizor de imagini
- Circuit Designer
- Aplicație pentru teste grilă

Planul abordării independente

- Solicitare Proiect → scopul documentului, conţinutul documentului
- Descrierea sumară a produsului software
- Descrierea soluţiei → arhitectura soluţiei, tehnologii şi tooluri de dezvoltare
- Experienţa echipei în domeniul produsului
  - Cunoştinţele tehnice ale echipei (Advanced, Medium, Beginner)
  - Experienţa echipei
- Estimări resurse necesare
  - Determinarea rolurilor din cadrul echipei (ex: Developer, Project Manager, etc.)
  - Determinarea distribuţiei echipei per roluri (câte persoane au acelaşi rol?)
  - Determimarea efortului depus de fiecare rol într-o săptămână (x ore/săpt)
- Prezentare calendar livrări propuse
- Motivare echipă.

Abordarea Concurențială (“Competitive”)

- Realizarea unui sistem de simulare a competițiilor între mașini de Formula 1.
- Echipele vor alinia la startul unei curse câte două mașini fiecare. Pentru implementare se menționează o serie de caracteristici și parametrii. Exemplu de parametrii:
  - Culoarea (specifică echipei)
  - Acceleratie (m/s2) = ACC
  - Frana (m/s2) = BRK
  - Viteza maxima (m/s) = SPD
  - Manevrabilitate (rad/s) = STR
  - Constrângere (default): 10*ACC+10*BRK+SPD+200*STR = 400
  - Modul de strategie conducere vehicul (AI)

Desfășurarea competiției

- Vor fi desfășurate un număr de curse egal cu numărul de echipe. Fiecare echipă va alinia în cursă două mașini cu setări (posibil) diferite și strategii (posibil) diferite;
- Competiția cu un mecanism bazat pe AI. Punctaje acordate: loc 1 – 7 p, loc 2 – 4p, loc 3 – 3p, loc 4 – 2p, loc 5 – 1 p, ultimul loc - -1p, descalificare: -2p;
- Între masina cu cel mai bun rezultat al unei echipe (MAX) și cea cu rezultatul mai slab (MIN) punctajul acumulat de echipa va fi: (MAX * 7 + MIN * 3) / 10
- După terminarea curselor se efectuează clasamentul prin însumarea punctelor obținute
- Primele două echipe vor participa într-o nouă cursă, pentru departajare, fiecare cu câte o singură mașină

Componente. Permisiuni la nivelul interfeței – aplicației

- Încărcarea hărților
- Alocarea punctelor și orientărilor de start, a numărului de ture, a timpului de referință, a sensului de parcurgere, etc.
- Alegerea culorilor participanților
- Conexiunea cu serverului și actualizarea continuă a situației în funcție de pozițiile venite de la server pentru fiecare mașină
- Nu efectuează calcule ci doar citește poziții specificate prin coordonate în sistem metric și afisează mașinile în pozițiile specificate utilizând culoarea selectată pentru fiecare echipă
- Afișează rezultatele cursei, punctejele obtinute și clasamentul provizoriu
- Afisează harta, vehiculele, timpul scurs de la start, parametrii fiecărui vehicul și ordinea în cursă. Afișarea se face în timp real, în mod grafic, full-screen

Componente. Server

- Inițializează conexiuni cu interfață cât și cu cele 2**X**N mașini ale celor N echipe în cazul unei confruntări normale sau ale celor 2 mașini pentru finală. Conexiunea celor 2**X**N procese-mașini se pot iniția de la mașini către server (fizica aplicației) sau invers. Probabil că este mai ușor conexiuni de la procese-mașini către.
- Așteaptă la inițializarea modulelor AI, după transmiterea hărții către acestea, un interval de 30 de secunde pentru calculul parametrilor mașinilor
- Comunică aplicației starea în cursă: ordine, timpi, poziții, parametri de rulare pentru fiecare vehicul în parte
- Citește periodic de 10 ori pe secunda de la fiecare modul de AI deciziile de conducere ale vehiculului asociat, calculează noile poziții ale acestuia, noul unghi de orientare al volanului și confirmă AI noile poziții ale tuturor participanților, viteza curentă și noile direcții de deplasare (X, Y, speed, steer)
- Limitează viteza de deplasare a unui vehicul la SPD
- Calculează noile poziții ale vehiculelor, unghiurile de orientare pe baza unei aproximări discrete corespunzătoare împărțirii unei cuante de timp de 1ms

Componente. Strategia de deplasare (AI)

- Comunica server-ului de 10 ori pe secunda decizia curentă de conducere a vehiculului si anume:
  - Franare în procent (0-100] din BRK sau accelerație în procent [0-100] din ACC
  - Cerere traiectorie de deplasare în unghiul [0-2*PI) (măsurat în valoare absolută față de **OX**), volanul va fi mișcat către poziția cerută cu viteza STR , în sensul ce asigură o rotație minimă

Evaluarea echipei

- Evaluarea se va realiza în cadrul a 4 milestone-uri:
  - S04 → Specificațiile proiectului (SRS – “Software Requirement Specifications”, SDD – “Software Design Document”, Arhitectură) (0.5 puncte)
  - S06 → Aplicația de vizualizare a mașinilor (0.5 puncte)
  - S10 → Serverul (fizica aplicației) (1 punct)
  - S14 → AI engine-ul și competiție; primele două echipe au echivalat examenul final cu punctaj maxim (1 punct)

Abordarea Colaborativă (“Synergistic”)

Sistem Automat de Analiză şi Extragere a Informaţiei din Documente

- Transformarea documentelor scrise (ex: cărţi, ziare, reviste, facturi, hărţi) în documente electronice (ex: PDF, RTF) utilizabile pe o gamă largă de dispozitive, inclusiv mobile
- Conservarea meta-informaţiei şi a aspectului original (arhitectura multi-layer a documentului)
- Generarea conţinutului paginilor, layout-ului şi a ierarhiei documentului
- Generarea informaţiilor relevante pentru efectuarea unor căutări inteligente

Intrare

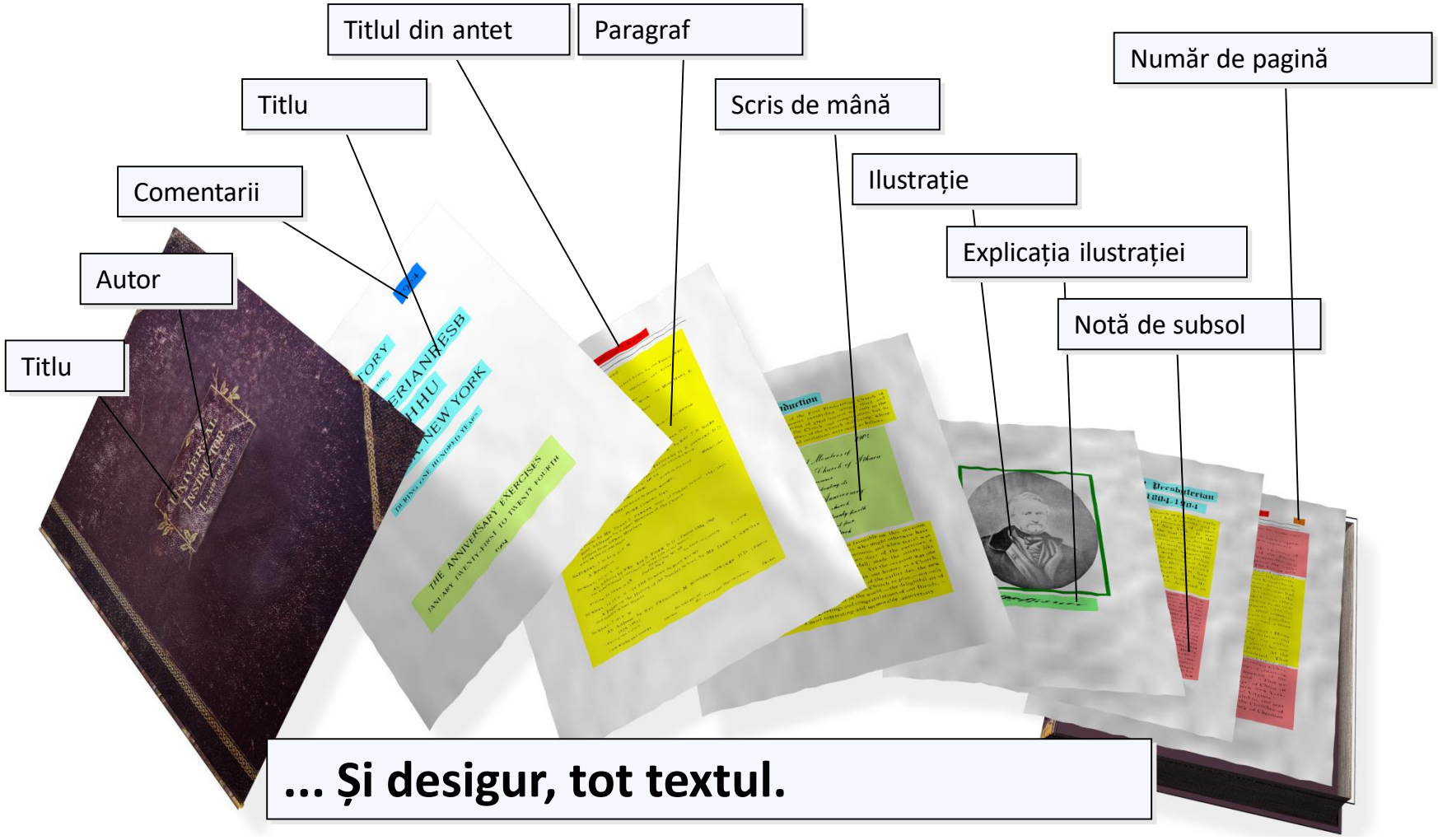
Ieșire

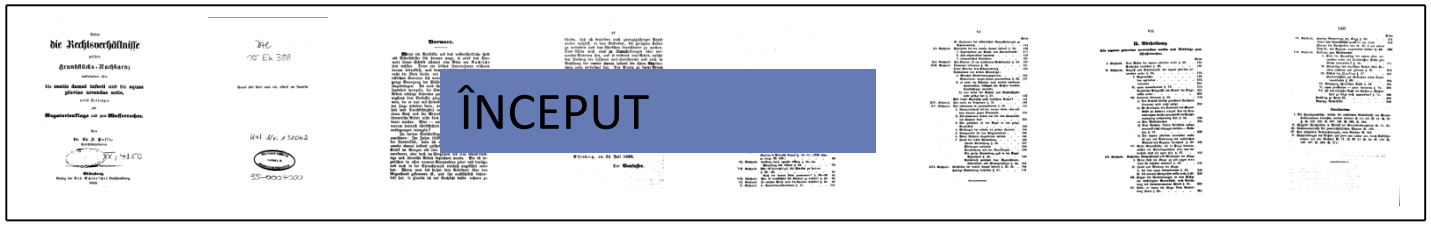
document

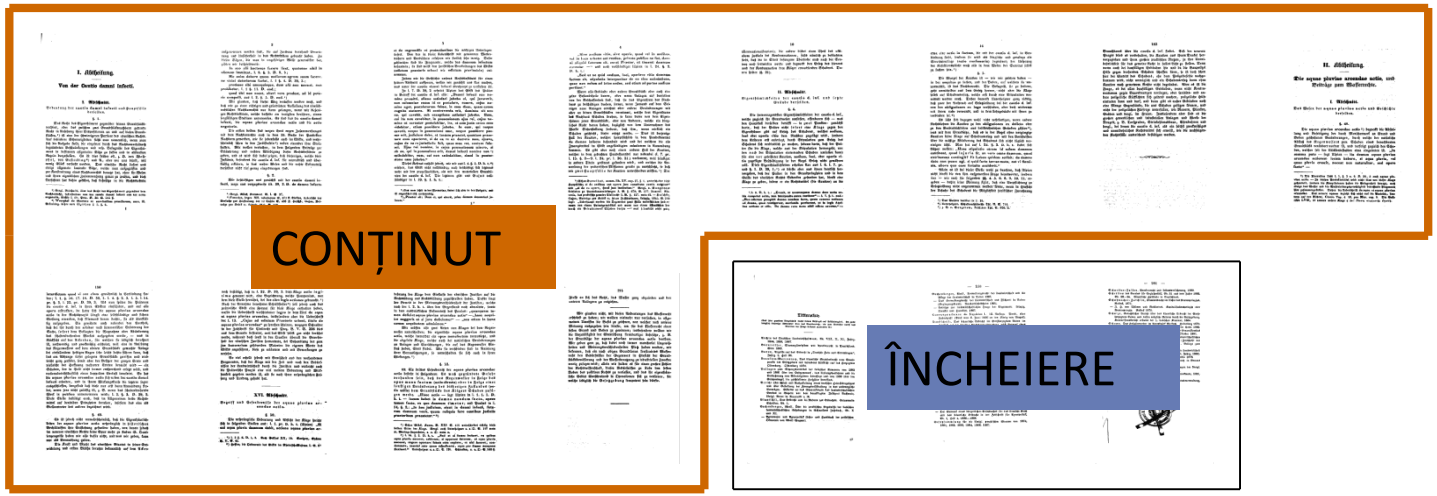
Scanare	Pre-procesarea imaginii	Corecție
	Analiza elementelor	
Import	Recunoașterea optică a caracterelor (OCR)	Export
	Analiza structurală	

Reguli
DB

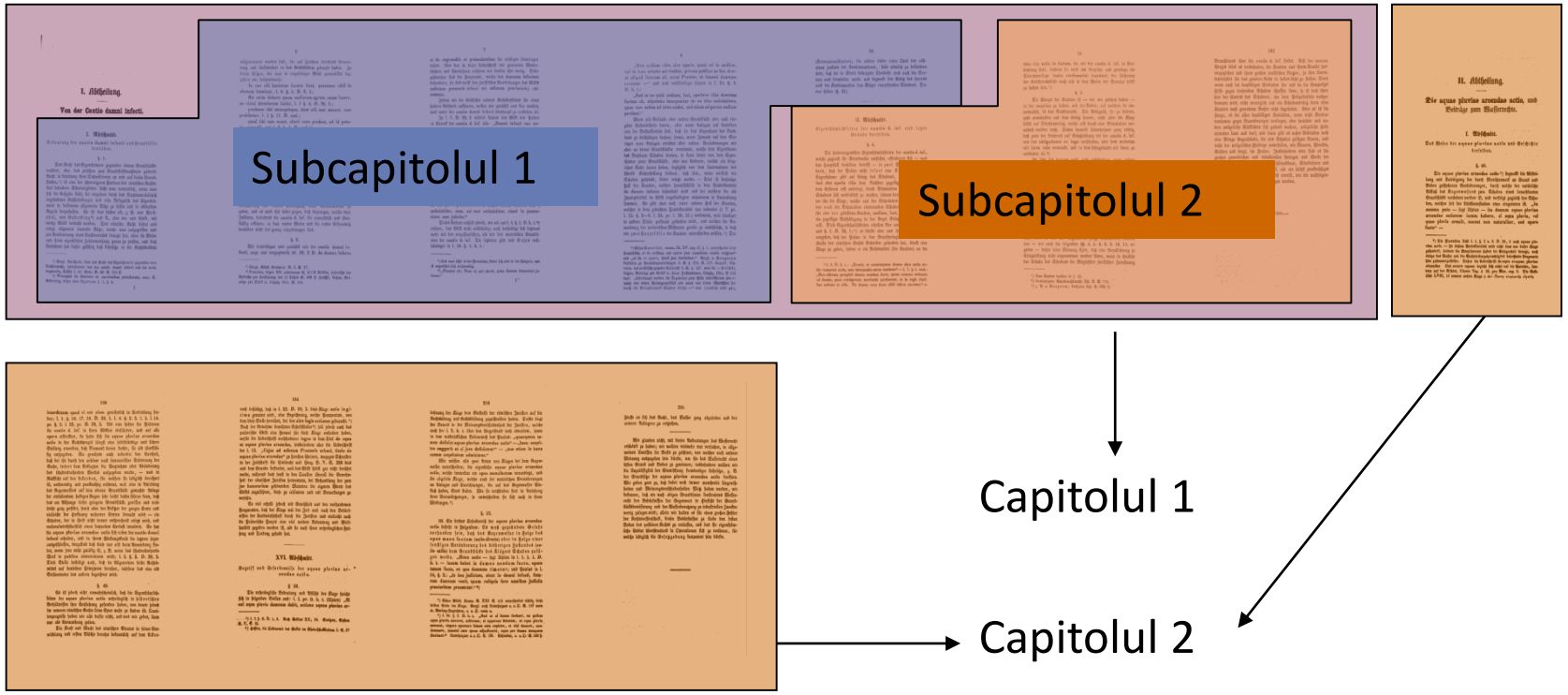
PDF
METS/ALTO
(TIFF+JPEG)
XML (MARC)


Titlul din antet
Paragraf
Titlu
Scris de mână
Număr de pagină
Comentarii
Ilustrație
Autor
Explicația ilustrației
Titlu
Notă de subsol
... Și desigur, tot textul.

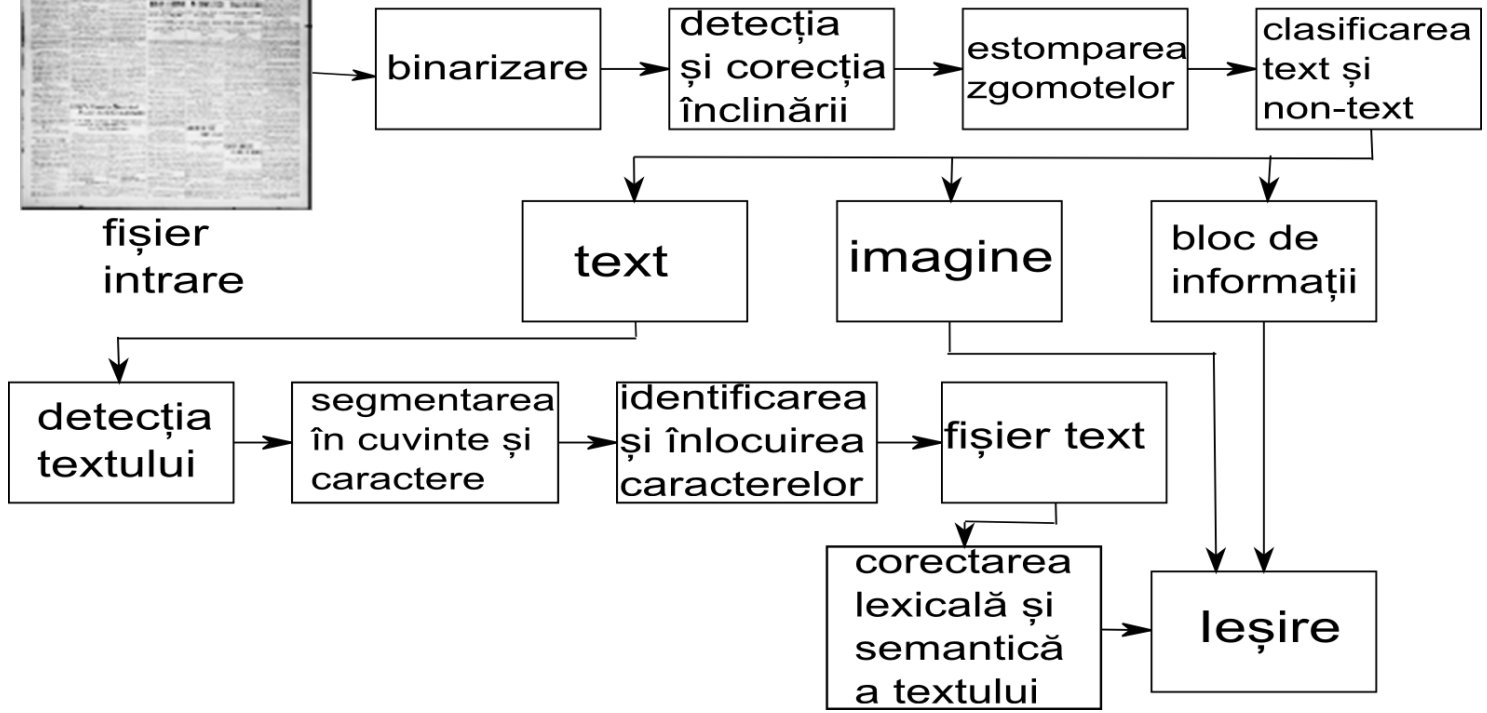

ÎNCEPUT


CONȚINUT

ÎNCHEIERE


Subcapitolul 1
Subcapitolul 2
Capitolul 1
Capitolul 2

fișier intrare



Abordarea Mixtă (“Mixed”)

- Construirea unui sistem de prelucrare de imagini bazat pe o strategie de votare
- Prelucrarea constă în transformarea unei imagini color (o pagină scanată de ziar) într-o imagine bitonală, astfel încât să fie evidențiat cât mai clar textul. Imaginea bitonală era trimisă spre OCR pentru a fi convertită în text digital. Cu cât erau recunoscute mai multe caractere, cu atât rezultatul era considerat mai bun.
- Echipa împărțită în patru celule: 3 celule care ofereau trei algoritmi diferiți de conversie și o celulă care combină cele trei rezultate într-unul mai bun decât oricare din cele trei rezultate inițiale.

Exemplu

(a) Imagini reprezentative din setul de dat cu:

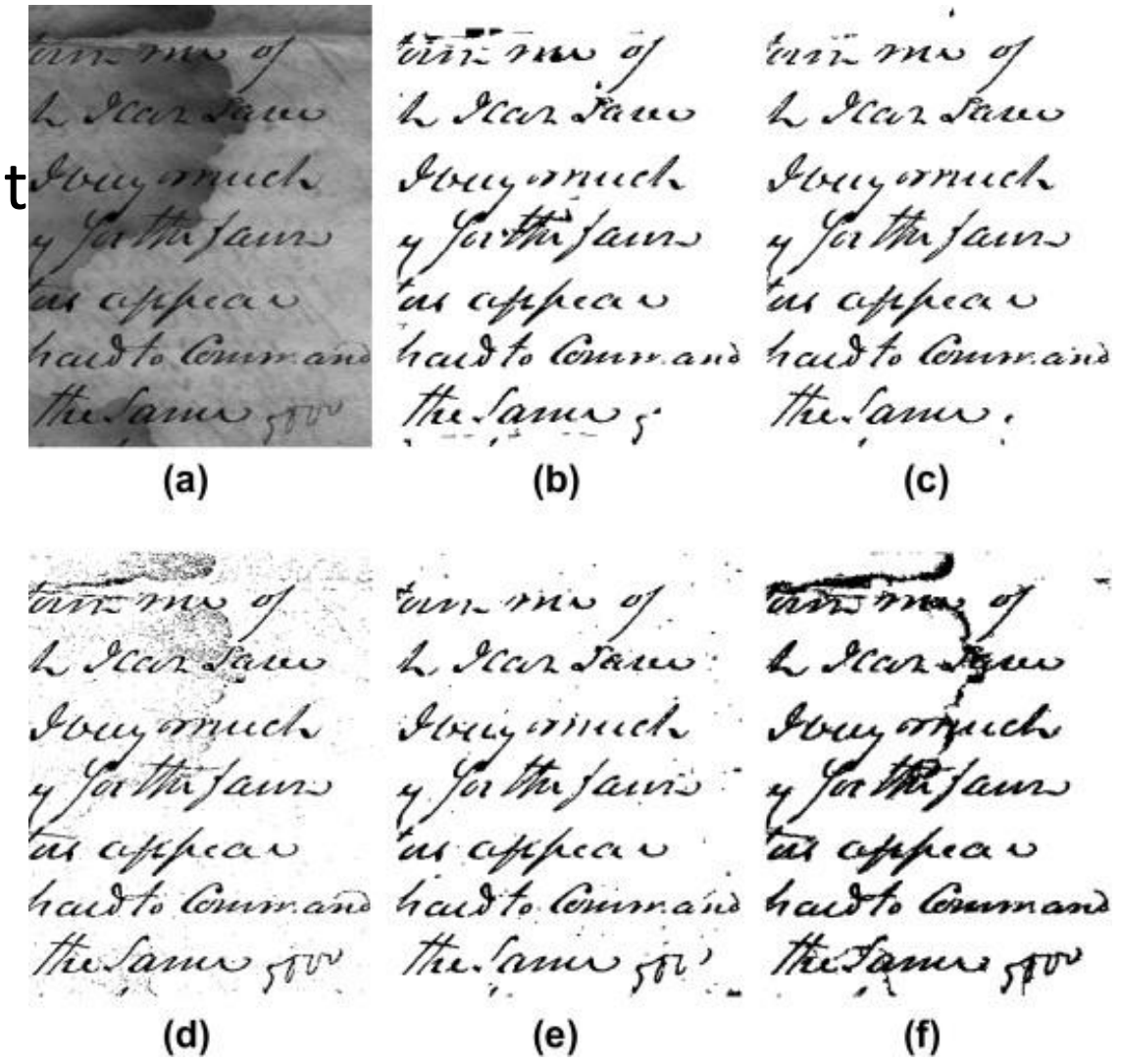
(b)Messaoud et al. (2011);

(c) Su et al. (2010);

(d) Ntirogiannis et al. (2009);

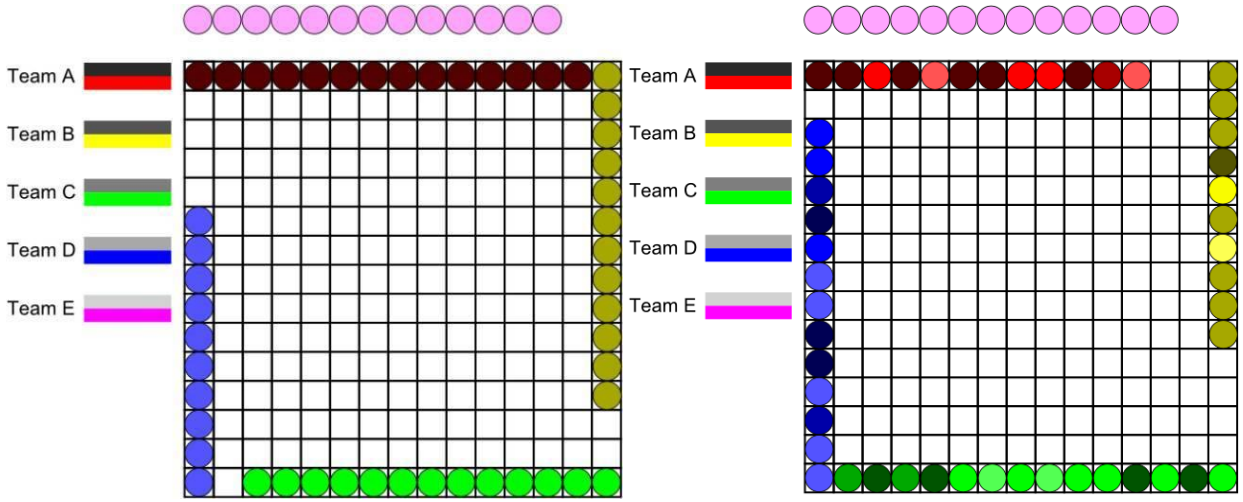
(e) Howe (2011);

(f) Gatos et al. (2008).



Abordarea Anarhică (“Anarchy”)

- Board game, în care un pion este executabilul unui singur student, un cod care conține euristici pentru jucare și mecanisme de comunicare cu serverul și ceilalti pioni.
- Studenții dintr-o grupa vor avea o luminozitate comună și o culoare, care se schimbă în funcție de echipa la care joacă.



Abordarea Artistica (“Originality”)

- Scopul proiectului este de a obține o imagine cu o valoare “artistică” de un cât mai ridicat nivel de exprimare utilizând doar generare procedurală, geometrie fractalică, și algoritmică simplă.
- Durata de rulare este setată la **60 s** și utilizarea a maximum **1KB date seed**.
- Evaluarea subiectivă de către o comisie stabilită.
- Trofeu simbolic **Van'Gogu**

Abordarea Sincrona (“Synchronous”)

Proiect 1

- Echipa: 7 oameni
- Primul proiect la Managementul proiectelor software se desfășoară în perioada laboratoarelor 3, 4, 5, 6. Începutul laboratorului 7 este termenul limită pentru definitivat proiectul. Proiectul va fi prezentat în forma sa finală în laboratorul 7

Abordarea Sincrona (“Synchronous”)

Proiect 2

- Echipa: 5 oameni
- Al doilea proiect la Managementul proiectelor software se desfășoară în perioada laboratoarelor 7, 8, 9, 10. Începutul laboratorului 11 este termenul limită pentru definitivat proiectul. Proiectul va fi prezentat în forma sa finală în laboratorul 11

Bibliografie

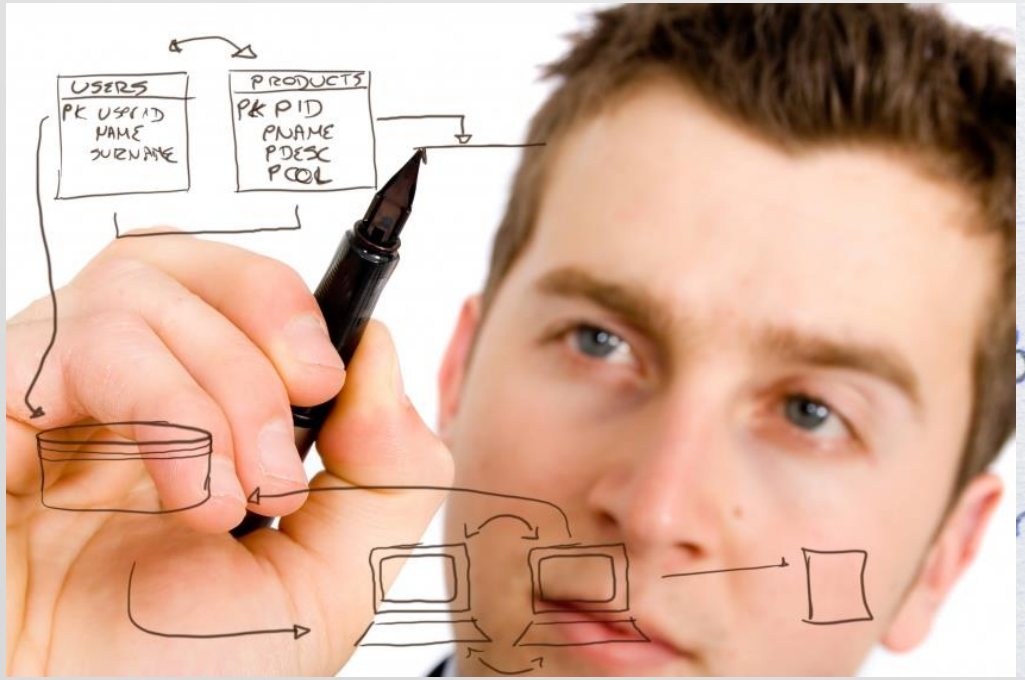
- TEXTBOOK OFICIAL : “*Introduction to Software Project Management*”, Adolfo Villaflorita, CRC Press, 2014, ISBN: 9781466559530

- “*Rapid Development*”, McConnell, Steve, Microsoft Press, 1996, ISBN 1-55615-900-5.

- “*Information Technology Project Management*”, Schwalbe, Kathy, 2nd ed., Course Technology, 2002, ISBN 0-619-03528-5 (include copie de evaluare 120-zile pentru MS-Project 2000).

- “*UML Distilled: A Brief Guide to the Standard Object Modeling Language*”, Fowler, Martin. 3rd ed., Addison-Wesley. ISBN 0-321-19368-7.

If you're 6 months late on a milestone due next week but nevertheless really believe you can make it, you're a project manager.



Obiective specifice

- Înțelegerea clară a problemelor, factorilor de succes și a riscurilor asociate cu dezvoltarea proiectelor în domeniul software
- Rolul și scopul managementului de proiect
- Asimilarea etapele și procesele din cadrul ciclului de viață al unui proiect
- Cunoașterea tehnicilor variate de planificare și de management al unui proiect software
- Evaluarea metodologiilor de bază în proiectarea, dezvoltarea, testarea și implementarea software-ului

Obiective specifice (2)

- Examinarea metodologiilor de bază în proiectarea, dezvoltarea, testarea și implementarea software-ului
- Examinarea tehnicilor variate de management a unei echipe de dezvoltare software
- Necesitatea și tehnicile corespunzătoare de management al utilizatorilor și ale așteptărilor acestora
- Utilizarea tehnicilor de planificare și dezvoltarea arhitecturii software și a ciclului de viață a produselor

« De ce » management software?

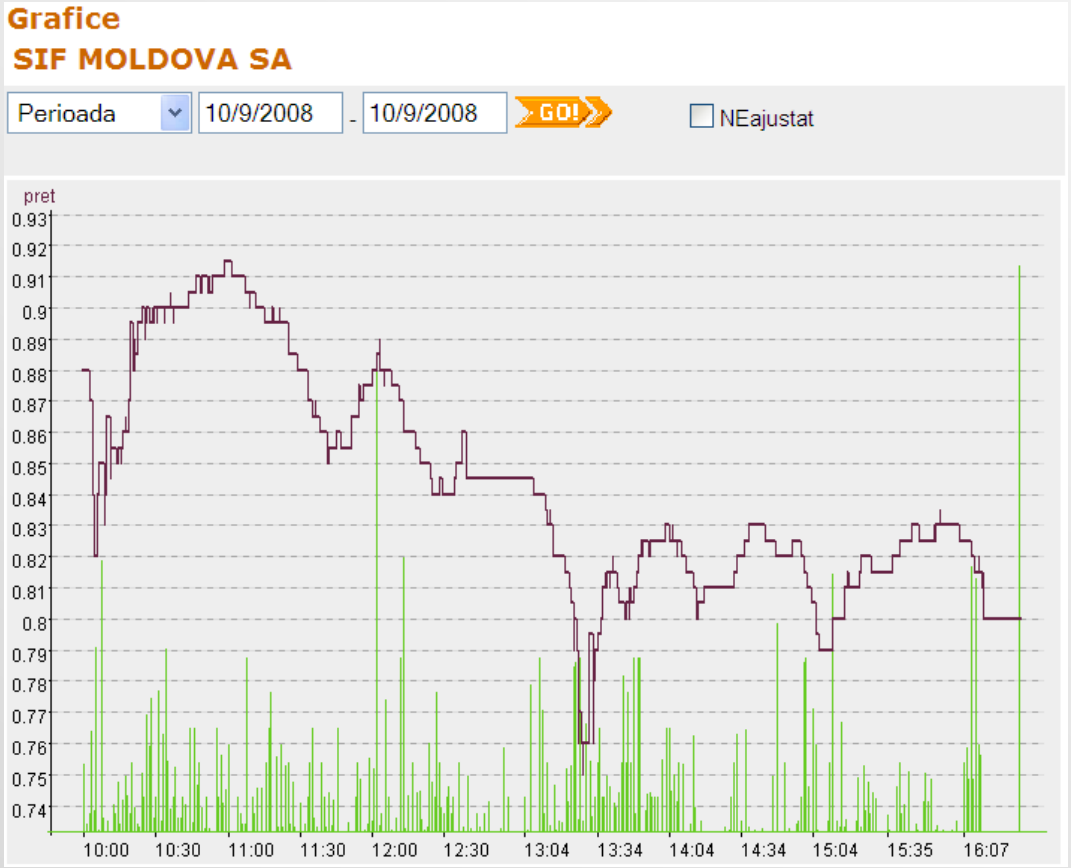
- Motivație
- Necesitate
- Obiective
- Impact

If you don't know where you're going, any road will take you there.

Criza dezvoltării software

- Erori grave
  - Sonde spațiale pierdute (Venus ’60, Marte 99)
  - Criza rachetelor – Cuba 1979
  - Rachetele Patriot 1991
  - Primul zbor Ariane 5 1996, artificii de 7 miliarde $
  - Aeroportul Denver 1994-1996
  - Anul 2000
  - Incidente în fiecare lună
    - bursa din Tokyo
    - accidente de circulație
- Proiectarea software
  - Livrarea în întârziere a tuturor proiectelor
  - Cost mult ridicat față de cel prevăzut
  - Livrarea unui produs de proastă calitate
  - Eșuarea în majoritatea cazurilor

Studiu de caz: Lucrare de doctorat



Sistem expert pentru prognoza acțiunilor bursiere

Studiu de caz: Lucrare de doctorat

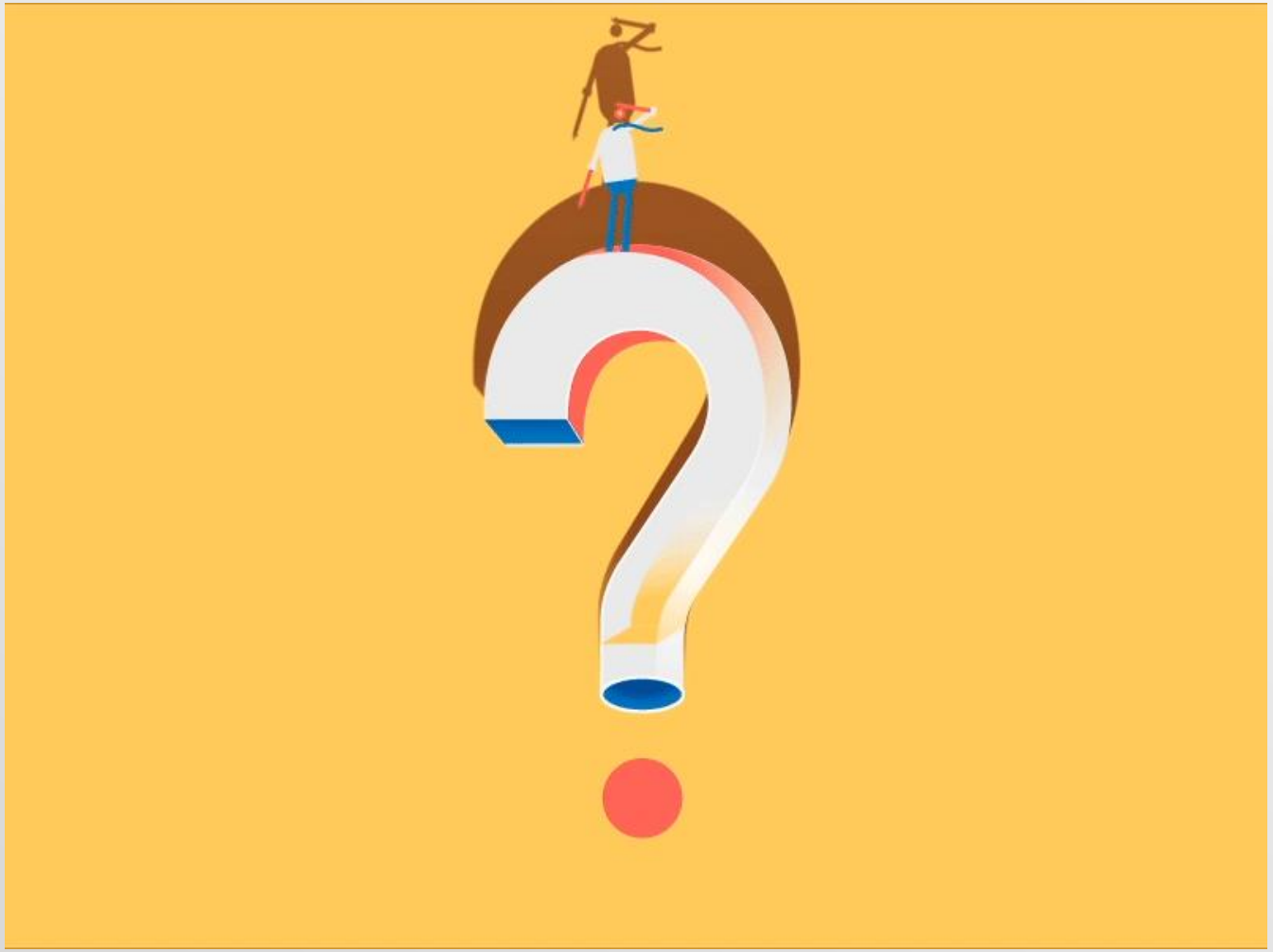
- Interval de estimare: 10 secunde
- Evaluare proces stochastic: Lanțuri Markov
- 70% predictie corectă. E suficient?
- 99% știință, 1% rutină

Concluzii

- Evaluare comision
- Omul de știință nu este neapărat înclinat practic
- Un bun manager asigură un produs complet și viabil

Good project managers know when not to manage a project.

Mulțumesc



Curs #1

Introducere

Motivație (1)

- Multe cursuri de Computer Science sau Electronică se concentrează doar pe aspectele tehnice și pe notații pentru buna dezvoltare a software-ului (ex: cum are loc procesul de testare; cum se scriu cerințele funcționale; cum se modelează sistemele; UML; Java)
- În schimb, pentru a obține software de calitate, este necesar un process bine definit și gestionat, ce organizează activitățile într-o manieră eficientă și controlată

Motivație (2)

- Să considerăm următoarele:
  - Redactarea corectă a cerințelor funcționale nu este de ajutor în cazul în care nu există un proces controlat pentru acceptarea și urmărirea modificărilor
  - Tehnicile de testare sunt inutile dacă nu mai există timp rămas pentru testare (ex: în cazul în care timpul necesar pentru dezvoltare a fost subestimat, iar lansarea software-ului a fost întârziată)
  - Dezvoltarea unui sistem software necesită execuția, monitorizarea și controlul diferitelor activități puțin sau deloc conectate cu scrisul codului (ex: instruirea utilizatorilor, prezentarea produsului software, gestionarea publicității si a comunicării, redactarea documentației)

Motivație (3)

- Dacă se dorește livrarea unui produs la timp și respectând bugetul, având de asemenea proprietățile de calitate dorite, sunt necesari:
  - Un proces pentru definirea planificării, a bugetului și punerea de comun acord a caracteristicilor de calitate a produsului
  - O listă de tehnici pentru definirea, stabilirea, planificarea, execuția și monitorizarea: obiectivelor, calității, timpului și costurilor

Competențe și obiective ale cursului

- Gestionarea unui proiect (software) necesită competențe, aptitudini și tehnici specifice
- O parte dintre întrebările ce vor putea fi răspunse la sfârșitul cursului sunt:
  - Cum pot estima cât de mult va dura completarea unui task?
  - Care este suma de bani pe care o voi solicita pentru un proiect?
  - Cum pot să mențin echipa motivată și să mă asigur că proiectele reprezintă pentru angajați o ocazie de a învăța, crește și avansa în carieră?
  - Cum pot gestiona riscurile unui proiect?
  - Cum evaluez dacă proiectul respectă timpul, bugetul, deadline-ul?
  - Cum controlez calitatea produsului final?

Managementul Proiectelor Software (MPS)

- Tehnicile de **Project Management** sunt multidisciplinare

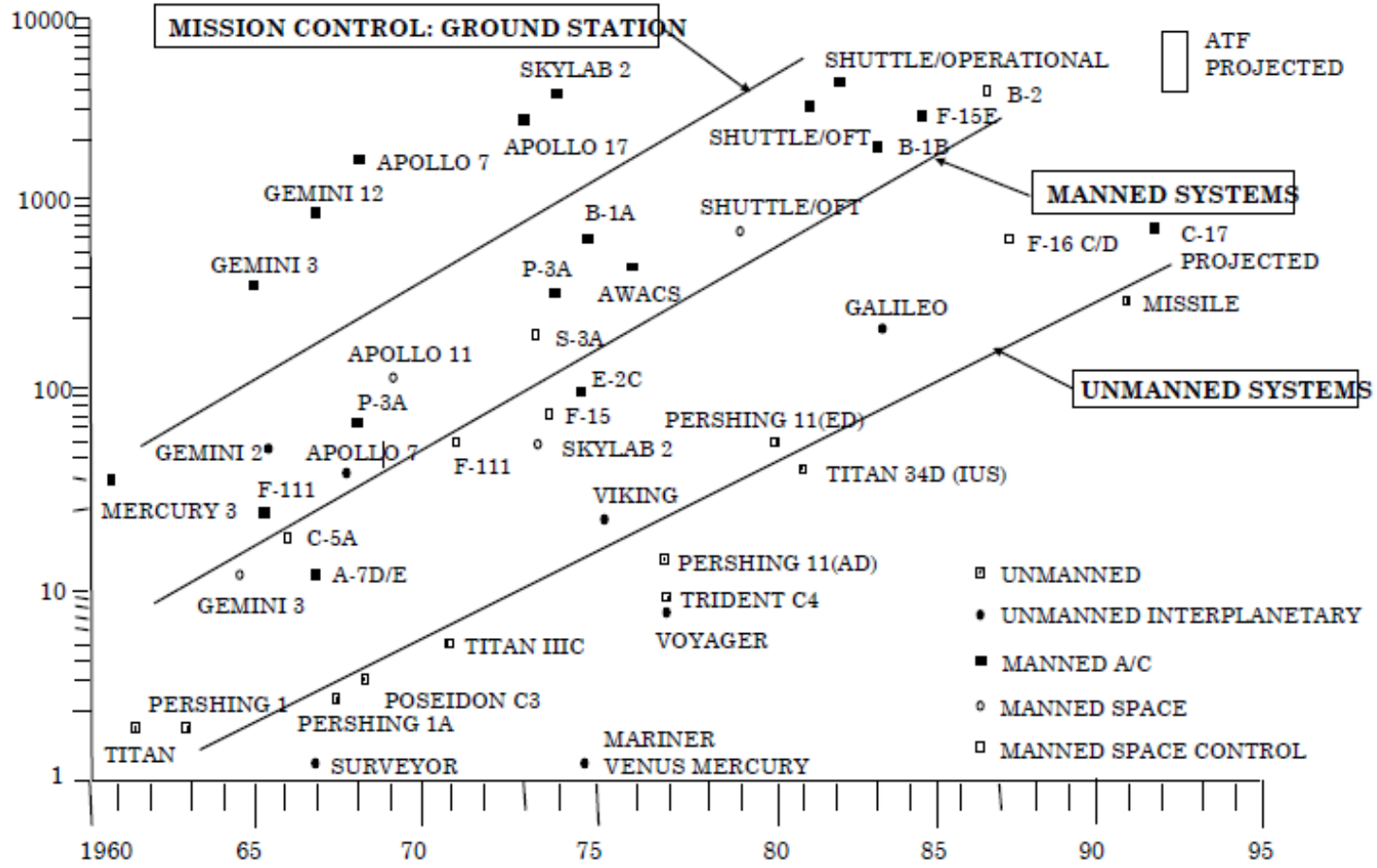
  - ceea ce veți învăța în acest curs este aplicabil pentru orice alt domeniu (al ingineriei).

- Există unele caracteristici care fac managementul proiectelor software interesant.

Managementul Proiectelor Software (MPS)

- Managementul proiectelor software este interesant și provocator pentru că:
  - Produsul este intangibil
  - Produsul este flexibil (ex: dimensiuni diferite, constrângeri diferite)
  - Multe proiecte software sunt irepetabile (pot fi realizate doar o data)
  - Procesul de dezvoltare este flexibil
  - Dimensiunea și complexitatea cresc exponențial
  - Viețile oamenilor pot depinde de buna funcționare a produselor software (ex: sistemul de control al unei aeronave) – sisteme critice de siguranță

Complexitate (1)



Sursă: “Mars and Beyond: NASA’s Software Challenges in the 21st Century”

Dr. Michael R. Lowry. NASA Ames Research Center. 5 decembrie 2003
web.cecs.pdx.edu/~black/S3S/MichaelLowry.ppt

Complexitate (2)

- Întregul ansamblu al rachetei Saturn V (propulsoarele, modulul de comanda și modulul lunar) avea o capacitate de calcul mai mică decât un telefon mobil standard din ziua de azi.

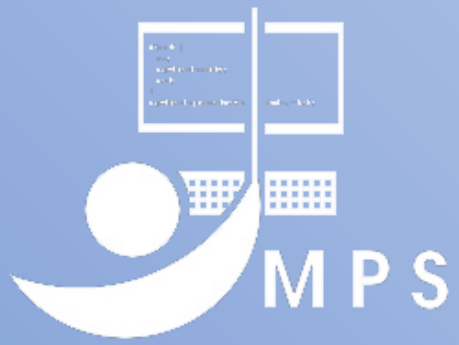
Sursa: **Apollo**

by Charles Murray and

Catherine Bly Cox

O scurtă istorie

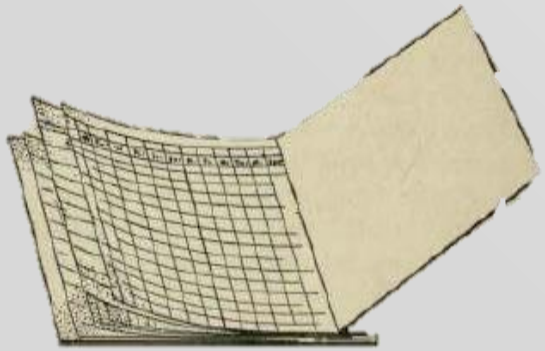
a Managementul Proiectelor (Software)



Frederick Winslow Taylor

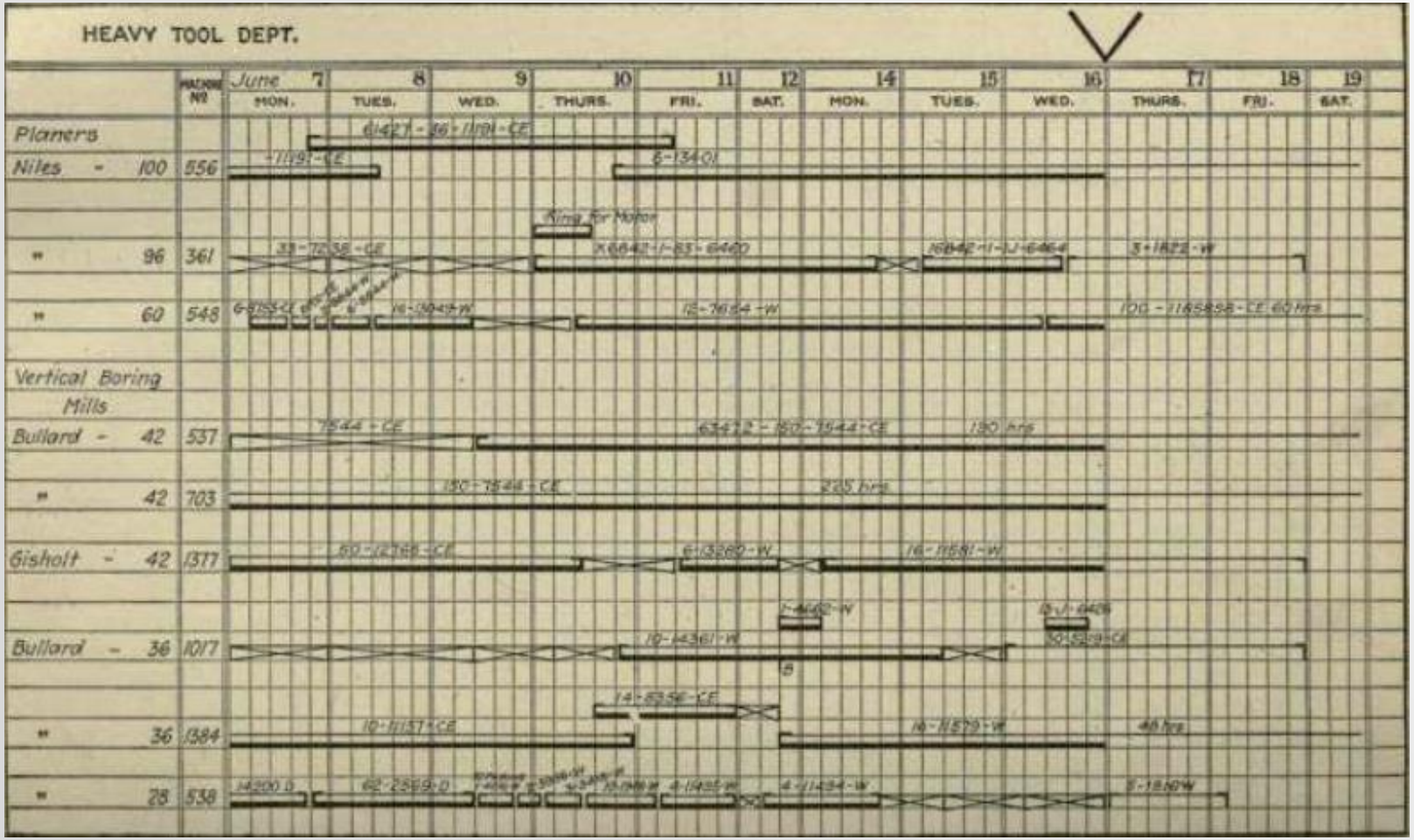
- “Scientific Management” al lui Taylor este prima teorie despre muncă și management
- Începutul anilor 1900
- Viziune negativă despre muncitori (atunci când fac task-uri repetitive, ei lucrează la acea capacitate minimă necesară pentru a scăpa nepedepsiți)
- Alte caracteristici interesante:
  - Definirea științifică a muncii
  - Selecția științifică a personalului
  - Împărțirea responsabilităților între muncitori și conducere
  - Stimulente și perioade de odihnă (pentru a crește eficiența muncitorilor)

Henry Gantt



- Diagramele Gantt încă sunt folosite în ziua de azi pentru planificarea proiectelor
- Definite în timpul Primului Război Mondial
- Folosite prima oară în planificarea și monitorizarea lucrului și progresului în construcția vapoarelor: distinctive între lucru și progress
- Cartea sa este disponibilă pentru descărcare pe archive.org

Exemplu de Diagrama Gantt



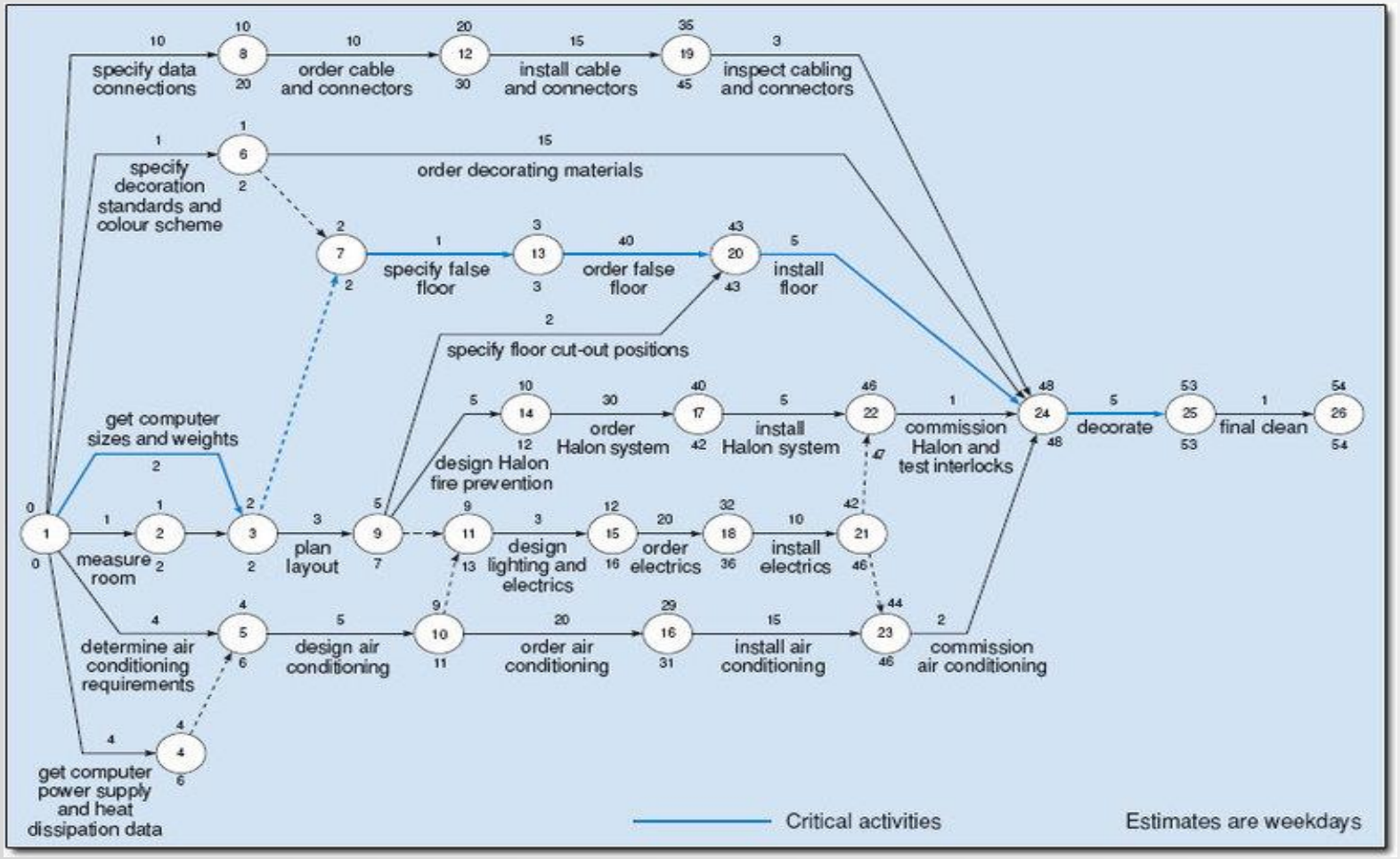
Sursă: “The Gantt chart, a working tool of management” – Clark, Wallace si Gantt

Anii ‘50 și ‘60

- Al Doilea Război Mondial:
  - **Proiectul Manhattan**: diagrame de flux de proces
- 1957 **CPM** (Critical Path Method – metoda drumului critic)
  - Algoritm matematic pentru planificarea unui set de activități ale unui proiect, folosit pentru planificarea activităților de mentenanță în fabrici
  - Dupont + Remington Rand => echipa UNIVAC
  - Nu a suferit modificări fundamentale până în prezent
- 1958 **PERT** (Program Evaluation and Review Technique)
  - Programul de rachete Polaris al marinei SUA (Booz Allen & Hamilton – firma de consultanță managerială – lucrând ca echipa ORSA pentru Lockheed Missile System)

Motivații pentru CPM si PERT

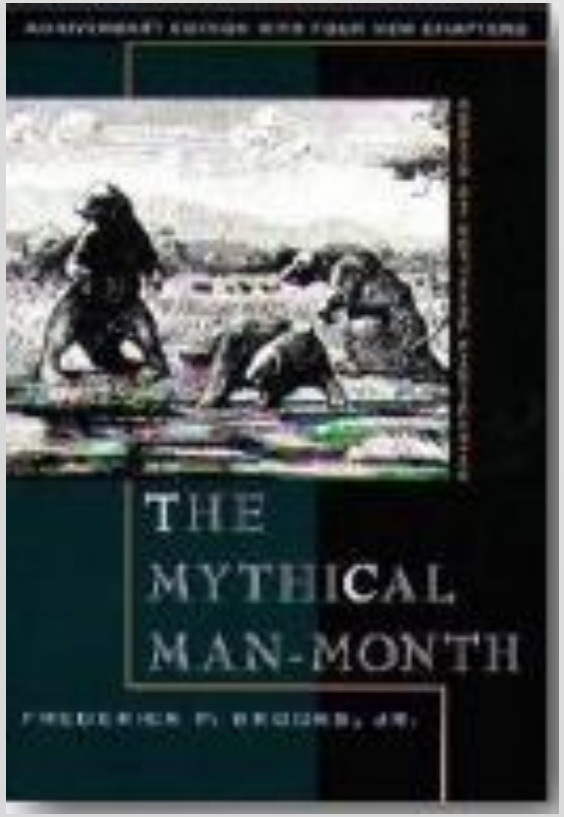
Ce activități pot întârzia finalizarea proiectului din următorul plan?



Anii ‘60 și ‘70

- 1960: contracte importante ale Guvernului SUA (Vietnam, centrale nucleare, NASA Apollo): standardizare și automatizare
  - PERT/COST și WBS devin obligatorii în proiectele sponsorizate de Guvern
  - Metoda Analizei Valorii (Earned Value Analysis - EVA) este definită
  - Managementul configurațiilor
  - Organizații de project management (PMI, IPMA) promovează profesia și tehnicile
  - (1961) IBM folosește PM în mod comercial

Anii ‘60 și ‘70



- 1970: Dezvoltarea Software intră în ecuație
  - EVA dezvoltat pentru monitorizarea programului și a costului
  - Modelul cascadă pentru devoltarea software
  - “The mythical man-month” subliniază numeroase capcane ale dezvoltării software

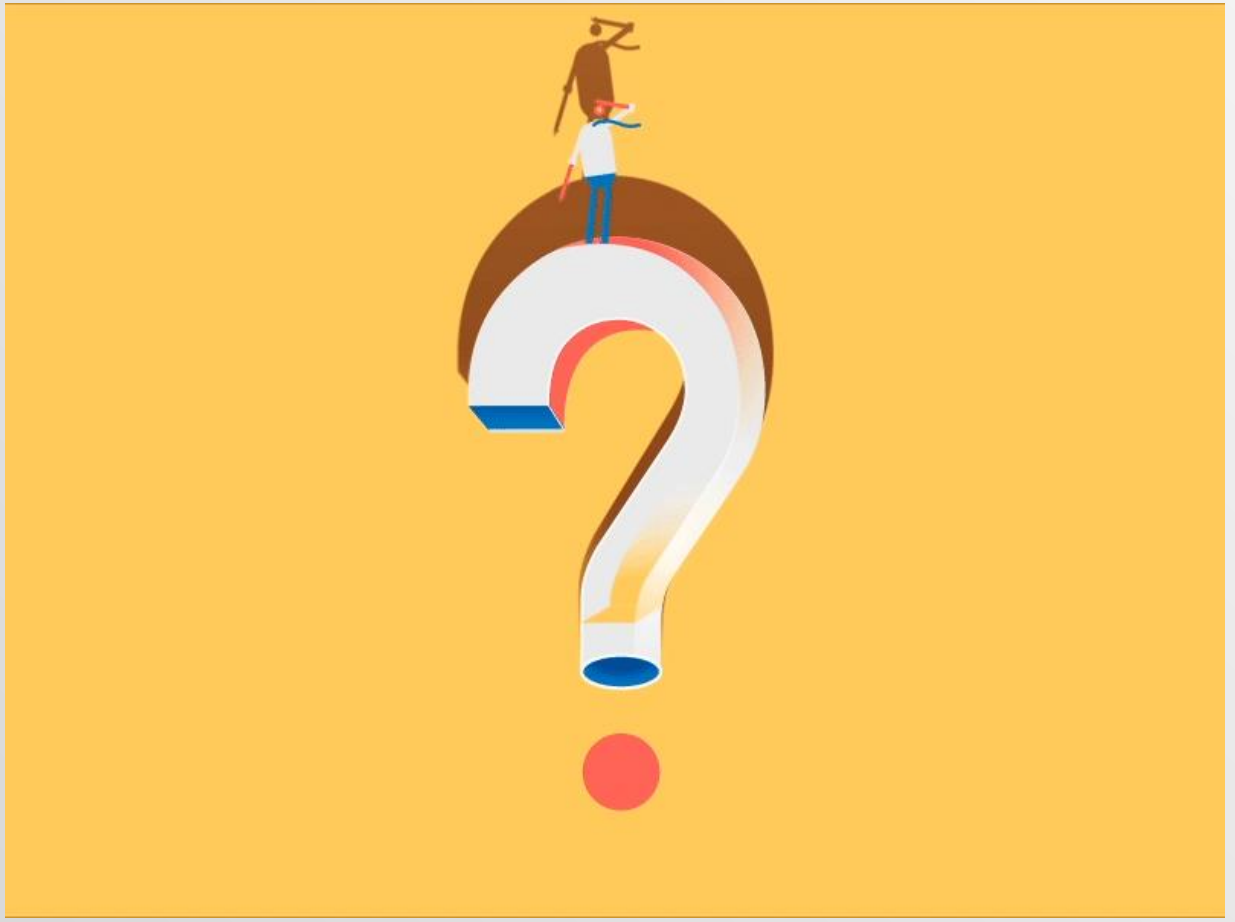
Revoluția Computerelor

- 1980
  - Proliferarea hardware-ului și a software-ului fac tool-urile de PM accesibile și pentru firmele mai mici
  - Capacitatea hardware crește exponential (Legea lui Moore) și la fel și software-ul
  - Modele de estimare (FP și COCOMO) sunt introduce pentru a prezice complexitatea software-ului

Revoluția Computerelor

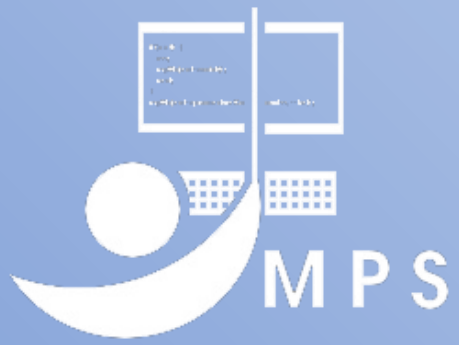
- 1990
  - Calitate totală
  - Organizații mai flexibile, mai rapide și mai receptive
- Astăzi
  - Aplicații Web și noi modele de distribuție a aplicațiilor
  - Dezvoltare prin componente și framework-uri
  - Agilitate, interacțiune rapidă, feedback constant

Mulțumesc



Proiecte și Stakeholderi

Numele jocului, jucătorii și (unele) reguli

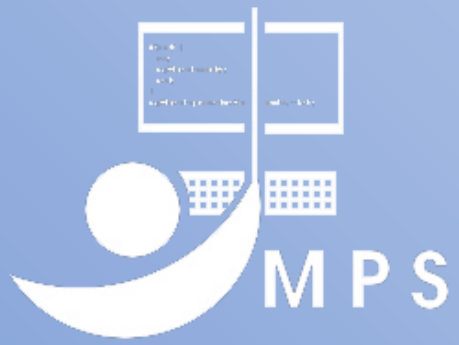


Obiective

- Înțelegerea noțiunii de proiect, a ciclului de viață al unui proiect
- Înțelegerea părților implicate (stakeholders) și a relațiilor dintre acestea
- Înțelegerea influenței organizațiilor asupra proiectelor

Ce este un proiect?

Numele jocului

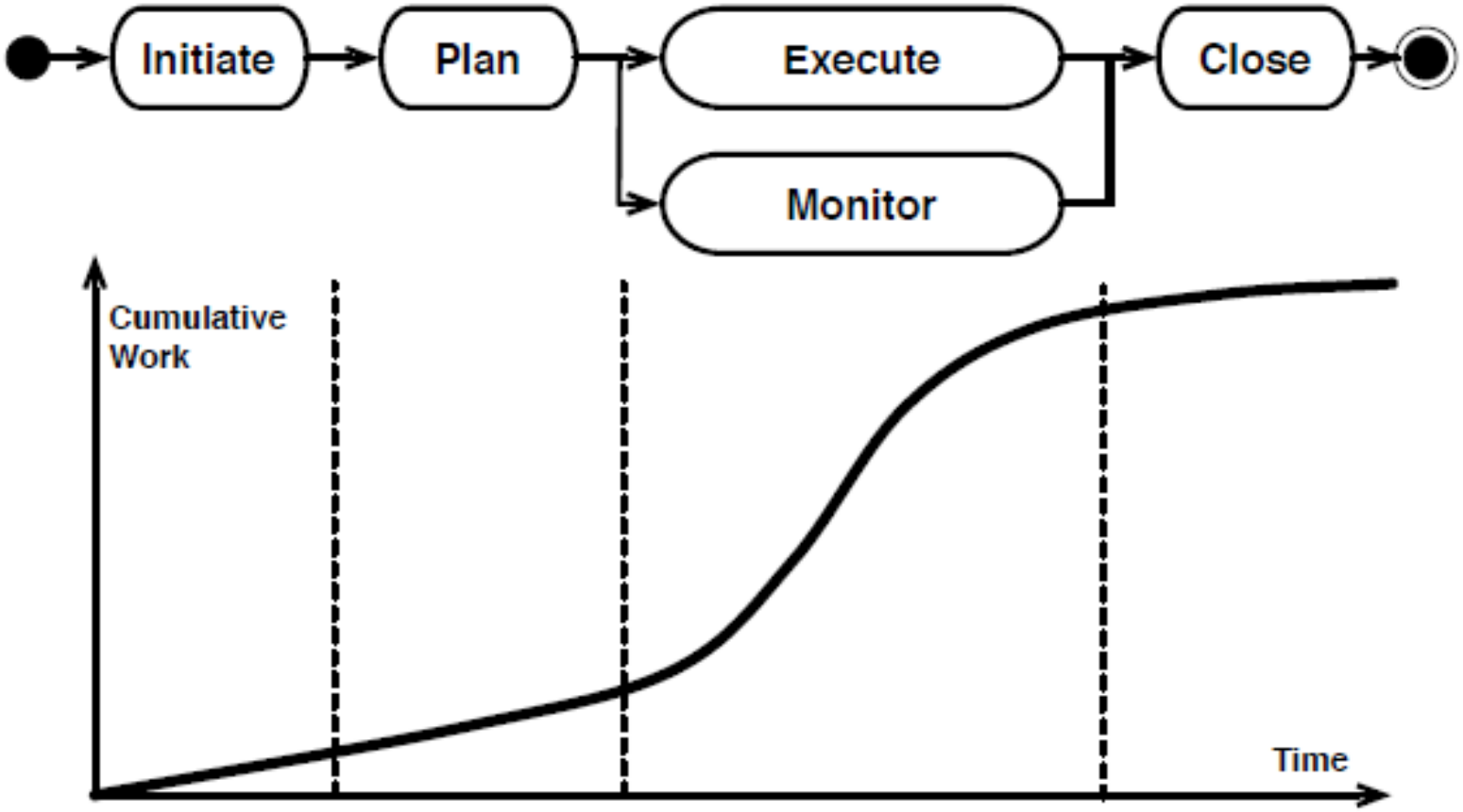


Un proiect este un efort **temporar** realizat pentru crearea unui produs, serviciu sau rezultat **unic**.

Caracteristicile unui proiect

- Temporar
  - Un început și un sfârșit definite (obiectivele sunt îndeplinite sau proiectul a fost finalizat – caz în care obiectivele nu sunt îndeplinite)
  - Rezultatele proiectelor nu sunt neapărat temporare (ciclul de viata al proiectelor)
- Produse, servicii sau rezultate unice
  - Un produs cuantificabil (ex: o componentă, ...)
  - Capacitatea de a realiza un serviciu (ex. printr-o funcție de business)
  - Un rezultat, de exemplu cunoștințe (strânse în documente, prezentări...)
- Elaborare progresivă
  - Dezvoltare prin pași și incremente (necesară pentru a păstra în vedere scopul proiectului)
- Constrângeri de resurse

Elaborare progresivă



Contextul managementului proiectelor

- Subproiecte
  - Proiectele pot fi împărțite în subproiecte (uneori numite și ele tot “proiecte” și administrate ca atare)
- Managementul proiectelor și programelor
  - Set de proiecte înrudite și manageriate într-o manieră coordonată pentru obținerea unui anumit tip de beneficiu
- Managementul portofoliului
  - Colecție de proiecte / programe fără legătură, dar grupate împreună pentru facilitarea managementului și îndeplinirea obiectivelor strategice

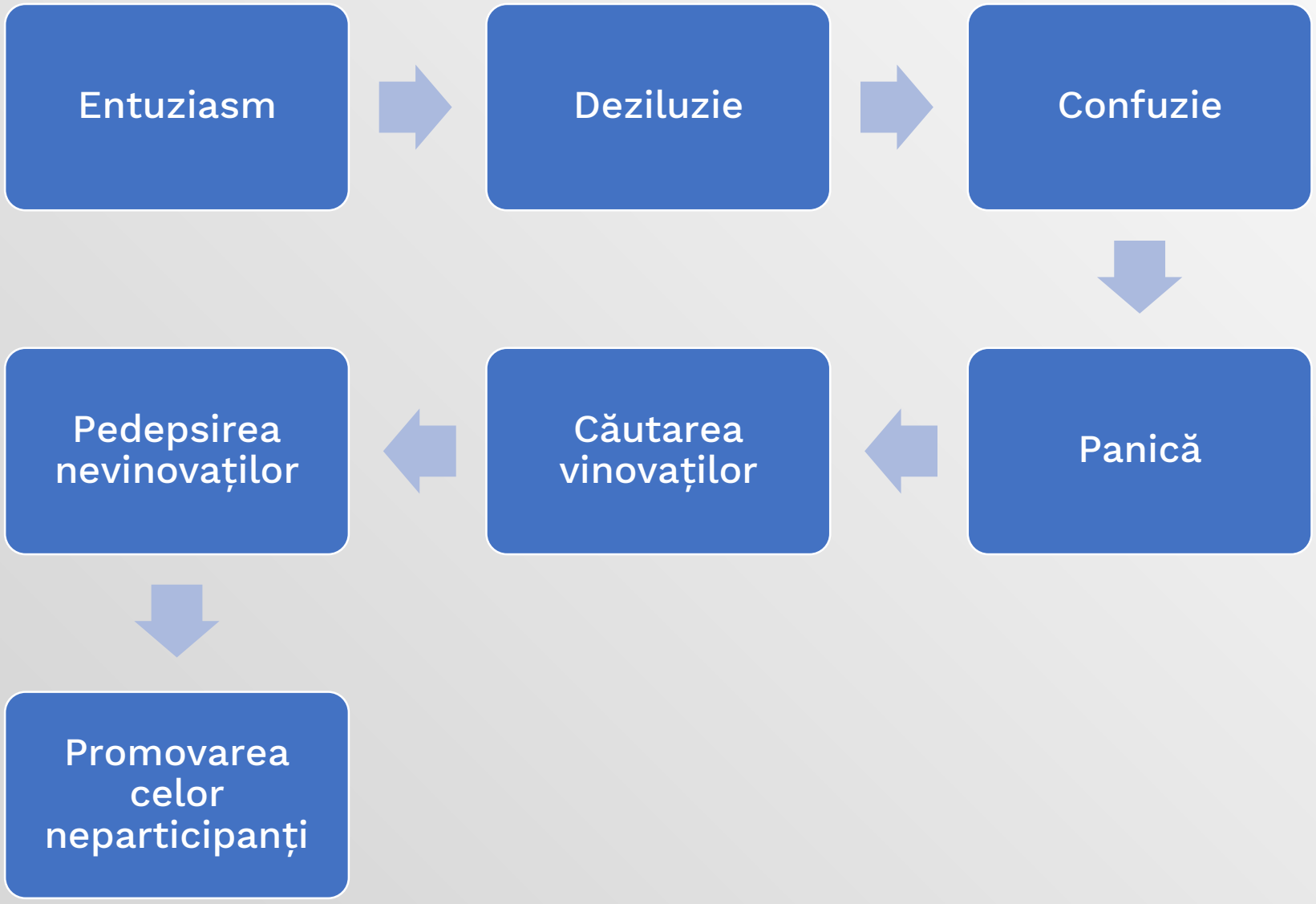
Proiecte și proceduri operaționale

- Lucrul poate fi pe bază de proiect sau operațional
- Caracteristici comune:
  - Realizat de oameni
  - Resurse limitate
  - Planificat, executat și controlat
- Diferențe
  - **Proiect**: obținerea obiectivelor și finalizare
  - **Operațional**: susținerea business-ului

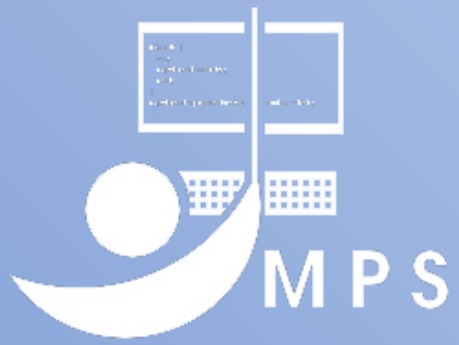
Exemple (și contraexemple)

- Pregătirea cinei
- Fabricarea unui mașini
- Design-ul unei mașini
- Redactarea unui articol
- Dezvoltarea unui sistem software
- Menținerea unui sistem software
- Managementul personalului

Cele 7 faze ale unui proiet



Proiecte de Dezvoltare Software



Tipuri de Proiecte de Dezvoltare Software

În cariera de project manager, o persoană poate fi implicată în diferite tipuri de proiecte de dezvoltare software, precum:

- Dezvoltarea de aplicații
- Reingineria proceselor și a sistemelor
- Integrarea sistemului
- Servicii de consultanță
- Instalare și training

A. Dezvoltarea de aplicații

- Scop: dezvoltarea unei aplicații (desktop, web, mobil, incorporate)
- Tipul cel mai distractiv 😂
- Tipuri de dezvoltare de aplicații:
  - **One-off-uri**: sisteme create special pentru client
  - **Off-the-shelf**: pentru satisfacerea nevoilor unei game largi de utilizatori
  - **Off-the-shelf customizat**: sisteme standardizate ce necesită o cantitate semnificativă de customizare pentru a putea fi folosite într-o organizație. Ex: sisteme de tip ERP (Enterprise Resource Planning)

B. Reingineria proceselor și a sistemelor

- Scop: schimbarea modalității de desfășurare a procedurilor operaționale din cadrul unei organizații în vederea realizării unui scop strategic (ex: îmbunătățirea calității, eficienței)
- De obicei proiecte ample, care necesită o analiză precisă a situației existente din punct de vedere al procedurilor, sistemelor, infrastructurii
- Necesită adesea introducerea unui sistem ERP și a activităților de integrare a sistemului și a datelor

C. Servicii de integrare a sistemului

- Scop: automatizarea fluxului informațional în cadrul sistemelor unei organizații
- Tipuri de integrare:
  - **Orizontal**: integrarea sistemelor ce realizează operațiuni similare
  - **Vertical**: integrarea sistemelor care automatizează diferite etape ale unei proceduri

Alte tipuri de proiecte

- D. Servicii de consultanță
  - Cerută de cele mai multe ori pentru a aduce expertiză din exterior echipei
- E. Servicii de instalare și training
  - Servicii legate de instalare și training pentru sisteme software specific
  - Remarcă: constituie de asemenea sursă de venit în dezvoltarea open source

Proiectele și mediul lor

Stakeholder

Un **stakeholder** reprezintă un individ sau o organizație ce este implicat(ă) activ într-un proiect, sau al cărui interes ar putea fi influențat (pozitiv sau negativ) în urma realizării sau finalizării proiectului.
(PMBOK)

Stakeholder

- Câteva caracteristici:
  - Pot avea influențe diferite și diverse niveluri de responsabilitate în timpul proiectului
  - Pot juca roluri diferite
  - Pot avea o influență pozitivă sau negativă asupra proiectului
  - Pot fi dificil de identificat
  - Lipsa intervenției lor poate influența în mod negativ proiectul (nevoie de identificare și implicare)
- Remarcă: managerul de proiect și echipa de proiect **sunt** și ei stakeholderi în cadrul proiectului, deși termenul este adesea folosit pentru a desemna “ceilalți” stakeholderi

Tipuri de stakeholderi

- Project managerul
- Echipa de proiect
- Sponsorul proiectului
- Organizațiile participante
- Partenerii
- Clientul
- Restul: oricine ar putea fi afectat de rezultatele proiectului

Stakeholderi Cheie

- Interni:
  - Membrii echipei de proiect: grupul ce implementează
  - Echipa de management a proiectului: membrii echipei direct implicați în activitatea de project management

- De mijloc:
  - Client / Utilizator: persoană sau organizație ce va utiliza rezultatele proiectului. Pot fi mai multe niveluri de utilizatori.
  - Sponsor: persoană sau grup ce furnizează resursele financiare
  - Organizație participant: organizația implicată cel mai mult în realizarea proiectului

- Externi:
  - Influenceri: persoane sau grupuri neimplicate direct în proiect, dar care i-ar putea influența cursul

Exercițiu – Identificare Stakeholderi

Identificați stakeholderii următoarelor proiecte:

- Construirea unui pod ce conectează o insulă de continent;
- Construirea unui depozit de deșeuri (groapă de gunoi)
- Construirea unei foi de calcul (spreadsheet) open source
- Dezvoltarea unei aplicații web pentru monitorizarea masei corporale
- Portarea OpenOffice (suita Office gratuita) pe Android

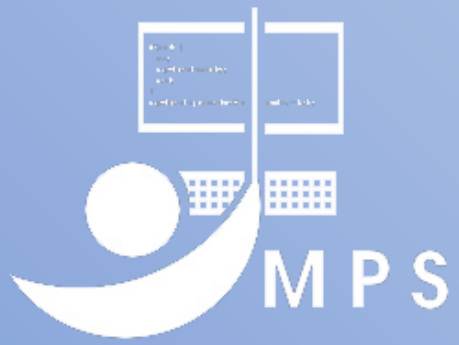
Managerul de Proiect

Persoana responsabilă cu gestiunea proiectului și a așteptărilor stakeholderilor.

Câteva competențe necesare:

- Comunicare și negociere
- (Oarecare) Predispoziție către riscuri
- Orientare către scop
- Abilități de leader
- Gândire creativă, în afara tiparelor
- Cunoștințe tehnice solide
- Corectitudine profesională
- Bun simț
- Stil

Organizarea Dezvoltării Proiectelor Software



Managementul Proiectelor Software

- **Managementul proiectelor software (Software project management)** reprezintă integrarea tehnicilor de management în dezvoltarea software

- Nevoia unei asemenea integrări își are rădăcinile în anii ‘60, în zilele “crizei software”, când specialiștii recunoșteau complexitatea crescătoare a livrării produselor software ce îndeplinesc specificațiile

Ce face un Software să fie considerat Produs?

- O aplicație pentru gestionarea filmelor vizionate
- Un sistem de contabilitate cu intrare dublă pentru acasa sau pentru mici intreprinderi
- Un serviciu web pentru gestionarea rezervărilor terenurilor de squash într-un club de squash
- Un sistem de planificare a resurselor într-o corporație
- Un sistem de frânare automat pentru trenuri

- Din ce componente este compus?
- Care ar fi efectele în cazul unui eșec?
- Ce competente sunt necesare pentru dezvoltare?
- Care este modelul de business asociat?

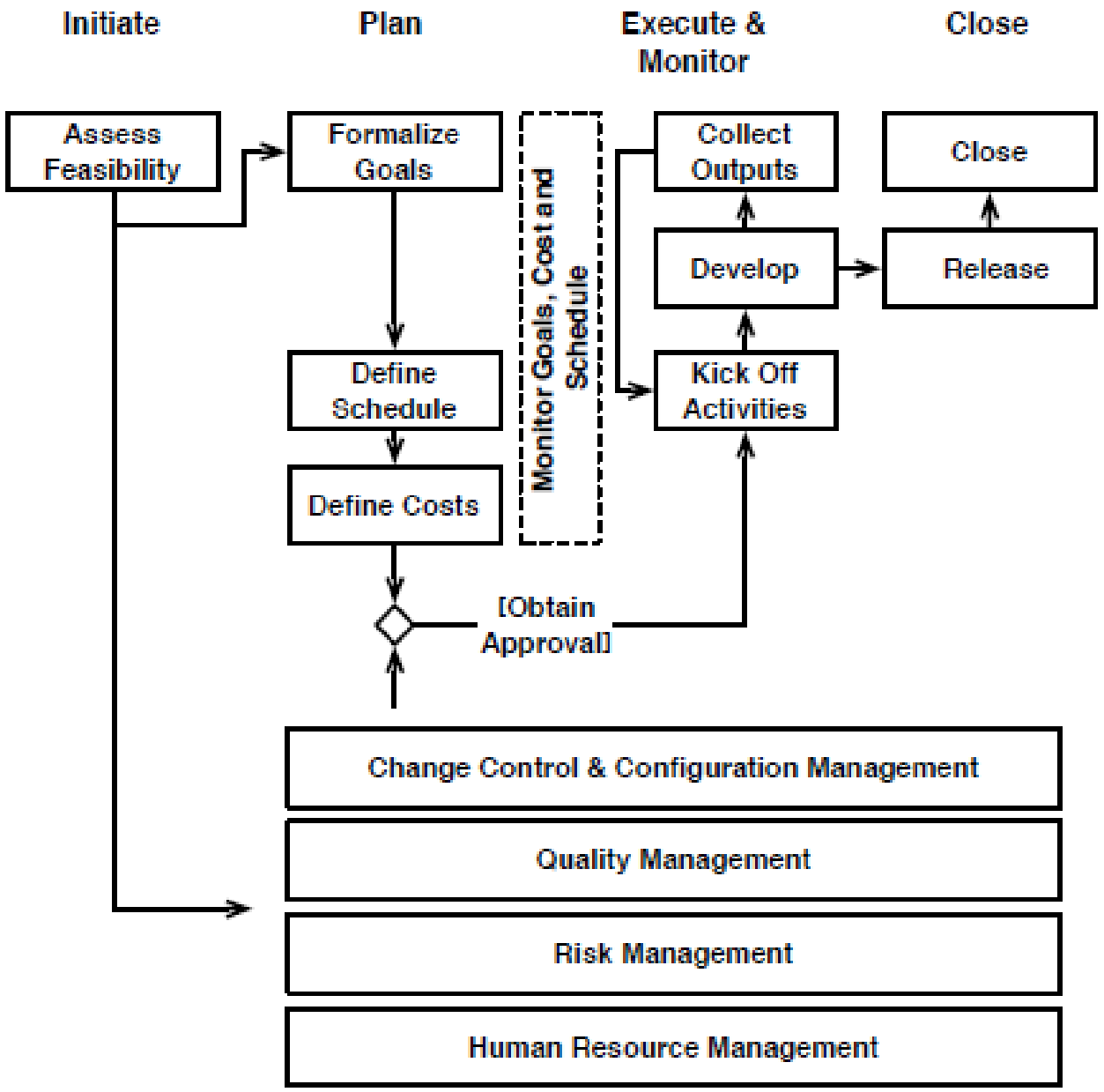
Framework de Dezvoltare Software

Un framework general trebuie să:

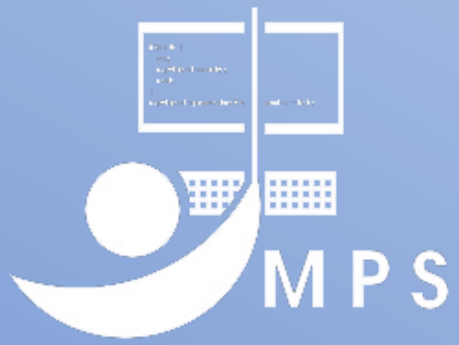
- Formeze o viziune comună despre scopurile ce trebuie realizate, caracteristicile rezultatelor proiectului și caracteristicile procesului de dezvoltare
- Structureze lucrul ca o perfecționare progresivă, de la specificații la scopuri
- Reducă impactul nesiguranțelor și a necunoscutelor
- Evidențieze orice deviere de la plan (scopuri, costuri, calitate)
- Asigure coerență și calitatea componentelor proiectului de-a lungul timpului în pofida necunoscutelor și a schimbărilor (sau a cererilor de schimbare)
- Motiveze echipa

Câteva preocupări

- Evaluarea fezabilității
- Managementul scopurilor
- Managementul timpului
- Managementul costurilor
- Controlul schimbărilor și managementul configurației
- Managementul calității
- Managementul riscurilor
- Managementul resurselor umane


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

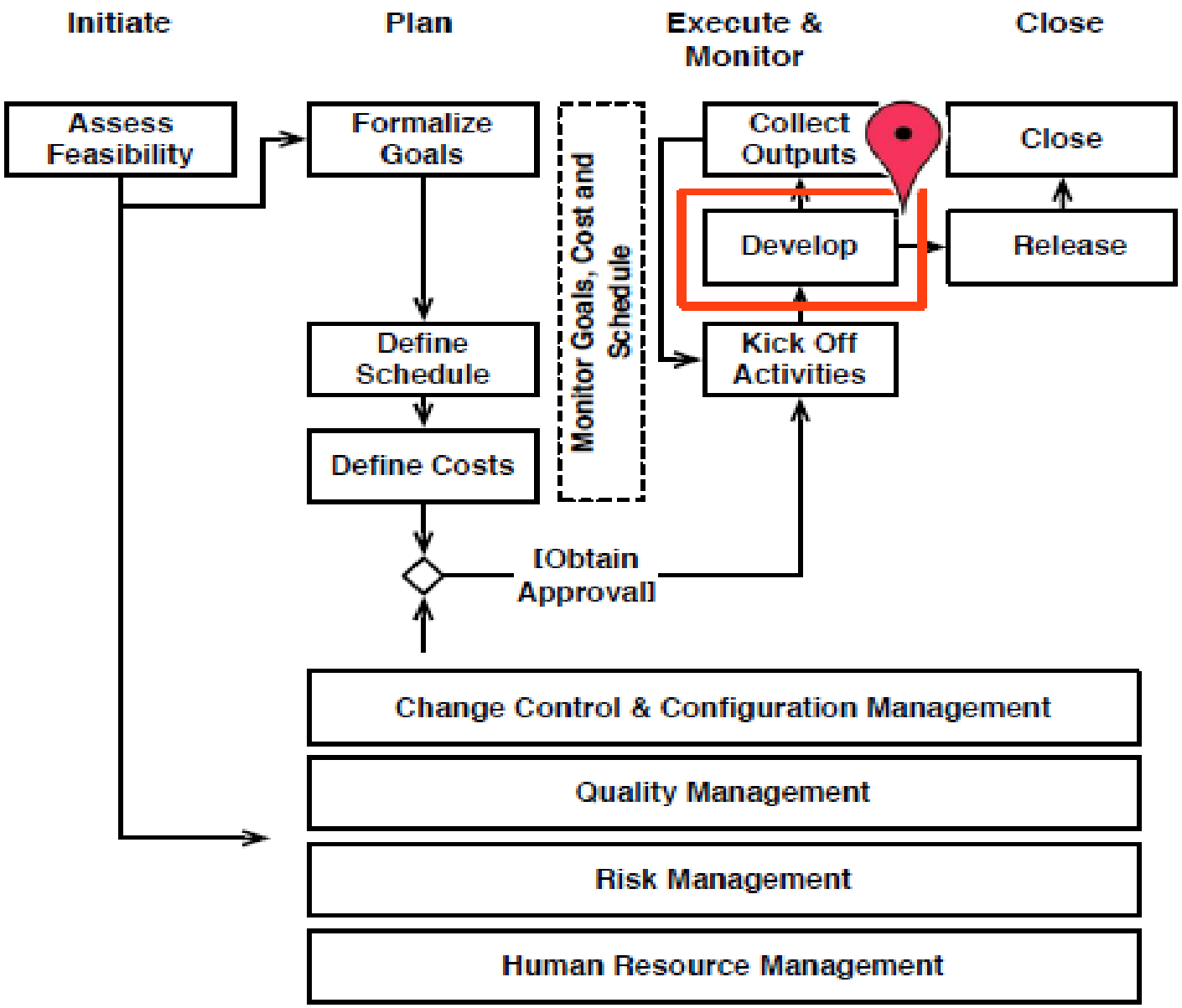
Procese (tradiționale) de dezvoltare a Software-ului



Good project management is not so much knowing what to do and when, as knowing what excuses to give and when.

Obiective

- O introducere subtilă și la nivel înalt în activitățile de dezvoltare a software-ului
- Descoperirea pietrelor de temelie necesare pentru producerea de software
- Remarci:
  - Acest curs nu îl inlocuiește pe cel de software engineering
  - Activitățile trebuie integrate într-un process coerent pentru a avea sens
  - Proiectele de dezvoltare a software-ului variază de la extrem de reduse la extrem de extinse … nu toate activitățile sunt la fel de utile sau relevante în orice context


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Define Schedule
Define Costs
Monitor Goals, Cost and Schedule
[Obtain Approval]
Collect Outputs
Develop
Kick Off Activities
Close
Release
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Prezentare generală

- Dezvoltarea de software necesită o perfecționare progresivă, ce transforma conceptul în operații prin următoarele faze:
  - Stabilirea cerințelor si a experienței utilizatorilor
  - Design
  - Implementare
  - Verificare și validare
  - Deployment
  - Operații și mentenanță
- O dată cu parcurgerea acestor faze, se iau anumite alegeri specifice; costul schimbărilor crește în consecință
- Procese diferite pun accentul pe fiecare activitate sau definesc ordinea în care aceste activități pot fi efectuate.

Managementul Cerințelor

Cerințe

- Scop: formarea unei viziuni comune despre caracteristicile sistemului ce trebuie construit

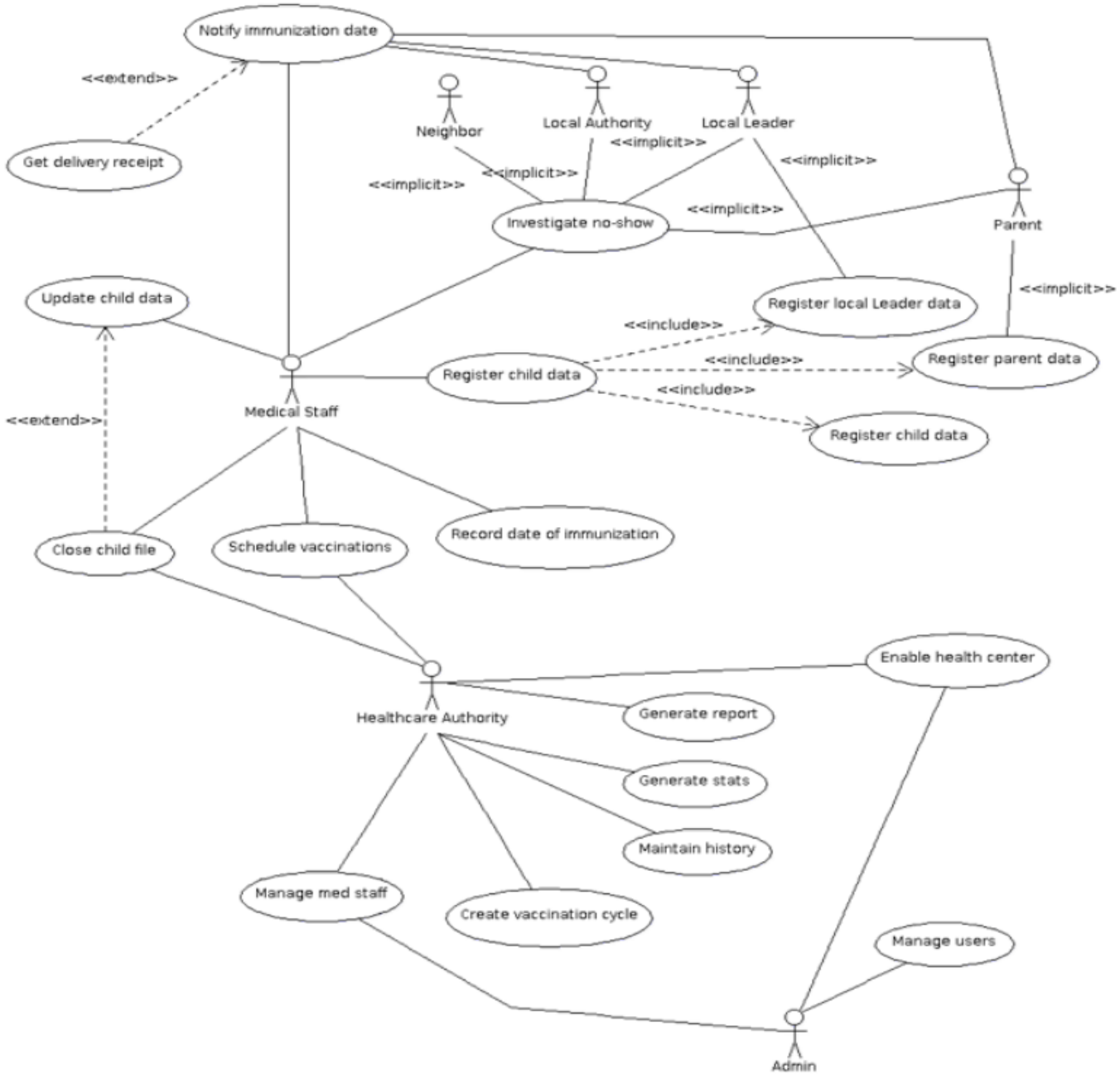
- Rezultat:
  - Lista de cerințe:
    - un document text
    - o listă de scenarii ale utilizatorilor
    - un set de diagrame (e.g. diagrame de cazuri de utilizare) și descrierile textuale corespunzătoare

Lista de cerințe

- Format:
  - Text liber sau structurat descriind funcționalitățile și celelalte proprietăți ale sistemului
- Avantaje:
  - Simplu de schițat și distribuit
  - Formatul poate fi utilizat pentru a păstra evidența schimbărilor (versionare)
- Dezavantaje:
  - Nu se concentrează pe interacțiunea cu utilizatorul: poate fi ușor de înțeles de către client
  - Ambiguități și incoerențe; interacțiuni între cerințe

Diagramele cazuri de utilizare (use case)

- Format:
  - Diagrame ce descriu interacțiunea dintre utilizatori și sistem
  - Descriere textuală a interacțiunii ca o secvență de pași
- Avantaje:
  - Intuitive, mai ușor de înțeles de către client
  - Se concentrează pe ceea ce face sistemul (functionalități)
- Dezavantaje:
  - Dificil de reprezentat și de păstrat evidența cerințelor non-funcționale
  - Gestionarea diagramelor necesită puțin mai multă muncă decât cea a textului


Notify immunization date
<<extend>>
Get delivery receipt
Neighbor
Local Authority
Local Leader
<<implicit>>
<<implicit>>
<<implicit>>
<<implicit>>
<<implicit>>
Investigate no-show
Parent
Update child data
Register local Leader data
<<include>>
<<implicit>>
Register child data
<<include>>
Register parent data
<<include>>
Medical Staff
<<extend>>
Register child data
Close child file
Schedule vaccinations
Record date of immunization
Enable health center
Healthcare Authority
Generate report
Generate stats
Maintain history
Manage med staff
Create vaccination cycle
Manage users
Admin

Povești de utilizator (de planificare)

- Format:
  - Descrieri textuale structurate ale funcționalităților: Din postura de [utilizator], vreau să fac [asta] pentru că [motiv]
- Avantaje:
  - Intuitiv, compact, mai ușor de înțeles de către client
  - Se concentrează pe ceea ce face sistemul (functionalități)
- Dezavantaje:
  - Dificil de reprezentat și de păstrat evidența cerințelor non-funcționale
  - Reprezintă o specificație partială (multe detalii trebuie stabilite în timpul implementării) – folosită în metodologiile Agile

Ingineria cerințelor

- Scop:
  - Definirea și menținerea cerințelor de-a lungul timpului
- Activități:
  - Elaborarea cerințelor (workshop-uri, brainstorming, grupuri țintă)
  - Structurarea cerințelor
  - Design-ul experienței utilizatorului
  - Validarea cerințelor

Structurarea cerințelor

- Scop: Imbunătățirea mentenanței cerințelor de-a lungul timpului
- Tool-uri:
  - Izolare și identificare: (gestionarea și manipularea mai ușoară a fiecărei cerințe)
  - Organizare și clasificare (e.g. FURPS)
  - Adnotări (prioritate, importanță, trasabilitate)

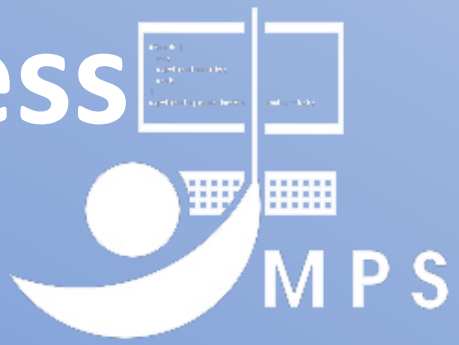
Design-ul experienței utilizatorului

- Scop: Asigurarea unei experiențe coerente și satisfăcătoare asupra diferitelor componente ale unui sistem software, incluzând design-ul, interfața, interacțiunea și manualele
- Tool-uri:
  - Analiza centrată pe utilizator: înțelegerea felului în care utilizatorii interacționează cu sistemul (grupuri țintă, experimente)
  - Design centrat pe utilizator: specificarea felului în care utilizatorii vor interacționa cu sistemul (mock-ups)

Validarea cerințelor

- Găsirea (și soluționarea):
  - Inconsistențelor
    - scenariul 1: R1. A; …; Rn: diferit de A
    - scenariul 2: R1. oricare ar fi x. A(x); …; Rn: A(c) fals
  - Cerințelor incomplete
    - comportamentul nu este specificat pentru anumite cazuri și situații (adesea situații non-nominale)
  - Duplicatelor
    - aceleași cerințe sunt descrise de două ori (posibil în feluri diferite)

Modelarea procesului de business si reingineria



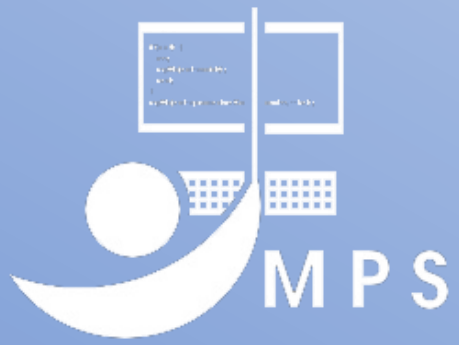
Organizații și Software

- Software-ul trebuie creat în așa fel încât să se potrivească în structura operațională a unei organizații
- Totusi: software-ul poate de asemenea schimba felul în care funcționează o organizație
- Modelarea procesului de business determină felul în care functionează o organizație
- Reingineria procesului de business planifică felul în care funcționează o organizație, în vederea eficientizării operațiunilor sale.

Modelarea procesului de business

- Proces complex, este uneori planificat și organizat ca un proiect independent
- Efectuat prin interviuri, analiza documentelor, shadowing
- Informații de colectat:
  - Structura organizațională: lanțul de responsabilitate
  - Procesele de business
  - Infrastructura IT existentă: hardware, sisteme, baze de date
  - Entități de business: datele produse și procesate de către organizație

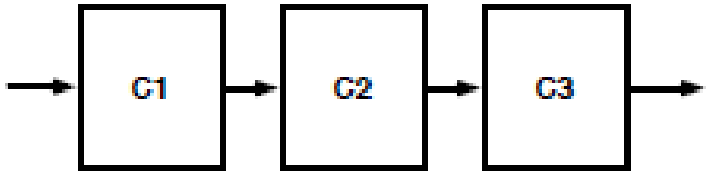
Design-ul sistemului



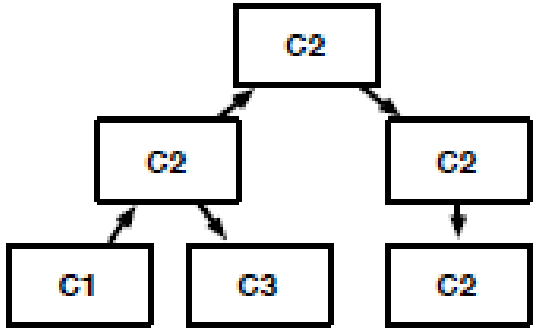
Design-ul sistemului

- Scop: Definirea structurii software-ului ce trebuie construit (= arhitectura sistemului)
- Ieșiri:
  - componentele ce constituie sistemul
  - funcțiile implementate de către fiecare componentă
  - cum sunt interconectate componentele
- Activitatea este relevanta si din motive manageriale: arhitectura sistemului produce descompunerea naturală a muncii

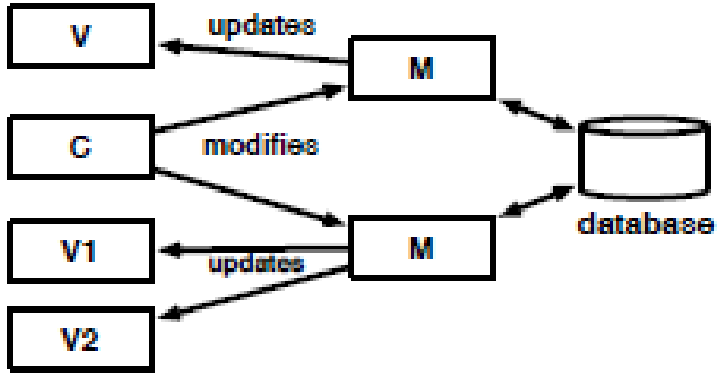
Modele Arhitecturale



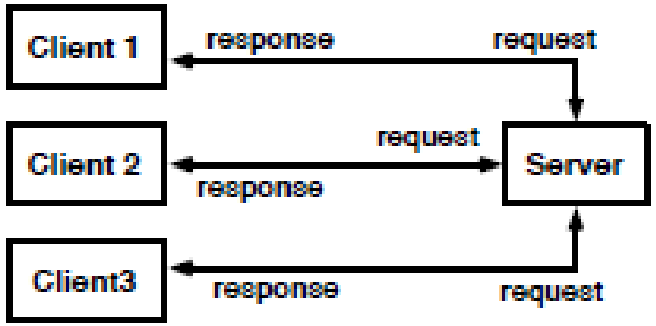
A Pipeline Architecture



A Layered Architecture



A Data-Centric applicaton with two MVCs



A client-server Architecture

Modele Arhitecturale

- “Pipe and filter”
  - Alcătuirea unităților de procesare a datelor
  - Important: specificațiile I/O
- Pe niveluri / ierarhic
  - Ierarhia componentelor
  - Important: controlul și fluxul informației; resposabilitățile blocului
- Data-Centric
  - MVC: date, prezentare, logică
  - Important: modelul de date, operațiuni
  - Multe aplicații web sau desktop folosesc acest stil architectural
- Client-server
  - Server (principalele funcții) și client (servicii de tip request)
  - Important: protocolul de comunicație / specificațiile serviciului

Implementare

Implementare

- Scop: Scriere de cod
- Câteva activități relevante pentru project management ce au loc în timpul implementării:
  - Colectarea metricilor de productivitate și mărime
  - Colectarea metricilor de calitate
  - Utilizarea standardelor de documentatie și scriere de cod
  - Practici de managementul codului (versionare; standard de lansare a codului)

Verificare și validare

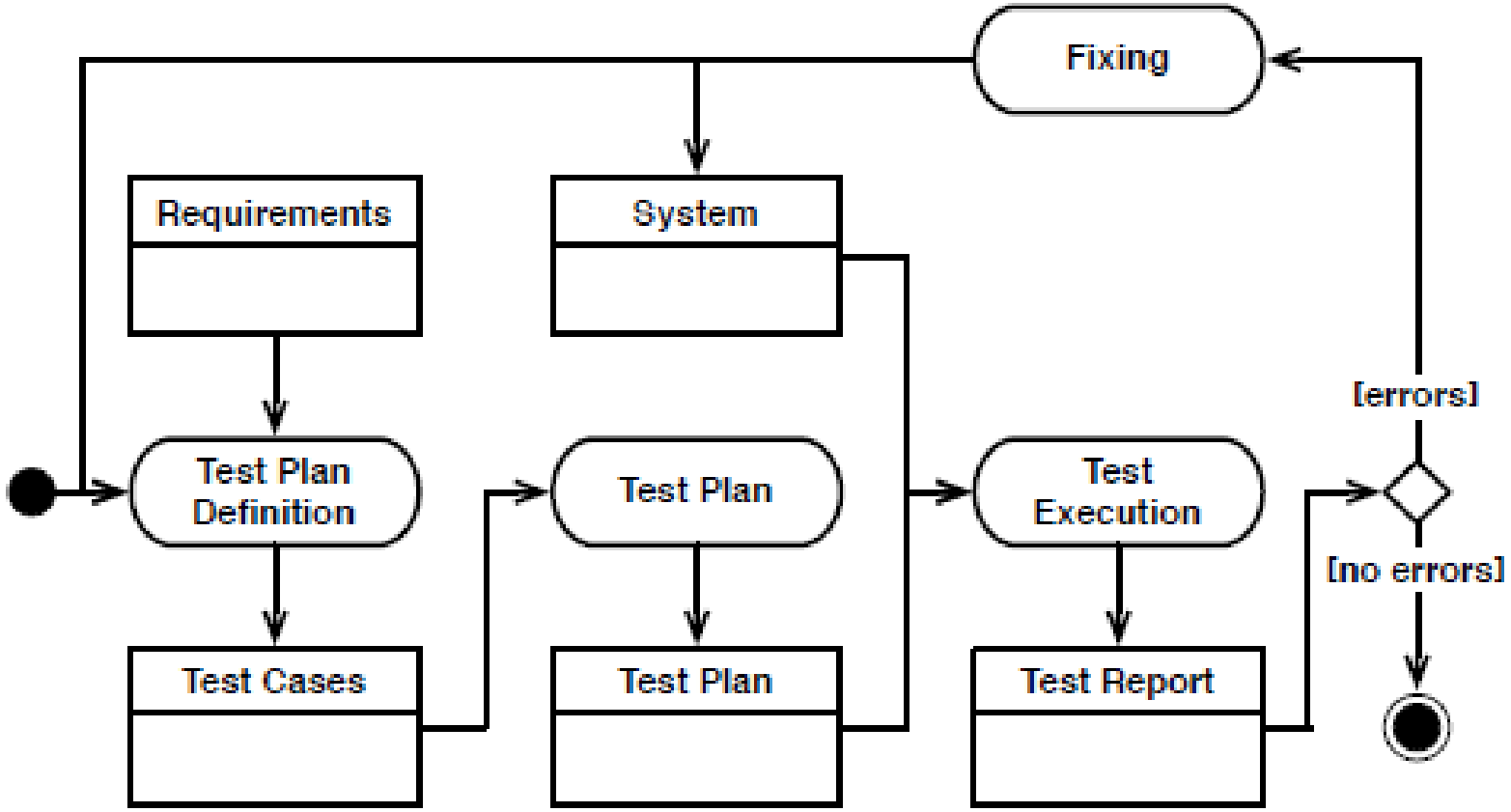
Verificare și validare

- Validare = construim sistemul care trebuie?
- Verificare = am construit sistemul cum trebuie?
- Cunoscute colectiv sub acronimul V & V
- Parte a managementului calității
- Principala (dar nu singura) metodă de a realiza V & V pentru sistemele software este testarea

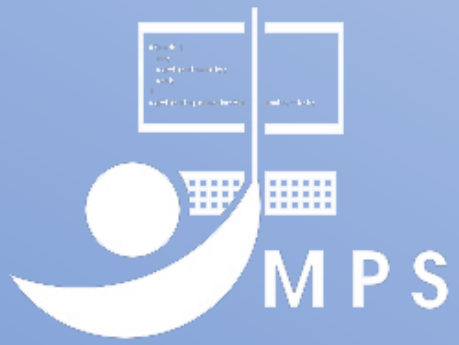
Tipuri de testare

- Unit testing
  - Scop: o secvență de cod, ca de exemplu o clasă
- Testare de integrare
  - Scop: interacțiunea dintre două componente
  - Bug-ul Mars Climate Orbiter: două componente au folosit diferite unități de măsura (metric și imperial) => ~ 400 milioane dolari pierduți
- Testarea sistemului
  - Scop: sistemul se comportă precum se aștepta și implementează în mod correct toate cerințele
  - Cazuri de test
- Testarea utilizabilității
  - Scop: verifică dacă experiența utilizatorului și interacțiunea sunt intuitive, eficiente și satisfăcătoare
  - Folosită pentru a reduce probabilitatea erorilor umane (sisteme de siguranță critică)

Procesul de testare a sistemului



Deployment



Deployment

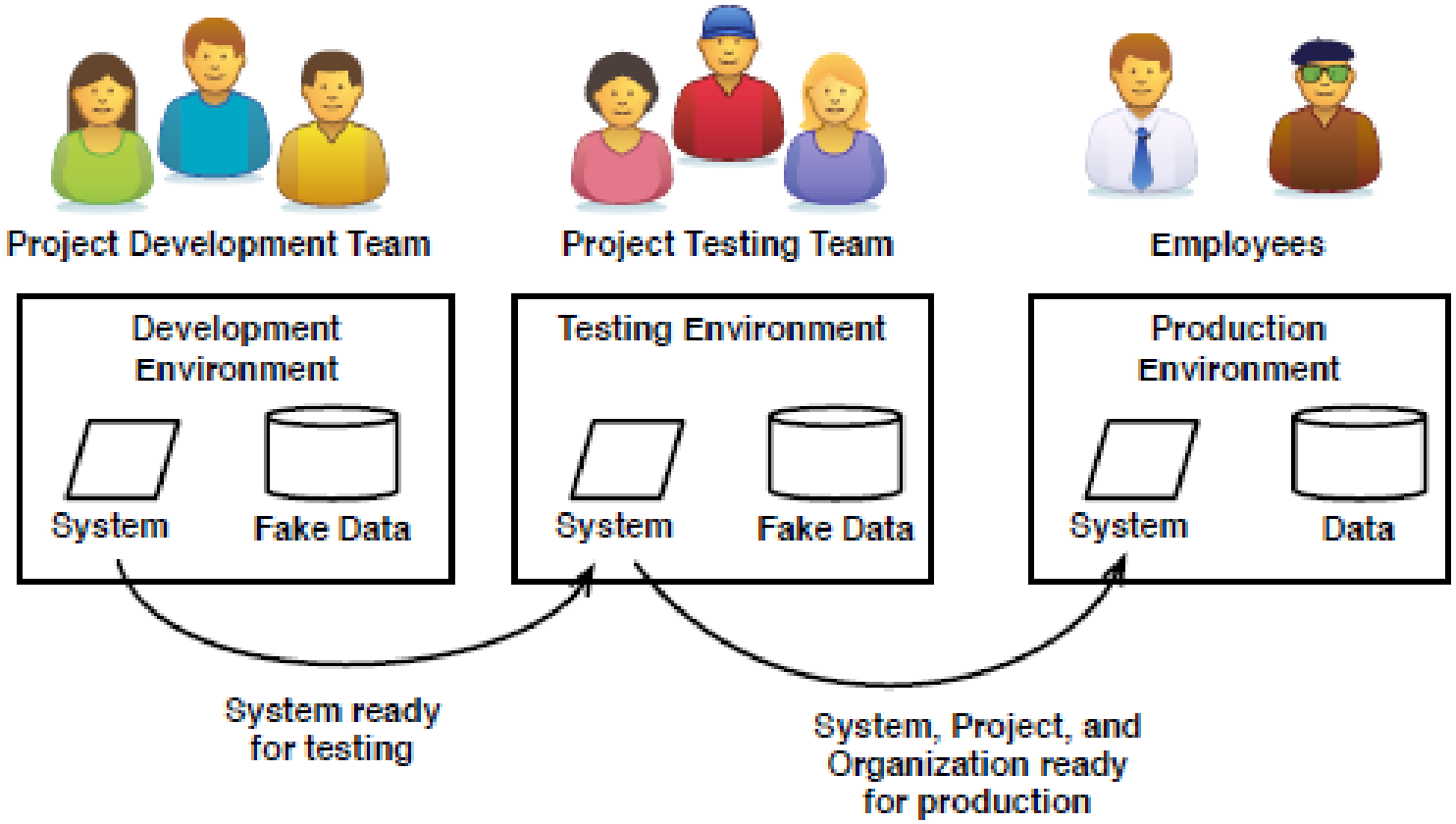
- Scop:
  - A instala noul sistem și a-l face operațional
- Câteva roluri:
  - Asigurarea continuității operațiunilor de business
  - Migrarea datelor
  - Tranziția către operațiuni și mentenanță

- Factori de luat în considerare
  - Factorul uman: oamenii sunt gata să utilizeze sistemul?
  - Factorul datelor: sunt disponibile pentru noul software toate datele necesare pentru ca sistemul să ruleze?
  - Factorul hardware: sunt toate interfețele gata și funcționale?

Abordări

- Cut-over: noul system il inlocuieste pe cel vechi
- Abordare paralelă: vechiul și noul sistem operează simultan pentru o perioadă
- Pilotare: noul sistem este instalat pentru un număr limitat de utilizatori sau pentru o unitate de business specifică
- Abordarea în faze: funcțiile sunt lansate incremental

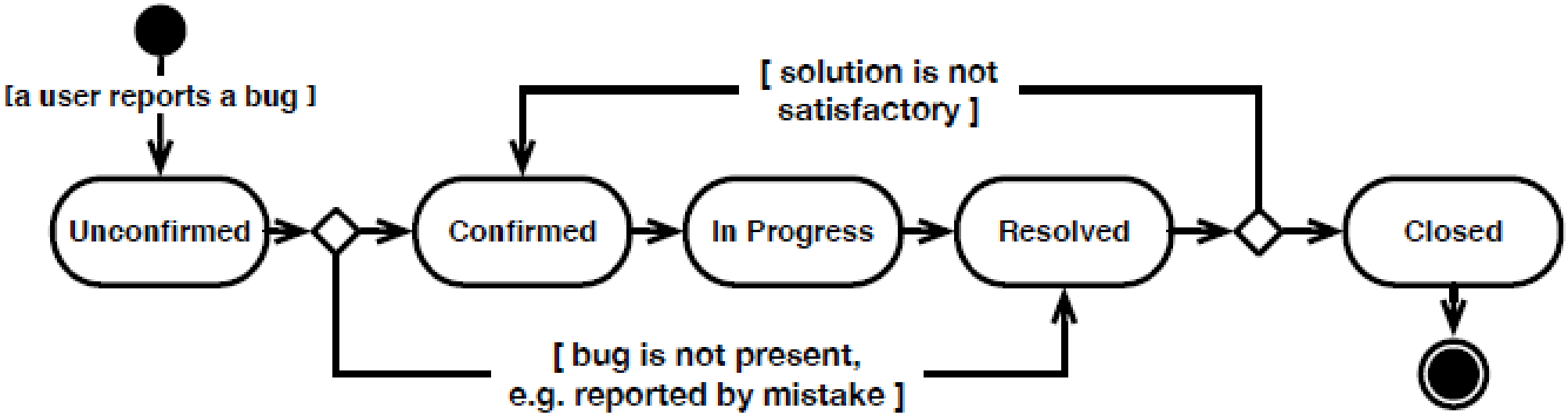
Gestionarea evoluției Software-ului



Operațiuni și mentenanță

Operațiuni și mentenanta

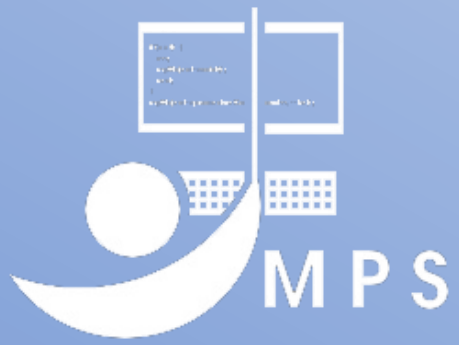
- Scop: Asigurarea bunei funcționări a sistemului
- Activități:
  - Oferirea de Suport Tehnic
  - Monitorizarea performanței sistemului
  - Colectarea și gestionarea tichetelor (clarificări, bug-uri, cereri de îmbunătățire)
  - Declansarea activităților de mentenanță


[a user reports a bug]
[solution is not satisfactory]
Unconfirmed
Confirmed
In Progress
Resolved
Closed
[bug is not present, e.g. reported by mistake]

Tipuri de mentenanță

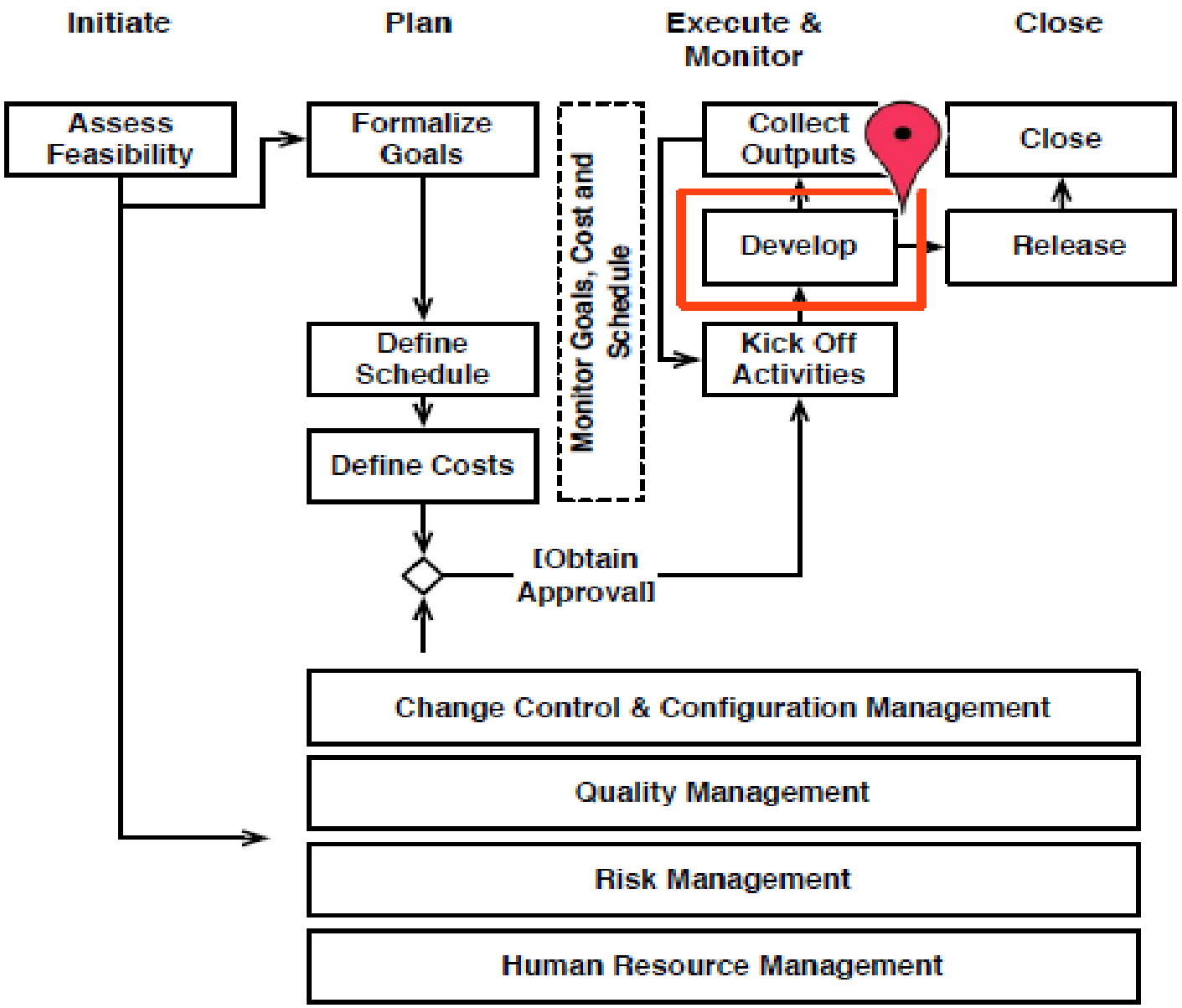
- Corectivă, referitoare la repararea unei probleme apărute după lansarea sistemului
- Preventivă, referitoare la repararea unei probleme descoperite, dar neproduse încă (sau, cel puțin, nesemnalizate de către utilizatori)
- Adaptivă, referitoare la adaptarea sistemului la condiții externe schimbate
- Perfectivă, referitoare la îmbunătățirea unor caracteristici ale sistemului, ca de exemplu, a performanțelor

Activități (tradiționale) de dezvoltare a Software-ului



Obiective

- O introducere subtilă și la nivel înalt în activitățile de dezvoltare a software-ului
- Descoperirea pietrelor de temelie necesare pentru producerea de software
- Remarci:
  - Acest curs nu il inlocuieste pe cel de software engineering
  - Activitățile trebuie integrate într-un process coerent pentru a avea sens
  - Proiectele de dezvoltare a software-ului variază de la extrem de reduse la extrem de extinse … nu toate activitățile sunt la fel de utile sau relevante în orice context


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Define Schedule
Define Costs
Monitor Goals, Cost and Schedule
[Obtain Approval]
Collect Outputs
Develop
Kick Off Activities
Close
Release
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Prezentare generală

- Dezvoltarea de software necesită o perfecționare progresivă, ce transformă conceptul în operații prin urmatoarele faze:
  - Stabilirea cerințelor și a experienței utilizatorilor
  - Design
  - Implementare
  - Verificare și validare
  - Deployment
  - Operații și mentenanță
- O dată cu parcurgerea acestor faze, luăm anumite alegeri specifice; costul schimbărilor crește în consecință
- Procese diferite pun accentul pe fiecare activitate sau definesc ordinea în care aceste activități pot fi efectuate.

Procesul Software

- Procesul Software: **un set coherent de activități pentru specificarea, design-ul, implementarea și testarea sistemelor software.**
- Procesul software gestionează tranziția de la concept la produs
- Un model de process software reprezintă **reprezentare abstractă** a unui proces. Acesta prezintă o descriere a procesului dintr-o perspectivă particulară.
- Caracteristicile unui proces:
  - "Building blocks" (blocurile de construcție)
  - Ordinea în care blocurile sunt executate
  - Intrări / ieșiri ale fiecărei activități
  - Nivelul de formalitate (când și cum se realizează deplasarea către următoarea activitate; ce trebuie urmărit)

Building Blocks: ce

Modelarea mediului / business-ului	Descrie mediul (oamenii, organizația, celelalte sisteme) în care software-ul va rula	Descriere textuală, diagrame (BPMN, UML)
Cerinte (“ce” – ul)	Descrie calitățile externe ale software-ului (ce văd utilizatorii) și constrangerile	Descriere textuală, layouts GUI, cazuri de utilizare, povești de utilizator etc.

Exemplu

Modelarea Business-ului

1. Conducatorul unității primește cererea de cheltuieli

2. Conducatorul unității autorizează și trimite mai departe către administrație

3. Administrația verifică disponibilitatea bugetului și trimite mai departe către achiziții

4. Achizitiile continuă procesul

5. …

Cerințe

1. Sistemul trebuie să îi permită conducatorului unității să autorizeze cererea de cheltuieli

2. …

FURPS

- Funcționalitate
- Uzabilitate
- Fiabilitate (Reliability)
- Performanță
- Suportabilitate

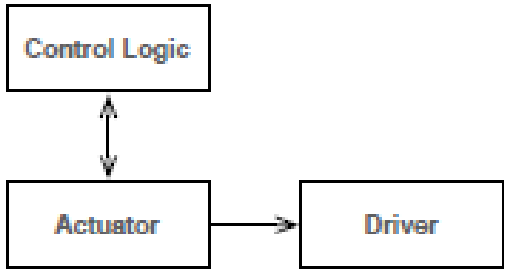
Building Blocks: cum

Design architectural **cum**	Descrie cum este sistemul structurat pe componente	Diagrame (clase, componente, …)
Implementare	Implementarea cerințelor conform structurii definite în design-ul arhitectural	Cod

Exemplu

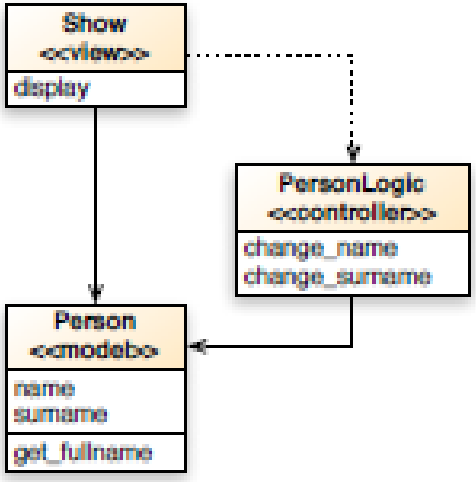
Câteva notații

- Diagrame de flux de date (data flow)
- Descompunere ierarhică
- Diagrame de stare
- Diagrame de clase
- …



Obiective

- Din ce componente este alcătuit un sistem (logic / fizic)
- Cum interacționează
- Cum se comportă
- …



Building Blocks: v&v

Verificare (“functioneaza cum trebuie?”)	Verifică dacă implementarea funcționeaza conform așteptărilor	Raport (și specificațiile testelor – cod sau text)
Validare (“este ceea ce trebuie?”)	Verifica dacă sistemul face ceea ce era prevăzut în cerințe	Raport (și specificațiile testelor – de obicei text)

Diferite scopuri / reprezentari:

- unitate (testul unei secvențe de cod)
- componenta / system (testarea unei componente): specificații textuale / scripturi executabile

Diferite tipuri:

- testare white box (se cunoaște codul sursă)
- testare black box (nu se cunoaște structura internă)

Exemple

- Liste textuale / executabile de teste
- Adesea sub formă:
  - cu sistemul într-o anumită stare
  - dacă unele evenimente se întâmplă (e.g. interacțiune cu utilizatorul)
  - când un anume output este așteptat
- Exemplu:
  - După logarea cu succes ca utilizator simplu
  - Dacă utilizatorul apasă butonul admin, este trimis către o pagină de eroare ce afișează utilizatorul nu este autorizat
- Poate fi automatizat (chiar și cu GUI)

Building Blocks: going live

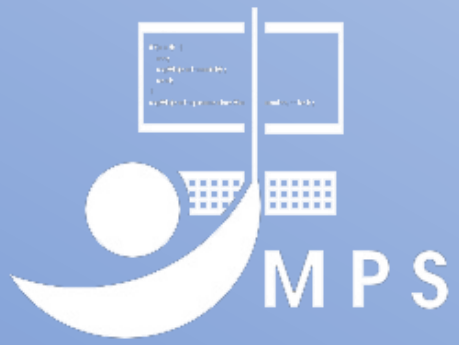
Documentație pentru utilizator	Manualul de utilizare și alte informatii pentru a ajuta utilizatorii (functionalități și utilizarea sistemului)	Manual
Distribuire și Deployment	Activități pentru a se lansa live și a ajunge la clienti	Pachetul de instalare a software-ului, Appstore, Website, Aplicații Web
Mentenanță	Ce se face după lansarea software-ului (menționat uneori; abordările moderne de software development consideră mentenanță ca un nou proiect)	

(Unele) Preocupări transversale

Definiția Planului de Business	Cum veți face bani din sistem	
Managementul Configurației	Activități legate de menținerea coerenței sistemului atunci când evoluează	Lista de obiecte, repository
Managementul Proiectului	Activități necesare pentru a menține proiectul sub control	Acest curs ☺
Imbunatatirea Procesului	Măsurarea performanțelor și asigurarea îmbunătățirii personale în ceea ce privește producerea de software	

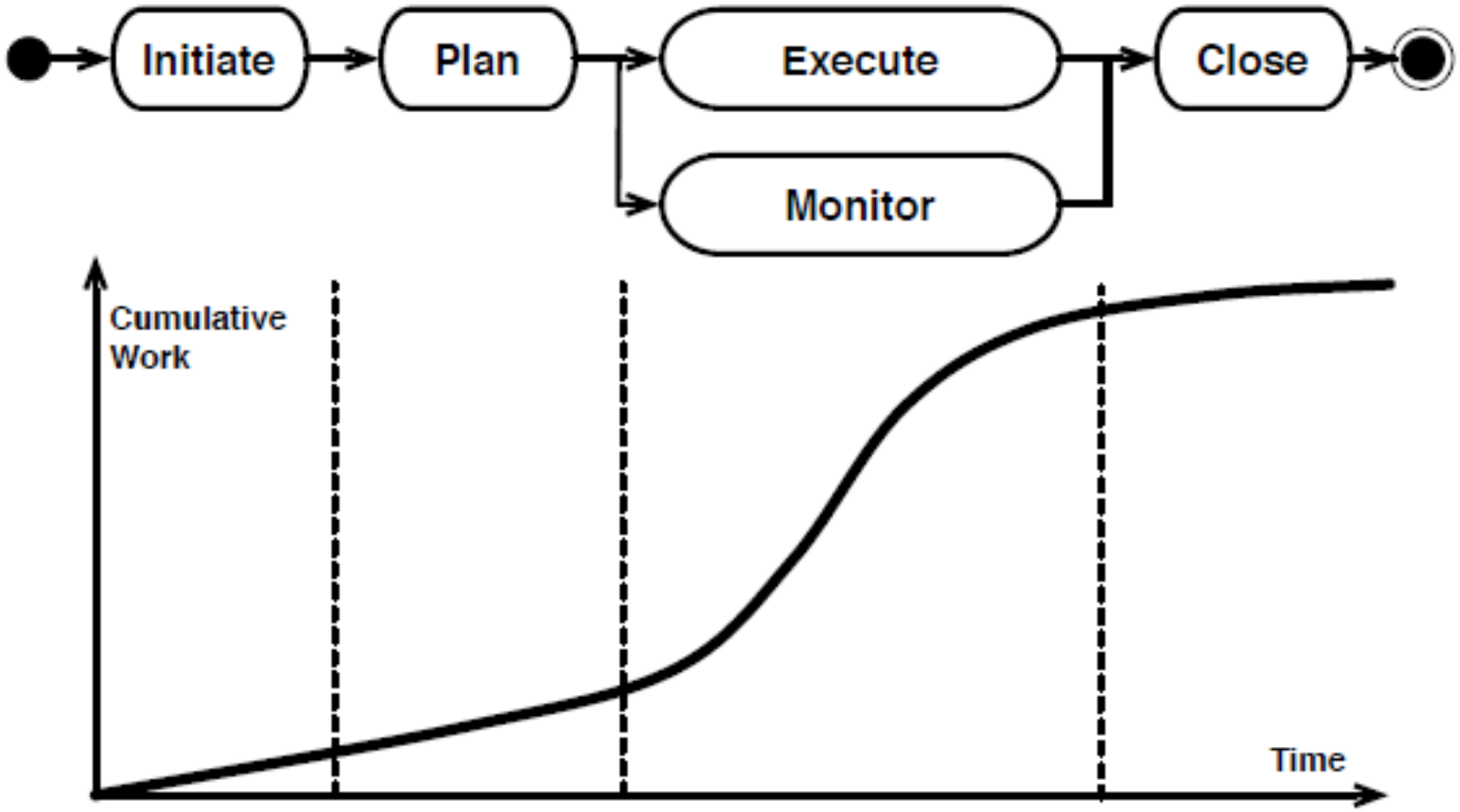
Ce este un proiect?

Numele jocului



Un proiect este un efort temporar **realizat pentru crearea unui produs, serviciu sau rezultat** unic**.**

Elaborare progresivă



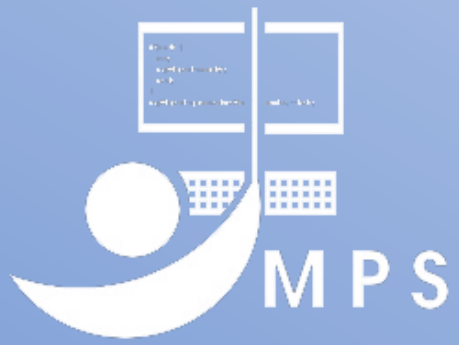
Proiecte și proceduri operaționale

- Lucrul poate fi pe bază de proiect sau operațional
- Caracteristici comune:
  - Realizat de oameni
  - Resurse limitate
  - Planificat, executat și controlat
- Diferențe
  - Proiect: obținerea obiectivelor și finalizare
  - Operațional: susținerea business-ului

Exemple (și contraexemple)

- Pregătirea cinei
- Fabricarea unui mașini
- Design-ul unei mașini
- Redactarea unui articol
- Dezvoltarea unui sistem software
- Menținerea unui sistem software
- Managementul personalului

Proiecte de Dezvoltare Software



Tipuri de Proiecte de Dezvoltare Software

În cariera de project manager, o persoană poate fi implicată în diferite tipuri de proiecte de dezvoltare de software, precum:

- Dezvoltarea de aplicații
- Reingineria proceselor și a sistemelor
- Integrarea sistemului
- Servicii de consultanță
- Instalare și training

A. Dezvoltarea de aplicații

- Scop → dezvoltarea unei aplicații (desktop, web, mobil, incorporate)
- Tipul cel mai distractiv ☺
- Tipuri de dezvoltare de aplicatii:
  - **One-off-uri** → sisteme create special pentru client
  - **Off-the-shelf** → pentru satisfacerea nevoilor unei game largi de utilizatori
  - **Off-the-shelf customizat** → sisteme standardizate ce necesita o cantitate semnificativa de customizare pentru a putea fi folosite intr-o organizatie. Exemplu: sisteme de tip ERP (**Enterprise Resource Planning**)

B. Reingineria proceselor și a sistemelor

- Scop: schimbarea modalității de desfășurare a procedurilor operaționale din cadrul unei organizații în vederea realizarii unui scop strategic (ex: îmbunătățirea calității, a eficienței)
- De obicei proiecte ample, care necesită o analiză precisă a situației existente din punct de vedere al procedurilor, sistemelor, infrastructurii
- Necesită adesea introducerea unui sistem ERP și a activităților de integrare a sistemului și a datelor

C. Servicii de integrare a sistemului

- Scop: automatizarea fluxului informational in cadrul sistemelor unei organizatii
- Tipuri de integrare:
  - **Orizontal** → integrarea sistemelor ce realizeaza operațiuni similare
  - **Vertical** → integrarea sistemelor care automatizeaza diferite etape ale unei proceduri

Alte tipuri de proiecte

D. Servicii de consultanță

- Cerute de obicei pentru a îmbunătăți comunicarea cu clienții (know-how despre competențele companiei)

E. Servicii de instalare si training

- Servicii legate de instalare și training pentru sisteme software specific
- Remarcă: constituie de asemenea sursă de venit în dezvoltarea open source

Proiectele și mediul lor

Jucatorii

Un stakeholder reprezintă un individ sau o organizație ce este implicat(ă) activ într-un proiect, sau al cărui interes ar putea fi influențat (pozitiv sau negativ) în urma realizarii sau finalizării proiectului.
(PMBOK)

Jucătorii

- Câteva caracteristici:
  - Pot avea influențe diferite și diverse niveluri de responsabilitate în timpul proiectului
  - Pot juca roluri diferite
  - Pot avea o influență pozitivă sau negativă asupra proiectului
  - Pot fi dificil de identificat
  - Lipsa intervenției lor poate influenta in mod negativ proiectul (nevoie de identificare si implicare)
- Remarcă: managerul de proiect si echipa de proiect **sunt** și ei stakeholderi în cadrul proiectului, desi termenul este adesea folosit pentru a desemna “ceilalți” stakeholderi

Tipuri de Stakeholderi

- Project managerul
- Echipa de proiect
- Sponsorul proiectului
- Organizațiile participante
- Partenerii
- Clientul
- Restul: oricine ar putea fi afectat de rezultatele proiectului

Stakeholderi Cheie

- Interni:
  - Membrii echipei de proiect: grupul ce realizează munca
  - Echipa de management a proiectului: membrii echipei direct implicați în activitatea de project management

- De mijloc:
  - Client / Utilizator: persoana sau organizație ce va utiliza rezultatele proiectului. Pot fi mai multe niveluri de utilizatori.
  - Sponsor: persoană sau grup ce furnizează resursele financiare
  - Organizatie participant: organizația implicată cel mai mult în realizarea proiectului

- Externi:
  - Influenceri: persoane sau grupuri neimplicate direct în proiect, dar care i-ar putea influența cursul

Exercițiu – Identificare Stakeholderi

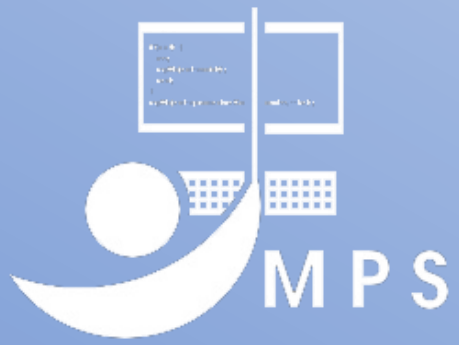
- Identificati stakeholderii următoarelor proiecte
  - Un proiect pentru construirea unui pod ce conectează o insulă de continent
  - Un proiect pentru construirea unui deposit de deșeuri (groapă de gunoi)
  - Un proiect pentru construirea unei foi de calcul (spreadsheet) open source
  - Un proiect pentru dezvoltarea unei aplicații web pentru monitorizarea masei corporale
  - Un proiect pentru a porta OpenOffice (suita Office gratuita) pe Android

Managerul de Proiect

- **Managerul de Proiect**
  - Persoana responsabilă cu gestiunea proiectului și a așteptărilor stakeholderilor

- **Câteva competențe necesare:**
  - Comunicare și negociere
  - (Oarecare) Predispoziție către riscuri
  - Orientare către scop
  - Abilități de leader
  - Gândire creativă, în afara tiparelor
  - Cunoștinte tehnice solide
  - Corectitudine profesională
  - Bun simț
  - Stil

Organizarea dezvoltării proiectelor software



Managementul Proiectelor Software

- **Managementul proiectelor software (Software project management)** reprezintă integrarea tehnicilor de management în dezvoltarea software
- Nevoia unei asemenea integrari își are rădăcinile în anii ‘60, în zilele “crizei software”, când specialiștii recunosteau complexitatea crescătoare a livrării produselor software ce îndeplinesc specificațiile

Ce face un Software să fie considerat Produs?

- O aplicație pentru gestionarea filmelor vizionate
- Un sistem de contabilitate cu intrare dublă pentru acasă sau pentru mici întreprinderi
- Un serviciu web pentru gestionarea rezervărilor terenurilor de squash într-un club de squash
- Un sistem de planificare a resurselor într-o corporație
- Un sistem de frânare automată pentru trenuri

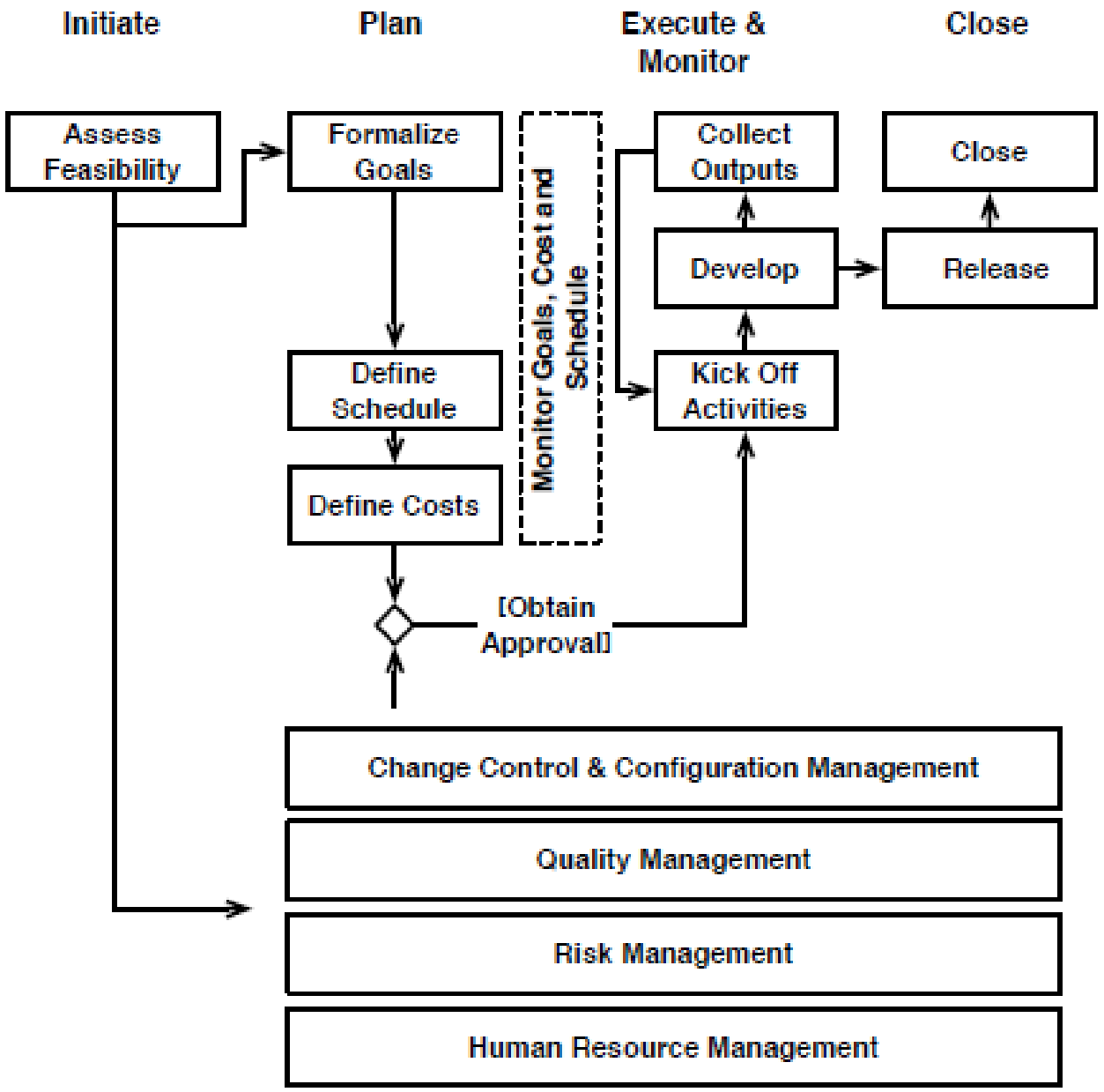
- Din ce componente este compus?
- Care ar fi efectele în cazul unui eșec?
- Ce competente sunt necesare pentru dezvoltare?
- Care este modelul de business asociat?

Framework de Dezvoltare Software

- Un framework general pentru MPS trebuie să:
  - Formeze o viziune comună despre scopurile ce trebuie realizate, caracteristicile rezultatelor proiectului și caracteristicile procesului de dezvoltare
  - Structureze lucrul ca o perfectionare progresivă, de la specificații la scopuri
  - Reducă impactul nesiguranțelor și a necunoscutelor
  - Evidențieze orice deviere de la plan (scopuri, costuri, calitate)
  - Asigure coerență și calitatea componentelor proiectului de-a lungul timpului în pofida necunoscutelor și a schimbărilor (sau a cererilor de schimbare)
  - Motiveze echipa

Câteva preocupări

- Evaluarea fezabilității
- Managementul scopurilor
- Managementul timpului
- Managementul costurilor
- Controlul schimbărilor și managementul configurației
- Managementul calității
- Managementul riscurilor
- Managementul resurselor umane


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Inițierea unui proiect

Fezabilitatea și autorizarea unui proiect

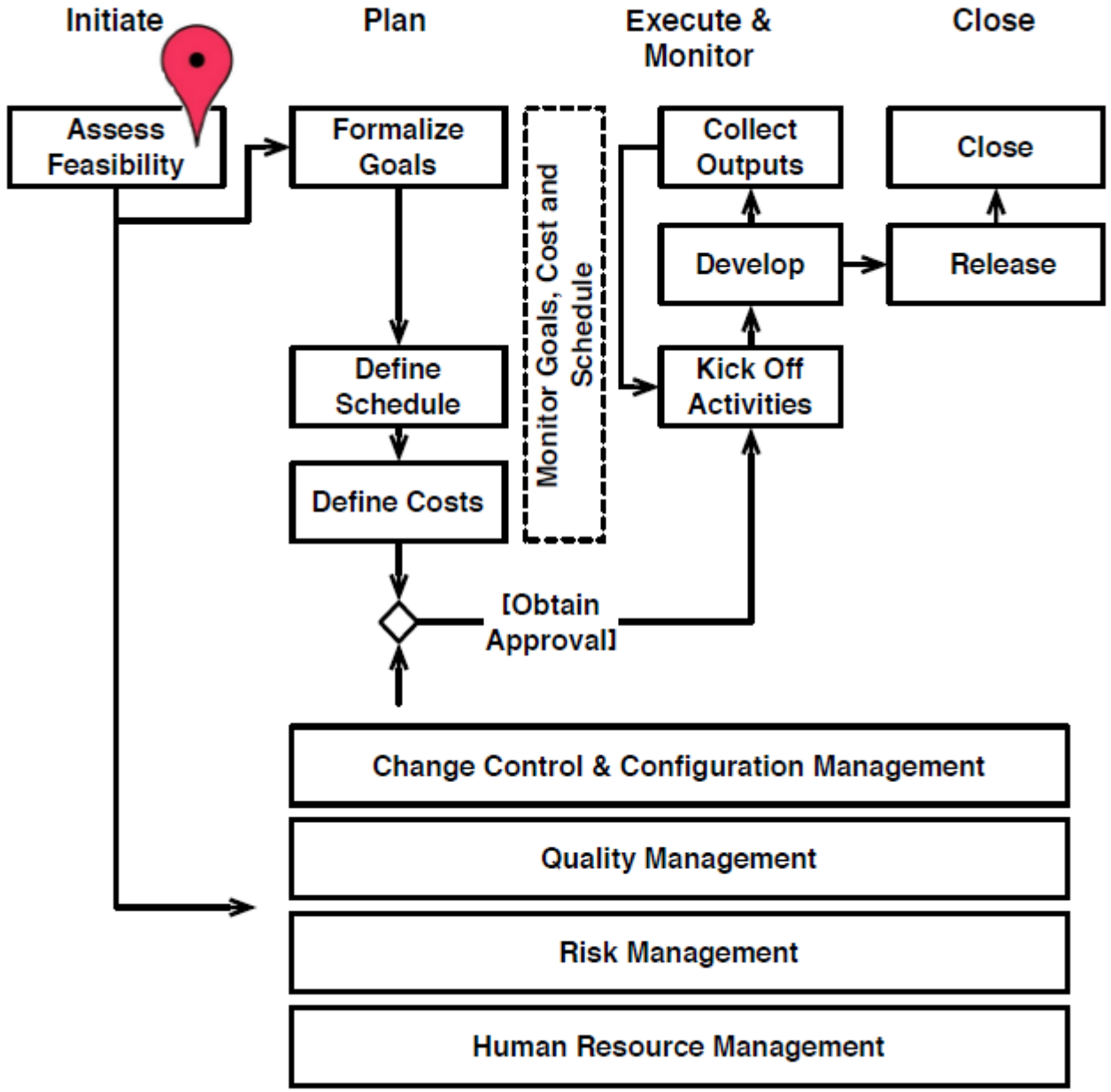
Obiective

- Înțelegerea tehnicilor calitative și cantitative pentru a selecta dintre anumite proiecte
- Înțelegerea tehnicilor calitative și cantitative pentru a alege cea mai bună alternativă dintre diferite implementări ale aceluiași proiect
- Înțelegerea redactării unui Studiu de Fezabilitate
- Alegerea între dezvoltare internă sau externă (a face sau a cumpăra)

A badly planned project will take three times longer than expected - a well planned project only twice as long as expected.

Cum începe un proiect?

- Inițierea de către un stakeholder (o companie, un potențial client) condus de o anumită nevoie (piață, socială, legală, tehnologică etc.)
- Limitele și procesele nu sunt întotdeauna clare sau formalizate
- Primele activitati realizate pentru a:
  - Decide obiectivele
  - Înțelege valorile și riscurile (pentru organizație și pentru celelalte părți implicate)
  - Alege o abordare a proiectului


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Valoarea proiectului și riscurile

PERFORMANCE REVIEW
YOU NEED TO GET BETTER AT ANTICIPATING PROBLEMS.
IF I COULD ANTICIPATE PROBLEMS, I WOULDN'T HAVE AGREED TO WORK FOR YOU.
YOU SEEM ANGRY. I DID NOT SEE THAT COMING.
GET US SOME RISK MANAGEMENT SOFTWARE.
WHAT CAN RISK MANAGEMENT SOFTWARE TELL YOU THAT MY COMMON SENSE AND EXPERIENCE CAN'T?
DATA.
STOP FAILING THE TURING TEST!
www.dilbert.com
scottadams@aol.com
© 2001 Scott Adams, Inc./Dist. by UFS, Inc.

Valoarea proiectului și riscurile

- Două caracteristici principale determină dacă un proiect merită să fie pornit sau nu:
  - Valoarea generată de proiect
  - Riscurile asociate proiectului
- Semnificația valorii și a riscului depind de o mulțime de factori
- Valoarea și riscurile pot fi evaluate calitativ sau cantitativ
- Proiectele (și oamenii) sunt imprevizibile
- Garbage in = Garbage out

Valoarea proiectului și riscurile

- Valoarea Proiectului
  - Valoarea directă și indirectă generate de proiect
  - Durabilitatea rezultatelor proiectului
  - Alinierea la obiectivele strategice ale unei organizații
- Riscurile proiectului
  - Disponibilitatea resurselor
  - Sincronizare
  - Dificultăți tehnice și incertitudini

Valoare - Directă, Indirectă

- Măsoară ieșirile (outputs) și urmările pozitive sau negative ale unui proiect
- Unele metrici de luat în considerare includ:
  - Venituri, directe și indirecte
  - Impact social și asupra mediului
  - Imagine și publicitate
  - Cunoștintele dobândite
- Sunt strict legate de modelul de business și de sustenabilitatea rezultatelor proiectului.

Sustenabilitate

- Capacitatea de a susține proiectul și rezultatele sale după ce acesta a luat sfârșit
- Luarea în considerare a costurilor operaționale ale rezultatelor unui proiect, precum și a felului în care aceste rezultate supraviețuiesc după sfârșitul proiectului reprezintă considerații importante pentru a afla dacă un proiect merită demarat sau nu
- Sustenabilitatea este adesea trecuta cu vederea, mai ales când executarea proiectului generează venituri

Alinierea cu obiectivele strategice

- Măsoară cât de important sau relevant este un proiect pentru organizație
- Prioritatea, resursele alocate, suportul intern, oportunitățile pentru echipa de proiect după finalizarea proiectului sunt toate afectate de cât de strategic este proiectul pentru organizație.

Riscuri - Disponibilitatea Resurselor

- Proiectele necesită **disponibilitatea de resurse umane, financiare și tehnice** în anumite intervale de timp
- Deși poate fi dificil de preîntâmpinat nevoia exactă de resurse, o verificare a nevoilor proiectului este esențială
- Aspecte ce trebuie luate în calcul: resursa necesară, disponibilitatea curentă, proiecții despre nevoia și disponibilitatea viitoare, prioritatea și importanța proiectului

Riscuri - Timing

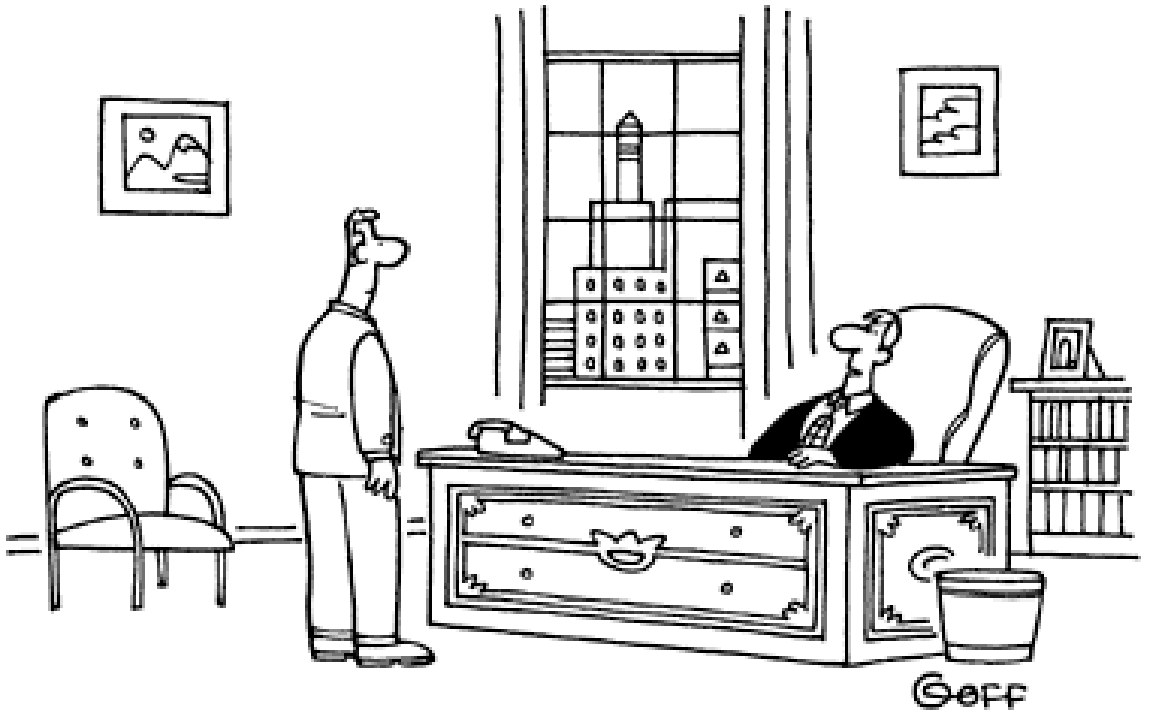
- Multe proiecte au **ferestre de timp specifice pentru livrarea rezultatelor**
- Livrarea prea devreme sau prea târzie poate face ca rezultatele proiectului să fie inutile
- De exemplu, să luăm în considerare cursa dintre firmele rivale ce livrează produse similare

Riscuri - Dificultati Tehnice si Incertitudini

- Succesul multor proiecte se bazează pe **capacitatea reală de a rezolva anumite provocări tehnice**, atunci când este cazul
- A înțelege care sunt aceste provocări reprezintă un factor important în determinarea riscurilor proiectului

Tehnici pentru evaluarea valorii și a riscurilor

© 2000 Ted Goff www.tedgoff.com



"I had to spend the money I budgeted for your raise on a therapist after listening to your endless complaining."

Perioada de rambursare

Perioada de rambursare reprezintă timpul necesar pentru a dobândi un câștig financiar egal cu investiția inițială.

- Măsurat în luni sau ani
- Atunci când se utilizează aceasta metodă, sunt alese proiectele / opțiunile care minimizează perioada de rambursare

Exemplu

	Project A	Project B	Project C
Year 0	€ (50,000.00)	€ (20,000.00)	€ (15,000.00)
Year 1	€ 30,000.00	€ (10,000.00)	€ 15,000.00
Year 2	€ 30,000.00	€ 10,000.00	€ 1,000.00
Year 3	€ 1,000.00	€ 60,000.00	
Year 4	€ 1,000.00	€ 50,000.00	
Expenses	€ (50,000.00)	€ (30,000.00)	€ (15,000.00)
Gains	€ 62,000.00	€ 120,000.00	€ 16,000.00
Profit	€ 12,000.00	€ 90,000.00	€ 1,000.00
Payback	2 years	3 years	1 year

Remarcă: notații în stil contabil, numerele negative cu roșu, între paranteze

Discuție

Avantaje

- Date simple, ușor accesibile
- Reduce expunerea la risc
- Eficiență în special în proiecte de tehnologie înaltă
- Favorizează beneficiile pe termen scurt

Dezavantaje

- Dificil de utilizat pentru proiecte pe termen lung
- Bazate doar pe fluxul numerar
- Nu cuantifică expunerea la risc
- Nu ține cont de câștigurile totale

Slăbiciunile rambursării

- Proiecte diferite pot avea aceeași perioada de rambursare, dar profiluri diferite în returnarea investițiilor
- Aceste profiluri nu sunt luate în considerare de către tehnică, dar ar putea face diferența dintre două proiecte

Slăbiciunile rambursării

Aceeași perioadă de rambursare, dar **Project A** aduce mai mulți bani mai repede (și reduce riscurile)

Year	Project A	Project B
Year 0	€ (10,000.00)	€ (10,000.00)
Year 1	€ (5,000.00)	€ (5,000.00)
Year 2	€ 10,000.00	€ 5,000.00
Year 3	€ 5,000.00	€ 10,000.00

Slăbiciunile rambursării

Perioade de rambursare diferite, **Project B** are perioada de rambursare mai scurtă, dar aduce mai puțini bani

Year	Project A	Project B
Year 0	€ (10,000.00)	€ (10,000.00)
Year 1	€ (5,000.00)	€ (5,000.00)
Year 2	€ 5,000.00	€ 5,000.00
Year 3	€ 5,000.00	€ 11,000.00
Year 4	€ 20,000.00	

Randamentul investitiei
(Return on Investment – ROI)

ROI calculează profitul anual mediu și îl transformă în procent al investițiilor totale

- Profit = Câștiguri – Investiții
- Profit Anual = Profit / Durată
- ROI = Profit Anual / Investiții
- Când se utilizează ROI, se alege proiectul cu cel mai mare ROI

Exemplu

Să presupunem că avem următoarele date pentru un proiect și trebuie să decidem dacă îl demarăm sau nu

	Project A	Project B	Project C
Year 0	€ *(50,000.00)*	€ *(20,000.00)*	€ *(15,000.00)*
Year 1	€ 30,000.00	€ *(10,000.00)*	€ 15,000.00
Year 2	€ 30,000.00	€ 10,000.00	€ 1,000.00
Year 3	€ 1,000.00	€ 60,000.00	
Year 4	€ 1,000.00	€ 50,000.00	

Exemplu

Soluție: Project B (cel mai mare ROI)

Project A

- Profit = 62000 – 50000 = 12000
- Profit anual = 12000/4 = 3000
- ROI = 3000/50000 = 6%

Project B

- Profit = 120000 – 30000 = 90000
- Profit anual = 90000/4 = 22500
- ROI = 22500/30000 = 75%

Project C

- Profit = 16000 – 15000 = 1000
- Profit anual = 1000/2 = 500
- ROI = 500/15000 = 3%

Fluxurile de numerar actualizate
Inflație

- Valoarea banilor scade de-a lungul anilor (inflație!) conform formulei:

$$\text{Discount Factor} = \frac{1}{(1+i)^n}$$

- Astfel, banii investiți acu ere peste 5 ani
- DCF (Discounted Cash Flow – Fluxurile de Numerar Actualizate) reprezintă tehnici care iau în calcul inflația
- De unde vine inflația? Din devalorizare.

http://en.wikipedia.org/wiki/Inflation

Valoarea actualizată netă (Net Present Value – NPV)

Valoarea Actualizata Neta folosește discount-uri asupra sumelor din viitor pentru a realiza o comparație mai realistă între investițiile prezente și câștigurile viitoare.

Valoarea actualizată netă - Exemplu

Rata de discount ipotetică: 10% (i-ul din formula)

$$\text{Discount Factor} = \frac{1}{(1+i)^n}$$

Year (n)	Cash Flow	Discount Factor	Present Value
0	€ (35,000.00)	1.00	€ (35,000.00)
1	€ 10,000.00	0.91	€ 9,090.91
2	€ 15,000.00	0.83	€ 12,396.69
3	€ 20,000.00	0.75	€ 15,026.30
Expenditure	**€ (35,000.00)**		**€ (35,000.00)**
Gains	**€ 45,000.00**		**€ 36,513.90**
Profit	**€ 10,000.00**		**€ 1,513.90**

Valoarea actualizata netă – Discuție

Avantaje

- Date mai exacte despre profit și pierderi

Dezavantaje

- Folosește o rată de discount fixă (poate fi nerealistă)
- Favorizează proiectele pe termen scurt

Matrice de scor

- Metodele financiare (rambursare, ROI, NPV) țin cont doar de datele financiare
- Matricele de scor permit luarea în considerare și a altor factori
- Se bazează pe un set standardizat de criterii și ponderi, care evidențiază caracteristicile relevante ale unui proiect
- O evaluare calitativă a scorurilor proiectului în funcție de fiecare criteriu permite poziționarea criteriului pe o scară și comparația cu proiecte trecute sau rivale

Matrice de Scor - Exemplu

Factor	Value	Weight	SUM	Comment
The project aligns with the strategic objectives	YES	2	2	
The project has a profit > 20%	NO	4	0	
Payback period < 2 years	YES	5	5	
Enlarges the customer base	YES	2	2	
The project requires a standard technology	NO	3	0	
The quality constraints are simple to meet	YES	1	1	
The timing is not too tight	NO	4	0	
We have skilled personnel to do the work	YES	5	5	
			15	

- Valorile pot fi binare (Da / Nu) sau numere (e.g. de la 1 la 5) și măsoara cât de bine îndeplineste proiectul cerința
- Ponderea (weight) măsoară cât de important este un factor pentru decizia finală

Discuție

Avantaje

- Simplă
- Încurajează standardizarea și luarea obiectivă a deciziilor
- Ajută la discutarea și evaluarea caracteristicilor proiectului
- Lărgește gama evaluării
- Nu favorizează proiectele pe termen scurt

Dezavantaje

- Un model simplu poate încuraja dezvoltarea unor liste lungi și inutile
- Factori diferiți au aceeași importanță (cu excepția cazului în care matricea de ponderi este folosită)

Avertisment

- Nu toate matricele de scor sunt la fel de bune
- Mai jos aveți un exemplu de matrice nepotrivită. De ce?

Factor	Value	Weight	SUM	Comment
The project has a profit > 20%	YES	3	3	
The project is highly risky	NO	3	0	
			3	

Avertisment

Asigurați-vă că toate aspectele influențează decizia fie pozitiv, fie negativ;
sau puteți folosi scoruri cu semn diferite!

Analiza SWOT

- Tehnica lui Albert Humphrey
- Analiza sistematica a:
  - Punctelor tari (Strengths)
  - Punctelor slabe (Weaknesses)
  - Oportunităților (Opportunities)
  - Amenințărilor (Threats)

  … pentru a înțelege fezabilitatea unui proiect și/sau a crea scopuri realizabile ale proiectului
- Adesea prezentată ca o matrice 2x2, cu fiecare celula listând toate elementele de un anume tip

Analiza SWOT

Sursă: http://en.wikipedia.org/wiki/File:SWOT_en.svg (cc license)

SWOT - Factori de luat în calcul

Puncte tari

- Competențe
- Puncte de vânzare

Puncte slabe:

- Dezavantaje
- Metodologie
- Timing
- Deficiențe de capabilitate

Oportunități

- Trenduri pe piață și în industrie
- Slăbiciunile competitorilor

Amenințări

- Trenduri pe piață și în industrie
- Tehnologiile competitorilor
- Sustenabilitate

Analiza părților implicate (stakeholders)

- Scop: a înțelege care sunt părțile implicate într-un proiect și care este influența lor asupra proiectului
- Diferite tehnici disponibile
- Una dintre tehnici organizează părțile implicate într-o matrice 2x2 în care:
  - o dimensiune măsoară puterea pe care o exercită un stakeholder (slabă sau ridicată)
  - cealaltă dimensiune măsoara interesul pe care îl are un stakeholder în proiect (negativ sau pozitiv)
- Aceasta permite definirea politicilor de management specifice pentru diferitele părți implicate

Evaluarea sustenabilității

- Această analiză are scopul de a permite înțelegerea costurilor operaționale ale rezultatelor unui proiect
- Reprezintă uneori o activitate specifică a proiectului
- O analiză preliminară a sustenabilității poate ajuta decizia de a alege una dintre implementarile proiectului
- Aspecte ce trebuie luate în calcul include **modelul de business** și **pragul de rentabilitate**

Studiul de fezabilitate

Studiul de fezabilitate

Reprezintă un document ce permite autorizarea formală a unui proiect și legarea lui la scopurile organizației

- Gama largă de rezultate: de la puține la câteva sute de pagini (în funcție de complexitate și formalitate)
- Studiul de fezabilitate poate fi văzut ca un proiect în miniatură, conturând informația principal ce va fi detaliată de-a lungul proiectului
- Baza pentru selecția proiectului: conducerea trebuie să aleagă ce proiecte vor fi activate

Scopurile studiului de fezabilitate

- Identificarea:
  - obiectivele proiectului
  - constrângerilor proiectului
- Evaluarea valorii și a riscurilor
- Asigurarea alinierii proiectului la:
  - obiectivele clientului
  - obiectivele organizației principale
- A demonstra că obiectivele proiectului
  - pot fi indeplinite respectând contrângerile de calitate, costuri și timp

Document de fezabilitate - Structură

- O declarație a muncii, ce descrie ce va realiza proiectul
- Obiectivele de business (valoarea) ale proiectului sau rezultatele și informațiile despre modelul de business, dacă sunt relevante
- Un sumar al bugetului proiectului, care prognozează cheltuielile și câștigurile
- Un sumar al milestone-urilor, mai exact o planificare grosieră ce include cele mai importante evenimente
- O analiză a părților implicate
- Riscurile proiectului
- Posibile alternative ale proiectului, de exemplu decizii de tipul a face sau a cumpăra
- O evaluare a proiectului și a alternativelor sale

Fezabilitate – considerații aditionale

Documentul de fezabilitate are valoare pentru:

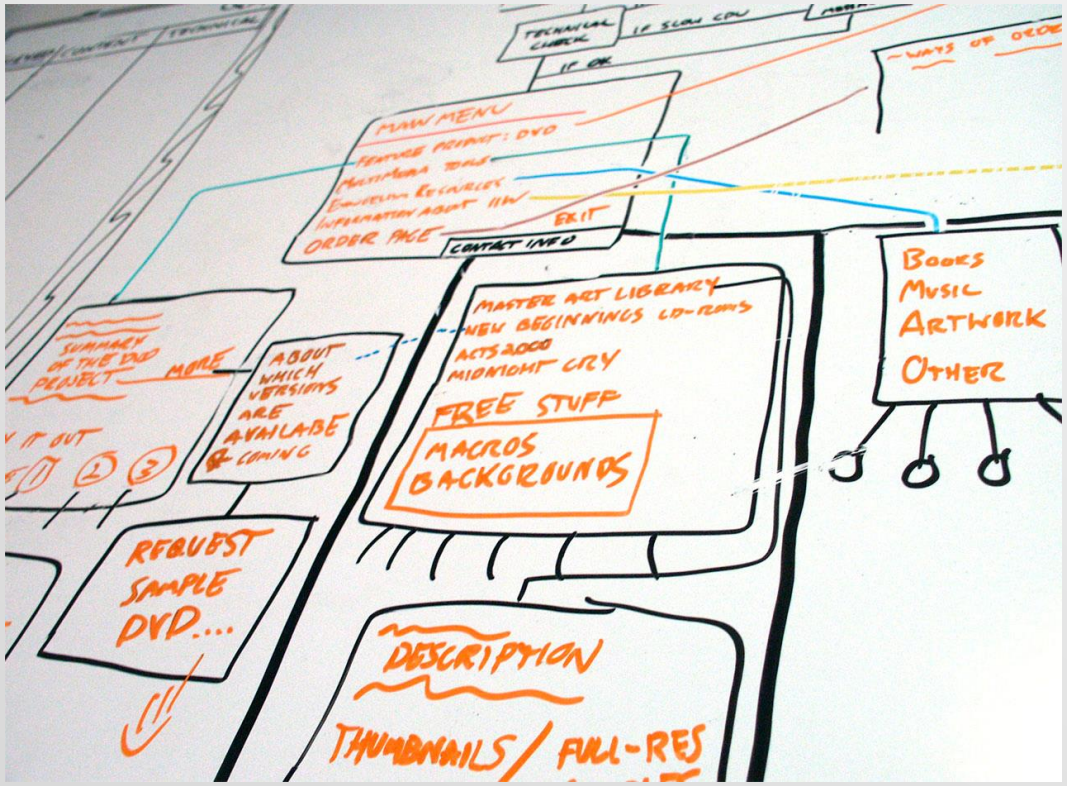
- **Client** – îl ajută să înțeleagă ceea ce urmează să fie realizat, precum și perspectivele pe termen scurt sau lung
- **Organizația principală** – o ajută să înțeleagă dacă are sens să se continue un proiect
- **Managerul de proiect** – îl ajută să înțeleagă dacă proiectul va fi sau nu în zona sa de confort (și să ia astfel o decizie obiectivă cu privire la a realiza sau nu proiectul)

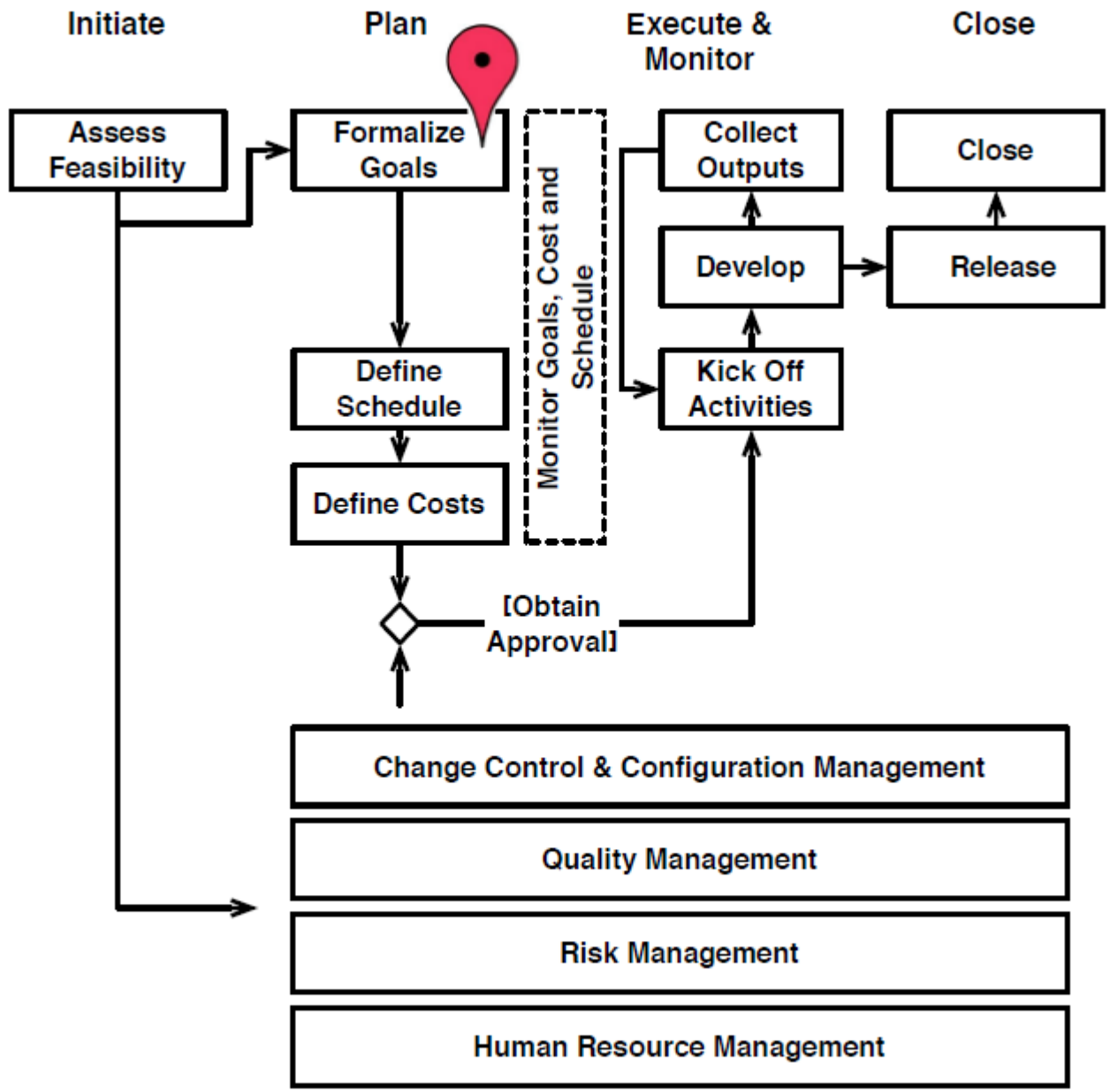
Procesul de aprobare al proiectului

- Procesul ce duce la aprobarea proiectului este mai mult sau mai puțin structurat conform practicilor organizației principale
- Este organizat după următorii pași:
  - Dupa primirea unei cereri, identificarea unui manager de proiect (preliminar)
  - Managerul de proiect pregătește un studio de fezabilitate care este aprobat de către client și principalele părți implicate
  - Managerul de proiect trimite documentele pentru autorizare
  - Documentul este analizat și o decizie formală este luată
  - Managerul de proiect definitiv este ales și proiectul trece în faza de planificare

Formalizarea obiectivelor proiectului

The first myth of management is that it exists.




Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Formalizarea obiectivelor proiectului

- Definirea obiectivelor proiectului (domeniul de aplicare al proiectului) reprezintă una dintre primele și cele mai importante activități din cadrul unui proiect
- Domeniul de aplicare al unui proiect:
  - Asigură că proiectul include doar (și toată) munca necesară
  - Stabilește baza muncii ce va fi efectuată
  - Definește un document de referință pentru acceptarea proiectului
- Definirea domeniului de aplicare al proiectului începe în timpul studiului de fezabilitate
- Obiectivele proiectului și alinierea proiectului la scopul său continuă pe tot parcursul ciclului de viață al unui proiect

Documentul de aplicare a proiectului

Acest document conține:

- Obiectivele si cerintele proiectului - descriu ceea ce vrem să realizăm prin proiect, principalele sale caracteristici și rezultate
- Presupuneri și constrângeri - ce descriu condițiile ce trebuie îndeplinite pentru a asigura succesul proiectului
- Rezultatele proiectului și punctele de control - ce descriu rezultatele proiectului și, în unele cazuri, o aproximare a timpului de livrare

Obiectivele și cerințele proiectului

- Obiectivele și cerințele proiectului sunt baza de definire a:
  - Bazei muncii de efectuat (comparație între Work Breakdown Structure si Change and Configuration Management)
  - Criteriile de acceptare a proiectului (comparație între închiderea proiectului și managementul calității)
- Strict legate de cerințele software
- Uneori este util să includă și ceea ce este în afara domeniului de aplicare a proiectului

Obiectivele proiectului

Make the objectives **SMART!**

- **Specifice** - Clare și concise
- **Masurabile** – Ușor de utilizat, măsuratori pentru a înțelege dacă scopul a fost atins. O dată, un număr, o formula (keep it simple!)
- **De comun acord** - Obiectivele trebuie sa fie suficient de specifice pentru ca echipa să le îndeplinească
- **Realiste** - Obiectivele trebuie să fie realiste. Obiectivele nerealiste stabilesc așteptări nerealiste și fac echipa apatică
- **Legate de timp** - Trebuie să aibă un început și un sfârșit. Dacă nu poate fi stabilit un sfârșit, este probabil vorba despre muncă operațională.

Obiectivele proiectului

Make the objectives **russian (MoSCoW)!**

- **M:** Must Have
  - Esențiale
- **S:** Should Have
  - Importante, dar nu esențiale
- **C:** Could Have
  - De dorit
- **W:** Won't Have
  - Nu vor fi realizate

Obiectivele proiectului - MoSCoW

- Trebuie să ne asigurăm că **acele criterii pentru succesul clasei M depind** de factori ce ne sunt sub control.
- Exemple negative (M nu e sub controlul managerului de proiect):
  - Sistemul va avea 1000 de utilizatori în prima luna (De ce tool-uri dispune managerul de proiect pentru a asigura indeplinirea acestui obiectiv?)
  - Viteza de introducere a datelor de către utilizatori va crește de zece ori (Cum se poate asigura managerul de proiect ca această creștere va fi atinsă?)
- Uneori contează foarte mult cuvintele alese pentru descrierea obiectivelor. Consecințele pot fi costisitoare.

Presupuneri

- **Presupunerile** sunt condiții considerate ca fiind adevărate, dar care în realitate pot fi false
- Presupunerile nu se afla sub controlul managerului de proiect, dar **pot fi sub controlul unora dintre părțile implicate în proiect**
- În acest caz, presupunerile pot fi folosite pentru a defini datoriile și obligațiile părților implicate
- **Constrângerile** sunt limitări cunoscute. Ele explică de ce alegem unele obiective în locul altora și de ce munca este structurată într-un anume fel.

Ieșirile proiectului - Milestone, Livrabile

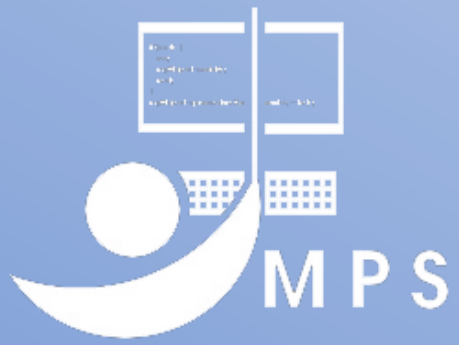
- Ieșirile Proiectului (Outputs) definesc ceea ce proiectul va realiza și când
- **Milestone** - un eveniment semnificativ din cadrul proiectului
  - Identifică puncte critice în proiect și în planificare
  - Adesea folosit în momente de review sau de livrare
  - Poate fi legat de termeni contractuali, constrângeri de calendar, **livrabile**
- **Livrabil** - un produs unic, măsurabil și verificabil al muncii
  - Poate fi intern sau extern
  - Poate avea diferite diseminări și niveluri de formalitate
  - In diagramele Gantt, ele interconectează adesea sarcini (output-ul unei sarcini este un livrabil ce constituie input-ul sarcinii următoare)

Ieșirile proiectului - Milestone, Livrabile

În practicile curente, adesea milestone-urile și livrabilele sunt folosite în mod interschimbabil (amândouă sunt folosite pentru a identifica produse; milestone-urile pot reprezenta produse cheie)

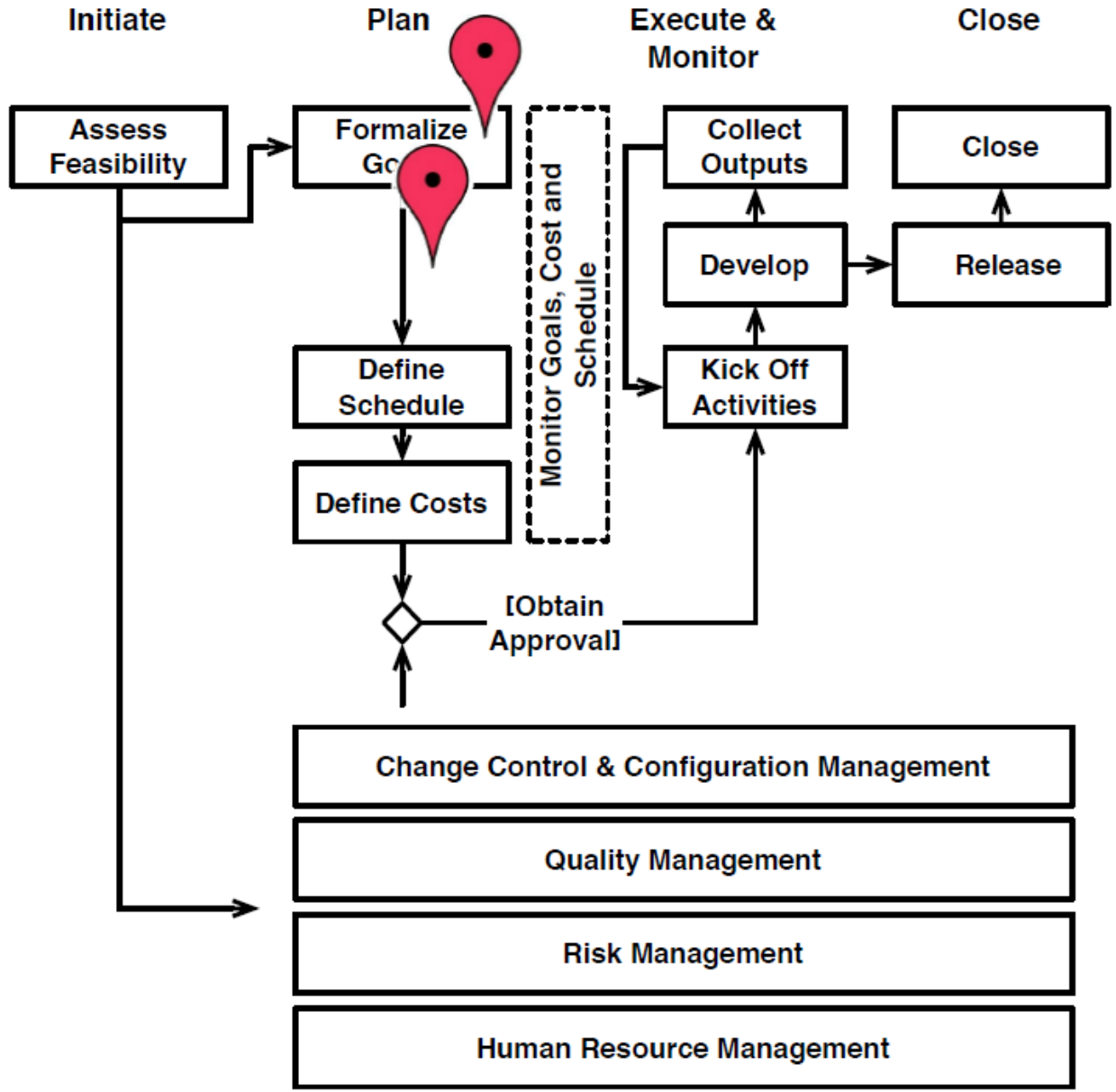
Ambele au durata **zero** in planificare

Work Breakdown Structure (WBS)



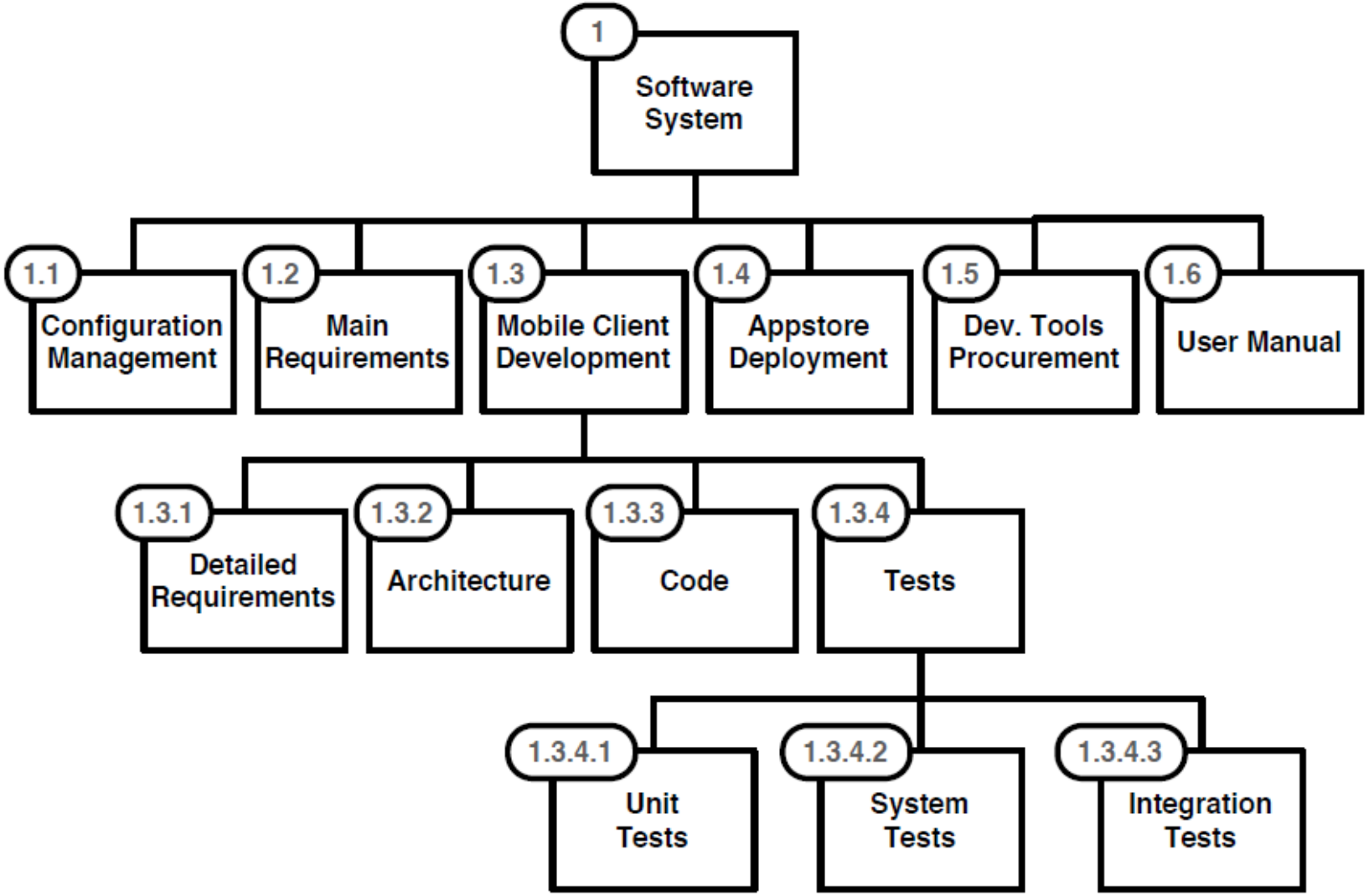
Obiective

- Înțelegerea tranziției de la obiectivele proiectului la planificarea proiectului
- Introducere în notația specifică pentru Work Breakdown Structure


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Go
Define Schedule
Define Costs
Monitor Goals, Cost and Schedule
Collect Outputs
Develop
Kick Off Activities
Close
Release
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Work Breakdown Structure (WBS) reprezintă o descompunere ierarhică (orientată spre livrabile) a muncii ce trebuie efectuate de către echipa de proiect pentru realizarea obiectivelor proiectelor și crearea livrabilelor dorite.

Exemplu WBS


1
Software System
1.1
Configuration Management
1.2
Main Requirements
1.3
Mobile Client Development
1.4
Appstore Deployment
1.5
Dev. Tools Procurement
1.6
User Manual
1.3.1
Detailed Requirements
1.3.2
Architecture
1.3.3
Code
1.3.4
Tests
1.3.4.1
Unit Tests
1.3.4.2
System Tests
1.3.4.3
Integration Tests

WBS – Observații

- Două formate:
  - Arbore graphic (Vision, Graffle, LibreOffice, …)
  - Schemă textuală (Microsoft Word, Text Editor, Outliner, …)
- Folosește un sistem numeric decimal pentru identificarea elementelor (Ex: 3.1.5)
- Arată legăturile de tipul este inclus în
- Nu include dependențe sau durate

De ce este util?

Un WBS stabilește baza pentru:

- Definirea lucrului ce va fi efectuat într-un proiect
- Stabilirea felului în care diferite activități sunt legate de obiectivele proiectului
- Stabilirea unui cadru pentru definirea, alocarea și monitorizarea lucrului și a costurilor
- Identificarea elementelor organizatorice necesare pentru finalizarea lucrului

WBS – Reguli generale

- Totul este la locul lui:
  - Regula 100% - asigurarea ca toate elementele lucrului sunt incluse (WBS-urile orientate catre produs sunt mai potrivite pentru aceasta regula)
  - Regula ER (Excludere Reciprocă): asigurarea că nu există suprapuneri în definirea elementelor
  - Nu este nevoie de echilibrare: nu este necesar ca toate drumurile să meargă până la același nivel
- Calitatea WBS-ului este ridicată
  - Coerenta: task-urile din acelasi pachet (work package) trebuie să aibă același scop
  - Cuplare: dependențele dintre pachete trebuie minimizate, astfel încât membrii echipei să poată lucra independent
  - Continuitate: producerea de pachete ar trebui să fie full-time pentru maximizarea eficienței
  - Cost: pachetele de la nivelul inferior ar trebui să necesite munca unui singur om, pe o perioadă între o săptămână și o lună

Cand ne oprim?

- Raspuns simplu: la nivel de pachet (care poate fi compus din mai multe activități elementare pe care nu este necesar sa le urmărim)
- Totusi: cat de mare este un pachet?
  - Conform “DOD and NASA Guide to PERT COST”: frunzele dintr-un WBS nu ar trebui să necesite mai mult de 3 luni de muncă sau 100.000 dolari cheltuiți
  - Conform altor standarde: 1-2 săptămâni pentru 1-2 persoane
- Nivelul de detalii depinde în schimb de dimensiunea proiectului

Tipuri de WBS (1/3)

- WBS de Produs
  - Evoluează conform structurii ieșirilor ce trebuie produse
  - Poate porni de la un Product Breakdown Structure
- WBS de Proces
  - Evoluează conform fazelor de organizare ale proiectului
  - De exemplu: Cerințe, Analiză, Design, Testare
- WBS Hibrid: ambele de mai sus
  - Îmbină procesul și produsul
  - De exemplu: fazele ciclului de viata la niveluri mai înalte; componente la niveluri joase

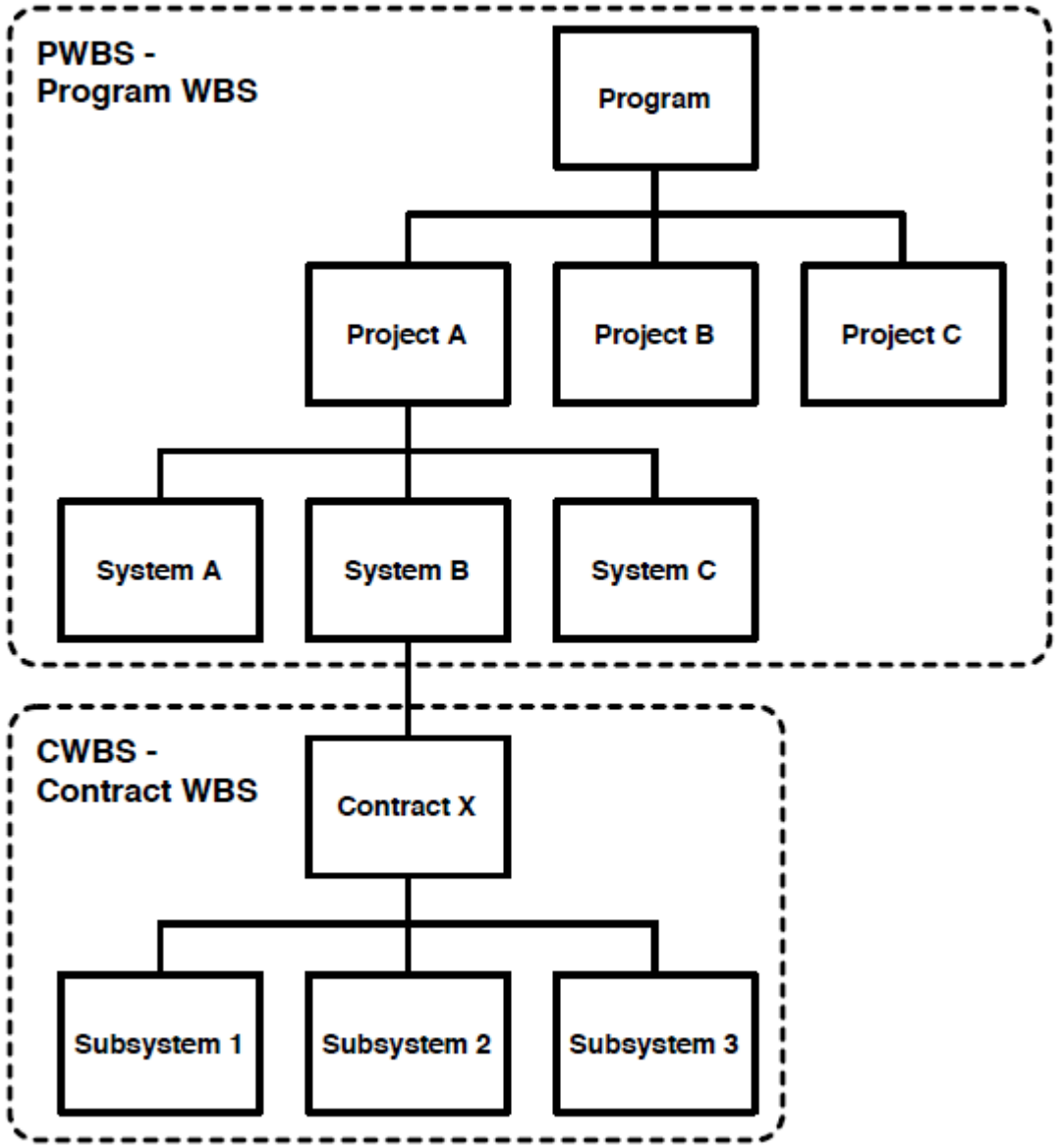
Tipuri de WBS (2/3)

- WBS de Organizare și WBS Geografic
  - Nivelurile înalte sunt unități organizatorice
  - Nivelurile joase reunesc munca ce se află sub responsabilitatea unei unități
  - Pot fi utile pentru proiectele multidisciplinare
- WBS Geografic
  - Nivelurile înalte sunt reprezentate de echipe distribuite geographic (e.g. echipa din NY, echipa din Roma)
  - Nivelurile joase reunesc munca ce se afla sub responsabilitatea unei echipe

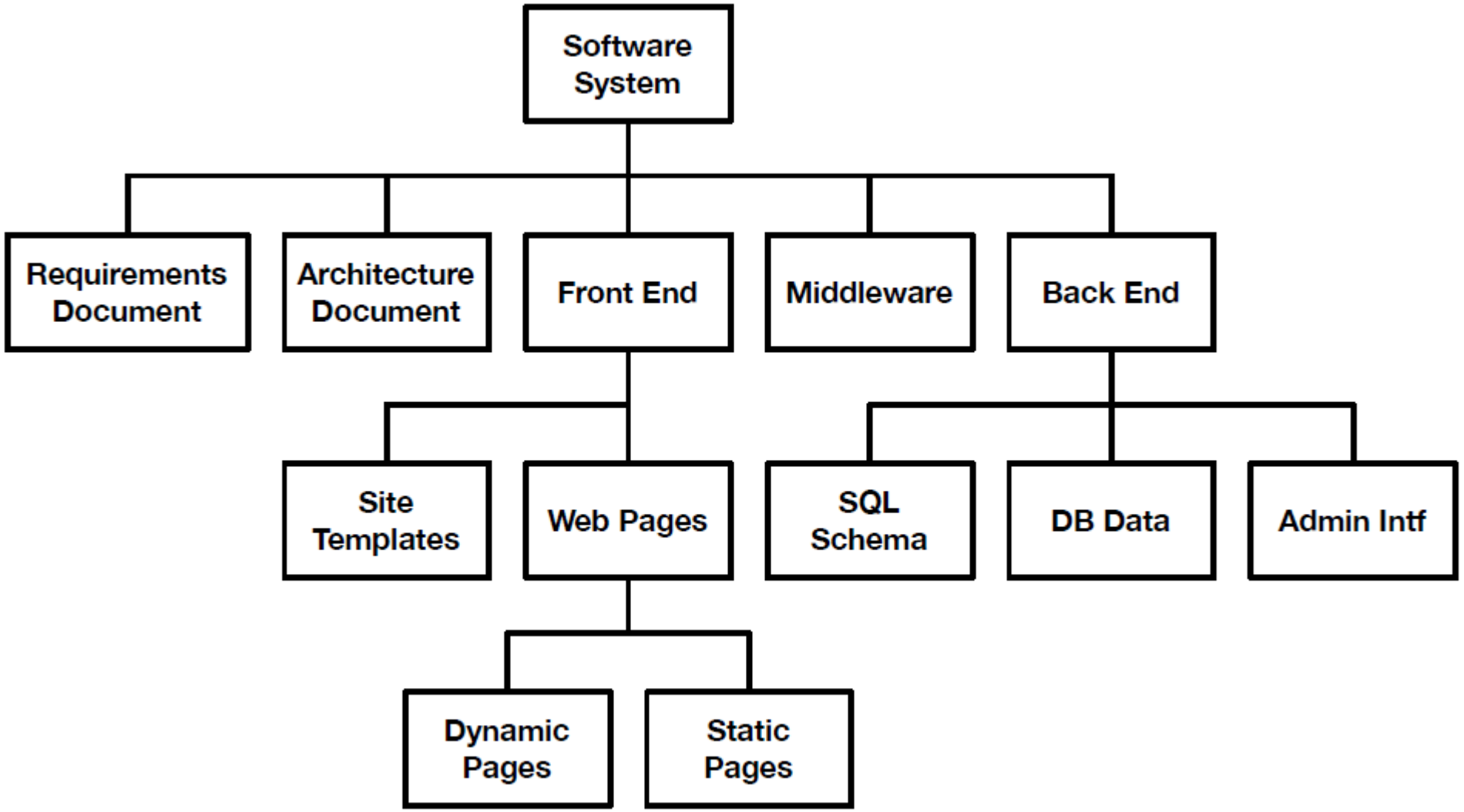
Remarcă: conform PMBOK, acestea nu sunt considerate WBS-uri. Sunt într-adevăr mai rar întâlnite.

Tipuri de WBS (3/3)

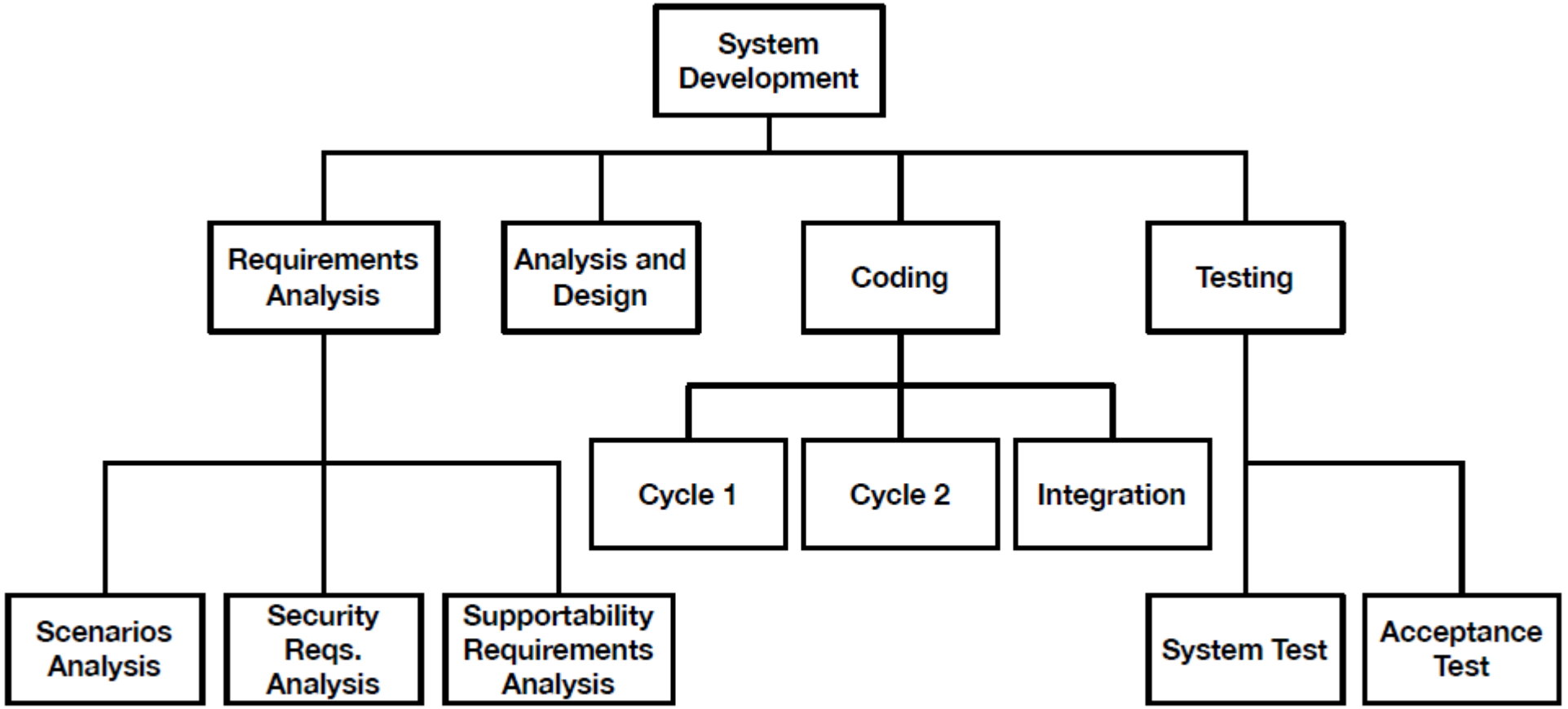
- PWBS - WBS de Program, folosit pentru coordonarea tuturor proiectelor (sistemelor)
- CWBS - WBS de Contract, baza pentru subcontractarea dezvoltării sistemului



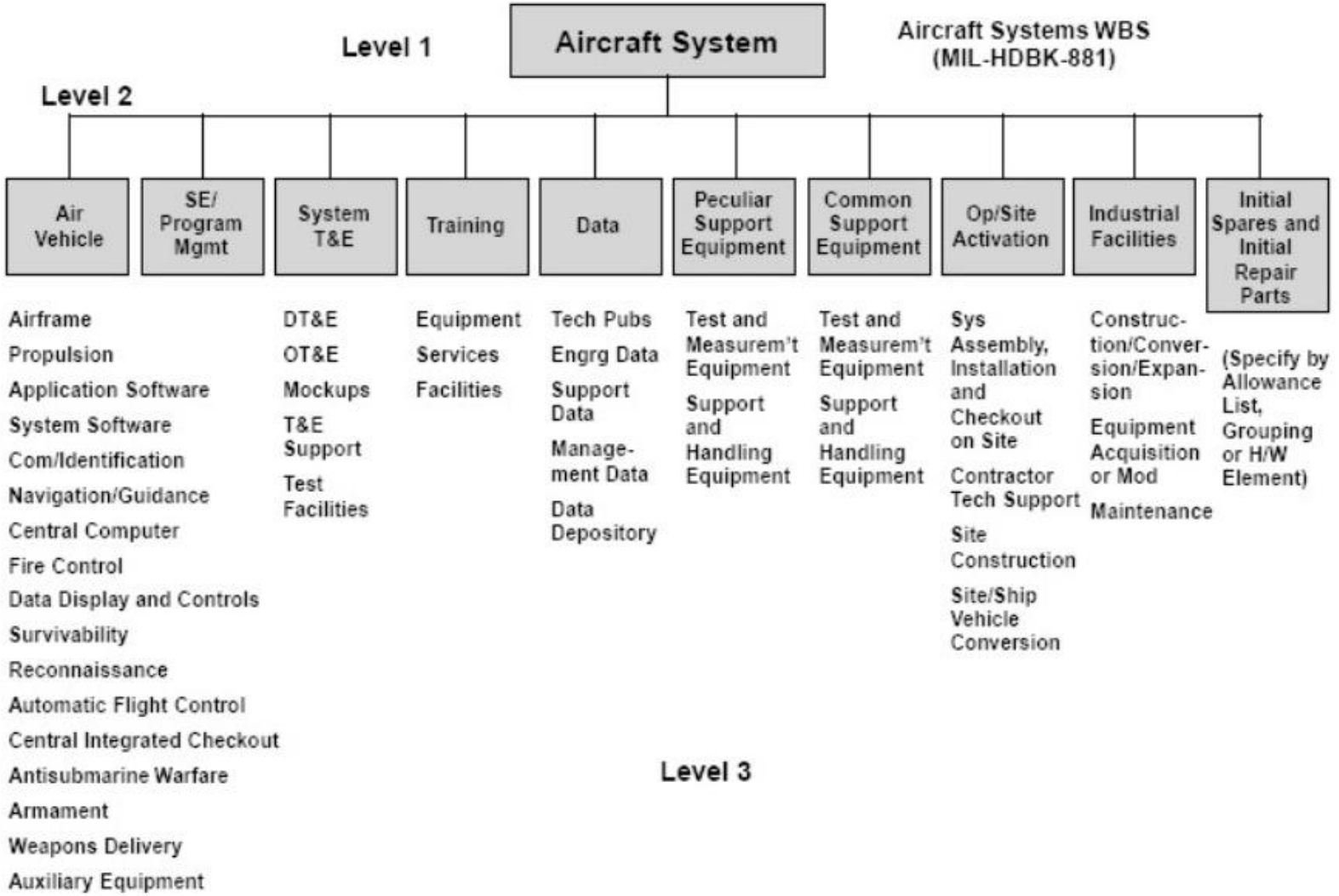
WBS de Produs - Exemplu



WBS de Proces - Exemplu



Exemplu WBS (MIL-HDBK-881)



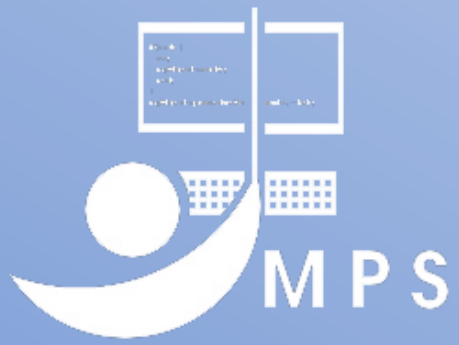
Dicționarul WBS

- Dicționarul WBS ajută la specificarea amanunțită a intrarilor unui WBS
- Poate conține titlu, număr, descriere detaliată a elementului, cantități, muncă asociată, elemente contractuale
- Reguli generale:
  - poate fi realizat pentru fiecare intrare a arborelui
  - urmariți definiția: creșteți cantitatea de detalii în timp ce coborâți în arbore
  - o practică bună este realizarea dicționarului pentru frunze (work packages)

Dicționarul WBS

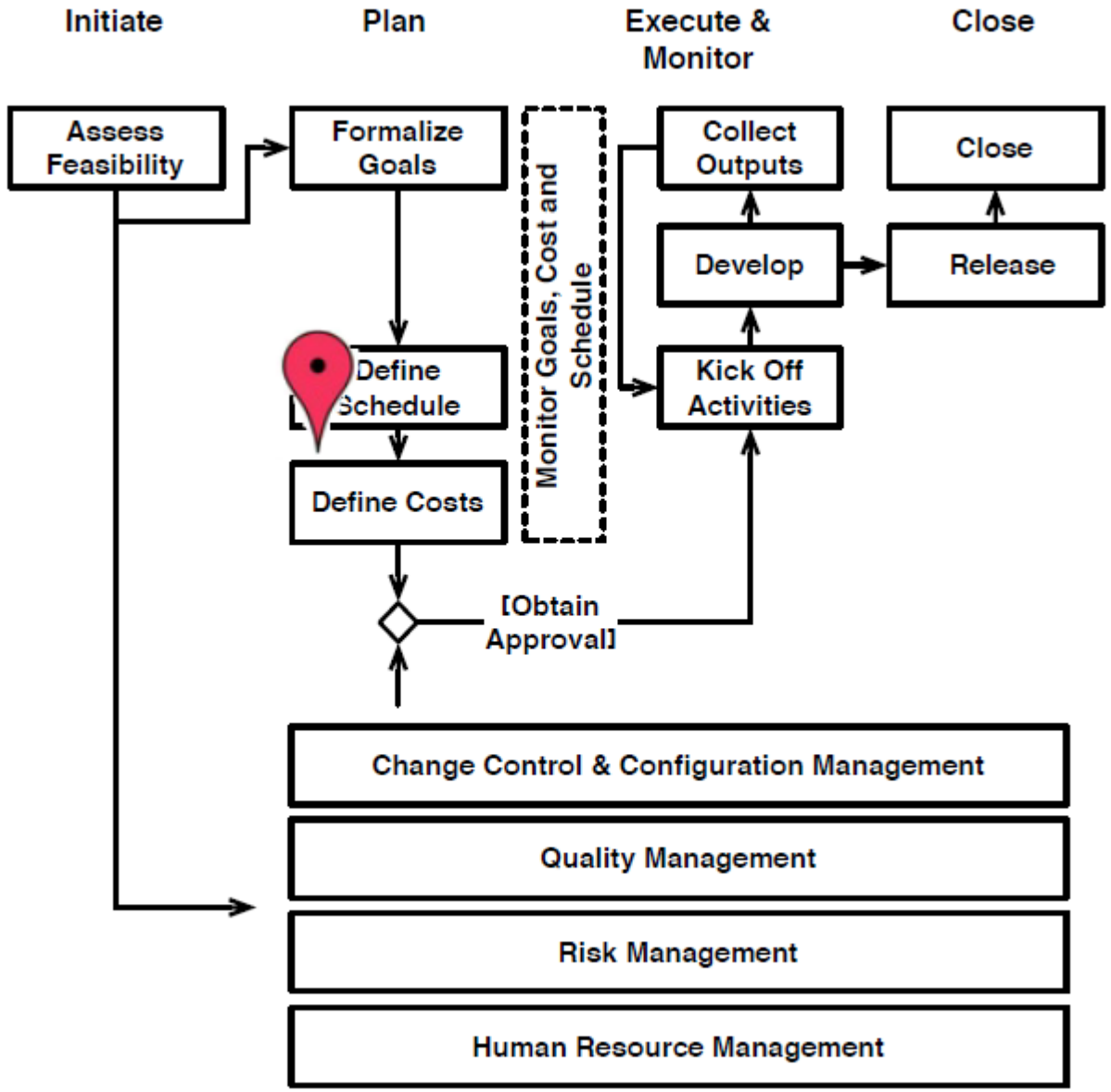
Work package number	1	**Start date or starting event:**	Month 1								
Work package title	Case Study Requirements and Experimentation Site Assessment										
Activity type	RTD										
Participant number	1	2	3	4	5	6	7	8			
Participant short name	P1	P2	P3	P3	P4	P5	P6	P7			
Person-months per participant	1	0	0	9	6	0	2	0			
Objectives											
Description of work	Task 1. Task 2. ...										
Deliverables	D1.1. D1.2.										
Milestones	M1.1. M1.2.										

Tehnici Algoritmice pentru Estimare: Function Points



Predictions are hard, especially about the future.

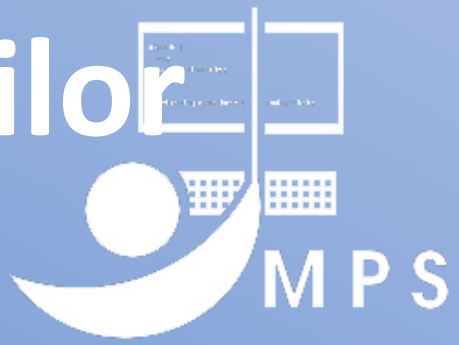
Yogi Berra


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Define Schedule
Define Costs
Monitor Goals, Cost and Schedule
Collect Outputs
Develop
Kick Off Activities
Close
Release
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Obiective

- Înțelegerea principiilor de bază ale tehnicilor algoritmice și, în mod detaliat, a metodei de estimare Function Point

Recapitulare: Principiile Tehnicilor Algoritmice



Introducere

- Scop: găsirea unei metode pentru a determina sistematic efortul (durata) necesar pentru un task/proiect arbitrar
- Ideal:
  - Identificarea unui set de caracteristici **măsurabile** ale unui proiect ce determină efortul/durata acestuia
  - Definirea unei funcții ce calculează **efortul/durata** pe baza caracteristicilor menționate mai sus

$$f(x_1, \ldots, x_n) = e$$

Problemă: cum calculăm f, x_1, …, x_n?

Soluție

- Analizarea proiectelor/seturilor de date existente; fiecare proiect este reprezentat de un vector:

$$< a_1, \ldots, a_n, \mathit{efort} >$$

- Găsirea corelărilor dintre (unele) variabile din seturile de date:

$$f(a_1, \ldots, a_k) \propto \mathit{efort}$$

- Găsirea mijloacelor adecvate de măsurare pentru variabile la începutul proiectului (astfel încât putem aplica funcția unui nou proiect)

Metrici de productivitate

Există două clase principale de metrici folosite pentru a măsura efortul necesar pentru dezvoltarea unei aplicații:

- Metrici bazate pe funcții - măsoară numărul și complexitatea funcțiilor de dezvoltat
- Metrici bazate pe dimensiune - măsoară dimensiunea unei aplicații (în materie de număr de linii de cod, de exemplu)

Metrici bazate pe funcții

Avantaje

- Complexitatea funcțională poate fi estimată atunci când cerințele sunt finalizate
- Nu depind de limbajul de programare

Dezavantaje și critici

- Dificil de legat de productivitate
- Mai dificil de măsurat decât metricile bazate pe dimensiune

Metrici bazate pe dimensiune

Avantaje

- Pot fi măsurate cu precizie (o dată ce avem acces la codul sursă și am adoptat standardele de codare)
- Instrumente de măsurare disponibile
- Mai ușor de legat de productivitate

Dezavantaje și critici

- Sunt cunoscute cu exactitate atunci când sistemul este construit (… un pic prea târziu pentru a le putea folosi pentru estimare)
- Sensibile la limbajul de programare și la standardele de codare
- Nu măsoara complexitatea codului

Ce este o linie de cod?

```
System.out.println("Hello world");
```

```
if (a == 1) {
return true;
}
else {
return false;
}
```

```
return a == 1 ? true : false
```

```
if (a == 1) { return true; } else { return false; }
```

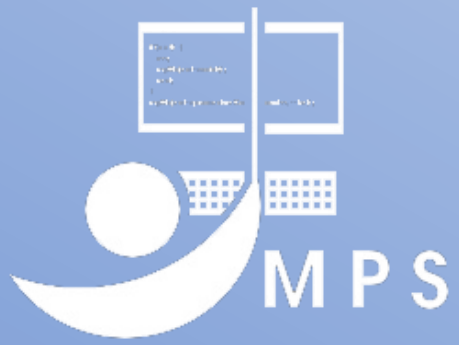
Exemple de productivitate

- Din Software Engineering, Ian Sommerville:
  - **Sisteme integrate in timp real**
    - 40-160 LOC/P-month (linii de cod / lună-persoană)
  - **Programe de sistem**
    - 150-400 LOC/P-month (linii de cod / lună-persoană)
  - **Aplicatii comerciale**
    - 200-9s00 LOC/P-month (linii de cod / lună-persoană)

Exemplu - https://en.wikipedia.org/wiki/Source_lines_of_code

- Red Hat Linux 7.1 contine aprox. 30 milioane SLOC (source lines of code), ar fi avut nevoie de 8000 de ani-persoană dacă ar fi fost dezvoltat conform practicilor de proprietate convenționale

Function Points



Function Points

- Bazate pe o combinație de caracteristici de program derivate din cerințe (metrici bazate pe funcții)
- Intâi propuse de Albrecht în anii ‘70
- Prima aplicabilitate în sistemele de business
- Evoluate pentru a include o gamă largă de sisteme și aplicații
- Baza de utilizatori largă și un IFPUG (International Function Point Users Group) pentru instruire în aplicarea tehnicii

Procesul pe scurt

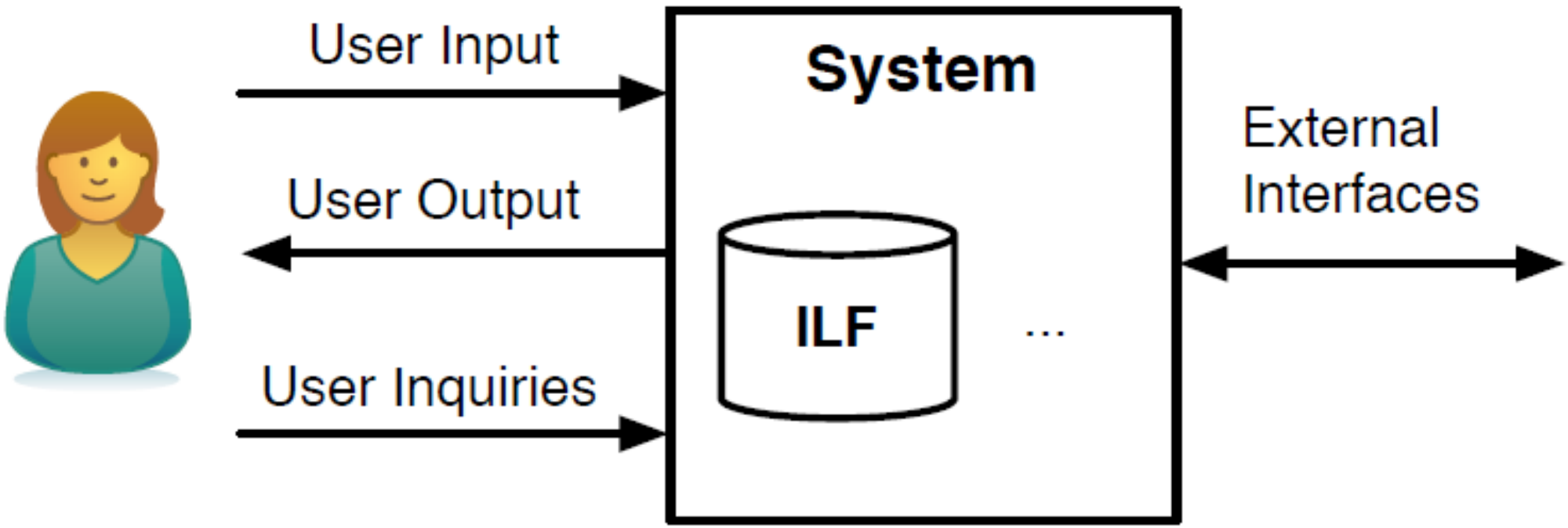
1. Definirea graniței analizei
2. Măsurarea unui set de caracteristici legate de cerințele funcționale ale sistemului și folosirea unei formule pentru determinarea Punctelor Funcționale Neajustate (Unadjusted Function Points – UFP)
3. Măsurarea unui set de caracteristici legate de cerințele non-funcționale: calcularea valorii factorului de ajustare (value adjustment factor – VAF)
4. Calcularea Punctelor Functionale (Ajustate) – FP:

   FP = f(VAF, UFP)

UFP - Intrări

- Intrările utilizatorului - numărul total de input-uri ale utilizatorului și complexitatea lor
- Ieșirile utilizatorului - numărul total de output-uri ale utilizatorilor și complexitatea lor
- Cererile utilizatorului - numărul total de intrări ale utilizatorului care generează un răspuns din partea software-ului, ca de exemplu numărul de cuvinte, rezultatele căutarii sau statusul software-ului
- Fișiere Logice Interne - numărul total de fișiere create și utilizate în mod dinamic de către sistem
- Interfețe externe - numărul total de fișiere externe care conecteaza software-ul cu un sistem extern. De exemplu, dacă software-ul și un dispozitiv comunică între ele, avem 1 interfață externă

Numărul total al fiecărei categorii este clasificat ca fiind ușor, mediu sau complex.


User Input
User Output
User Inquiries
System
ILF
...
External
Interfaces

VAF - Intrări

- Sistemul are nevoie de salvări și backup-uri de încredere?
- Sistemul are nevoie să comunice date?
- Elaborarea este distribuită?
- Performanța este un factor critic?
- Sistemul va funcționa într-un mediu bine cunoscut?
- Introducerea datelor este inclusă în tranzacții complexe?
- Datele sunt complexe?
- Elaborarea internă este complexă?
- Codul trebuie să fie reutilizabil?
- Proiectul include de asemenea instalarea și conversia datelor?
- Sistemul trebuie să fie utilizat în diferite organizații?
- Sistemul trebuie să fie ușor de menținut si de utilizat?

Raspunsuri de la 0 (irelevant) la 5 (extrem de relevant)

Formule pentru Function Points

$$UFP = \sum_{i=1}^{5} \begin{bmatrix} k_i^E & k_i^A & k_i^C \end{bmatrix} \cdot \begin{bmatrix} n_i^E \\ n_i^A \\ n_i^C \end{bmatrix}$$

$$FP = UFP \cdot (0.65 + 0.01 \cdot \sum_{i=1}^{14} C_i)$$

Comentarii despre formule

- **UFP** reprezintă suma ponderată a input-urilor externe, output-urilor externe, …
  - Valoarea constantelor (k^E, k^A, k^C) este dată de către metodă
- **FP (ajustate)** iau în considerare caracteristicile generale ale sistemului (cele 14 intrebări)
  - FP în intervalul **[0.65 UFP, 1.35 UFP]**

UFP
Caracteristici de program

- Numărarea UFP-urilor se face privind cerințele
- Analiștii se uită după urmatoarele elemente:
  - DET: tipuri de elemente de date (data element types)
  - FTR: tipul de fișier referit (file type referenced)
  - RET: tipul elementului înregistrat (record element type)
- Tabelele convertesc numerele de mai sus în 5 caracteristici
- Trainingul este necesar pentru aplicarea corectă a tehnicii

ILF/EIF		DET	
RET	1..19	20..50	> 50
0..1	Low	Low	Average
2..5	Low	Average	High
>5	Average	High	High

UI		DET	
FTR	1..4	5..15	> 15
0..1	Low	Low	Average
2	Low	Average	High
> 2	Average	High	High

UO/EQ		DET	
FTR	1..5	6..19	> 19
0..1	Low	Low	Average
2..3	Low	Average	High
>3	Average	High	High

FP: Caracteristici de program

- Joel Henry (“Introduction to SPM”) propune o abordare practică de numărare:
  - UI: numărul de casuțe de dialog, clasificat drept simplu, mediu și complex
  - UO: numărul de rapoarte, clasificat drept simplu, mediu și complex
  - UI: numărul total de intrări ale utilizatorului ce generează un răspuns din partea software-ului
  - ILF: numărul de fișiere și structuri de date interne complexe, clasificat drept simplu, mediu și complex
  - EIF: numărul de fișiere externe și dispozitive conectate, clasificat drept simplu, mediu și complex
- Regula de baza: realizează o estimare grosieră

Ce se întamplă o dată de avem FP-urile?

- O estimare FP măsoara printr-un număr dimensiunea sistemului ce trebuie construit
- Este posibil acum:
  - Să se lege FP direct de efort, prin măsuri de productivitate, dacă organizația păstrează acest gen de date:
    - FP pe persoană, pe lună
    - FP per lună calendaristică
  - Să se convertească FP-ul în SLOC și folosirea unei alte metode de estimare (e.g. COCOMO)
  - Să se utilizeze FP-ul pentru a verifica estimările furnizate de alte tipuri de estimare

Metrici de productivitate ale FP-urilor

- Fiecare companie ar trebui să pornească propriul program de măsurare a productivității
- Câteva date și studii “generale” sunt disponibile (e.g. http://www.softwaremetrics.com/Articles/estimatingdata.htm)
- Productivitate se schimbă cu dimensiunea sistemului (http://www.softwaremetrics.com/Articles/estimatingdata.htm):
  - 1.3 ore/FP pentru un sistem de 50 FP
  - 12.1 ore/FP pentru un sistem de 7000 FP
  - 133.6 ore/FP pentru un sistem de 15000 FP

Relația dintre FP și SLOC

- FP pot fi de asemenea legate de SLOC prin metrici FP/SLOC
- Aceasta permite:
  - Aplicarea altor tehnici algoritmice (e.g. COCOMO)
  - Măsurarea progresului folosind SLOC-uri

Language	Avg	Median	Low	High
ABAP (SAP)	18	18	16	20
Access *	36	38	15	47
Ada	154	-	104	205
Advantage	38	38	38	38
Assembler *	209	203	91	320
C *	148	107	22	704
C++ *	59	53	20	178
C# *	58	59	51	66
Clipper *	40	39	26	53
COBOL *	80	78	8	400
ColdFusion	68	56	52	105
Cool:Gen/IEF *	37	35	10	180
Culprit	51	-	-	-
Datastage	67	79	16	85
DBase IV	52	-	-	-
Easytrieve+	33	34	25	41
Excel	47	46	31	63
FORTRAN	90	118	35	-
J2EE *	57	50	50	67
Java *	55	53	9	214

Sursă: http://www.qsm.com/resources/function-point-languages-table

FP - Concluzii

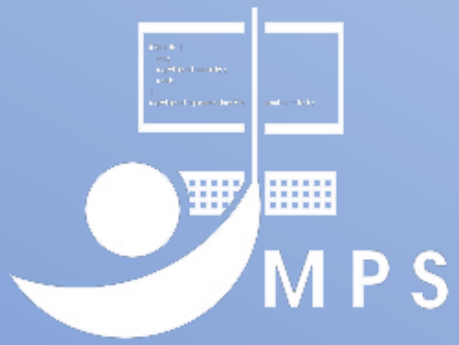
Dezavantaje

- necesită training specific pentru a fi aplicat corect
- poate introduce rigiditate în procesul de dezvoltare (mai ales în cazul în care este folosită această tehnică pentru stabilirea prețurilor, trebuie luată în considerare o schimbare a cerințelor)

Avantaje

- aduce o măsura **obiectivă** în complexitatea sistemului și poate fi folosită pentru punerea de acord în stabilirea prețurilor
- poate fi calculate după cerințe! (efortul de dezvoltare al unui sistem este mai dificil de estimat decât redactarea cerințelor)
- analiza, chiar și atunci când este derulată naiv, permite o mai bună înțelegere a sistemului de construit

COCOMO – Constructive Cost Modeling



Modelul COCOMO

- O familie de modele empirice bazate pe analiza proiectelor diferitelor companii
- Lungă istorie de la COCOMO-81 (1981) până la COCOMO-II (1999, 2000)
- Extins pentru a acoperi diferite procese de dezvoltare și alte aspecte, precum calitatea (COQUALMO)

Modelul COCOMO

- COCOMO este bazat pe o măsură fizică (linii de cod sursă)
- Estimarile devin mai precise atunci când trecem la faza de dezvoltare
- Erori de estimare:
  - Estimările inițiale pot fi eronate cu un factor de 4x
  - Atunci când avansăm în procesul de dezvoltare, estimările devin mai precise (și modelul ia în calcul parametri mai detaliați)

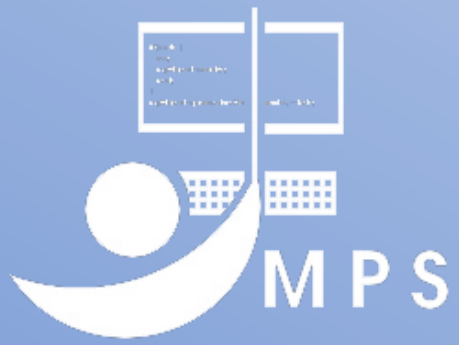
COCOMO – Structura Generala

- Toate modelele COCOMO au aceeași structură de bază

$$\mathrm{OUTPUT} = A \cdot (\mathrm{size})^B \cdot M$$

- OUTPUT-ul poate fi efort sau timp
- Măsură fundamentală este dimensiunea codului (exprimată în linii de cod sursă)
- Dimensiunea codului are un efect exponențial asupra efortului și dimensiunii (dar foarte aproape de 1)
- Diferiți factori de ajustare sunt folosiți pentru a face modelul mai precis

COCOMO 81



COCOMO 81 - Introducere

- Combinație de 3 modele cu diferite niveluri de detaliu și complexitate:
  - **de bază:** estimare rapidă, stadiu de dezvoltare incipient
  - **intermediar:** mai precis, necesită mai multe caracteristici ale produsului, stadiu de dezvoltare mai avansat
  - **avansat:** cel mai detaliat, necesită mai multă informație
- În toate modelele COCOMO:
  - 1 persoană-lună = 152 ore de lucru
  - SLOC este DSI (delivered source instructions)

  (doar codul este livrat către client. ex. testarea unitară, utilitățile nu sunt luate în considerare)

COCOMO 81 - Tipuri de Proiecte

COCOMO 81 face distincție între 3 tipuri de proiecte:

- **organic**: echipe mici, mediu familiar, aplicații bine înțelese, cerințe non-funcționale simple → ușor
- **semi-detașat**: echipa de proiect poate avea experiență mixtă, sistemul poate avea constrângeri non-funcționale semnificative, organizația poate avea mai puțină familiaritate cu aplicația → mai dificil
- **incorporat (embedded)**: constrângeri puternice, incluzând regularizări locale și proceduri operaționale; neobișnuit pentru echipe să aibă o experiență puternică asupra aplicației → hard

COCOMO 81 - Model de Bază

$$PM = A_{PM} \cdot (KSLOC)^{B_{PM}}$$

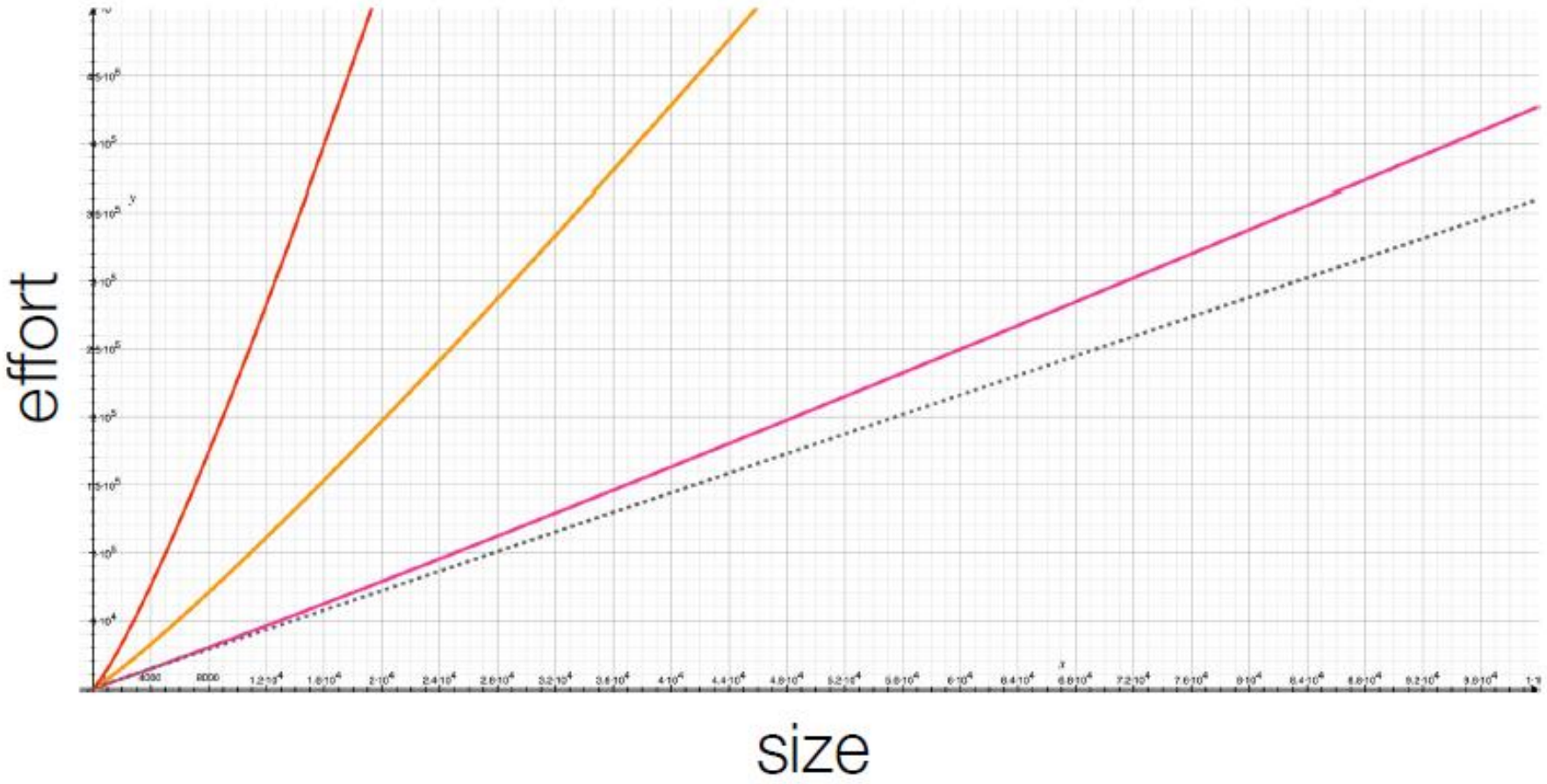
$$TDEV = A_{TDEV}(PM)^{B_{TDEV}}$$

unde:

- KSLOC: mii de linii de cod sursă livrate
- M este egal cu 1 (astfel nu apare în formule)

	A	B	A	B
Organic	2.4	1.05	2.5	0.38
Semi-detached	3	1.12	2.5	0.35
Embedded	3.6	1.2	2.5	0.32

Efectul Exponential COCOMO vs. liniar



Exemplu de aplicație

- Estimarea de 50 KDSI pentru un proiect organic
  - PM = 2.4 (50)^1.05 ~= 146 mm
  - TDEV = 2.5 (371.54)^0.38 ~= 16 luni
  - Echipă = 371.54 / 23.69 ~= 9 persoane
- Efectul diferiților parametri ai proiectului:

	A	B	A	B	PM	TDEV	Team
Organic	2.4	1.05	2.5	0.38	146	16.6	8.8
Semi-detached	3	1.12	2.5	0.35	240	17.0	14.1
Embedded	3.6	1.2	2.5	0.32	394	16.9	23.3

COCOMO - Intermediar

- Folosește o caracterizare mai fină, incluzând **atribute** (multiplicatori de efort) pentru a lua în calcul:
  - cerințe funcționale și non-funcționale
  - atribute ale proiectului
- **Multiplicatorii de efort** sunt organizați în 4 clase și 15 sub-elemente
- Importanța fiecărui atribut este **evaluată calitativ** între 1 (foarte scăzută) și 6 (foarte ridicată)
- Fiecare valoare corespunde unui multiplicator, în intervalul [0.7, 1.66] (multiplicator < 1 implică reducerea costului)
- Toate valorile **sunt multiplicate împreună** pentru a ajusta efortul

COCOMO 81 - Model Intermediar

$$PM_{nominal} = A_{PM} \cdot (KSLOC)^{B_{PM}}$$

$$PM = PM_{nominal} \cdot \Pi_{i=i}^{15} EM_i$$

$$TDEV = A_{TDEV}(PM)^{B_{TDEV}}$$

	A	B	A	B
Organic	3.2	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	2.8	1.2	2.5	0.32

Model Intermediar - Parametri

Effort Adjustment Factors		*Very_Low*	*Low*	***Nominal***	*High*	*Very_High*	*Extr_High*
Product Attributes							
Required Software Reliability	RELY	0.75	0.88	**1.00**	1.15	1.40	
Database Size	DATA		0.94	**1.00**	1.08	1.16	
Product Complexity	CPLX	0.70	0.85	**1.00**	1.15	1.30	1.65
Computer Attributes							
Execution Time Constraints	TIME			**1.00**	1.11	1.30	1.66
Main Storage Constraints	STOR			**1.00**	1.06	1.21	1.56
Virtual Machine Volatility	VIRT		0.87	**1.00**	1.15	1.30	
Computer Turnaround Time	TURN		0.87	**1.00**	1.07	1.15	
Personnel Attributes							
Analyst Capability	ACAP	1.46	1.19	**1.00**	0.86	0.71	
Applications Experience	AEXP	1.29	1.13	**1.00**	0.91	0.82	
Programmer Capability	PCAP	1.42	1.17	**1.00**	0.86	0.70	
Virtual Machine Experience	VEXP	1.21	1.10	**1.00**	0.90		
Programming Language Experience	LEXP	1.14	1.07	**1.00**	0.95		
Project Attributes							
Use of Modern Programming Practices	MODP	1.24	1.10	**1.00**	0.91	0.82	
Use of Software Tools	TOOL	1.24	1.10	**1.00**	0.91	0.83	
Required Development Schedule	SCED	1.23	1.08	**1.00**	1.04	1.10	

Atribute

- Atribute:
  - PRODUCT = RELY * DATA * CPLX
  - COMPUTER = TIME * STOR * VIRT * TURN
  - PERSONNEL = ACAP * AEXP * PCAP * VEXP * LEXP
  - PROJECT = MODP * TOOL * SCED
- Impactul parametrilor este între [0.09, 73.28]
- Managerul de proiect (sau echipa) estimează valorile parametrilor pentru a prezice efortul
- Exemplu:
  - Dacă “fiabilitatea necesară a software-ului” este scăzută, efortul prezis este 0.88 din cel calculat cu formula de bază

COCOMO 81 - Modelul Avansat detaliat

- Modelul detaliat
  - are multiplicatori mai detaliați pentru fiecare fază de dezvoltare
  - organizează parametrii în mod ierarhic, pentru simplificarea calculului în cazul sistemelor alcătuite din mai multe module
- Proiectele sunt organizate în 4 faze:
  - Planificarea Cerințelor și Design de Produs (RPD)
  - Design Detaliat (DD)
  - Cod și Testare Unitară (CUT)
  - Teste de Integrare (IT)
- Estimările sunt realizate în fiecare fază
- Datele astfel obținute sunt agregate pentru a da estimarea totală

COCOMO 81 - Modelul Avansat detaliat

Exemplu de parametru:

Cost Driver	Rating	RPD	DD	CUT	IT
ACAP	Very Low	1.8	1.35	1.35	1.5
	Low	0.85	0.85	0.85	1.2
	Nominal	1	1	1	1
	High	0.75	0.9	0.9	0.85
	Very High	0.55	0.75	0.75	0.7

COCOMO - Faza de mentenanță

- Modelul COCOMO poate fi de asemenea aplicat pentru a prezice efortul în timpul mentenanței sistemului.

mentenanța sistemului → mici *actualizări* și *reparații în* timpul vieții operaționale ale unui sistem

- Mare parte dintre parametrii de dezvoltare se aplică atât la dezvoltare cât și la mentenanță (câteva excepții: SCED, RELY, MODP)
- Un input essential este reprezentat de estimarea ACT-ului

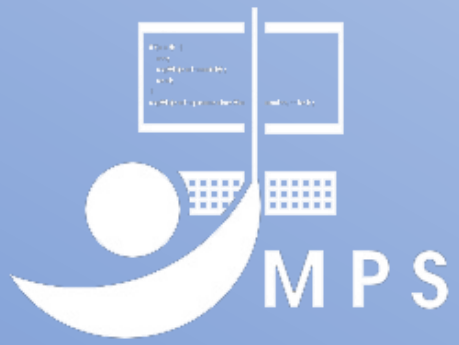
ACT - annual change traffic – porțiunea de cod sursă ce se schimbă în decursul unui an

COCOMO 81 – Mentenanță

$$ACT = \frac{\%Added + \%Modified}{100}$$

$$PM = ACT \cdot PM_{nom} \cdot EAF_{maint}$$

COCOMO II



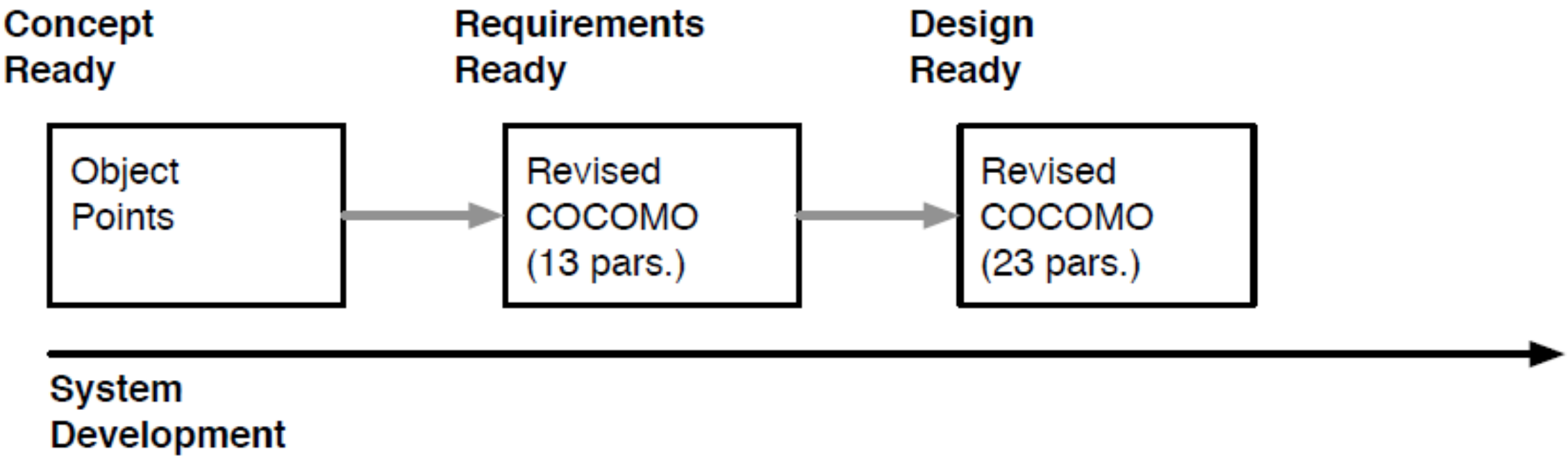
COCOMO II

- COCOMO este bazat pe COCOMO 81 și ia în considerare:
  - Noi procese de dezvoltare (e.g. spirală)
  - Flexibilitate crescută în dezvoltarea software (e.g. reutilizarea codului, generarea automată de cod)
  - Nevoia de a lua decizii în condiții de informații incomplete
  - Date noi cu privire la proiecte (nu neapărat o nevoie, cât o oportunitate (161 proiecte vs. 61)

COCOMO II - Modele

- COCOMO II include o serie de submodele ce produc estimări din ce în ce mai detaliate ale software-ului
- Submodelele COCOMO II sunt:
  - **Application Composition Model**: pentru prototipuri
  - **Early Design Model**: folosit atunci când cerințele sunt disponibile dar design-ul nu a început încă
  - **Post-architecture Model**: folosit o dată ce arhitectura sistemului a fost creată și avem la dispoziție mai multe informații cu privire la sistem
- Mai mult:
  - **Reutilizarea modelului**: folosit pentru a calcula efortul necesar pentru integrarea componentelor reutilizabile

COCOMO II - Stadiile modelelor



COCOMO II - Modelele ED și PA

$$PM_{NS} = 2.94 \cdot (SIZE)^{E} \cdot \Pi_{i=1}^{n} EM_i$$

$$E = 0.91 + 0.01 * \sum_{j=1}^{5} \mathrm{SF}_i$$

$$TDEV_{NS} = 3.67 * (PM_{NS})^{F}$$

$$F = 0.28 + 0.01 * \sum_{j=1}^{5} \mathrm{SF}_i$$

- Toate constantele pot (și trebuie) să fie ajustate ce valori dependente de organizație.

  Recomandat: 2.94 multiplicator de efort și 3.67 multiplicator de planificare

- Diferența între ED și PA este reprezentată de numărul de parametri.

- Exponentul depinde de factorii de ajustare. (față de COCOMO 81 în care era o simplă constantă)

COCOMO II - Multiplicatori de efort

Între 7 (Early Design) și 17 (Post Architecture) în funcție de nivelul de detalii necesar

Exemplu:

	Early Design cost drivers	Post-Architecture cost drivers (Counterpart combined)
Product reliability and complexity	RCPX	RELY, DATA, CPLX, DOCU
Required reuse	RUSE	RUSE
Platform difficulty	PDIF	TIME, STOR, PVOL
Personnel capability	PERS	ACAP, PCAP, PCON
Personnel experience	PREX	AEXP, PEXP, LTEX
Facilities	FCIL	TOOL, SITE
Required Development Schedule	SCED	SCED

Sursă: http://www.ifi.uzh.ch/req/courses/seminar_ws02/reports/Seminar_4.pdf

COCOMO II - Factori de scalare

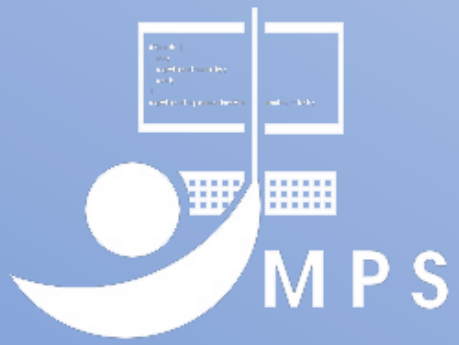
Exponentul este calculat prin furnizarea de răspunsuri calitative la următorii factori:

- **Precedentă**: cat de inedit este proiectul pentru organizatie
- **Flexibilitate**: flexibilitatea dezvoltarii (rigiditatea conformitatii cu cerintele)
- **Coeziunea echipei**
- **Maturitatea Procesului**: maturitatea conform chestionarului CMMI

Scale Factors (W_i)	Very Low	Low	Nominal	High	Very High	Extra High
PREC	thoroughly unprecedented	largely unprecedented	somewhat unprecedented	generally familiar	largely familiar	throughly familiar
FLEX	rigorous	occasional relaxation	some relaxation	general conformity	some conformity	general goals
RESL	little (20%)	some (40%)	often (60%)	generally (75%)	mostly (90%)	full (100%)
TEAM	very difficult interactions	some difficult interactions	basically cooperative interactions	largely cooperative	highly cooperative	seamless interactions
PMAT	Weighted average of "Yes" answers to CMM Maturity Questionnaire					

Sursă: http://www.ifi.uzh.ch/req/courses/seminar_ws02/reports/Seminar_4.pdf

Tehnici Algoritmice Concluzii



COCOMO – Considerație

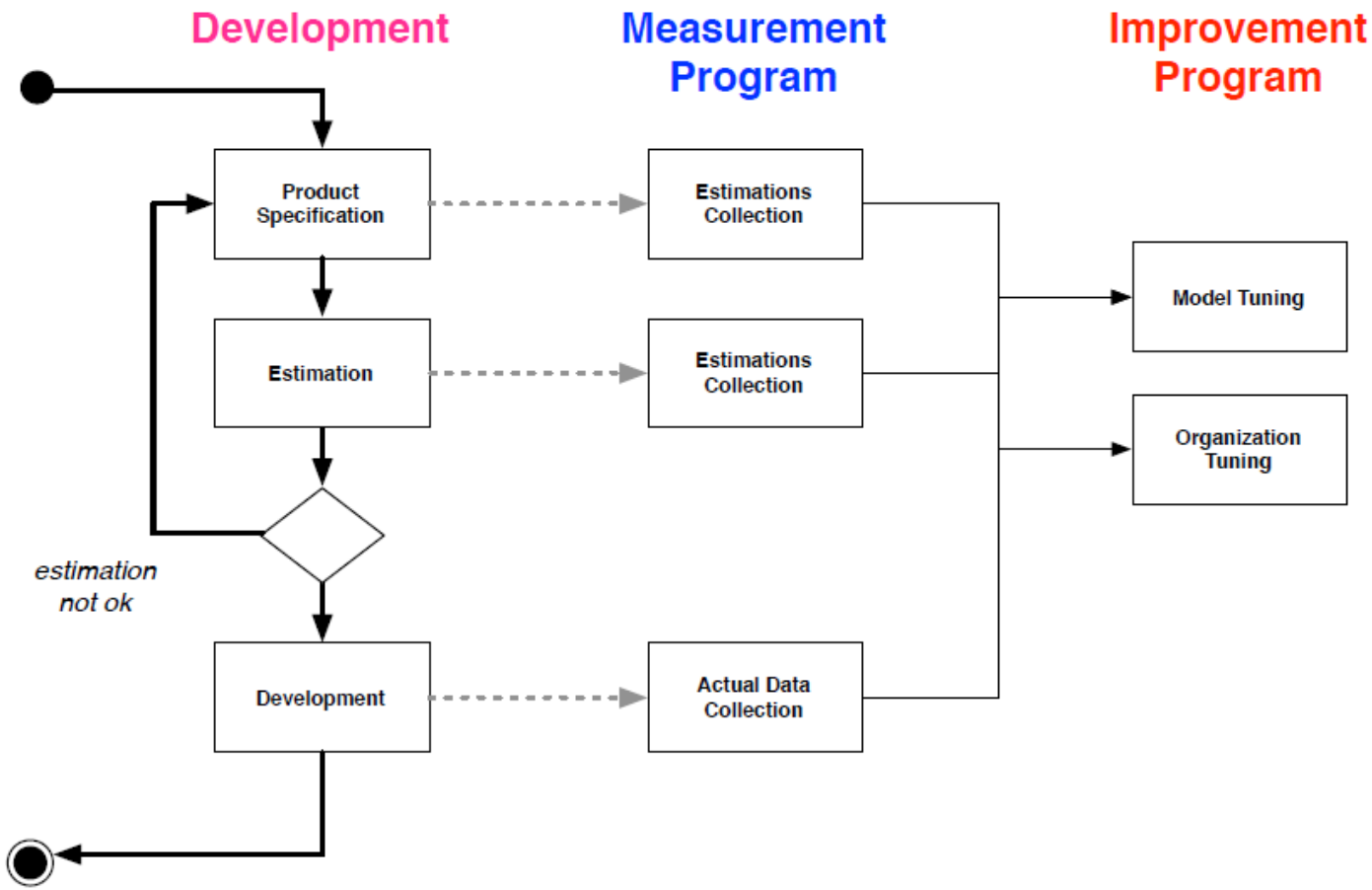
- O serie de modele din ce în ce mai complexe
- COCOMO calculează ambele D și E, iar forța de munca este derivata din D și E (adesea estimăm E, decidem M și calculam doar D)
- Estimările costurilor proiectului pot fi auto-îndeplinite: estimarea definește bugetul, iar produsul este ajustat pentru a respecta bugetul
- O aplicație precisă necesită ca organizațiile să își seteze propriile programe de măsurare (pentru rafinarea parametrilor)
- Modelele trebuie să fie adaptate la tehnologiile schimbatoare, ținând cont ca tehnologia se schimbă rapid… Poate fi dificil să le păstrăm actualizate la zi.

Tehnici Algoritmice - Recapitulare

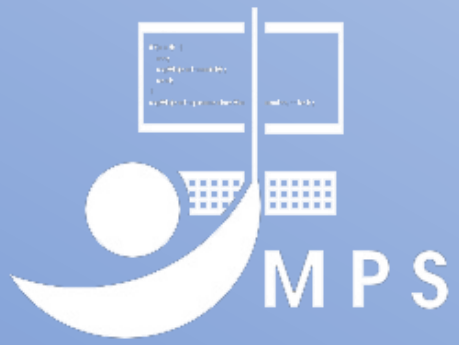
- Bazate pe caracteristicile sistemului și metricile de productivitate colectate în proiectele trecute
- Modele diferite:
  - Function Points:
    Req ☞ UFP ☞ FP, MM/FP

    Req ☞ UFP ☞ SLOC/UFP and COCOMO
  - Object Points
    Screens, Reports, Modules ☞ NOP, NOP/Month
  - COCOMO
    SLOC ☞ PDEV, TDEV ☞ Team Size = PDEV / TDEV

Fabrica de îmbunătățiri



Object Points



Limbaje de generatia a 4-a

- Un set de limbaje de nivel înalt (fiecare “generație” se îndepărteaza de codul mașină)
- Ele pot fi distinse între:
  - Tool-uri de gestionarea datelor: ele presupun utilizarea de funcții sofisticate pentru manipularea datelor (ex: SAS, SPSS, Mathematica, Matlab, ABAP)
  - Limbaje de generare de rapoarte și formulare, ce specifică formatul datelor și output-ul ce va fi generat (ex: SQL, Oracle Forms)
  - Generatoare automate de cod din tool-uri de tip CASE - Computer Aided Software Engineering (ex: Rails)

Object Points

- Metrici pe baza de funcții pentru limbaje de generatia a 4-a
- Calcule mai simple față de metoda Function Points

Referințe

Kauffman, Wright, Zweig, Automating Output Size and Reuse Metrics in a Repository-Based Computer Aided Software Engineering (CASE) Environment IEEE Trans. Software Engineering, 20(3), p. 169-186, 1994

Calculul în metoda Object Points

- Similar cu Function Points
- Calculează numărul de ecrane și le clasifică în 3 categorii → simplu, mediu, complex
- Calculează numărul de rapoarte și le clasifică în 3 categorii → simplu, mediu, complex
- Determină numărul de module ce trebuie dezvoltate
- Folosește matrice de ponderi pentru a însuma valorile de mai sus, luând în considerare codul reutilizat
- O formula transformă OP-urile în măsuri de productivitate

Clasificarea ecranelor și a rapoartelor

- Niveluri de complexitate ale OP pentru ecrane

	Number and sources of data tables		
Number of Views Contained	Total < 4	Total < 8	Total 8+
<3	simple	simple	medium
3-7	simple	medium	difficult
8+	medium	difficult	difficult

- Niveluri de complexitate ale OP pentru rapoarte

	Number and source of data tables		
Number of Sections Contained	Total < 4	Total < 8	Total 8+
0-1	simple	simple	medium
2-3	simple	medium	difficult
4+	medium	difficult	difficult

Sursa: http://yunus.hacettepe.edu.tr/~sencer/objectp.html

Metoda de Calcul pentru Object Points

	Weight		
Type	Simple	Medium	Difficult
Screen	1	2	3
Report	2	5	7
Modules	10	10	10

$$NOP = \sum_{i=1}^{3} \begin{bmatrix} k_i^s & k_i^m & k_i^d \end{bmatrix} \cdot \begin{bmatrix} n_i^s \\ n_i^m \\ n_i^d \end{bmatrix} \cdot \frac{(100 - r)}{100}$$

$$EFFORT = \frac{NOP}{PROD}$$

Unde:
r = procentul de componente reutilizate
PROD→ productivitatea

Estimări ale productivității

- Productivitatea **între 4 și 50 object points / lună,** depinzând de mentenanța tool-ului și capabilitatea dezvoltatorului:

Developer Experience	Very Low	Low	Nominal	High	Very High
PROD	4	7	13	25	50

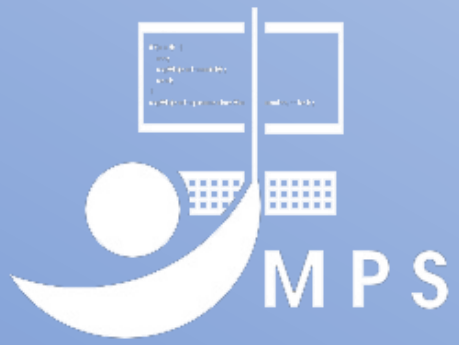
- Alte surse indică până la **75 OP/luna**

Sursă: http://yunus.hacettepe.edu.tr/~sencer/objectp.html

Observații

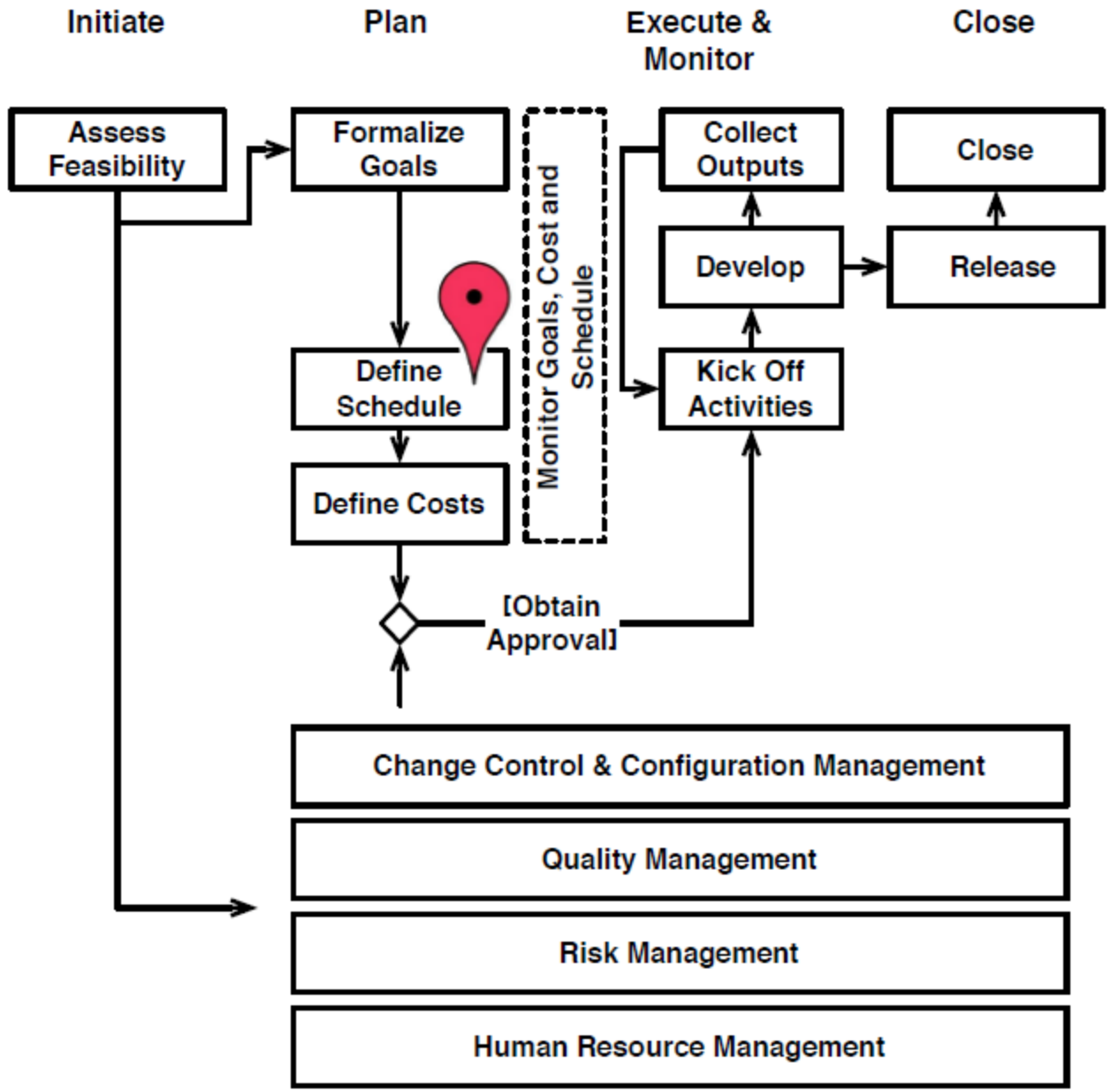
- Simplu de estimat → să ne amintim de abordarea lui Joel Henry pentru calculul de Function Points
- OP ia în calcul și codul reutilizat → fapt apărut și în unele elaborări ale modelului COCOMO

Planificarea proiectului



Obiective

- Transformarea WBS-ului într-o planificare
- Înțelegerea dependențelor dintre activități
- Învățarea tehnicii Drumului Critic (Critical Path)
- Învățarea uniformizării resurselor


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Prezentare generală

- Avem:
  - Un WBS (activități)
  - Estimarile de efort (durată) pentru fiecare element din WBS
- Vrem să planificăm activități astfel încât să putem cunoaște când începe și când se termină fiecare activitate, când avem nevoie de resurse, când livram
- Proces:
  - Identificarea constrângerilor (dependențelor)
  - Alocarea și uniformizarea resurselor
  - Găsirea drumului critic și iterarea până cand planul este satisfăcător
- Ieșire: diagrama Gantt

Diagrama Gantt modernă

- Outline textual + graf al calendarului
- Activitățile drept bare (posibil adnotate cu nume și resurse)
- Livrabile (diamante)
- Activitățile pot fi grupate (informația grupului este derivată din activități de nivel mai jos)
- Dependențe între sarcini

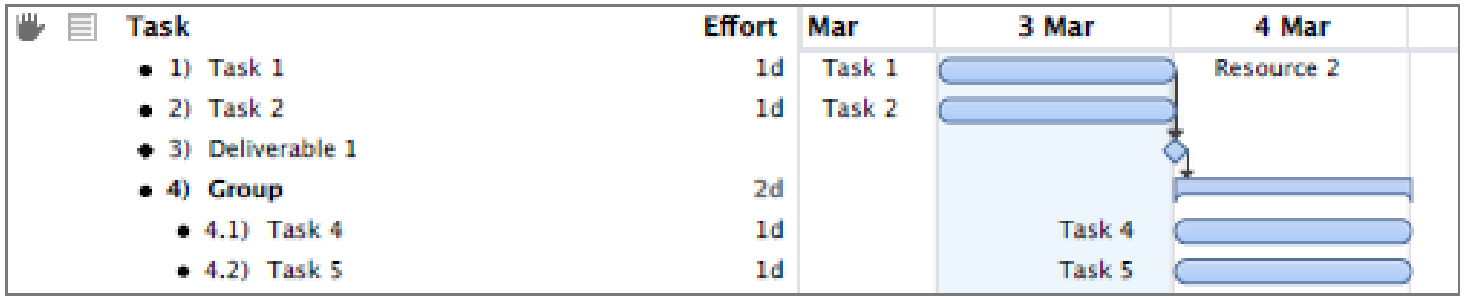
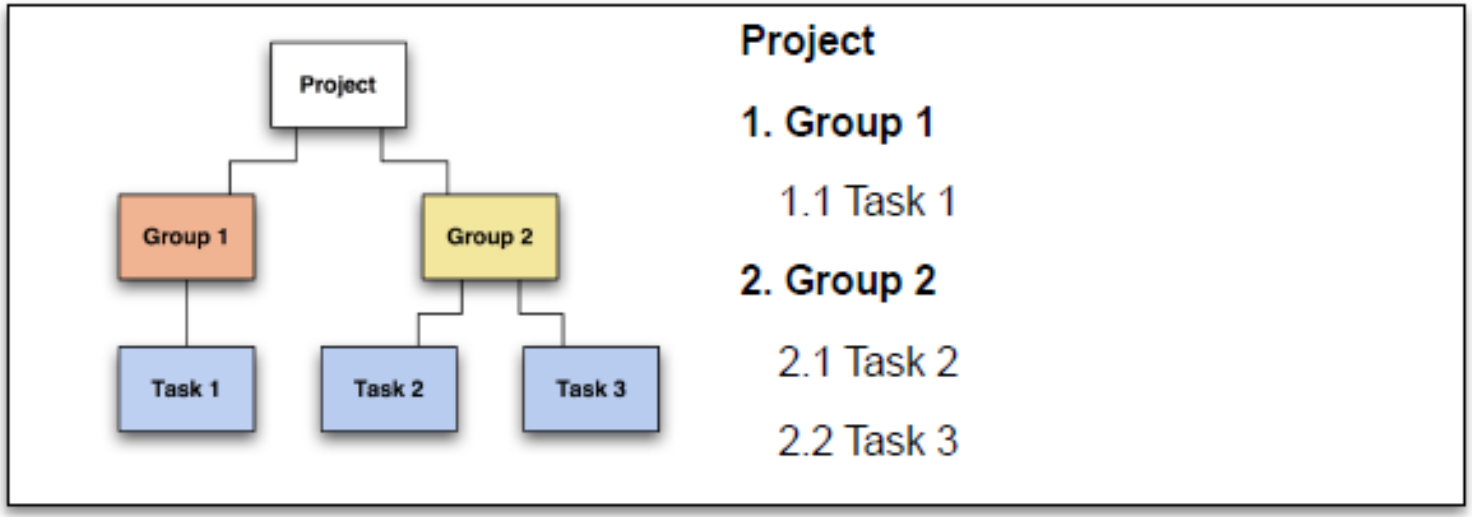
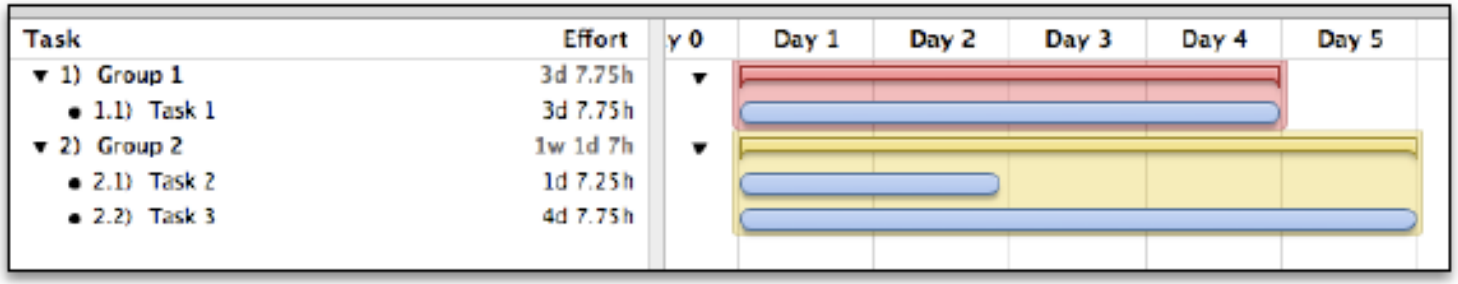


Diagrama Gantt modernă și WBS





Identificarea constrângerilor
/dependențelor

Identificarea dependențelor

- Executarea activităților este constrânsă de logica planului → nu construim acoperișul înainte ca fundația și structura casei să fie gata
- Dependente puternice (hard) și slabe (soft)
- Când folosim tool-uri de planificare:
  - Se specifică doar dependențele puternice
  - Cele slabe sunt de obicei inserate de tool-ul de planificare

Dependențele puternice

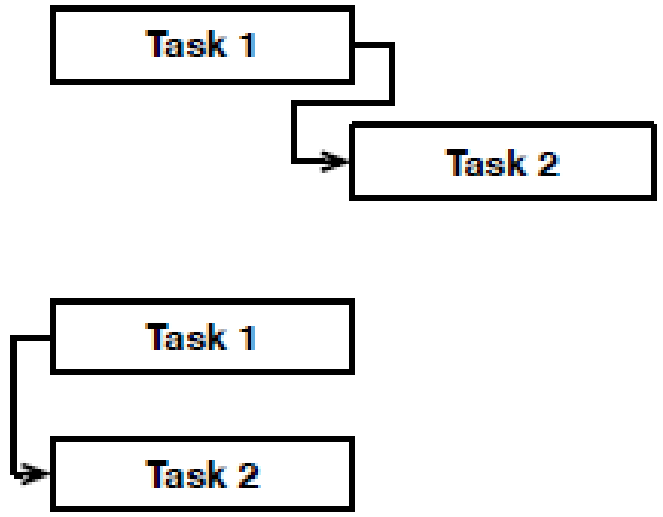
- Nu se poate face mare lucru în ceea ce le privește
- Pot deriva:
  - Fie din logica proiectului (ex. testarea trebuie să vină după scrierea codului)
  - Fie din dependențe externe (ex. semnarea unui contract; o aliniere particulară a planetelor este necesară pentru lansarea unei nave spațiale)
- Eliminarea dependențelor puternice poate fi realizată cu un anumit cost (ex. creșterea riscului, refacerea muncii)

Dependențele slabe

- Apar din cauza unei anumite alegeri făcute dintre toate planurile alternative posibile
- Pot deriva:
  - Fie din alegeri arbitrare (ex. managerul de proiect alege ordinea în care modulele vor fi dezvoltate)
  - Fie din disponibilitatea resurselor și uniformizare (ex. managerul de proiect sau tool-ul de planificare pun în secvență două sarcini ce se bazează pe aceeași resursă)
- De remarcat că odată cu trecerea timpului, poate deveni dificil sau chiar imposibil să anulăm dependențe slabe (ex. o resursă împărțită între proiecte diferite)

Relații de dependență între task-uri

- Finish-to-Start (FS)
  - B nu poate începe înainte ca A să se termine
  - Cea mai des utilizată
- Start-to-Start (SS)
  - B nu poate începe înainte ca A să înceapă
  - Realizarea unui experiment; monitorizarea unui experiment
- Finish-to-Finish (FF)
  - B nu se poate termina înainte ca A să se termine
- Start-to-Finish (SF)
  - B nu se poate termina înainte ca A să înceapa (rar)



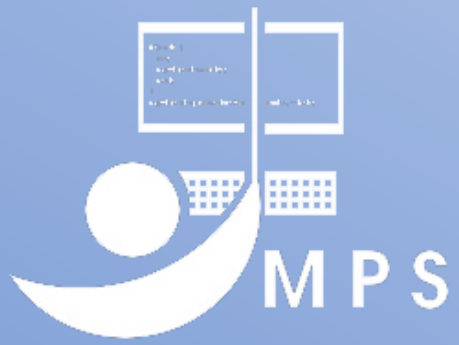
Timpi de lead și lag

- Dependențele dintre activități pot avea durată zero
- **Timp de lag** → întârzierea introdusă de dependența pozitivă (trece ceva timp între cele două sarcini)
- **Timp de lead** → durata dependenței este negativă (activitățile se suprapun parțial)

Reguli de bază

- Folosirea de milestones (și livrabile) pentru a marca în mod clar tranzițiile de fază (sau unele tranziții importante de la o activitate la alta)
- Incercarea de minimizare a dependențelor dintre task-uri (pentru a minimiza întârzierile cauzate de unele activități care le asteaptă pe altele să se încheie)
- Evaluarea alternativelor
- Unele activități ar putea depinde doar de calendar (și să fie constrânse de date)
- Luarea în considerare a tuturor dimensiunilor (cost, calitate, timp): reducerea timpului ar putea crește costurile, riscurile și compromite calitatea

Metoda drumului critic



Drumul critic

- Nu toate activitățile sunt la fel de importante sau critice într-un plan
- Metoda drumului critic alege acele activități care determină durata planului
- Aceste activități determină drumul critic
- Orice mică întârziere arbitrară într-o activitate din drumul critic poate întârzia data de finalizare a proiectului
- Calculul este bazat pe diagrame de rețea (reprezentare pe bază de graf a planului)

Diagrame de rețea

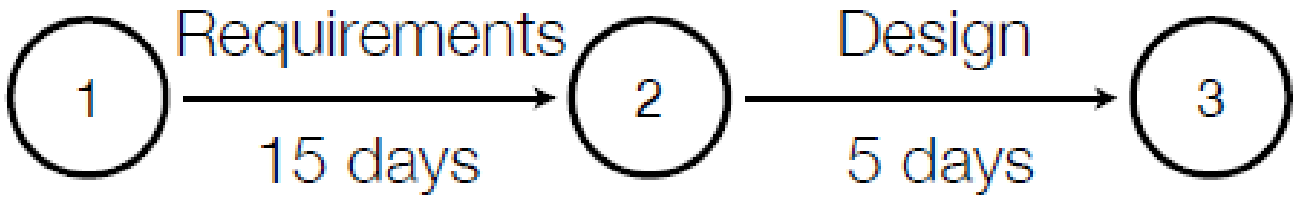
- Aparute în anii ‘50
- O reprezentare grafică a task-urilor este necesară pentru completarea proiectului (planul sub forma de graf)
- Vizualizarea fluxului sarcinilor și a relațiilor
- Două formate clasice:
  - AOA: Activity on Arc (sau Activity on Arrow)
  - AON: Activity on Node
- Convenții:
  - Fiecare task este etichetat cu un identificator și o durată (în unități standard, de ex zile)
  - Există un eveniment de început și unul de sfârșit
  - Timpul se deruleaza de la stânga la dreapta

Diagrame de rețea

- AOA (Activity on Arrow) sau ADM (Activity Diagramming Method):
  - Cercurile reprezintă Evenimente (ex. “start” sau “sfârșit” pentru un anumit task)
  - Liniile reprezinta Task-urile, ca de exemplu “Design”
- AON (Activity on Node) sau PDM (Precedence Diagramming Method):
  - Task-urile sunt în Noduri
  - Arcele reprezintă dependențele dintre task-uri

Formate Grafice

AOA (Activity on Arrow)



AON (Activity on Node)

…care este mai bun?

Comparații AOA/AON

- AOA folosit initial de Walker și Kelly pentru PERT
- AON mai flexibil și mai ușor de reprezentat
- AOA mai simplu de folosit pentru anumiți algoritmi

… ne vom rezuma in general la AON

Exemplu Comparație AOA/AON

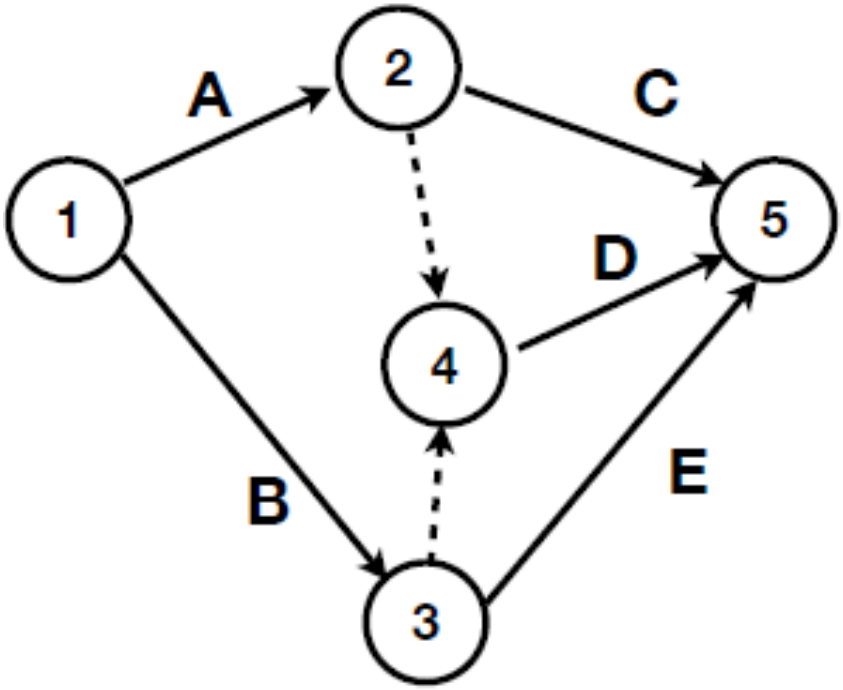
Să considerăm următorul plan:

Activity	Predecessors	Duration
A	None	3 months
B	None	4 months
C	A	3 months
D	A, B	1 month
E	B	2 months

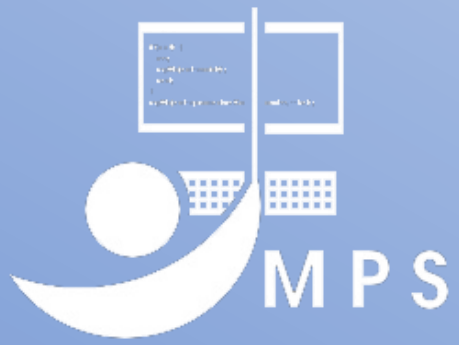
Exemplu Comparație AOA/AON

În notația AOA, unele dependențe pot necesita introducerea de arce sau noduri suplimentare (“dummy”) (*)

(*) De notat că un plan nu are un AOA unic asociat, din moment ce adăugam arce / noduri.

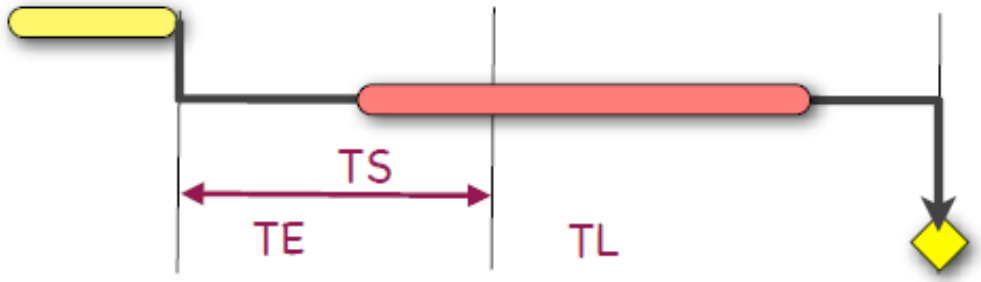


Calculul drumului critic



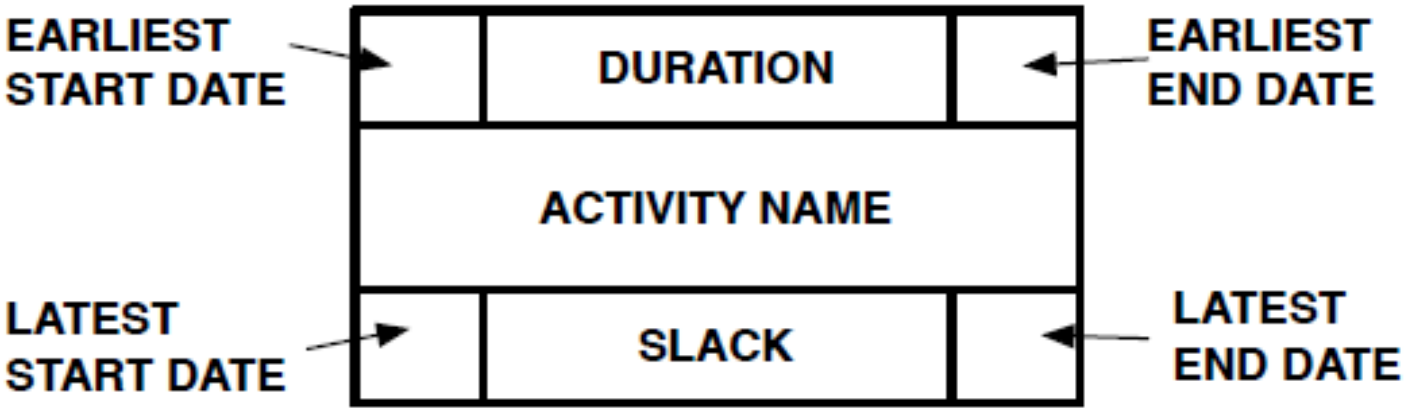
Stagnarea - Slack / Float

- Stagnare liberă
  - Stagnarea pe care o poate avea o activitate până să amâne urmatorul task
- Stagnare totală
  - Stagnarea pe care o poate avea o activitate până să întârzie între proiectul
- Timpul de Stagnare TS = TL – TE
  - TE = momentul de timp cel mai rapid în care poate avea loc un eveniment
  - TL = momentul de timp cel mai târziu în care se poate produce fără a extinde data de completare a proiectului sau a următoarei activități



Calcularea drumului critic

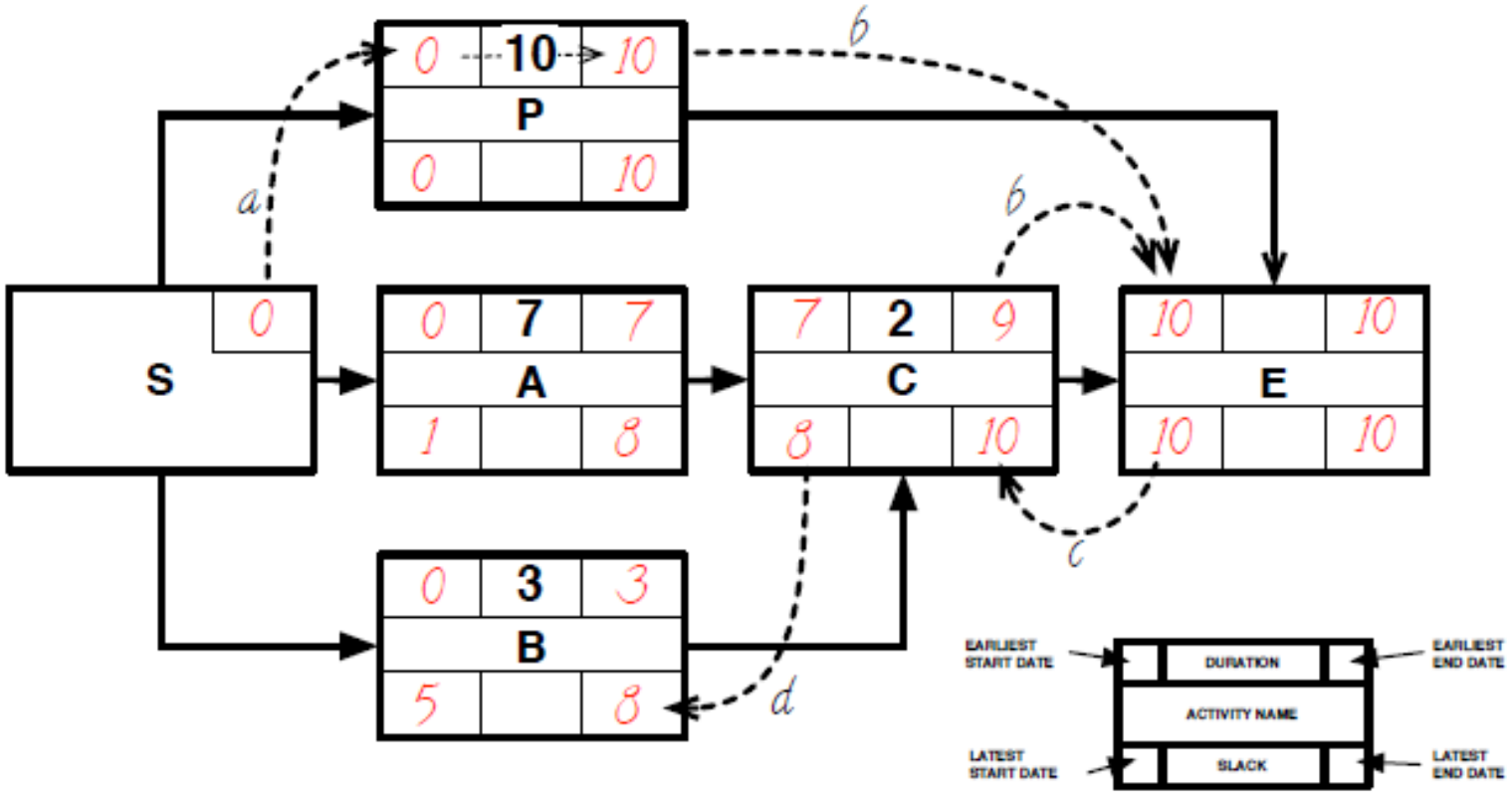
- Scop având un anumit plan (activități, durata, dependențe), să se determine stagnarea, data cea mai rapidă și cea mai târzie a fiecărei activități
- Notație → AON cu noduri reprezentate cum urmează:



Calcularea drumului critic

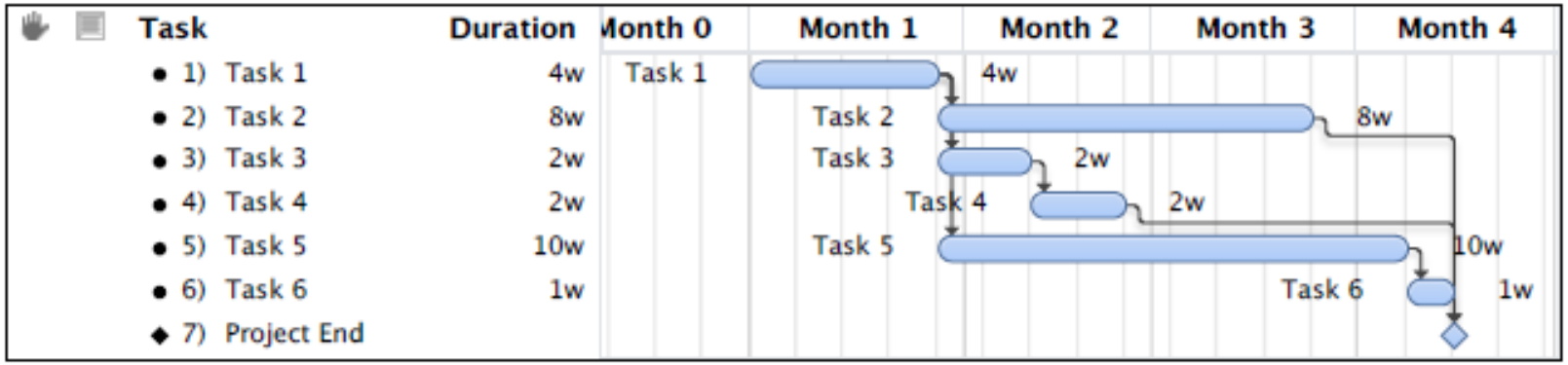
- O **trecere înainte** determină datele cele mai rapide de început și de sfârșit pentru fiecare activitate din plan
- O **trecere înapoi** determină datele cele mai târzii de început și de sfârșit pentru fiecare activitate din plan
- Diferența dintre cel mai rapid început (sfârșit) și cel mai târziu început (sfârșit) reprezintă stagnarea activității
- **Drumul critic** reprezintă drumul în care toate activitățile au stagnare zero
- Un plan are întotdeauna un drum critic → schimbarea planului schimbă activitățile care se găsesc în drumul critic

Exemplul 1


0
10
10
P
0
10
a
b
b
0
S
0
7
7
A
1
8
7
2
9
C
8
10
10
10
10
E
10
10
c
0
3
3
B
5
8
d
EARLIEST START DATE
DURATION
EARLIEST END DATE
ACTIVITY NAME
LATEST START DATE
SLACK
LATEST END DATE

Exemplul 2

- Abordare informală → descoperirea activităților care se pot deplasa într-un plan fără a schimba data de sfârsit a proiectului (ex. Task 3 nu este în drumul critic)
- Metoda drumului critic este subliniata automat de multe tool-uri de creare a diagramelor Gantt



Remarci privind metoda drumului critic

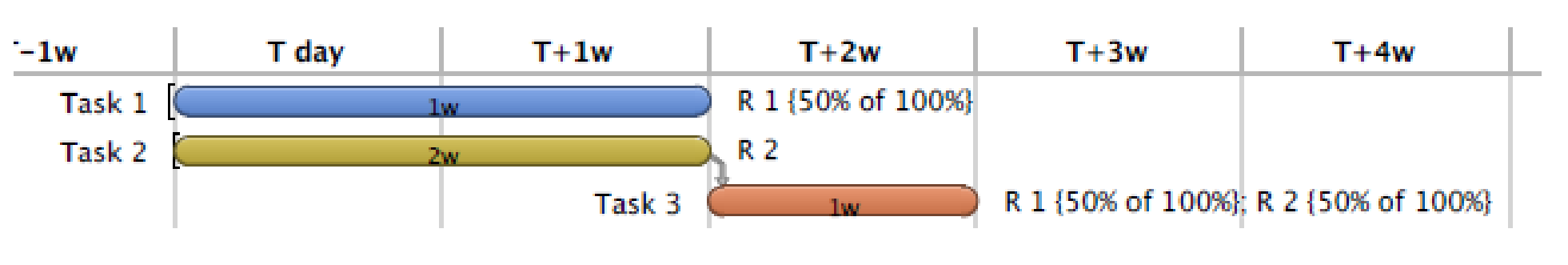
- Drumul critic se referă doar la durata și nu la alte caracteristici precum riscul sau dificultatea
- Activitățile ce nu se află în drumul critic **pot** întârzia planul, dacă întârzierea este suficient de lungă
- Atenție la drumurile (aproape) critice: o întârziere într-o activitate dintr-un drum non-critic poate crea un nou drum critic

Alocarea resurselor și uniformizarea resurselor

Un proces simplificat

- Intrări:
  - planul: activități, constrângeri, efort pentru fiecare activitate
  - echipa de proiect (număr, tipuri și disponibilitatea resurselor)
- Alocarea resurselor:
  - procesul prin care o resursă este alocată unui task, adică este însârcinată cu realizarea unei părți de munca (efort) definite în task
- Constrângeri:
  - conform disponibilității și nevoilor (ex. tipul de resurse necesare pentru o anumită activitate): fără supra-alocari (mai mult față disponibilitatea maximă) (uniformizarea resurselor)
- Daca nu se găseste nici o soluție, se pot varia unele ipoteze (ex. creșterea dimensiunii echipei)

Alocarea resurselor - Exemple



- Legenda:
  - fiecare slot: o săptămână
  - R1 alocat la Task 1 pentru 50% din timpul său
  - **R2 alocat la Task 2 full time**
  - R1 și R2 allocate la Task 3 pentru 50% din timpul lor
- Semnificație:
  - R1 va lucra 20 de ore în săptămânile 1 și 2 și 20 de ore în săptămâna 3
  - R2 va lucra **40 de ore în săptămânile 1 și 2** și 20 de ore in săptămâna 3

Utilizarea resurselor

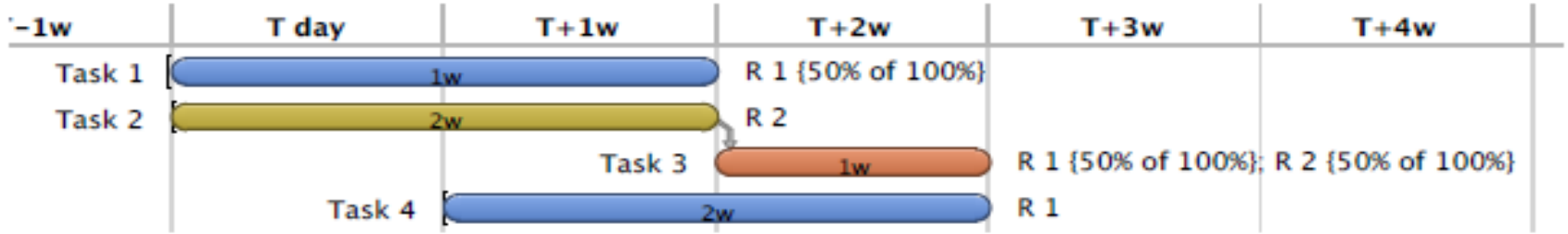
- Puterea de muncă → perioada de timp pentru care este necesară fiecare resursa la un moment dat
- Echipament → numărul de elemente necesare la un moment dat
- Materiale→ cantitatea de materiale necesare (consumate) la un moment dat

Cum se calculează?

- Utilizarea resurselor este calculată adunând cantitatea de muncă necesară pentru o perioadă dată
- Aceasta reprezintă o suma verticală cuprinzând toate sarcinile de muncă

Supra-alocare → o situație în care o resursă este folosită peste capacitatea sa maximă

Exemplu

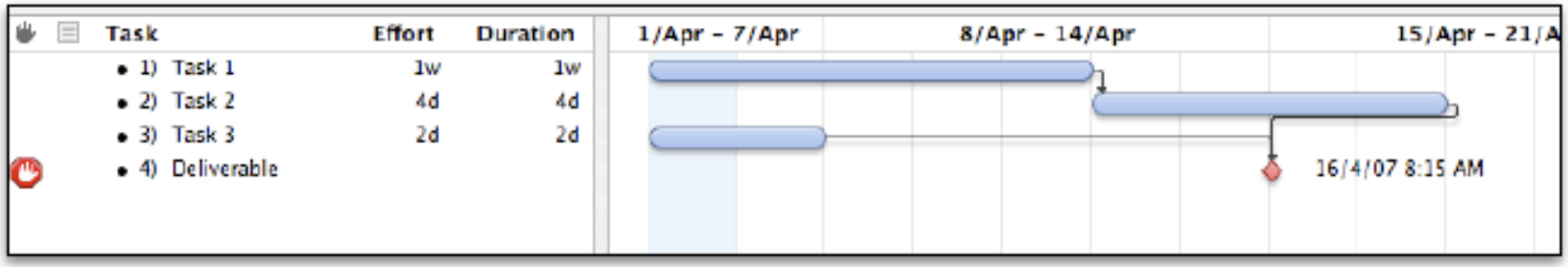


	hours	hours	hours	hours	hours
R1	20	20		T1	
			20	T3	
		40	40	T4	
Total R1	**20**	60	60		
R2	40	40		T2	
			20	T3	
Total R2	**40**	**40**	**20**		

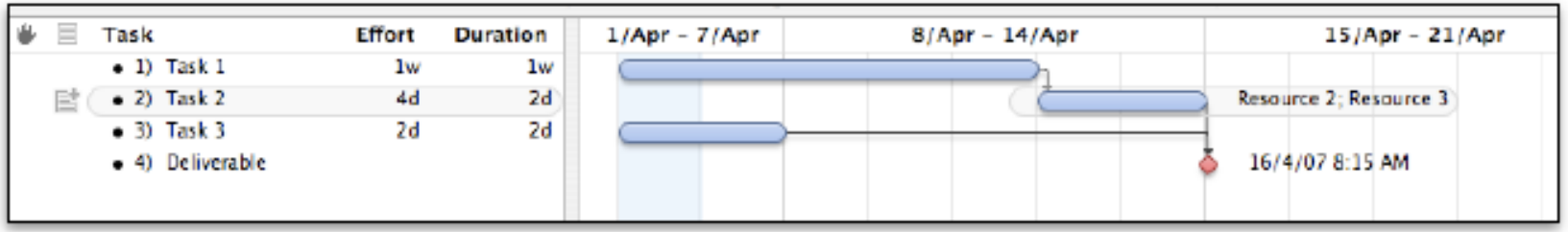
R1 este supra-alocată în W2 (T+1w) și W3 (T+2w)

Exemplu

- Reprezentăm planul subliniind constrângerile puternice. Livrabilele au datele de livrare fixe

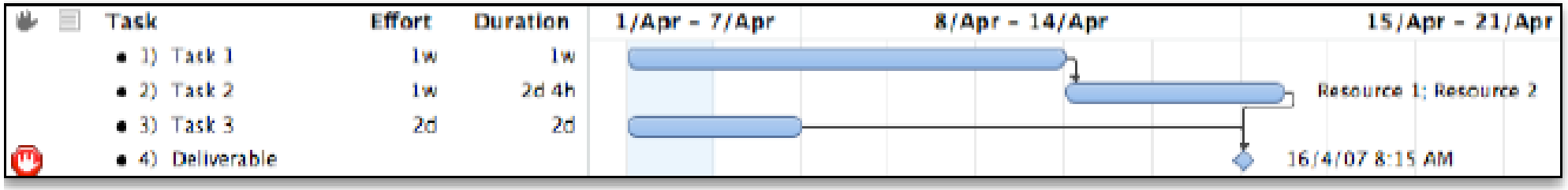


- Alocarea a două resurse pentru Task 2 permite satisfacerea constrângerilor

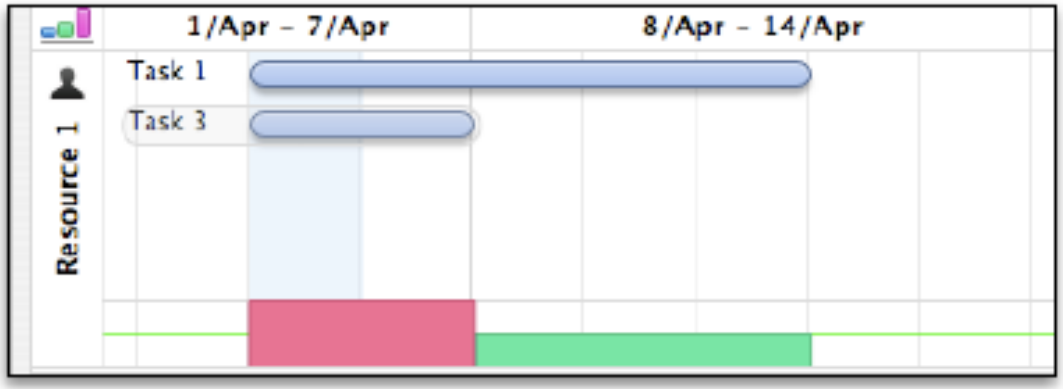


Exemplu

Problemă → Task 1 și Task 3 au nevoie de aceeași resursă



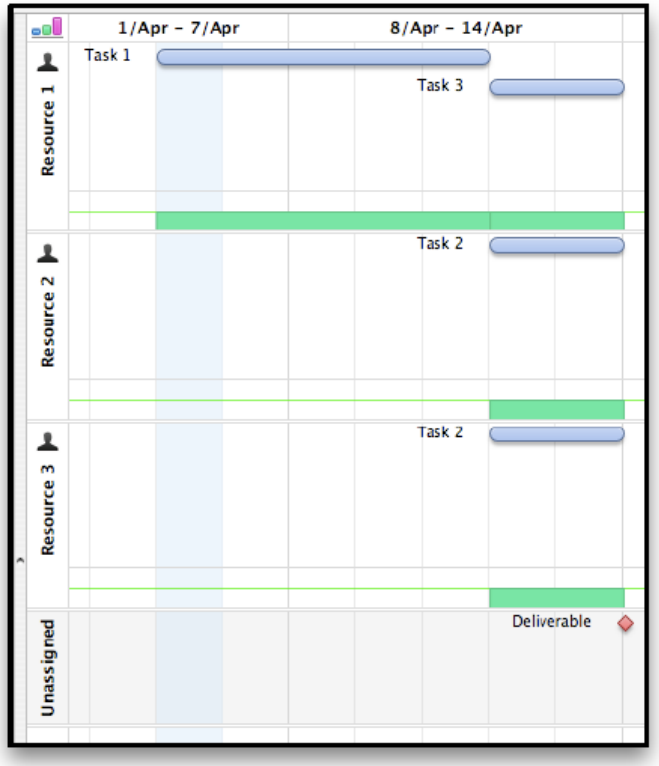
...supra-alocam Resursa 1



Exemplu

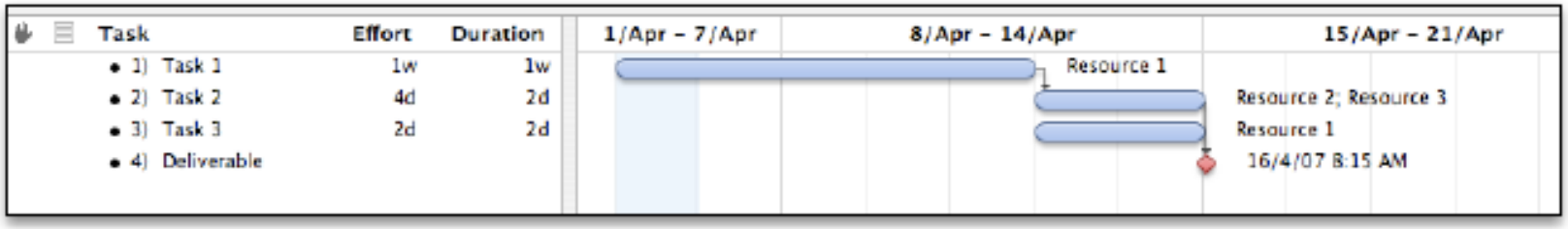
Soluția 1 → Uniformizarea resurselor, inserarea de constrângeri slabe în plan astfel încât nici o resursă nu este supra-alocata (nu muncește mai mult de 100%)

Soluția 2 → Tehnici de compresie (revenim pe acest subiect)



Câteva considerații:

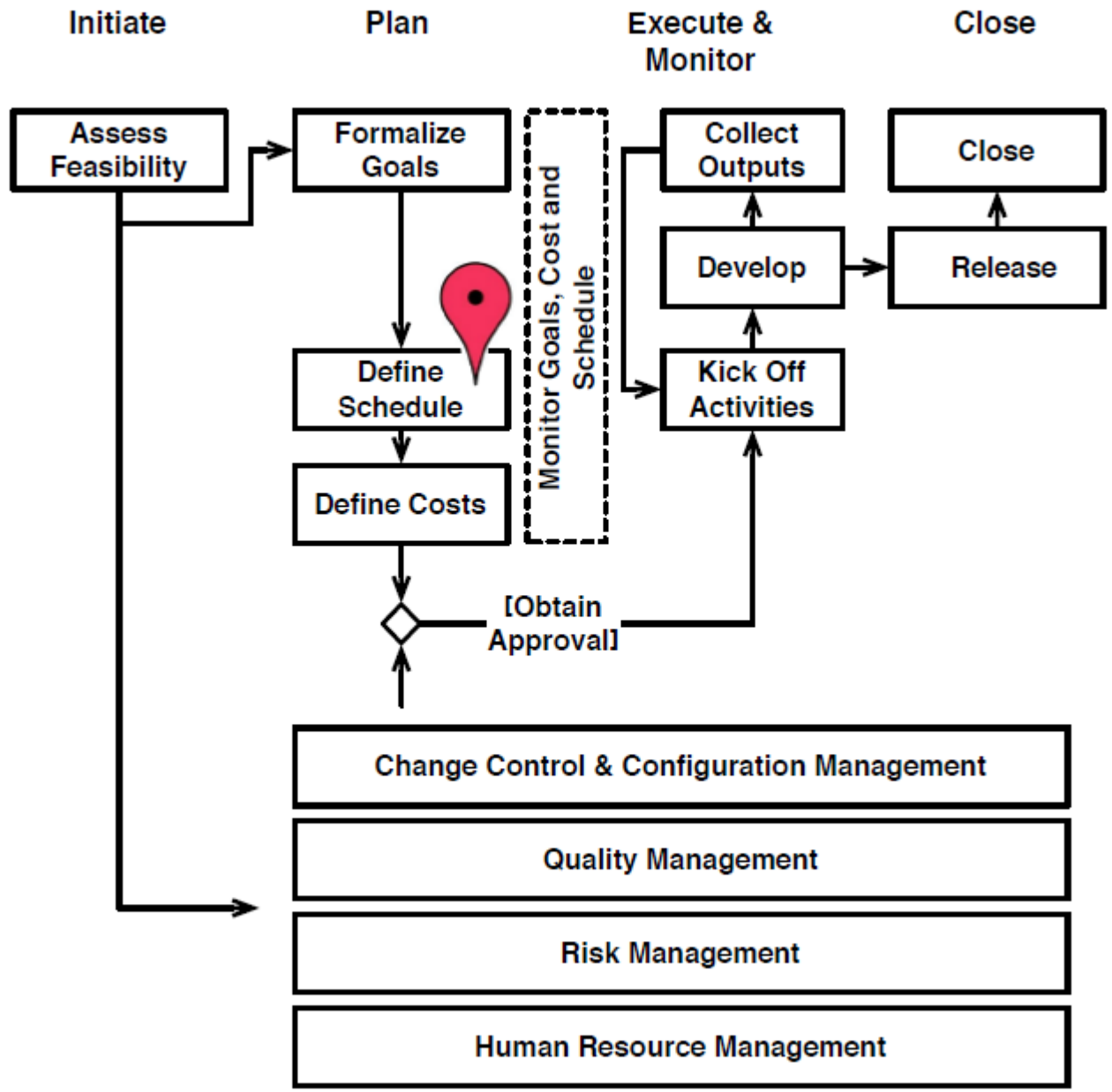
- Resursa 1 va lucra în proiect full time
- Resursa 2 și Resursa 3 sunt necesare doar spre sfârșitul proiectului (pentru Task 2)



Optimizarea planului

Obiective

- Învățarea tehnicilor pentru scurtarea unui plan
- Înțelegerea riscurilor apărute după scurtarea unui plan


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Considerație preliminară

- Prima versiune a planului va arăta cel mai probabil că nu vom fi capabili să livrăm conform deadline-ului stabilit de către sponsor
- În ipoteza ca planul este precis, există două opțiuni:
  - Proiectul nu se începe (în cazul în care nu este fezabil, ținând cont de constrângeri)
  - Proiectul este scurtat, folosind una dintre tehnicile descrise în acest curs
- A treia opțiune (revizuirea estimărilor) nu reprezintă o idee bună.

Realizarea unui plan fezabil

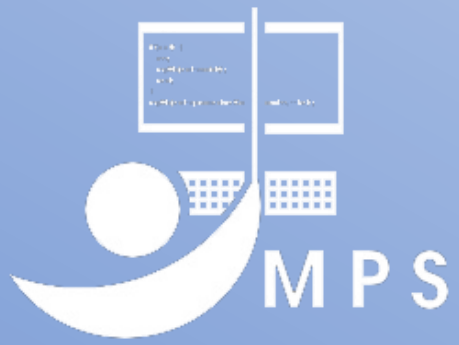
Schimbarea unor ipoteze pe care se bazează planul. De exemplu:

1. Reducerea scopului (transformă unele activități în mai scurte sau inutile)
2. Reducerea calității (transformă unele activități în mai scurte sau inutile)
3. Externalizarea unor activități (crește riscul și, posibil, costurile)

Acționarea asupra logicii planului:

4. Creșterea resurselor (poate face proiectul mai scump) [PROJECT CRASHING]
5. Evaluarea cursului alternativ al acțiunii (ex. schimbarea procesului de dezvoltare). Totuși: poate fi dificil în cazul în care echipa este obișnuită cu un anumit proces de dezvoltare
6. Substituirea activităților
7. Încălcarea regulilor: eliminarea constrângerilor puternice din plan (crește riscurile și, posibil, costurile) [FAST TRACKING - URMARIRE RAPIDA]
8. Lucrul asupra probabilității livrarii la timp **[**CRITICAL CHAIN MANAGEMENT - GESTIONAREA CRITICA A LANTULUI]

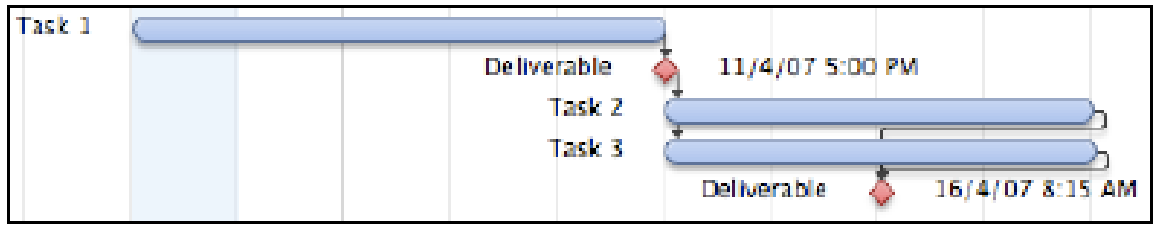
Metoda Project Crashing



Project Crashing

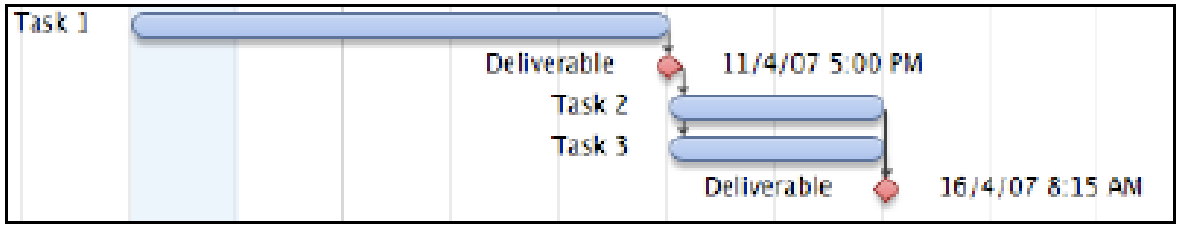
- Durata proiectului poate fi (adesea) redusă prin alocarea de mai multă muncă pentru activitățile proiectului, sub forma de lucru peste program, și prin alocarea de mai multe resurse, precum materiale, echipamente etc.
- Totuși, munca și resursele adiționale cresc costurile proiectului
- Astfel, decizia de reducere a duratei proiectului trebuie să fie bazată pe o analiză și pe compromisul dintre timp și cost
- Project Crashing reprezintă o metoda de scurtare a duratei proiectului prin reducerea timpului uneia sau a mai multor activități critice de proiect, obținând astfel un timp mai scurt decât timpul normal de activitate. Scopul este de a reduce durata proiectului și minimizarea costului crashing-ului

Project Crashing - Exemplu



cost = $ 3000
cost = $ 4000

- Project Crashing: alocăm mai multi oameni pentru Task-urile 2 și 3

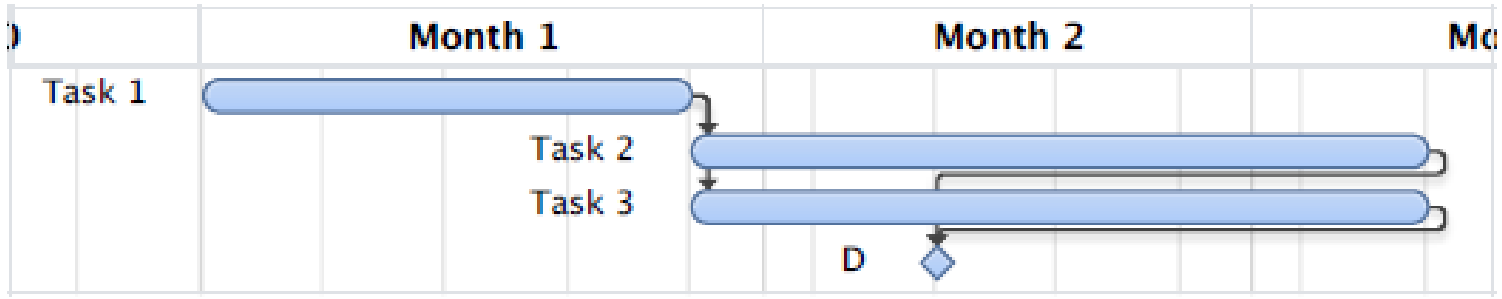


cost = $ 6000
cost = $ 8000

- Timp de Crash = 2 unități de calendar (ex. luni)
- Costul de Crash = (6000 + 8000) - (3000 + 4000) = 14000 - 7000 = 7000
- Crashing convenabil conform costurilor livrării întârziate

Project Crashing simplu - Exemplu

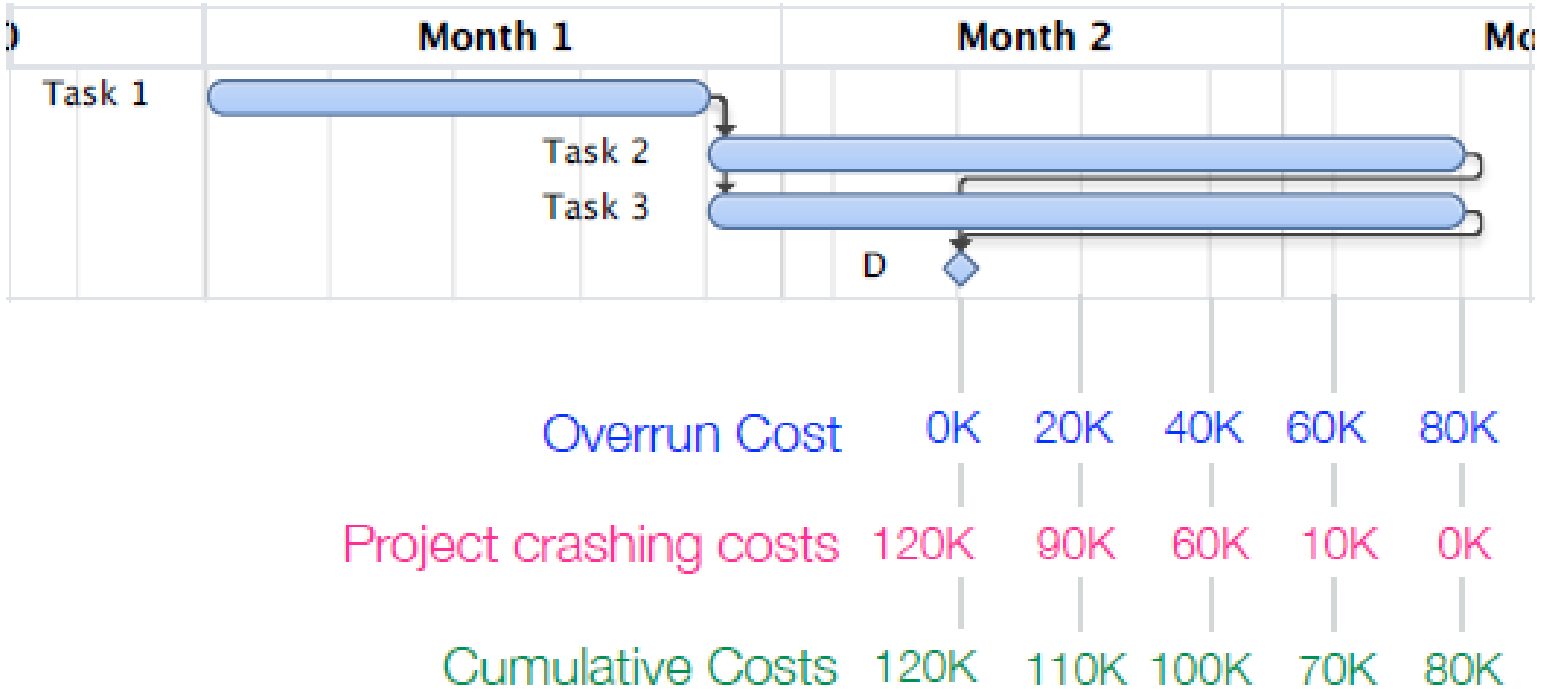
Proiectul este întârziat: se prespune că trebuie să plătim 20 de mii de euro pentru fiecare săptămână de întârziere



Putem reduce Task 2 și Task 3 prin alocarea de mai multe resurse: 10 mii pentru o săptamână, 60 de mii pentru două săptămâni, 90 de mii pentru trei săptămâni, 120 de mii pentru patru săptămâni.

Project Crashing simplu - Exemplu

Proiectul este întârziat: se presupune că trebuie să plătim 10 mii de euro pentru fiecare săptămână de întârziere



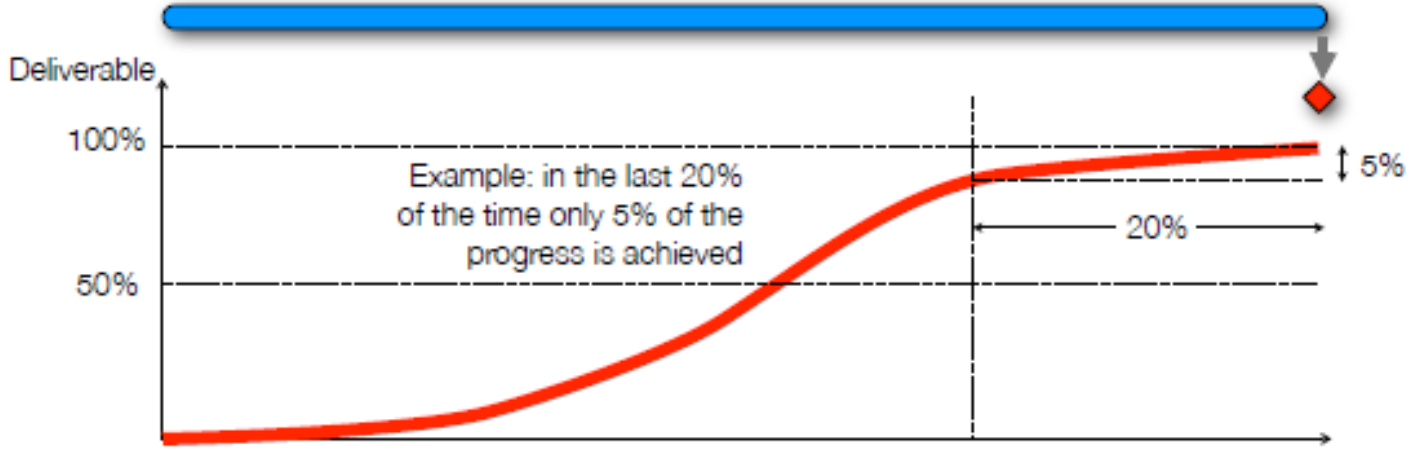
Project Crashing simplu - Exemplu

- Cea mai bună soluție (doar în cazul în care costurile contează) → scurtarea cu o săptămână (costurile cumulative au cea mai mică valoare)
- Alte considerente pot favoriza alte alegeri. De exemplu:
  - Livrarea la timp ar putea fi esențială pentru a păstra un client. Într-un asemenea caz, putem decide să suportăm cheltuieli mai mari
  - Putem alege să livrăm târziu (și să păstrăm activitățile așa cum sunt) pentru a evita lucrul peste program al echipei sau pentru ca riscurile asociate diminuării calității pot crește odată cu scurtarea activităților

Metoda Fast Tracking

Fast Tracking

- Metoda Fast Tracking este bazată pe faptul că livrabilele de activități sunt produse (și rafinate) incremental în timpul executării activității
- Exemplu: un document cu cerințe nu este scris în ultima zi a activității “redactarea cerințelor”. El este mai degrabă scris puțin câte puțin, în timp ce activitatea este executată

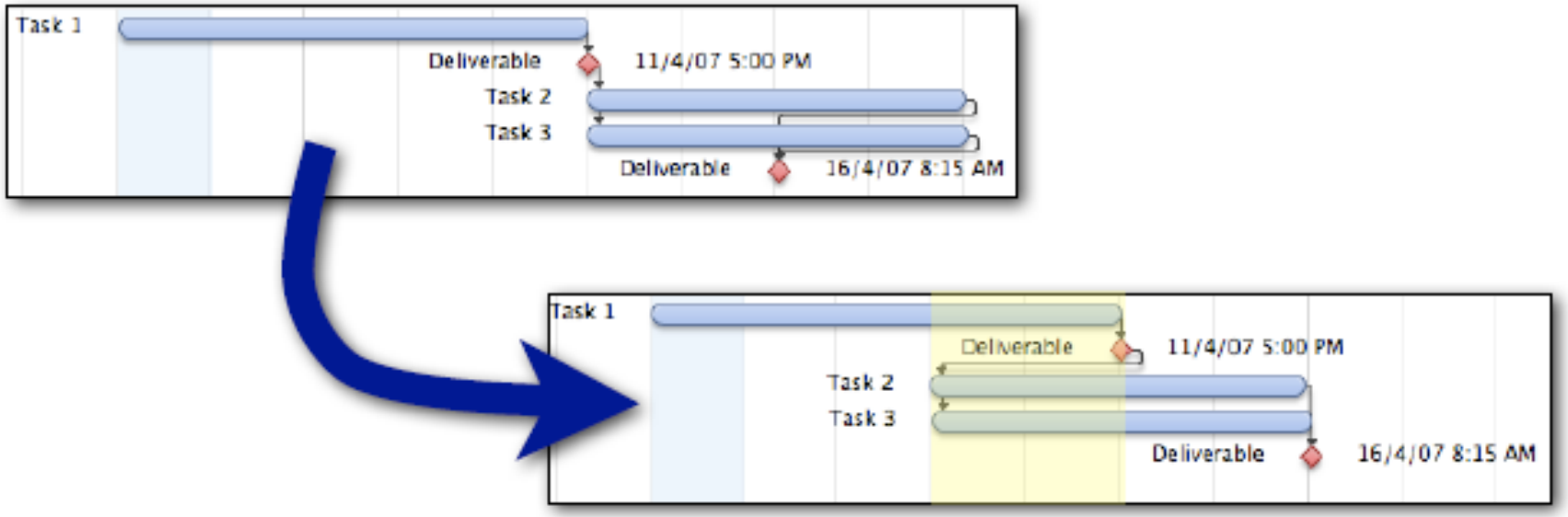


Fast Tracking

- Să considerăm exemplul din slide-ul precedent: dacă ultimele 5% nu reprezintă o parte critică a livrabilului, atunci putem începe mai devreme activitățile ce depind de documentul de cerințe.

Fast Tracking

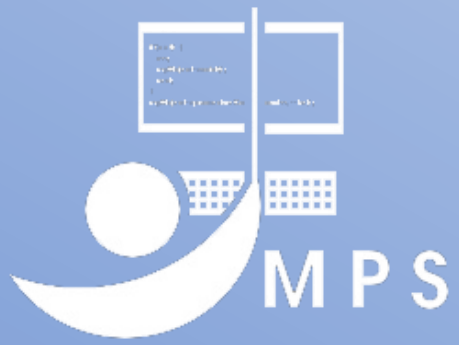
- Metoda Fast Tracking funcționează prin suprapunerea de activități ce în mod normal ar fi secvențiale



Fast Tracking - Probleme și reguli de bază

- Metoda Fast Tracking este riscantă și poate produce refaceri ale activităților
- Când decidem ce dependențe ar trebui sparte, trebuie să luăm în calcul:
  - Cum va progresa producerea de livrabile în timpul activității (va produce ieșiri intermediare?) și cum se va consolida (va fi stabilă ieșirea intermediară?)
  - Riscul implicat în schimbările aduse rezultatului (care va fi consecința refacerii activității? Cum va afecta restul planului?)

Metoda Critical Chain Management (CCM)



Critical Chain Management

- Metoda Critical Chain Management pornește de la ideea că estimările sunt simple presupuneri
- A spune că o activitate durează X (sau necesită efortul Y) înseamnă că **de cele mai multe ori**, activitatea va avea nevoie de X pentru a fi completată. **Vor fi însă și cazuri** în care activitatea poate dura mai mult sau mai puțin.
- Pentru a gestiona evenimentele neprevăzute (și a contrasta cu optimismul), managerii de proiect “umfla” estimările, aducându-le în zona pesimistă.

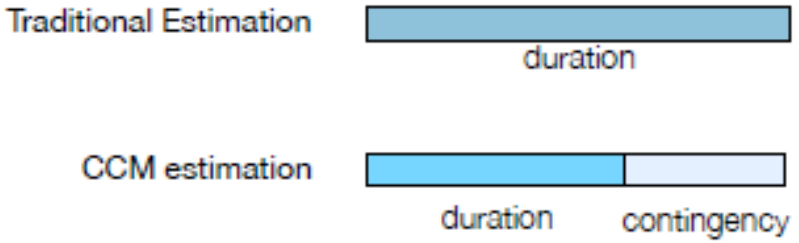
Considerente Statistice

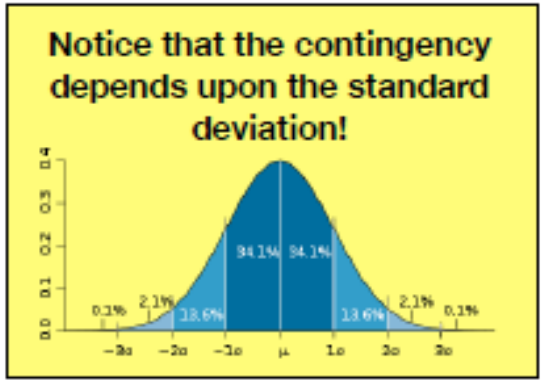
- Eficiența provine din:
  - Item 1 → **Discutăm cele mai probabile estimări, nu pe cele pesimiste:** în particular, numim eveniment neprevăzut diferența de durată dintre o estimare 50% probabilă și una 90% probabilă
  - Item 2 → Discutăm despre **lanțuri de activități. Deviația standard** a unui lanț de activități este mai mică față de suma deviațiilor standard ale activităților din lanț

$$\sigma_{a+b} = \sqrt{\sigma_a^2 + \sigma_b^2}$$

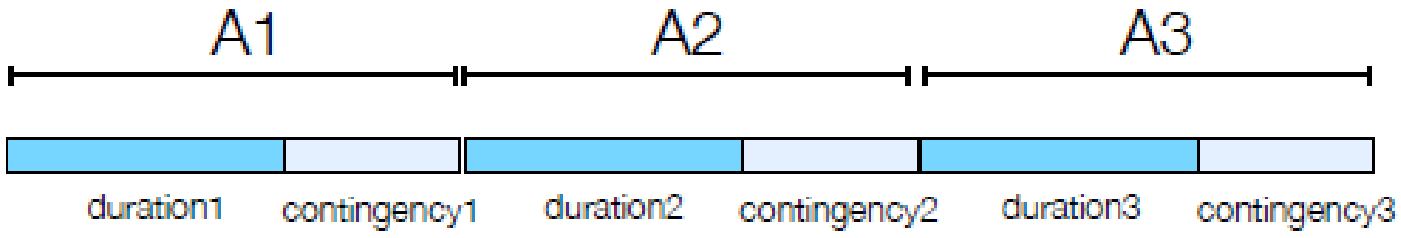
$$\sigma_{a+b} < \sigma_a + \sigma_b$$

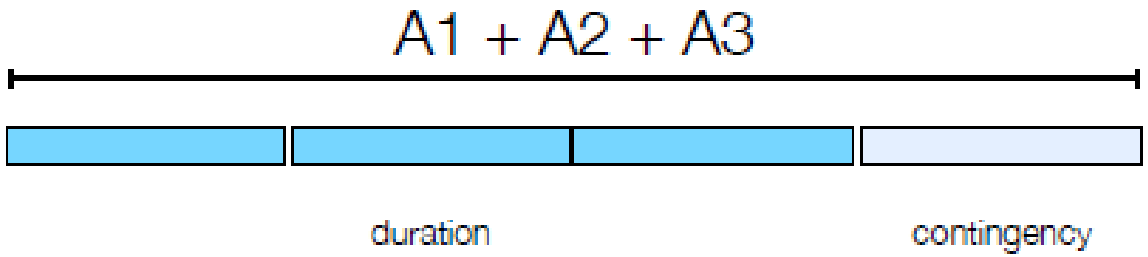
CCM - Item1 estimări





CCM - Item2 suma variantelor





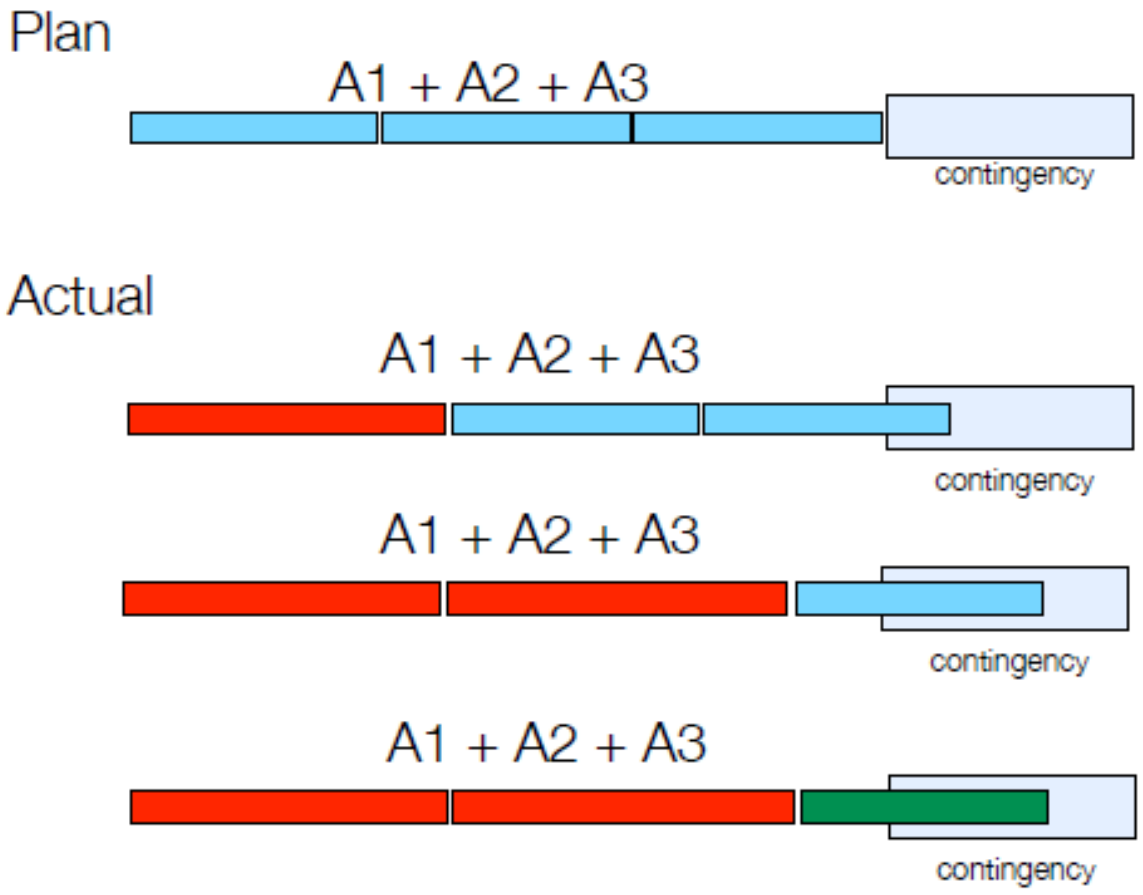
duration = duration1 + duration2 + duration3

contingency < contingency1 + contingency2 + contingency3

CCM - Ideea de bază

- Atunci când planificăm, noi discutăm și monitorizăm lanțurile și facem un buffer explicit de evenimente neprevăzute
- Item1 (durată + neprevăzute) + Item2 (suma variantelor) asigură faptul că planul este mai scurt față de cel obținut în abordarea standard
- În timpul executării planului, variația statistică va face ca unele activități să dureze mai mult decât era planificat. Aceste activități vor face ca lanțul să se reverse peste buffer-ul de evenimente neprevăzute, ceea ce este în regulă până într-un punct
- Managerul gestionează lanțurile, nu activitățile

Exemplu



- A1 durează mai mult față de cât era planificat și lanțul intră în buffer.
- A2 durează mai mult decât era planificat și lantul continuă să intre în buffer.
- Totuși A3 durează cât era planificat și o parte din buffer este economisită.

CCM - Gestionarea lanțurilor

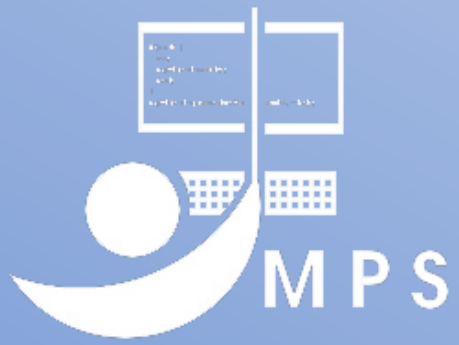
- Atunci când folosim CCM, două noțiuni importante apar:
  - **Definirea buffer-ului** → care lanțuri sunt cele mai bune de considerat sau, altfel spus, unde plasăm bufferele
  - **Gestionarea buffer-ului** → când trebuie să începem să ne îngrijorăm din cauza revărsării buffer-ului de evenimente neprevăzute

Gestionarea buffer-ului

Task Execution Status	Buffer Penetration: First Third	Buffer Penetration: Second Third	Buffer Penetration: Final Third
First Third	NO ACTION	Serious problem	**Very serious problem**
Second Third	NO ACTION	Could be a problem: identify and formulate solution	Serious problem
Final Third	**Very good! We are early**	NO ACTION	Monitor the situation

Fixarea costurilor și bugetarea

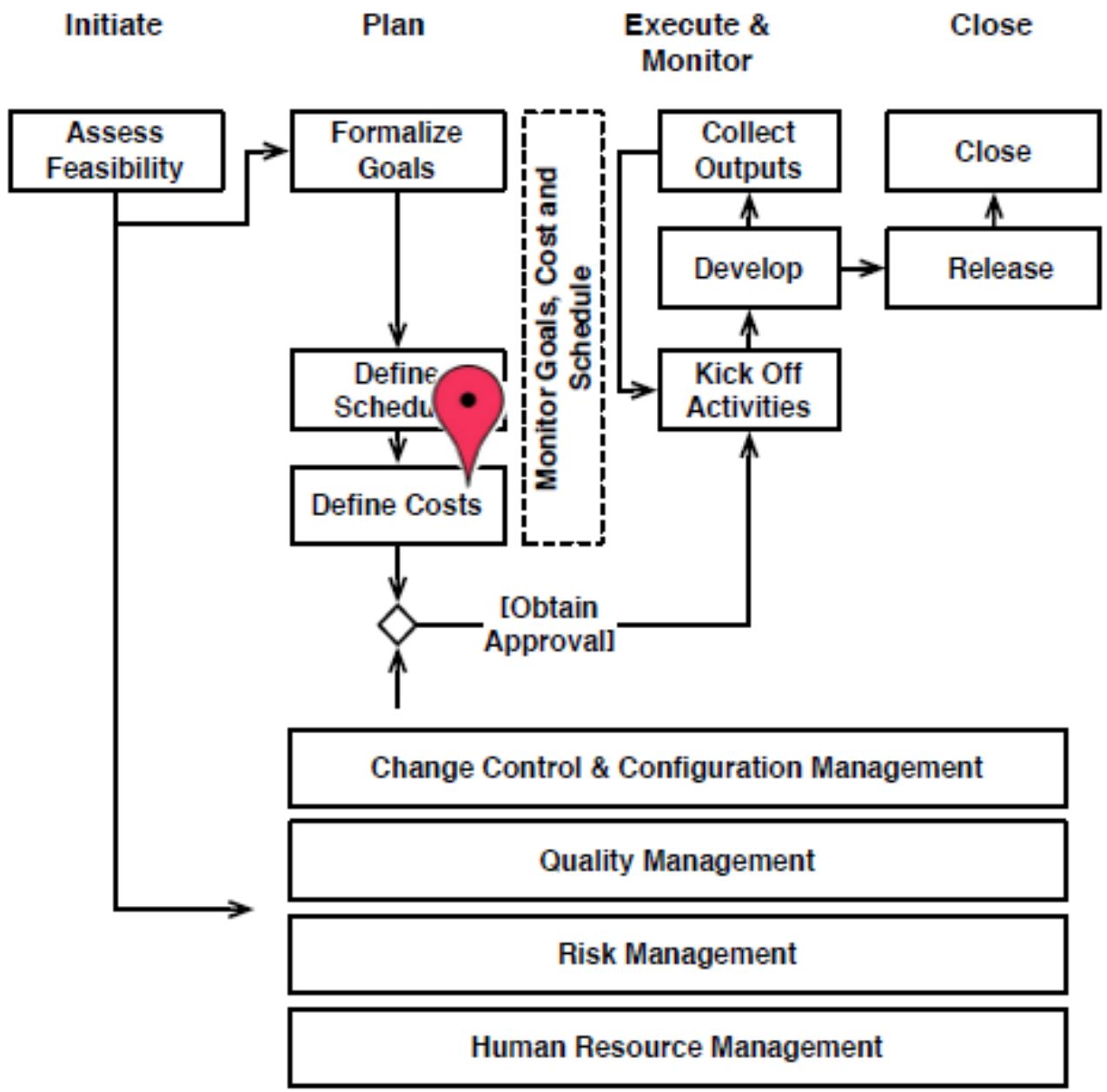
De la cost la valoare: metode și tehnici de stabilire a costului potrivit pentru software



ST COST COST CO
OST COST COST COS
ST COST COST COS
COST COST COS
WE'RE FANTASTICALLY EXCITED ABOUT THIS NEW PROJECT...WE THINK IT OPENS UP HUGE POTENTIAL MARKETS...
SHARE YOUR ENTHUSIASM FOR THE FUTURE
search ID: forn1496
© Original Artist
Reproduction rights obtainable from
www.CartoonStock.com

Obiective

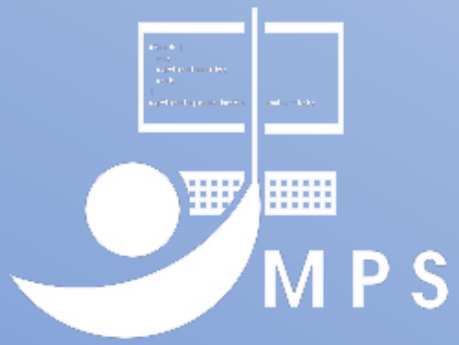
- Intrebări ce pot apărea:
  - Cât de mult costă dezvoltarea software-ului?
  - Respectă proiectul bugetul?
- Obiectivele cursului:
  - Bugetarea
  - Gestionarea costurilor proiectului


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Define Schedu
Define Costs
Monitor Goals, Cost and Schedule
[Obtain Approval]
Collect Outputs
Develop
Kick Off Activities
Close
Release
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Câteva definiții

- **Fixarea costurilor**: determinarea costurilor simple pentru livrarea proiectului
- **Bugetarea (și controlul costurilor):** determinarea nevoilor financiare ale unui proiect și pregătirea celor necesare pentru monitorizarea cheltuielilor (și a câștigurilor)
- **Fixarea prețurilor:** determinarea sumei de bani cerute pentru proiect
- **Costul Ciclului de Viata (Life Cycle Cost – LCC, uneori numit și Total Cost of Ownership):** costurile ce trebuie susținute pentru a opera (utiliza) sistemul în întregul său ciclu de viață

Fixarea costurilor proiectului software



Fixarea costurilor proiectului (software)

- **Costurile proiectului**
  - Cheltuielile pe care le vom suporta pentru a termina proiectul
  - Nu iau în calcul profitul
  - Constituite din costurile directe și indirecte
- **Structura elementului de cost**
  - Structura ierarhică ce definește obiectele de cost ce vor determina bugetul proiectului
  - Ajută structurarea fixarii costurilor și monitorizarea proceselor, reducând de asemenea riscul de a contabiliza de două ori aceleași cheltuieli

Costuri directe

- Costurile directe → costuri legate de realizarea proiectului
- Costuri directe pentru proiectele software:
  - **Personal:**
    - salariile persoanelor direct implicate în proiect (brute, nu nete)
  - **Materiale și stocuri:**
    - costurile materialelor necesare pentru a produce rezultatele proiectului
    - de obicei luate în calcul dacă proiectul are nevoi speciale
  - **Hardware și software:**
    - Sistemele necesare pentru a dezvolta produsul
    - de obicei luate în calcul dacă proiectul are nevoi speciale
  - **Transport, ședințe, evenimente**
  - **Alte costuri:**
    - cărți, training, închiriere echipament etc.

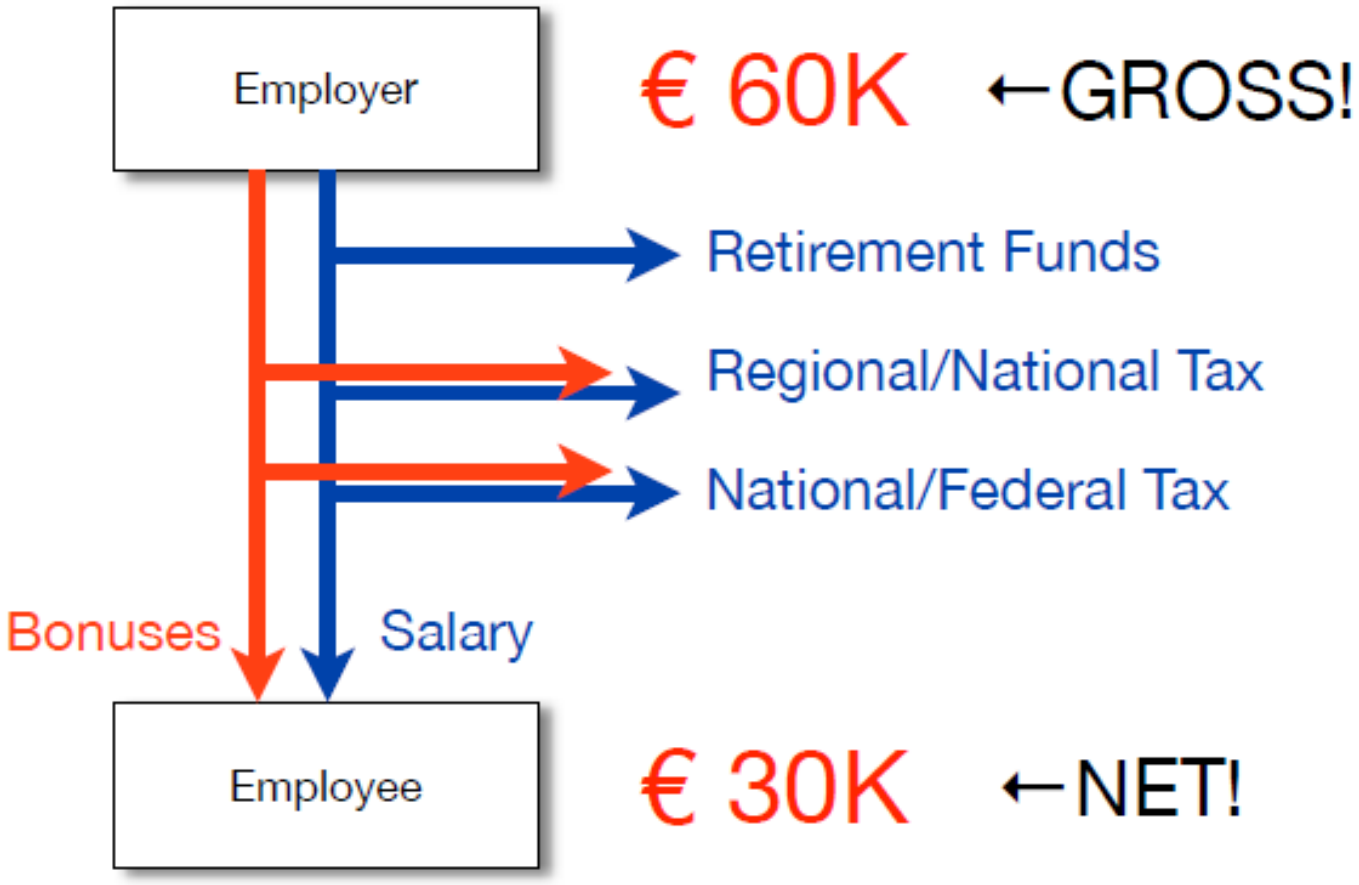
Fixarea costurilor proiectului (software)

- **Costurile indirecte** → cheltuieli necesare pentru a asigura funcționarea întreprinderii și a face munca posibilă
- Principalele elemente de cost în dezvoltarea de software:
  - Generale:
    - ❖ Costurile spațiului de lucru (închiriere, încălzire)
    - ❖ Consumabile
    - ❖ Echipament standard
    - ❖ Personal administrativ
  - De proiect:
    - ❖ Pentru proiectele mai mari, există elemente direct contabilizate în proiect

Calcularea costurilor indirecte

- Identificarea cheltuielilor ce contribuie la costurile indirecte
- Identificarea unei strategii pentru alocarea costurilor indirecte ale unui proiect
  - În funcție de efort (mai mult efort = mai multe costuri indirecte) – rata uniformă sau proporțional
  - În funcție de bugetul proiectului (drept procentaj din bugetul proiectului)
- În mod regulat
  - Evaluarea cheltuielilor apărute în anii anteriori
  - Estimarea cheltuielilor indirecte pentru anii ce urmează
  - Estimarea efortului pe care îl putem produce
  - Determinarea ratei forfetare

Costurile de personal - brut vs net



Costul proiectului reprezintă

$$\sum_{j=1}^{n} Hours_j * (PC_j + O_j) + \sum_{i=1}^{n} C_i$$

H	Numărul total de ore pentru profilul j
PC	Costul profilului j
O	Costurile generale pentru profilul j (Overhead)
C	Costul elementului I (ex. hardware)

Costul proiectului

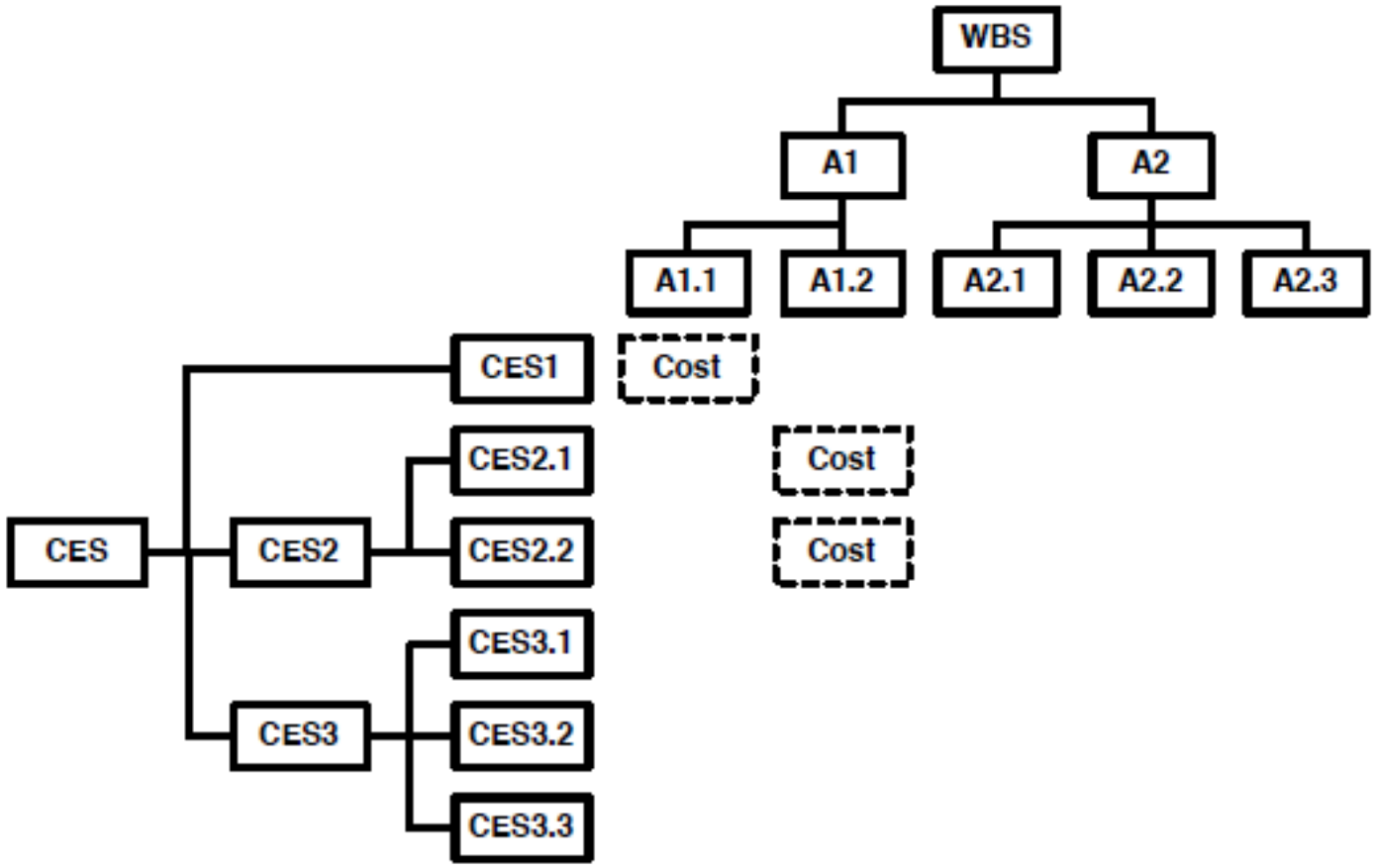
- Costurile proiectului pot fi privite din mai multe puncte de vedere:
  - Costul per element (de cât avem nevoie pentru hardware?)
  - Structura proiectului (de cât avem nevoie pentru redactarea cerintelor?)
  - Cheltuieli de-a lungul timpului (de cât avem nevoie în 2018?)

Gestionarea costurilor proiectului

Obiective și metode

- Obiective
  - Asigurarea disponibilității banilor atunci când trebuie cheltuiți
  - Monitorizarea cheltuielilor proiectului astfel încât el să rămână în buget sau pentru a permite luarea de măsuri corespunzătoare atunci când este nevoie
- Metode
  - Definirea unui flux numerar de bază
  - Autorizarea cheltuielilor
  - Păstrarea evidenței cheltuielilor

Costurile și structura proiectului



Costurile și timpul proiectului

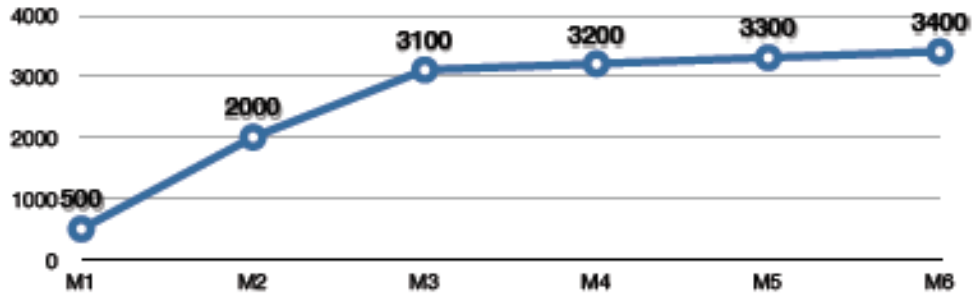
	Q1	Q2	Q3	Q4	Total
Expenses					
Expense 1	€ 10,000	€ 30,000	€ 50,000	€ 10,000	**€ 100,000**
Expense 2	€ 20,000	€ 40,000	€ 60,000		**€ 120,000**
Total Expenses	**€ 30,000**	**€ 70,000**	**€ 110,000**	**€ 10,000**	**€ 220,000**
Incomes					
Payment	€ 50,000			€ 200,000	**€ 250,000**
Total Incomes	**€ 50,000**	**€ 0**	**€ 0**	**€ 200,000**	**€ 250,000**
Balance	€ 20,000	-€ 70,000	-€ 110,000	€ 190,000	**€ 30,000**
Financial Need		-€ 50,000	-€ 180,000		

Costurile și timpul proiectului

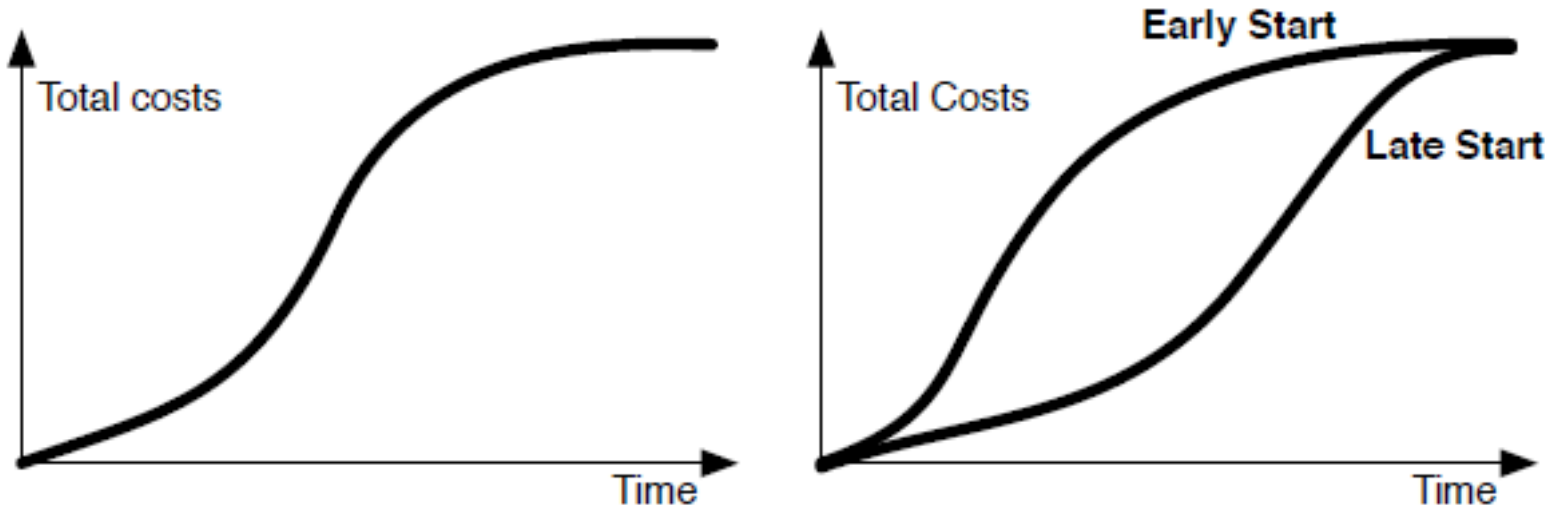
Name	Planned Cost	Time
A	1000	A
B	2000	B
C	400	C

A	1000	500	500			
B	2000		1000	1000		
C	400			100	100	100	100

Total	500	1500	1100	100	100	100
Cumulative	500	2000	3100	3200	3300	3400



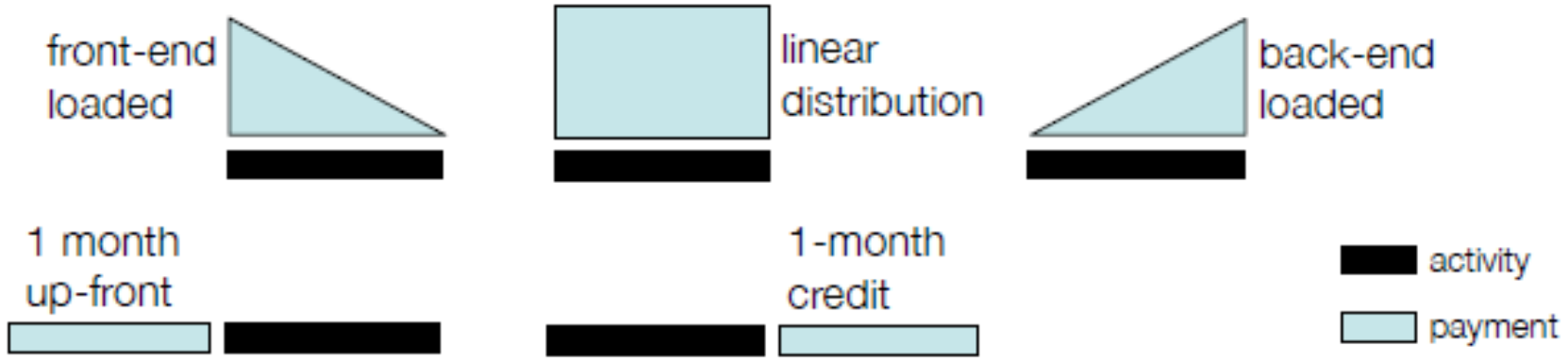
Costurile și timpul proiectului



Banana Shape

Profilul cheltuielilor

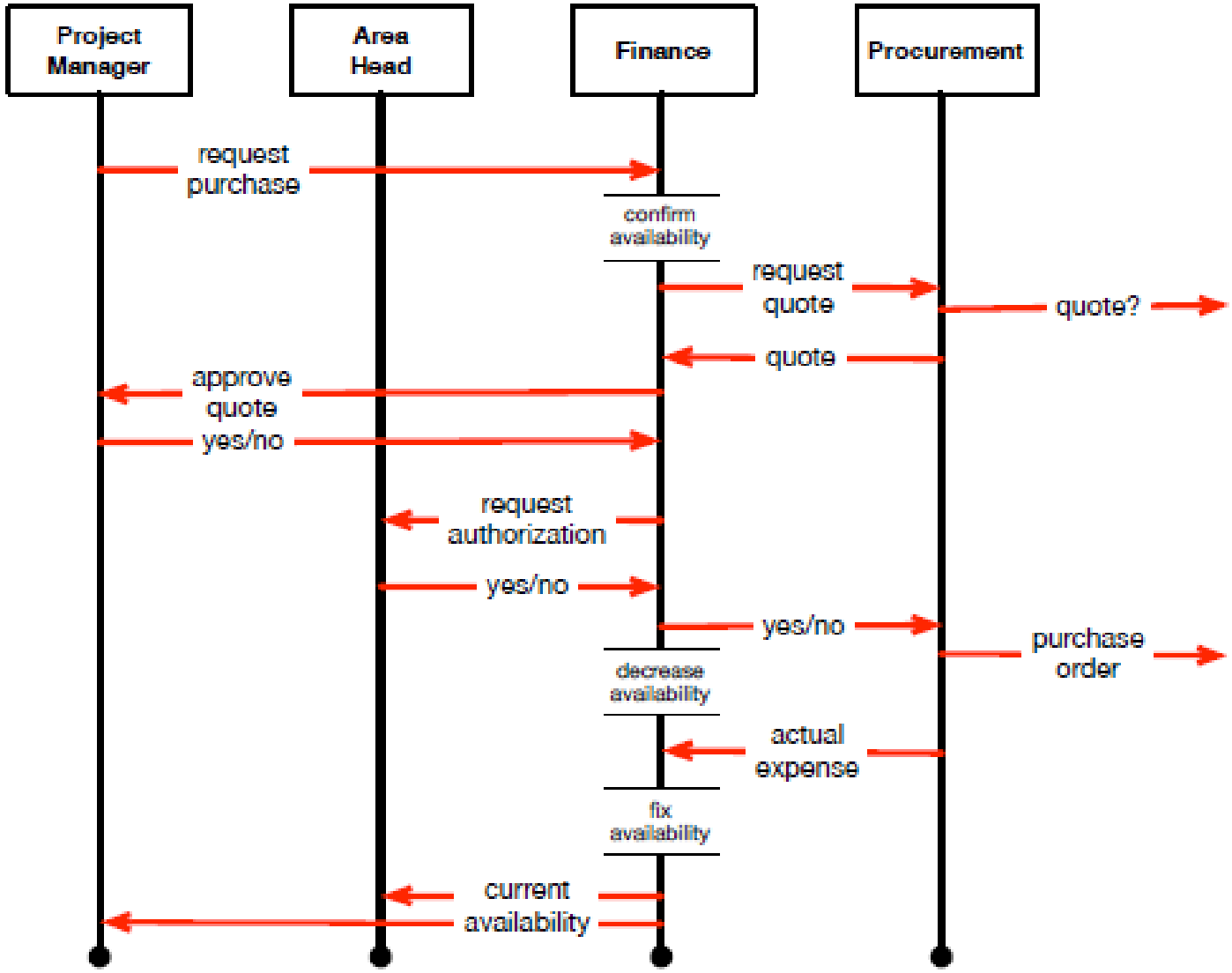
- În general, presupunem că volumul de muncă este distribuit uniform în timpul unei activități:
  - O resursă implicată într-o activitate ce necesită 40 de ore de efort și o durată de o săptâmană va lucra 8 ore pe zi
- Totuși, nu este întotdeauna acesta cazul, întrucât profile diferite pot fi definite pentru efort și cheltuieli



Autorizarea cheltuielilor

- Managementul proiectului și managementul financiar sunt de obicei atribuite unor structuri diferite
- Conform structurii organizaționale, puterea de a autoriza cheltuielile și plățile poate să țină doar de managerul de proiect sau să necesite un flux de lucru mai complex
- Regulile iau în calcul aspecte precum:
  - disponibilitatea fondurilor
  - daca acea cheltuială e bugetată sau nu
  - suma de bani (cheltuieli mai mari față de un anumit prag pot necesita o autorizare specială)

Autorizarea cheltuielilor
Exemplu



Raportul final

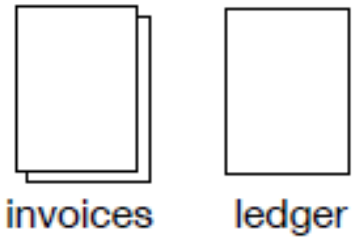
- La sfârșitul fiecărei perioade de raportare, se redactează documente despre statusul financiar al proiectului
- Două informații sunt disponibile:
  - Cheltuielile bugetate vs cheltuielile reale
  - Contabilizarea cheltuielilor
- Informația este folosită în moduri diferite:
  - Pentru a analiza **deviările** (ce diferențe au existat)
  - Pentru a evalua starea proiectului și a lua acțiunile corespunzătoare

Fluxul de lucru

Cheltuieli așteptate

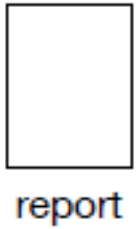
Budget	Y1	Y2	Y3
Personnel	4000	5000	3000
Hardware	1000	2000	
Subcontracting		1000	6000
…			

Cheltuieli reale



Budget	Y1	Y2	Y3
Personnel	**3000**	5000	3000
Hardware	500	2000	
Subcontracting		1000	6000
…			

Raport de final & Revizuire



Budget	Y1	Y2	Y3
Personnel	5000	4000	3000
Hardware	500	1000	
Subcontracting		1000	6000
…			

Informațiile de interes

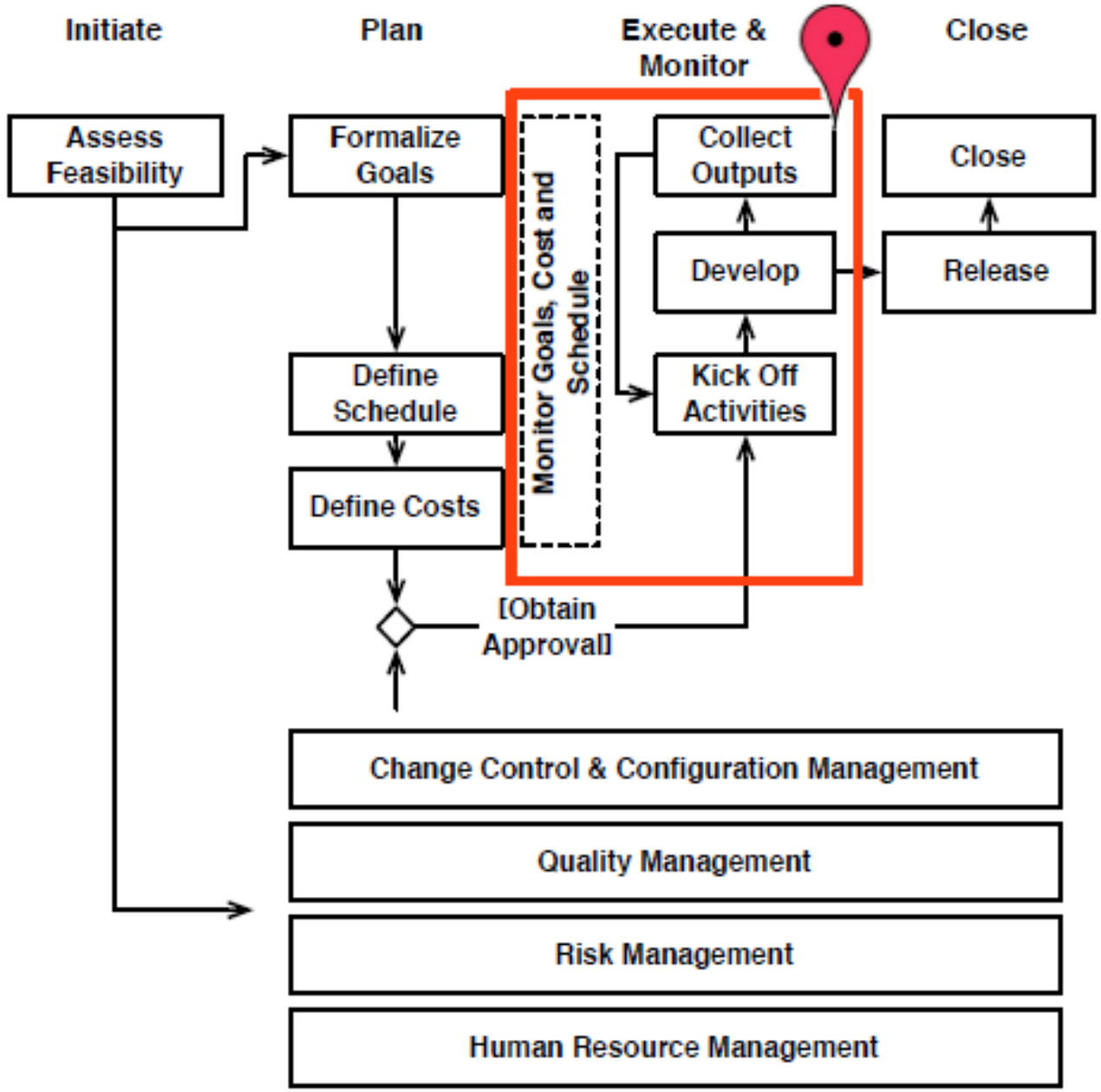
- Budget → cantitatea inițial bugetată
- Transferuri → variațiile realizate asupra bugetului
- Actualitate → cantitatea cheltuită în realitate

Budget	Budget	Variations	New	Spent	Available
Personnel	4000	+2000	6000	5000	1000
Hardware	3000	-2000	1000	0	1000
Subcontracting	1000		1000	400	600

Execuția, monitorizarea și controlul proiectului

Obiective

- Planurile sunt inutile dacă nu sunt puse în practică și, în timpul executării, monitorizate și actualizate pentru a reflecta situația reală
- Acest capitol prezintă:
  - Activitățile necesare pentru a pune un plan în practică
  - Tehnicile de monitorizare și controlare a planurilor (progres și costuri vs planificare și buget)
  - Earned Value Analysis, o tehnică ce îi permite managerului de proiect să monitorizeze un proiect într-o maniera integrată


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Define Schedule
Define Costs
Monitor Goals, Cost and Schedule
Collect Outputs
Develop
Kick Off Activities
Close
Release
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Executarea proiectului

Executarea proiectului

- Executarea proiectului reprezintă adevarata realizare a muncii
- Exista trei principale activități de management:
  - Demararea activităților
  - Strângerea rezultatelor activităților
  - Strângerea informațiilor despre starea proiectului

Demararea activităților

- Scop:
  - Asigurarea unui început formal pentru o parte semnificativă a proiectului
  - Asigurarea alinierii echipei la obiectivele și modalitățile de începere a activităților
- Principala modalitate este o ședinta de început
- În general, orice modalitate de comunicare poate fi folosită (dar poate fi mai puțin eficientă decât o ședință de început)
- Trebuie ales un nivel potrivit de granularitate

Strângerea rezultatelor activităților

- Scop:
  - Colectarea sistematică a rezultatelor proiectului (livrabile)
  - Ocazie de a evalua lecțiile învățate
- Pentru proiectele software, principala modalitate de colectare a rezultatelor proiectului este reprezentată de un repository + versionare
- O ședință de evaluare a lecțiilor învățate poate fi de asemenea folosită pentru a “formaliza” colectarea rezultatelor

Strângerea informațiilor despre starea proiectului

- Scop → colectarea sistematică a datelor pentru evaluarea statusului proiectului
- Poate fi realizată regulat (frecvența aleasă în funcție de dimensiunea proiectului)
- Poate fi realizată în funcție de nevoie (pentru evenimente exceptionale, ex. riscuri)
- Datele cantitative pot fi strânse pe baza metodelor de monitorizare
- Datele calitative (ex. moralul echipei, feeling-ul despre statusul sau dificultatea unui task) trebuie de asemenea colectate

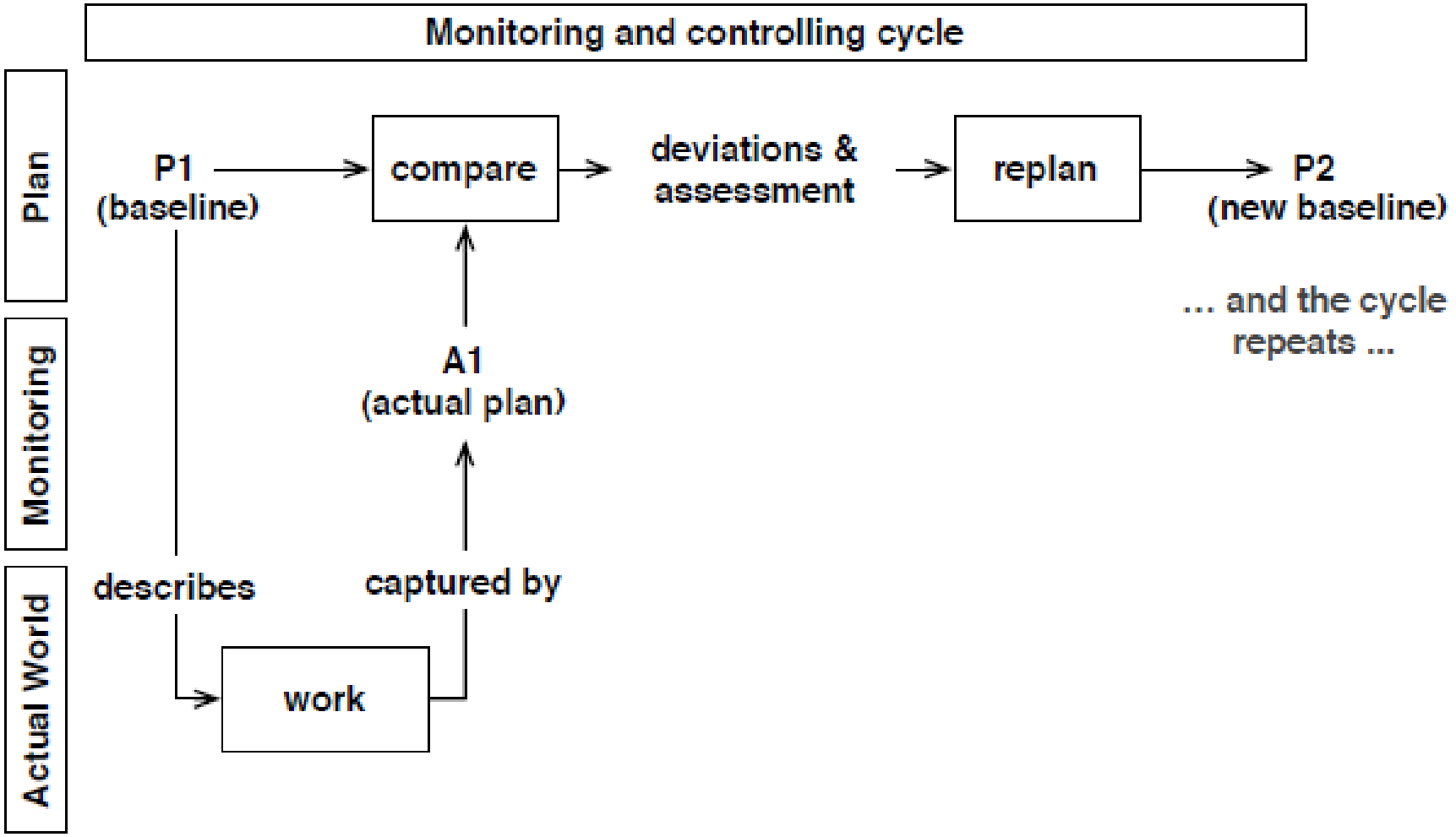
Monitorizarea și controlul proiectului

All stable processes we shall predict. All unstable processes we shall control.

John von Neumann

Introducere

- Obiective:
  - **Pentru proiect** → evaluarea statusului proiectului (scop, timp, cost, calitate etc.), analizarea deviațiilor, luarea de decizii corective, dacă este nevoie
  - **Pentru organizație** → colectarea datelor ajuta la construirea unui plan mai bun și mai exact pentru proiectele viitoare
- Proces (în mod regulat):
  - **Colectează** → Strânge datele despre statusul curent al proiectului
  - **Masoară și compară** → Compară cu un plan de baza, subliniază orice deviație, realizează o proiecție pe baza datelor curente
  - **Evaluează și replanifică** → Decide dacă sunt necesare acțiuni corective. Dacă da, planifică, documentează și ia acele acțiuni


Monitoring and controlling cycle
Plan
Monitoring
Actual World
P1
(baseline)
compare
deviations &
assessment
replan
P2
(new baseline)
... and the cycle
repeats ...
A1
(actual plan)
describes
captured by
work

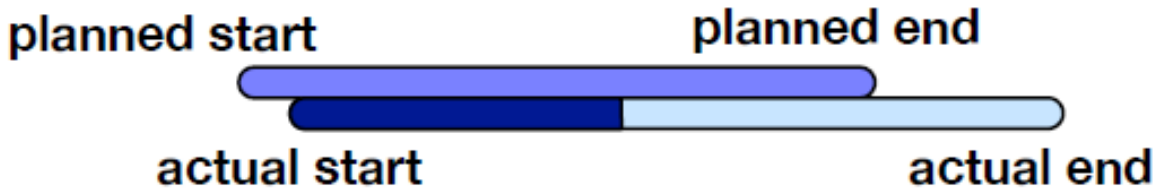
Abordări

- Concentrare:
  - Aici ne concentram pe planificare, costuri și progres
- Abordare neintegrată
  - Monitorizarea planificării → a înțelege dacă suntem înainte sau în urmă
  - Monitorizarea costurilor → a înțelege dacă suntem sub sau peste buget
  - Simple, dar viziuni parțiale
- Abordare integrata
  - Earned Value Analysis → metodă ce măsoară împreună planificarea, costurile și progresul
  - Mai complexă, dar viziune mai cuprinzătoare

Monitorizarea planificării

Concepte de bază

- Baza (valori planificate):
  - Un instantaneu al planului la un moment dat (planul la t1, planul la t2, …)
  - Se pot produce multe astfel de instantanee
- Valori reale
  - Statusul real al planificării
  - Începutul real, sfârșitul real, efortul/progresul real



Procesul

1. Construirea planului
2. Stabilirea unei baze
3. Evaluarea planului în mod regulat:
    1. Începutul real și sfârșitul real al unei activități
    2. Efortul real consumat pe activitate
    3. Progresul tehnic (poate fi dificil de evaluat)
4. Replanificare:
    1. Estimarea efortului și a duratei până la sfârșit
    2. Tehnica 1 → eficiență prin care efortul real a fost exprimat cu privire la efortul planificat
    3. Tehnica 2 → eficiență prin care progresul tehnic a fost exprimat cu privire la efortul planificat
    4. Împărtășirea noului plan și avansare la punctul 2

Colectarea datelor de efort

- Depinde de nivelul de formalitate; oamenii pot fi nevoiți să furnizeze date despre efortul consumat pe activități
- De obicei indicat să fie realizat saptămânal
- Activitățile planului trebuie sa aibă referințe
- Va conține și zgomot

John Doe	W1	W2	W3	W4
Requirements M1	30		6	
Requirements M2		30	6	
Meeting	2	2	6	2
Research	4	4	4	4
Indirect activities				2

Monitorizarea costurilor

Controlul costului abordarea simplă

- Tabelul de buget definește baza
- Costurile reale definesc statusul curent
- Poate fi împărțit de-a lungul anilor (sau a perioadelor de raportare)

CBS Item	Budgeted	Actual	Status	New Budget
Hardware	€10,000.00	€5,000.00	€5,000.00	€5,000.00
Software	€4,000.00	€2,000.00	€2,000.00	€2,000.00
Travel	€5,000.00	€6,000.00	-€1,000.00	**€1,000.00**
Project Bfr	€3,000.00		€3,000.00	**€1,000.00**
Total	€22,000.00	€13,000.00	€9,000.00	€9,000.00

Depășirile sunt rezolvate din alte fonduri sau din alte proiecte

Remarci

Avantaje

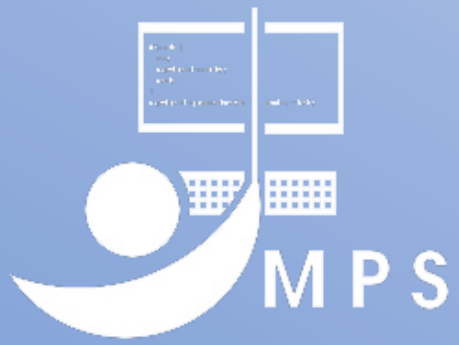
- Relativ simplu (totuși, există întârzieri între realizarea cheltuielilor și fluxurile de bani)
- Pentru anumite elemente, probabil cea mai bună metodă de monitorizare (ex. hardware, software etc.)

Dezavantaje

- Nu este suficient pentru a avea o idee despre statusul global al proiectului (va fi dus la bun sfârșit cu banii rămași?)

Earned Value Analysis

Analiza Valorii Câștigate



Earned Value Analysis

- **Earned Value Analysis** este o metodă ce furnizează o viziune de ansamblu asupra proiectului, măsurând **efortul planificat (costuri), progresul real (valoarea câștigată) și efortul (costurile actuale)** în materie de **valori monetare**
- Măsurarea planului, a muncii și a progresului folosind aceeasi unitate le face comparabile
- Util pentru că:
  - Progresul devine comparabil cu efortul
  - Bugetul și costurile actuale sunt puse în context (a fi sub buget nu este întotdeauna bine, dacă progresul tehnic este și el scăzut)

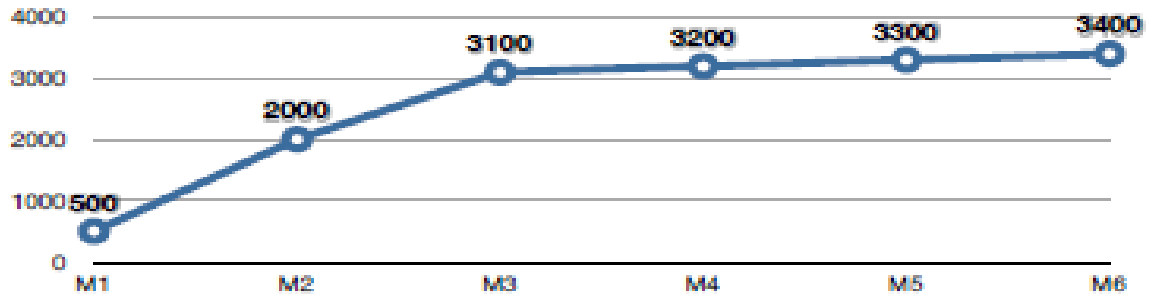
Presupuneri și definiții

- Presupuneri:
  - **Forță de muncă = Cost** → reprezentările grafice ale efortului sau costului sunt echivalente
  - Corolar: Forța de muncă reală = Costul real
  - **Progres = Bani**
- Definitii:
  - **Valoarea Planificată** → costurile cumulate planificate pentru proiect; numite de asemenea: **Costuri Bugetate ale Muncii Planificate**
  - Costuri Reale → costurile cumulate ce sunt suportate în realitate; numite de asemenea Costurile Reale ale Muncii Efectuate
  - **Valoarea Câștigată** → progresul real, exprimat drept cantitatea din valoarea planificată ce a produs rezultate

Calculul costului planificat

Name	Planned Cost	Time
A	1000	A
B	2000	B
C	400	C

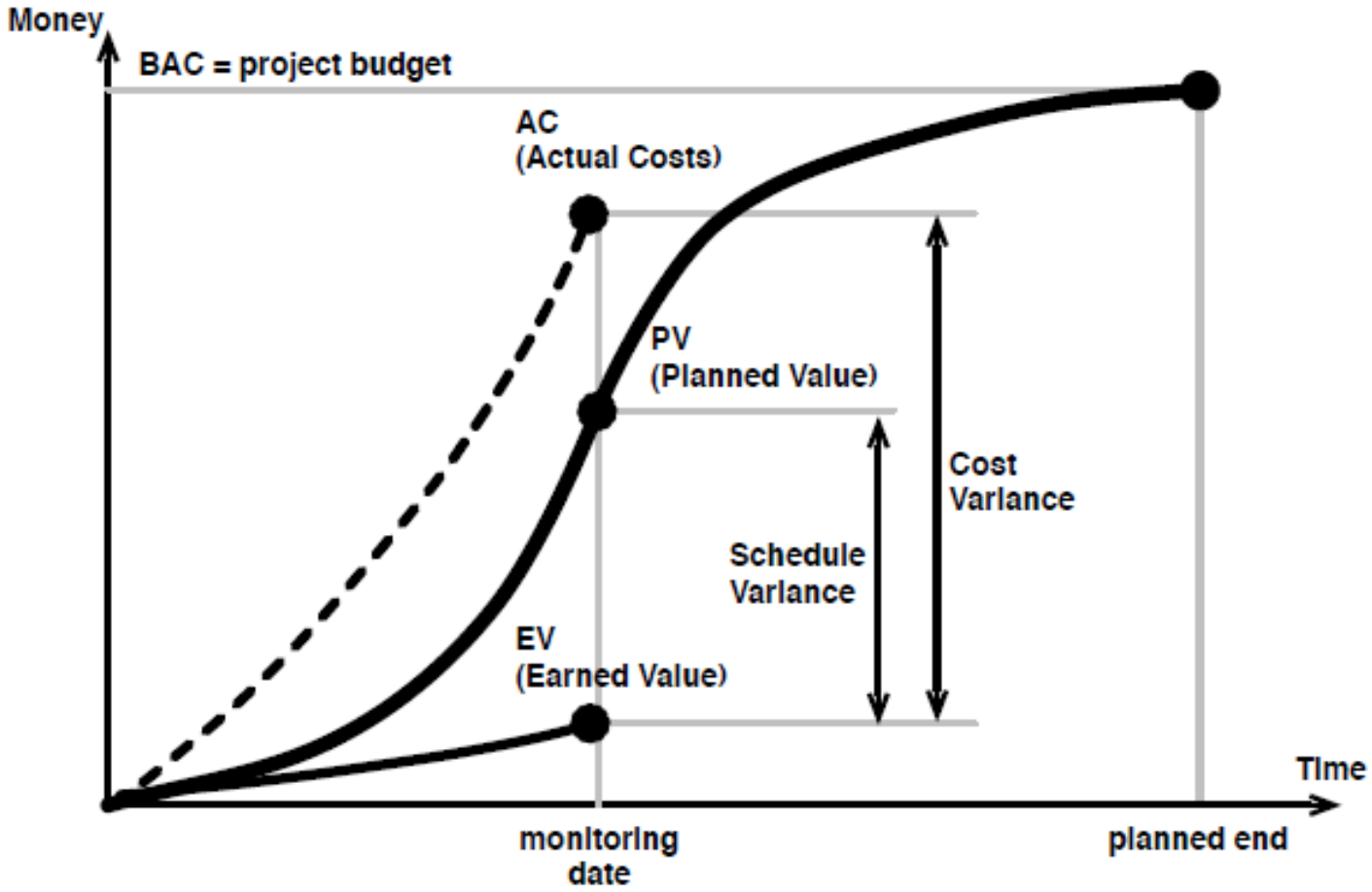
A	1000	500	500			
B	2000		1000	1000		
C	400			100	100	100	100
Total		500	1500	1100	100	100	100
Cumulative		500	2000	3100	3200	3300	3400



Calculul valorii câștigate

- Regula 1:
  - Valoarea câștigată ar trebui determinată prin examinarea produselor
- Regula 2:
  - **Regula 50/50** (50% din valoarea planificată la început, 50% la sfârșit)
  - **Regula 20/80** (20% la început și 80% la sfârșit)
  - **Regula 0/100** (0% la început și 100% la sfârșit)

Earned Value Analysis



Puncte și metrici interesante

- BAC = Budget at Completion (Bugetul la Completarea proiectului)
- SV: Schedule Variance (Varianța Programului)
  - O comparație între cantitatea de muncă realizată într-o anumită perioadă de timp și ce era planificat să fie realizat
  - O variantă negativă înseamnă că proiectul este în urma programului
- CV: Cost Variance (Varianta Costului)
  - O comparație între costul bugetat și cel real folosit pentru munca produsă
  - O variantă negativă înseamnă că proiectul depășește bugetul

Cost Performance Index (CPI)

- CPI (Cost Performance Index) compară valoarea câștigată cu totalul costurilor reale
- Cât câștigăm pentru fiecare Euro pe care îl cheltuim?

$$CPI_t = \frac{EV_t}{AC_t}$$

CPI > 1	Proiectul este eficient
CPI < 1	Proiectul este ineficient

Schedule Performance Index (SPI)

- SPI (Schedule Performance Index) compară munca realizată cu cea planificată
- Cât de repede progresează proiectul raportat la cât de repede ne-am fi așteptat noi?

$$SPI_t = \frac{EV_t}{PV_t}$$

SPI > 1	Proiectul este în fața planificării
SPI < 1	Proiectul este în urma planificării

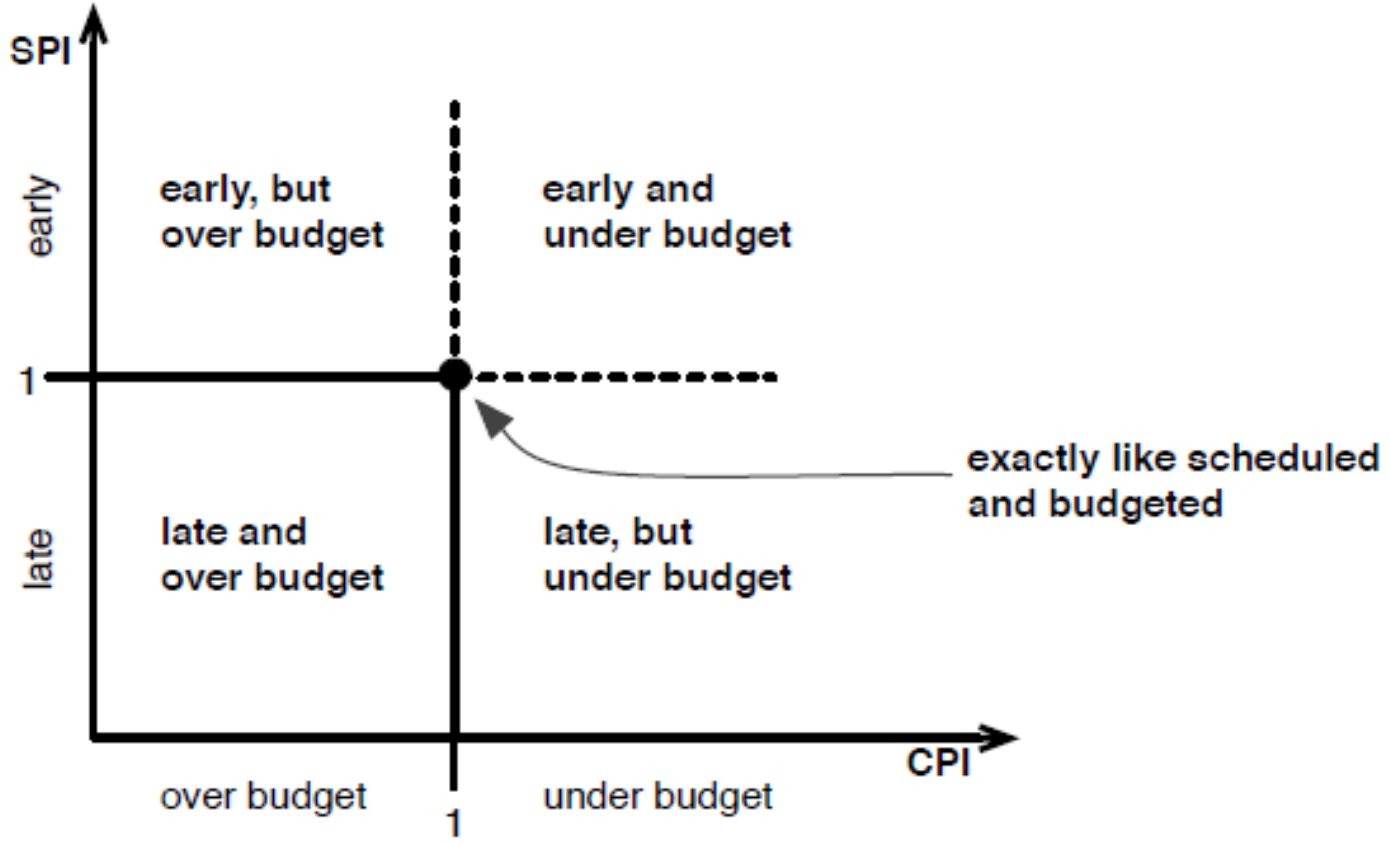
Cost Schedule Index (CSI)

- CSI (Cost Schedule Index) → CSI = CPI x SPI
- Cu cât este mai departe CSI de 1.0, cu atât devine mai puțin probabilă recuperarea proiectului

$$CSI_t = CPI_t * SPI_t$$

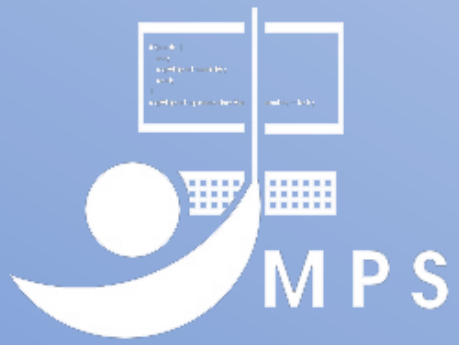
$$CSI_t = \frac{EV_t}{AC_t} * \frac{EV_t}{PV_t}$$

Masurarea indicatorilor SPI și CPI



Earned Value Analysis

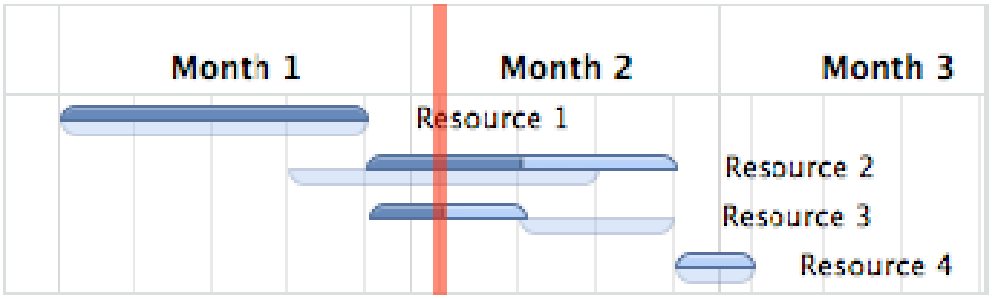
Exemplu



Exemplu

- Remarci:
  - Partea de jos → baza; partea de sus → real & progres
- Întrebări:
  - Suntem în întârziere?
  - Am depășit bugetul?

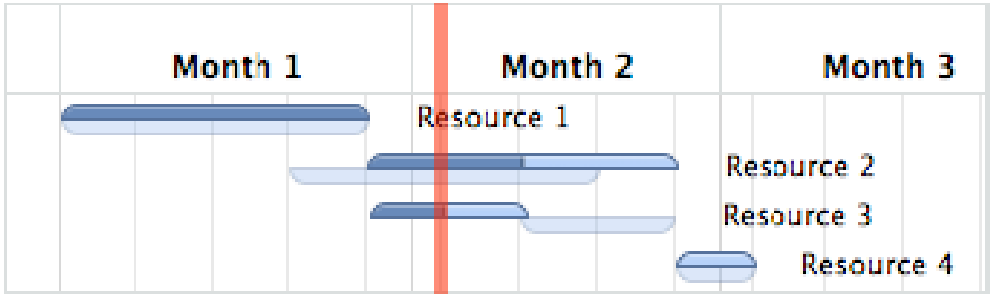
Task	Effort	Resources Cost	Task Cost
1) Paint Wall	4w	€3,200.00	€3,600.00
2) Paint Ceiling	4w	€3,200.00	€2,000.00
3) Refurnish	2w	€800.00	€400.00
4) Clean	1w	€400.00	



Exemplu

- Activitate 1: conform planificării (la timp)
- Activitate 2: începută târziu; în fața planificării
- Activitate 3: începuta devreme; progresul acelașii cu timpul trecut
- Activitate 4: neînceputa încă

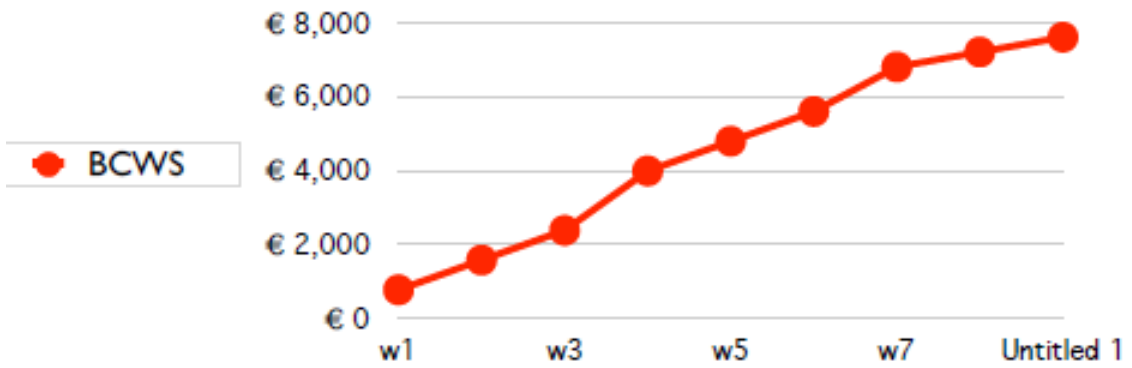
Task	Effort	Resources Cost	Task Cost
1) Paint Wall	4w	€3,200.00	€3,600.00
2) Paint Ceiling	4w	€3,200.00	€2,000.00
3) Refurnish	2w	€800.00	€400.00
4) Clean	1w	€400.00	



Exemplu - Valoarea Planificată (BCWS)

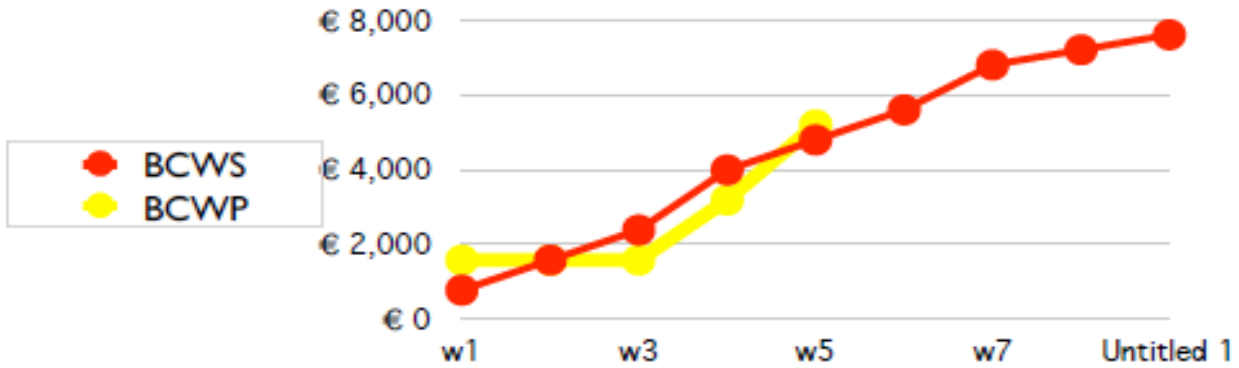
Task	Effort	Resources Cost	Task Cost
• 1) Paint Wall	4w	€3,200.00	€3,600.00
• 2) Paint Ceiling	4w	€3,200.00	€2,000.00
• 3) Refurnish	2w	€800.00	€400.00
• 4) Clean	1w	€400.00	

Paint Wall	800	800	800	800					
Paint Ceiling				800	800	800	800		
Refurnish							400	400	
Clean									400
Budgeted Cost of Work Scheduled (BCWS)	800	1600	2400	4000	4800	5600	6800	7200	7600



Exemplu - Valoarea Câștigată (BCWP)

Paint Wall	1600			1600					
Paint Ceiling				0	1600				
Refurnish					400				
Clean									
Budgeted Cost of Work Scheduled (BCWS)	1600	1600	1600	3200	5200				

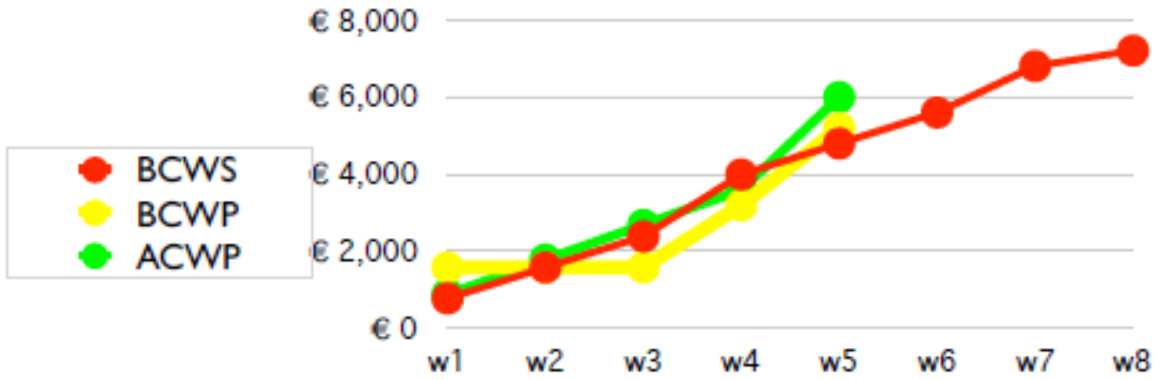


Comentarii

- In fața planificării în săptămâna 1, din cauza zgomotului regulii 50%-50% (analog pentru întârzierile din săptămânile 2 și 3)
- Săptămâna 4: suntem în urma planificării (activitatea 2 nu a început așa cum ne așteptam)
- Săptămâna 5: suntem din nou în fața planificării, din cauza activității 3
- Intrucât regula 50%-50% ia în calcul doar începutul și sfârșitul activităților, faptul că progresul din activitatea 2 este mai bun decât ne asteptam nu este luat în calcul în grafic.

Exemplu - Costurile Reale (ACWP)

Paint Wall	900	900	900	900					
Paint Ceiling				0	2000				
Refurnish					400				
Clean									
Actual Cost of Work Performed (ACWP)	900	1800	2700	3600	6000				



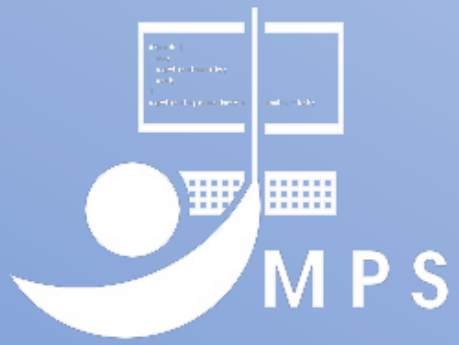
Recapitulare

Paint Wall	€800.00	€800.00	€800.00	€800.00	
Paint Ceiling				€800.00	€800.00
Refurnish					
Clean					
Budgeted Cost of Work Scheduled (BCWS)	€800	€1,600	€2,400	€4,000	€4,800
Budgeted Cost of Work Performed (BCWP)	€1,600	€1,600	€1,600	€3,200	€5,200
Actual Cost of Work Performed (ACWS)	€900	€1,800	€2,700	€3,600	€6,000
CPI (BCWP/ACWP)	178%	89%	59%	89%	87%
SPI (BCWP/BCWS)	200%	100%	67%	80%	108%
Cost Variance	€700	-€200	-€1,100	-€400	-€800
Schedule Variance	€800	€0	-€800	-€800	€400

Comentarii

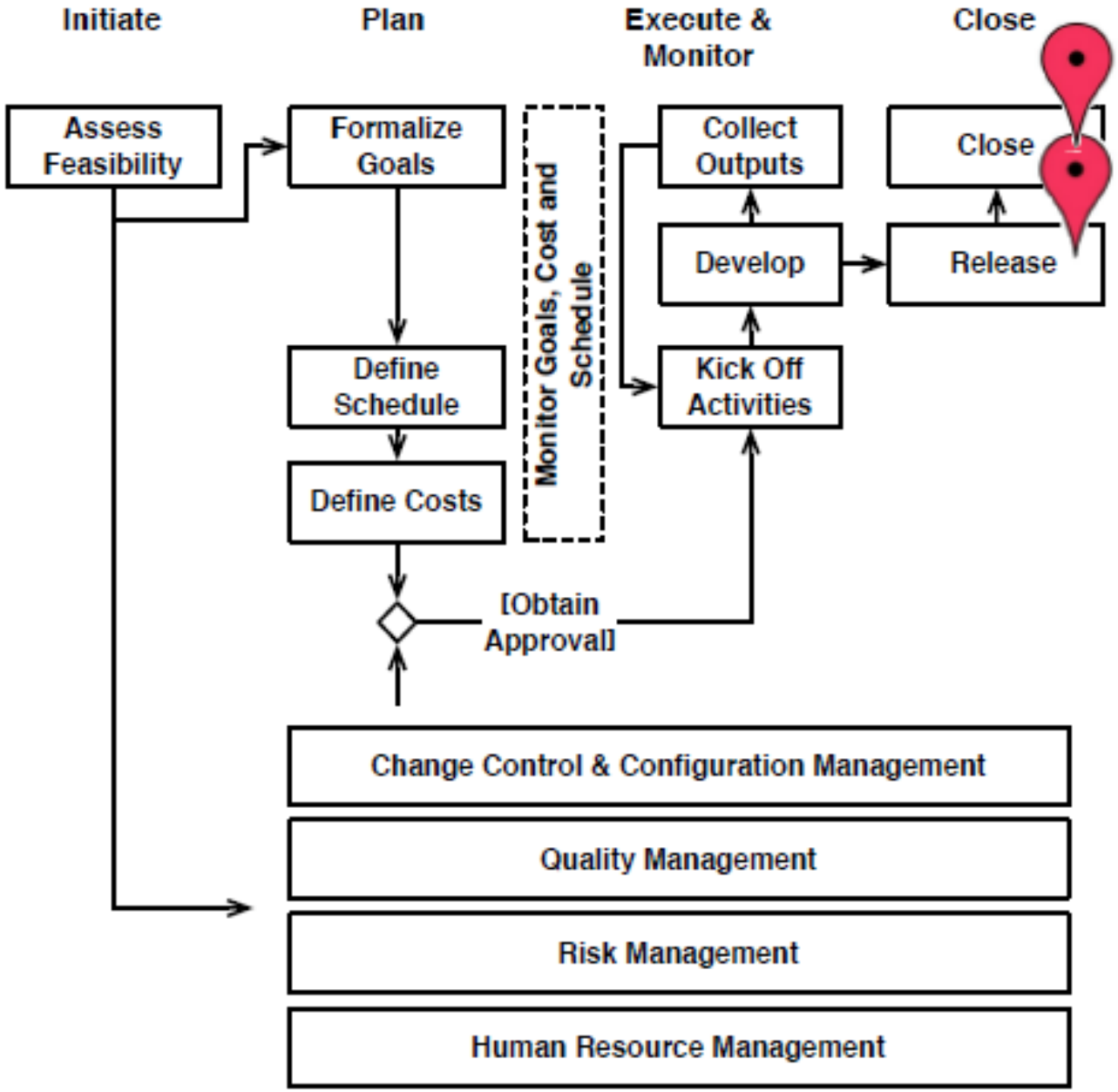
- Zgomot din cauza regulii 50%-50% (ex. săptămâna 1)
- Datele arată că noi suntem acum peste buget, dar devreme în planificare (ultima coloana)
- Totuși:
  - Eficiența reală a costurilor este, din cauza regulii 50%-50%, în activitatea 2 (am avut o creștere de 1600); datele vor fi mai exacte atunci când vom termina activitatea 2 (cheltuielile vor fi cel mai probabil 4000 Euro și BCWS 3200)

Finalizarea proiectului



Obiective

- Toate proiectele ajung la un sfârșit
- Multe dintre ele, totuși, trăiesc în uitare pentru o lungă perioadă de timp, nefiind active, dar nici închise (finalizate)
- Scopul unui bun manager de proiect este să se asigure că proiectele au fost finalizate cum trebuie
- Obiectivele acestui capitol:
  - Înțelegerea activităților ce trebuie efectuate pentru a închide un proiect
  - Înțelegerea motivelor pentru care proiectele nu sunt finalizate cum trebuie
  - Înțelegerea riscurilor apărute dacă un proiect nu este finalizat cum trebuie


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Tipuri de finalizari de proiect

- **Terminare prin intergrare și terminare prin adăugare**

Cazurile de succes: rezultatele proiectului integrate și/sau folosite ca intrări pentru alt proiect/producție

- **Terminare prin înfometare**

Proiectul se termină pentru ca s-au terminat resursele

- **Terminare prin extincție**

Proiectul terminat de către conducere, deoarece proiectul a eșuat (obiectivele neîndeplinite, înlocuit, neprofitabil)

Finalizarea proiectului

Finalizarea proiectului reprezintă ultima fază a unui proiect, în care ieșirile proiectului sunt trimise părților implicate, acordurile contractuale sunt gestionate cum se cuvine, iar registrele proiectului sunt stocate pentru referințe viitoare

- Scopuri principale:
  - Asigurarea posibilitatii utilizării ieșirilor proiectului
  - Asigurarea că nu există alte obligații viitoare / în așteptare
  - Învățare și creare de stocuri

De ce proiectele nu sunt finalizate cum trebuie

- Pentru proiectele care nu au succes:
  - Lipsă de interes din partea echipei
- Pentru toate proiectele:
  - Scăderea interesului din partea echipei
  - Costul realizării activităților necesare pentru finalizare
  - Activitățile de finalizare necesită puțină (sau chiar deloc) creativitate
  - Subestimarea cantității de cunoștințe implicite prezente
  - Subestimarea rapidității de pierdere a cunoștințelor
  - Repulsia eliberării resurselor din motive oportuniste
  - Factorul emoțional

Procesul de finalizare a proiectului

- Obținerea acceptului din partea clientului
- Instalarea livrabilelor proiectului
- Arhivarea livrabilelor vechi
- Documentarea proiectului
- Realizarea unei închideri definitive
- Realizarea auditului post-implementare
- Eliberarea staff-ului

Obținerea acceptului din partea clientului

- Acceptul ceremonial
  - Fără proceduri formale sau înregistrări formale pentru acceptarea livrabilelor proiectului
  - Scenarii: gentlemen agreement; atingerea deadline-urilor proiectului
- Acceptul formal
  - Procedură formală de acceptarea a livrabilelor proiectului
  - Testarea sistemului / aprobarea clientului

Audit post-implementare (post-mortem)

- Ne displace să repetăm aceleași greșeli
- Scopul acestui audit este de a analiza critic proiectul pentru a învăța și a îmbunătăți, evitând să facem aceleași greșeli
- Diferite formate și niveluri de formalitate sunt posibile
- Proiectele care nu au succes ne aduc o mulțime de informații
- Lecții utile și din proiectele de succes (ce a funcționat, ce am fi putut face mai bine)

Structura unui post-mortem

- **Conducerea de sondaje de proiect**

Evidențierea principalelor puncte tari și puncte slabe ale proiectului

- **Colectarea informației obiective**

Evidențierea măsurătorilor cantitative ale proiectului

- **Conducerea unei întâlniri pentru a extrage rezultatele**
- **Conducerea unei întâlniri de “istorie a proiectului”**

Găsirea cauzelor problemelor

- **Publicarea rezultatelor**

Asigurarea ca noi, organizația și echipa putem învăța din această experiență

Metrici post-mortem

- O evaluare cantitativă permite o interpretare mai precisă a proiectului
- Datele pot fi folosite si pentru estimări viitoare
- Metricile includ:

Metricile de Cost	Metricile de Planificare	Metricile de Calitate
Efortul Planificat și SLOC Estimat Efortul Real și SLOC Real Istoria schimbărilor cerințelor și a codului	Planificare Originală Planificare Finală Istoria evenimentelor ce au deraiat de la planificare	Erori în fiecare stadiu

Structura rezultatelor post-mortem

- Rezultatele unui audit post-mortem sunt publicate într-un document
- Acest document poate fi folosit pentru a disemina lecția învățată și poate fi folosit drept referință pentru viitoarele proiecte similare
- Un raport post-mortem poate fi organizat cum urmează:
  - Descrierea proiectului: informatii despre proiect, pentru a crea un context
  - Partea bună: ce a funcționat bine
  - Partea rea: cei mai rai 3 factori care au împiedicat echipele să atingă obiectivele
  - Partea urată: o recomandare pentru îmbunătățire

Eliberarea staff-ului

- Tranziția către noi activități poate dăuna echipei (să considerăm, de exemplu, că un proiect a lucrat ani de zile)
- Două aspecte importante:
  - Asigurarea recunoașterii corespunzătoare a experienței câștigate în timpul proiectului și a rezultatelor obținute
  - Asigurarea că sarcinile potrivite sunt atribuite membrilor echipei

Managementul schimbării

Obiective

- Importanța păstrării proiectului în același scop
- Cum cererile pot influența pozitiv sau negativ proiectul
- Tehnici de a gestiona schimbările

Cere schimbări și schimbările vor apărea în proiectul tău.

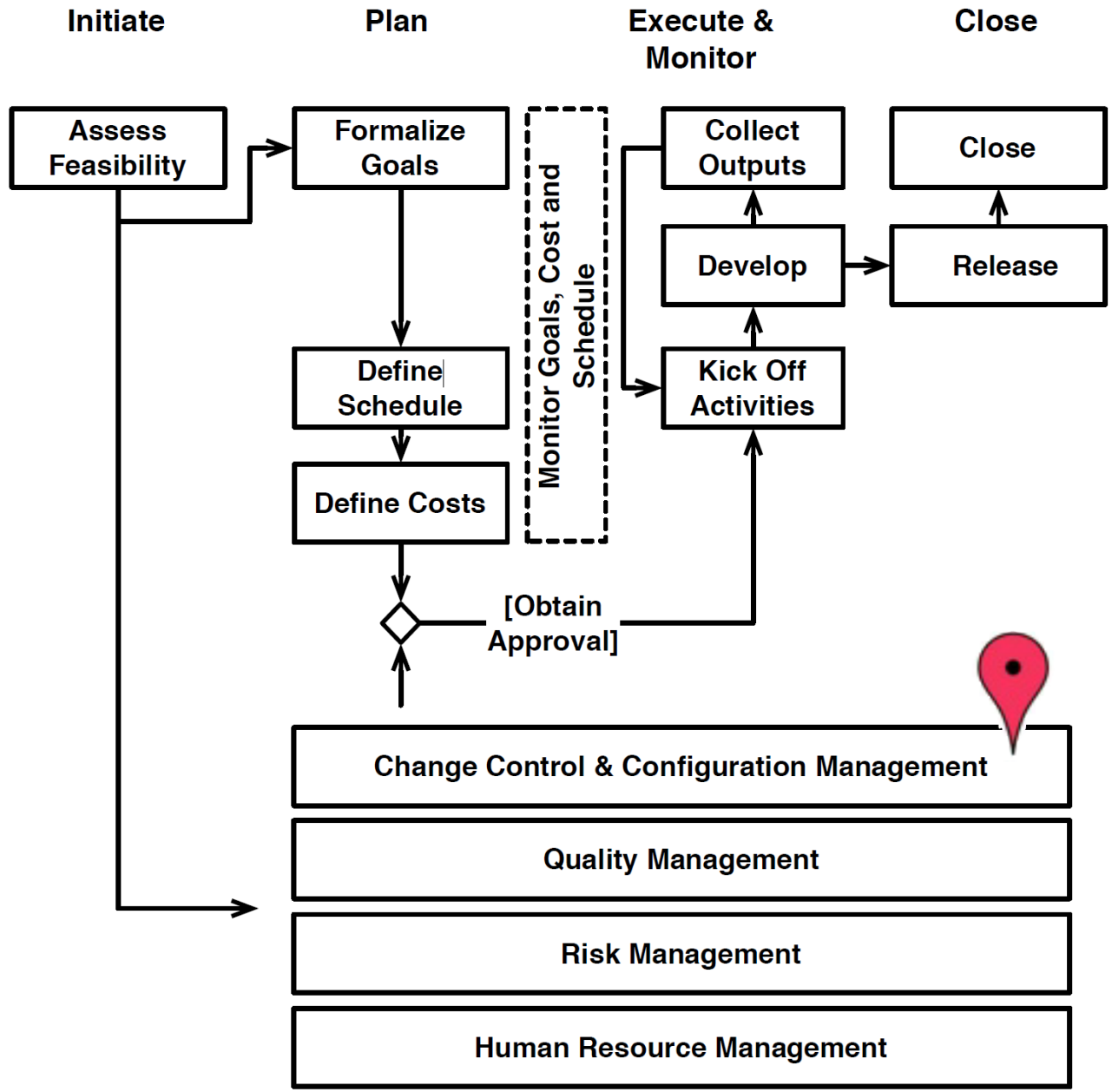
Framework

- Document formal ce cuprinde obiectivele proiectului
- Odată ce obiectivele sunt stabilite, atenția se va muta pe design-ul și implementarea pe faze ale proiectului, mentinând o continuă ajustare a acestuia
- Orice decizie ce alterează cursul acțiunii stabilite este o pertubație ce schimbă obiectivele, planurile, costurile, rezultatele, volumul de lucru etc.

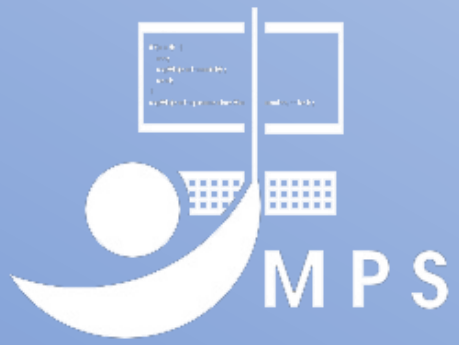
Concepte Fundamentale

- Change Control → un ansamblu de măsuri care să asigure că cererile de schimbare sunt tratate corespunzător

- Configuration Management → un ansamblu de măsuri care să asigure că obiectivul proiectului rămâne la fel de-a lungul timpului

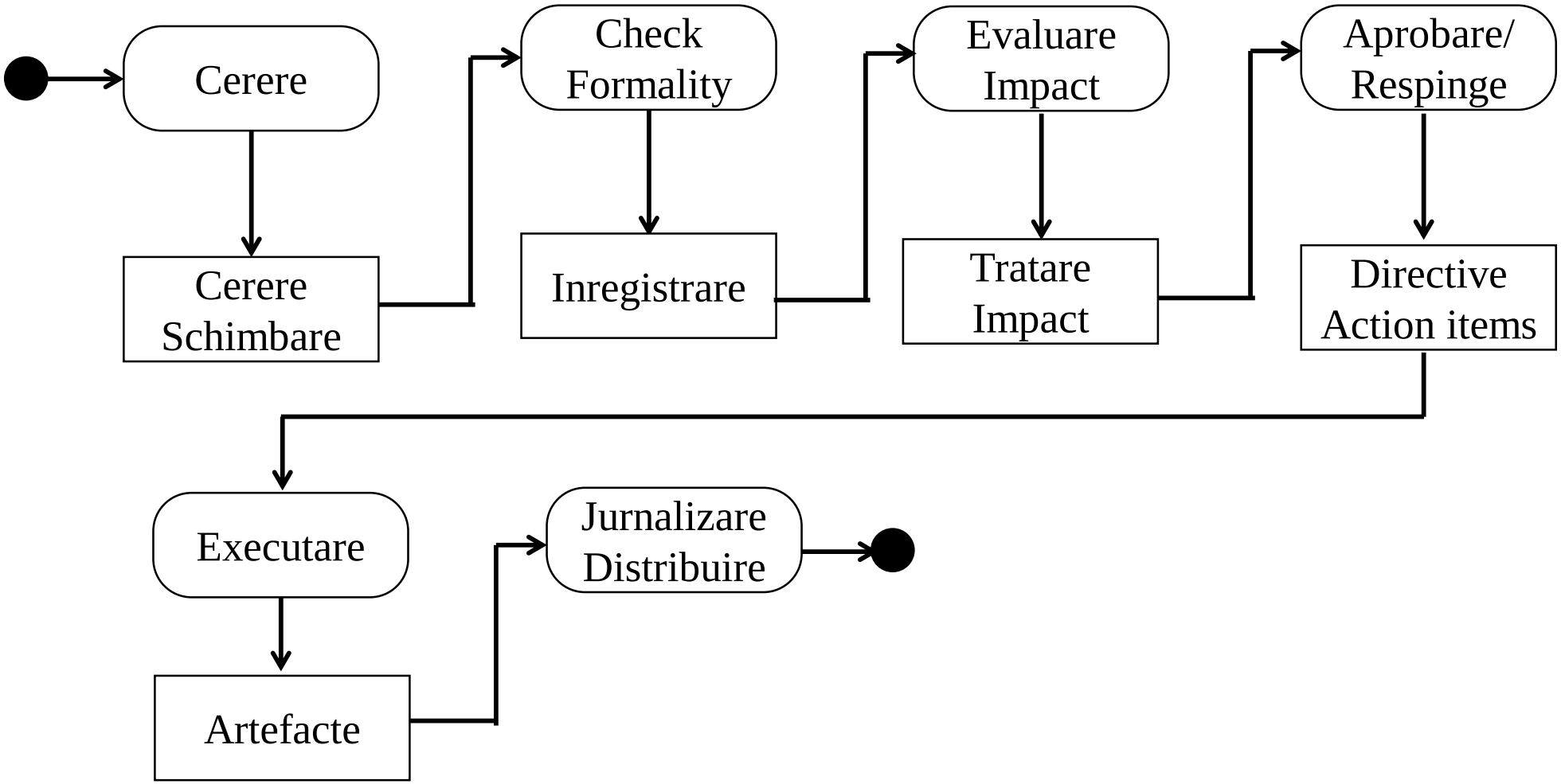

Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Define Schedule
Define Costs
Monitor Goals, Cost and Schedule
[Obtain Approval]
Collect Outputs
Develop
Kick Off Activities
Close
Release
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Change Control



Cauze cerere schimbare

- Inconsistența a cerintelor de proiect
- O mai bună dezvoltare a sistemului utilizat
- O oportunitate tehnică
- O provocare tehnică
- O modificare a mediului exterior
- Nerespectarea livrării proiectului


Cerere
Cerere Schimbare
Check Formality
Inregistrare
Evaluare Impact
Tratare Impact
Aprobare/ Respinge
Directive Action items
Executare
Artefacte
Jurnalizare Distribuire

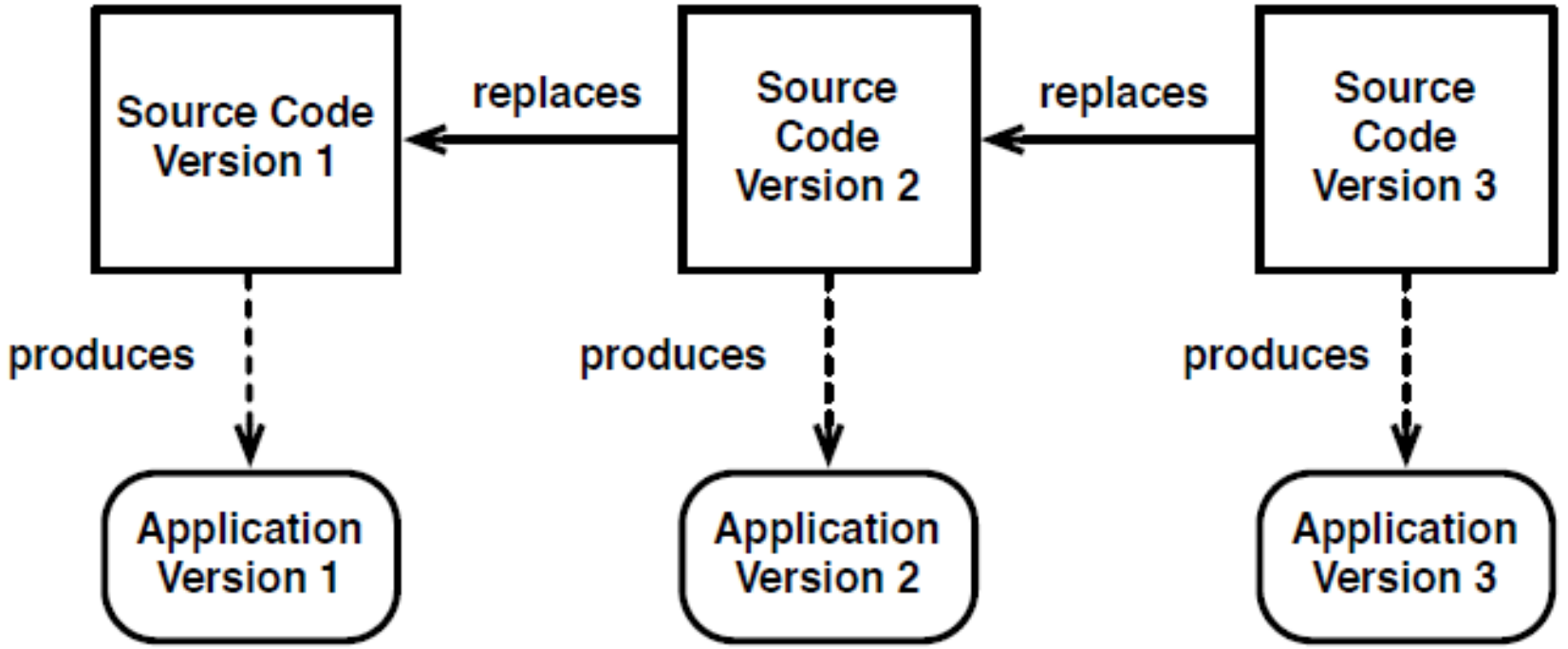
Comentarii

- Poate să existe un comitet executiv care să aprobe/respingă cereri
- Costurile și riscurile cresc odată cu înaintarea proiectului
- Procesul asigură existența unui jurnal și o procedură clară de pentru evaluarea impactului schimbării
- Schimbările și schimbările manageriale sunt controlate de tehnici agile (schimbările sunt considerate cerințe)

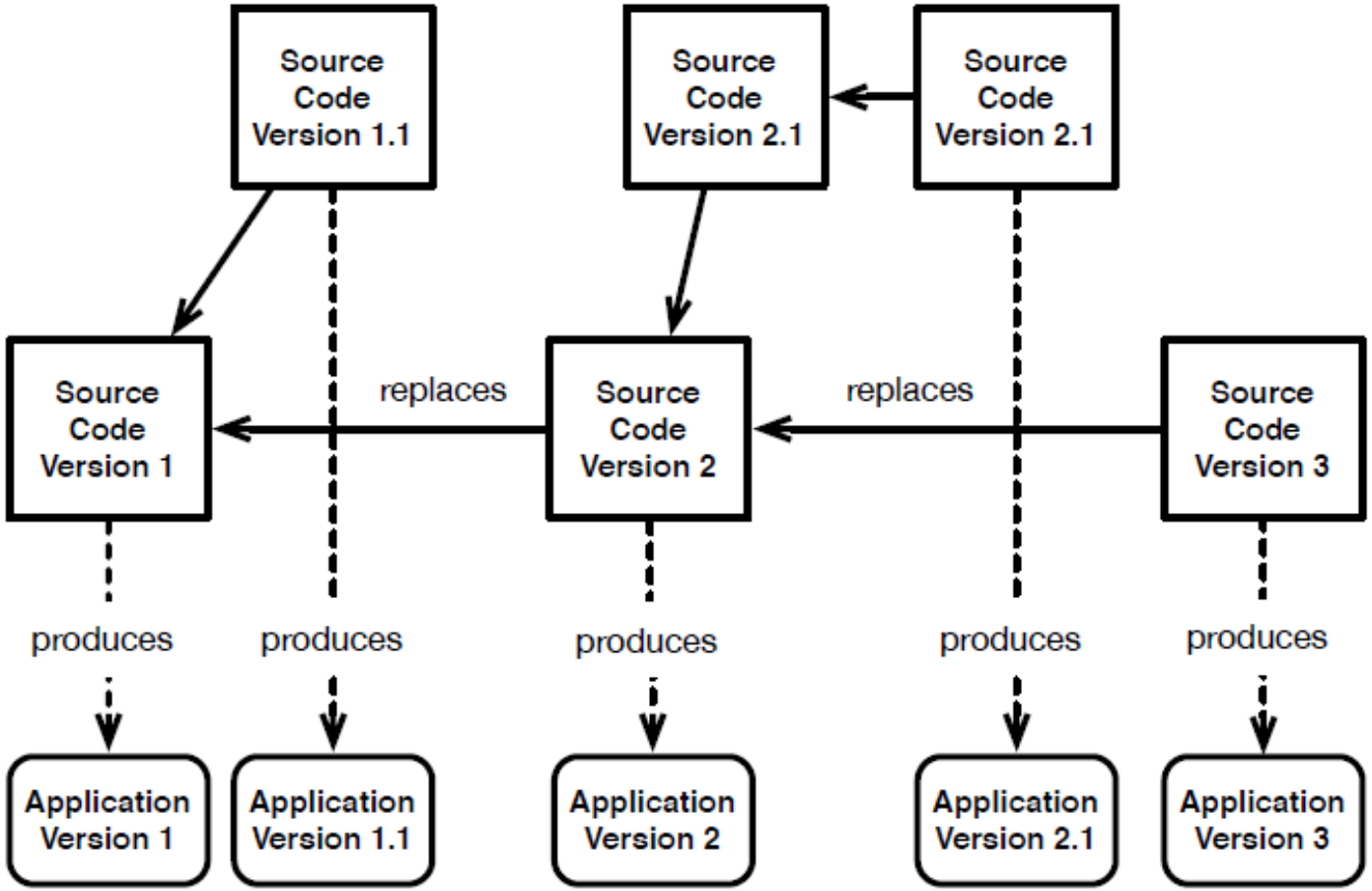
Evoluția modelelor software

- Sistemele software sunt alcătuite din mai multe artefacte:
  - Surse
  - Biblioteci
  - Biblioteci externe
  - Documentație
  - Scripturi
  - Baze de date etc.
- Sistemele software pot rula în mai multe configurații
- Două elemente cheie de complexitate trebuie adresate:
  - Identificarea artefactelor
  - Dezvoltare

Model de dezvoltare liniar



Branching Development Model



Comentarii

- Model de dezvoltare linear
  - Doar o singură versiune a aplicației poate rula la un anumit moment de tip

- Branching Development Model
  - Mai multe versiuni ale aplicației pot rula la un anumit moment de tip

Management de configurare

Management de configurare (CM)

Management de configurare – Configuration Management → Ansamblu de activități rulate în paralel pe procese de dezvoltare, al caror obiectiv este acela de a stabili și de a menține coerența sistemului de-a lungul timpului

Obiective principale - CM

- Construirea unui sistem dintr-un set consistent de componente
- Abilitatea de a retrage o anumită componentă software
- Abilitatea de a vizualiza istoricul modificărilor aduse sistemului
- Abilitatea de a face backup la o versiune anterioară

Remarcă: o legătură strânsă între procesul de managementul schimbării

Exemple

- Un bug este raportat de un utilizator pentru o aplicație care este pe piață de 10 ani
- Un client cere un upgrade a unui sistem inactiv ce a fost vândut în 2005
- E necesar să reproducem / înțelegem un comportament neobișnuit la un software de control din cadrul unei probe exploratorii din spațiu care orbitează în jurul planetei Jupiter

Pași și intrumente – stabilirea punctului de plecare

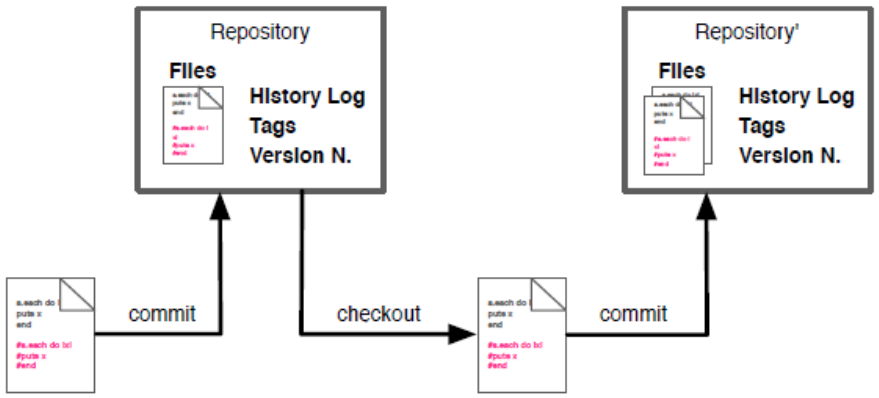
- Primul pas este acela de a stabili ce este un produs
- Un bun CM necesită:
  - Claritate în identificarea elementelor ce constituie produsul
  - Identificarea relațiilor dintre aceste elemente
  - Stabilește convenții de identificare și notare a versiunilor
  - Face snapshots → punctele de start/stop

Pași și intrumente – gestionarea schimbărilor

- Al doilea pas este păstrarea coerenței pe parcursul timpului
- Un bun CM necesită:
  - Definește punctul de start
  - Identifică și aprobă cererile de schimbare
  - Înregistrarea formală a modificărilor și menținerea unui istoric
  - Mentenanța vechilor versiuni

Version Control Systems

- Acesta implementează numeroase funcții
- Numai uneltele de proiectare nu sunt suficiente
- Concepte de baza
  - Working version → fișiere curente care sunt modificate
  - Repository → storage unde toate versiunile fișierelor sunt stocate împreună



Software Configuration Manager

Managementul riscului

Motivație

- În analiza unui proiect se ia în considerare partea financiara
- In documentul de management al domeniului am subliniat importanța realizarii obiectivelor noastre (A din SMART), dar este o mare diferență între realizabil și realizat.
- In etapa de planificare se ivesc diverse incertitudini (ex. estimare) care necesită o rezolvare generică (de ex. buffers)
- Se stabilește un plan, dar lumea este non-nominală, astfel apar modificări, atât negative, cât și pozitive

Managementul riscului

Managementul riscurilor colectează tehnici, metode cunoscute și procese care ajută la identificarea, evaluarea și monitorizarea riscurilor

- Obiectivele managementului riscurilor de proiect sunt:
  - Creșterea probabilității impactului evenimentelor positive
  - Scăderea probabilității și impactului evenimentelor nefavorabile proiectului.

Obiective

- Proiect suficient de bun pentru a fi abordat
- Rafinarea bugetului asignat
- Creșterea șanselor de a încheia proiectul cu succes:
  - în sfera de aplicare
  - de calitate
  - în bugetul alocat
  - la timp

Definiții

Traditional riscul este definit ca probabilitatea de a suferi pierderi

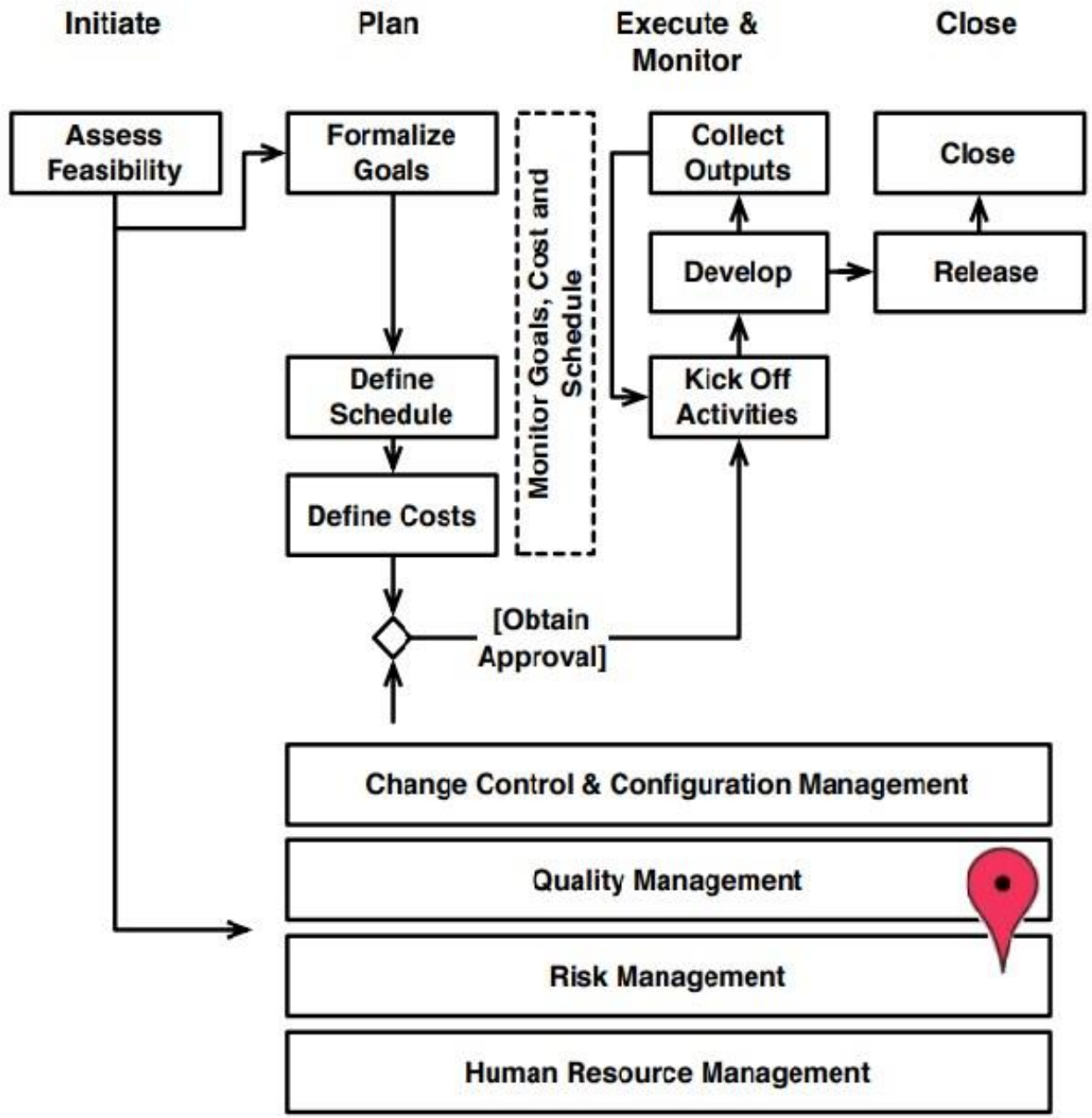
În managementul proiectelor riscul este un eveniment sau o condiție, care dacă se produce va avea un impact negativ sau pozitiv asupra proiectului

Managementul riscului

- Se folosește în mai multe domenii: finanțe, asigurări, inginerie (securitate)
- Riscul software este recunoscut de diferite standarde:
  - ISO / IEC 12207 (tehnologia informației - procesele ciclului de viață al software-ului)
  - UNI EN 29000-3 (ghid pentru aplicarea ISO 9001 la software dezvoltare și întreținere)
  - UNI ISO 10006 (Linii pentru gestionarea proiectelor)

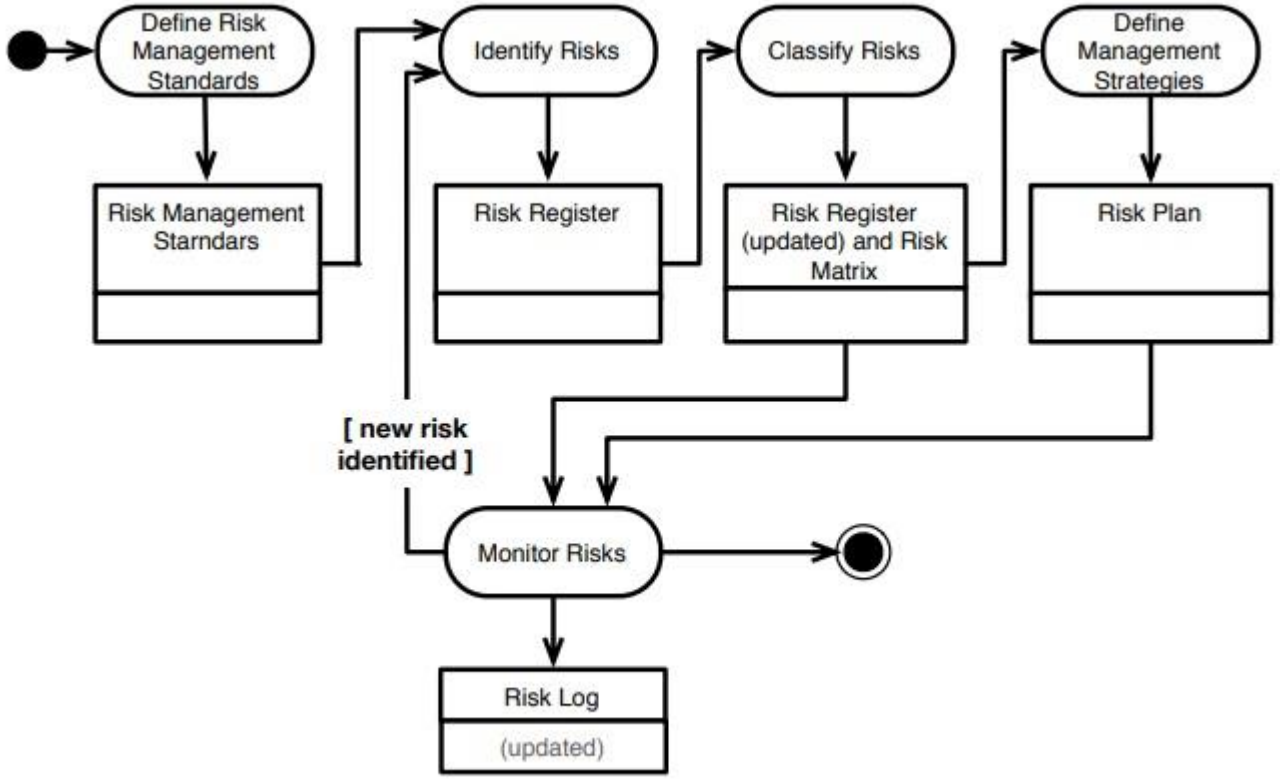
Obiectivele cursului

- Învățarea tehnicilor de identificare, evaluare, prioritizare, gestionare și controlarea riscurilor proiectului
- Învățarea celor mai frecvente riscuri în managementul de proiect
- Învățarea metodelor de protejare în condițiile apariției riscurilor


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Define Schedule
Define Costs
Monitor Goals, Cost and Schedule
[Obtain Approval]
Collect Outputs
Develop
Kick Off Activities
Close
Release
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Managementul Riscurilor
Managementul Proiectelor

Procesul de gestiune al riscurilor



Standarde

Scop descrierea modului de structurare și aplicare a managementului riscurilor

- Rezultat: document (sau seturi de documente și template-uri
- Parte din planul de management
- Ajută la definirea standardelelor și bunelor practici

Definirea standardelor de MR

- Documentul include :
  - Procedurile de actualizare
  - Procedurile de aplicare a planurilor de urgență
  - Împărțirea pe task-uri și pe persoane responsabile de acestea

- La noi schimbări
  - definirea probabilităților de risc și a impactului
  - format de raport

- Planul de management trebuie standardizat și adoptat de toată organizația
- Proiecte diferite implică diferite metode de gestionare a riscurilor

Identificarea riscurilor

Scop Înțelegerea riscurilor ce pot influența proiectul și documentarea caracteristicilor acestora

- Identificare riscurilor este un proces iterativ, riscuri noi pot apărea pe parcursul proiectului
- Rezultat listarea riscurilor, analiză calitativă / cantitativă pe baza riscurilor

Identificare riscurilor și clasificare

- Proces iterativ
  - Colectare
    - Identificarea riscurilor specifice proiectului
    - Descrierea riscului
  - Analiză
    - Identificare cauzelor
    - Definirea categoriei și probabilității
- Rezultat registru de riscuri

Analiza riscurilor
Calitative vs Cantitative

Scorul riscului de 70%

Evaluare

Simplă

Riscuri cu

Evaluam riscurile folosind un set de standarde definite în planul de risc management

Riscuri ce necesită răspuns

Produsul dintre impact si probabilitate

Nu sunt suficiente informații despre probabilitățile de risc

Evaluare obiectivă

Sistematică, cere timp

Oferă cifre ale impactului economic, cuantificată în timp, bani, planificare

Evaluare numerică, Analiză matematică

Date anterioare calitative

Tool-uri: Monte Carlo Simulation, Decision Tree Analysis

Risc cu impact negativ de 1k$ asupra costurilor

Metode de identificare a riscurilor

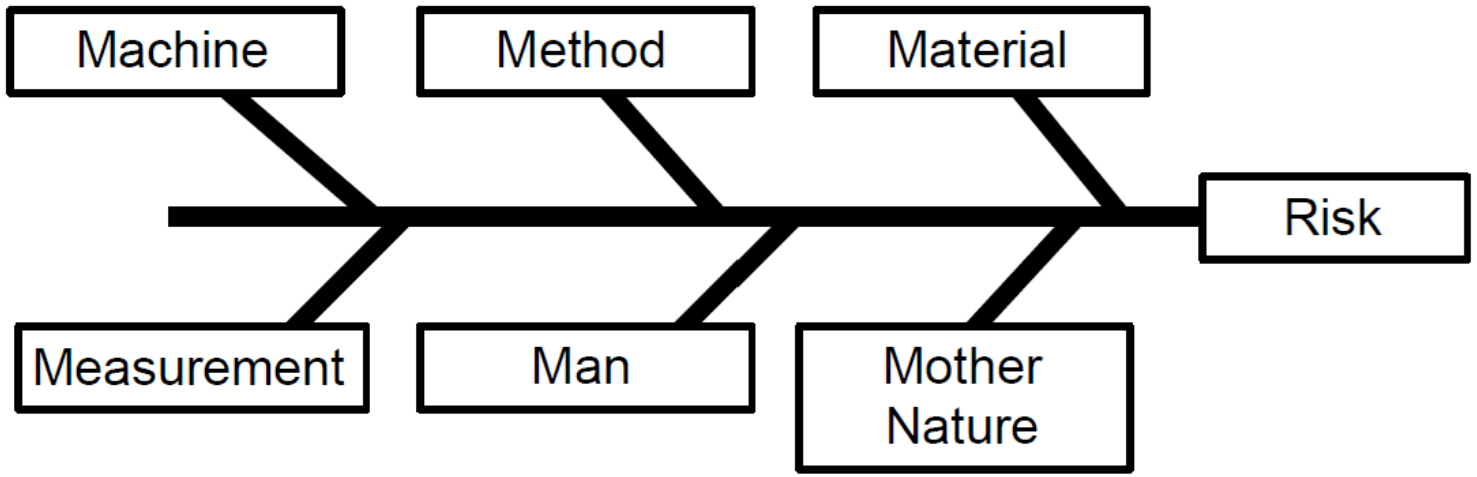
- Sedințe
- Analizarea documentelor
- Structuri de defalcare a riscurilor, liste de verificare, template-uri
- Analogii

Cele mai importante 10 cauze în eșecul unui proiect ale lui Boehm

- Scăderea personalului sau a subcontractanților
- Programul și bugetul nerealist
- Dezvoltarea unor funcții software / interfața cu utilizatorul greșite
- Gold Plating (greșirea priorităților)
- Controlul schimbării ineficient
- Riscuri tehnice

Tehnici de analiză a cauzelor de bază

- Diagrama cauza-efect (Ishikawa)
- Fault Trees / Failure Modes and Effect Analysis



Diagrame Fishbone - Puncte de pornire

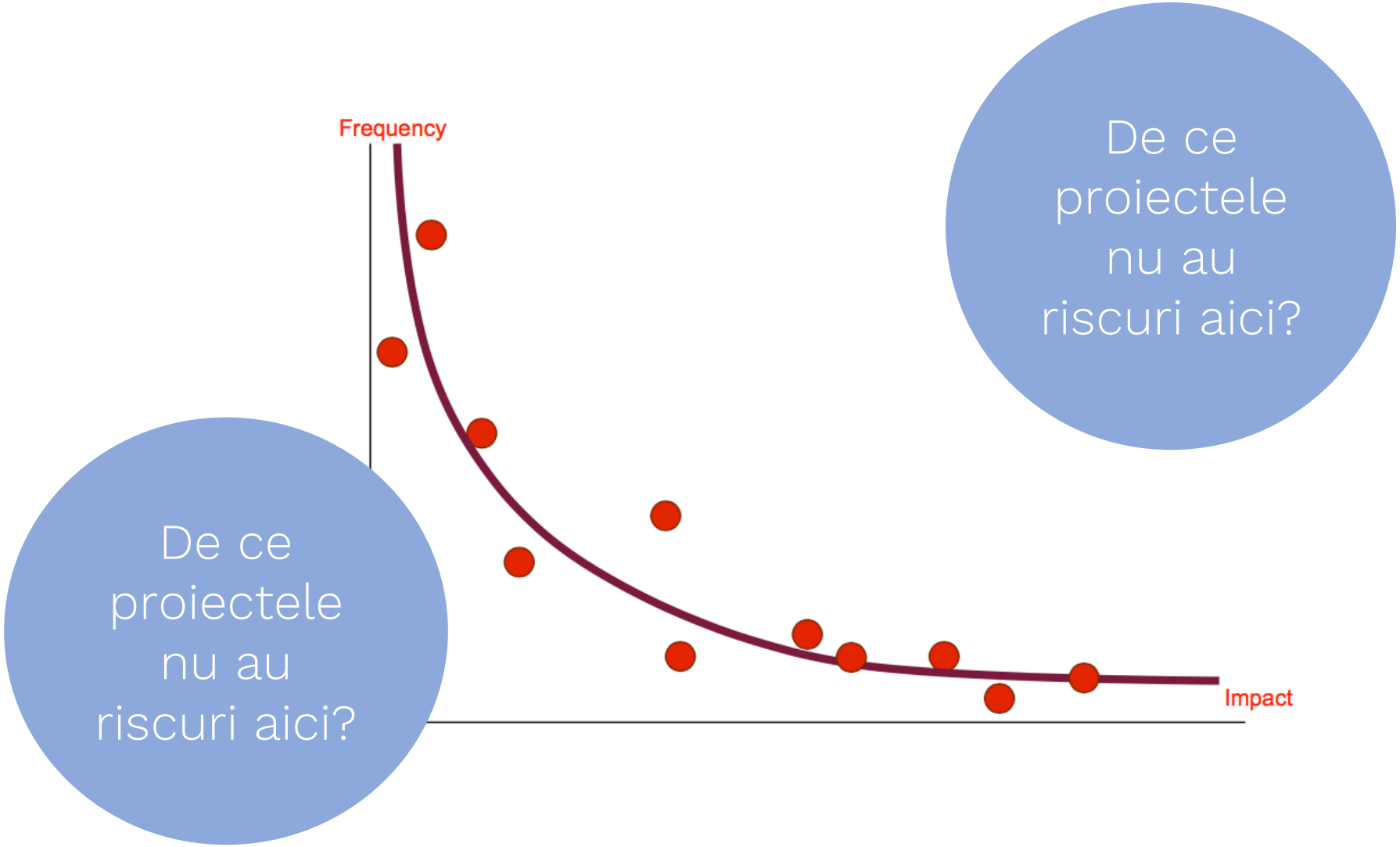
- Cei 6 M
  - Mașina, Metoda, Materialele, Măsurătorile, Man(Omul) și Mama Natura(Mediul de dezvoltare)
- Cei 8 P
  - Prețul, Promovarea, People(Oamenii), Procesele, Places(Locul), Politicile, Procedurile și Produsul
- Cei 4 S
  - Surroundings(Împrejurimi), Suppliers(Furnizori), Sisteme, Skills(Aptitudini)

Evaluarea riscurilor și strategiile de management al riscurilor

Evaluare a riscurilor

Scop prioritizați riscurile în funcție de impactul și asemănarea lor cu proiectul

Probabilitate / Impact



Tehnici

- **Analiza calitativă a riscului**
  - Mai simplu
  - Poate fi utilizată atunci când nu există informații exacte despre probabilitățile de risc
- **Analiza cantitativă a riscurilor**
  - Mai sistematică
  - Poate fi potrivită pentru analiza matematică
  - Furnizează cifre privind impactul economic al riscurilor

Analiza riscului calitativ

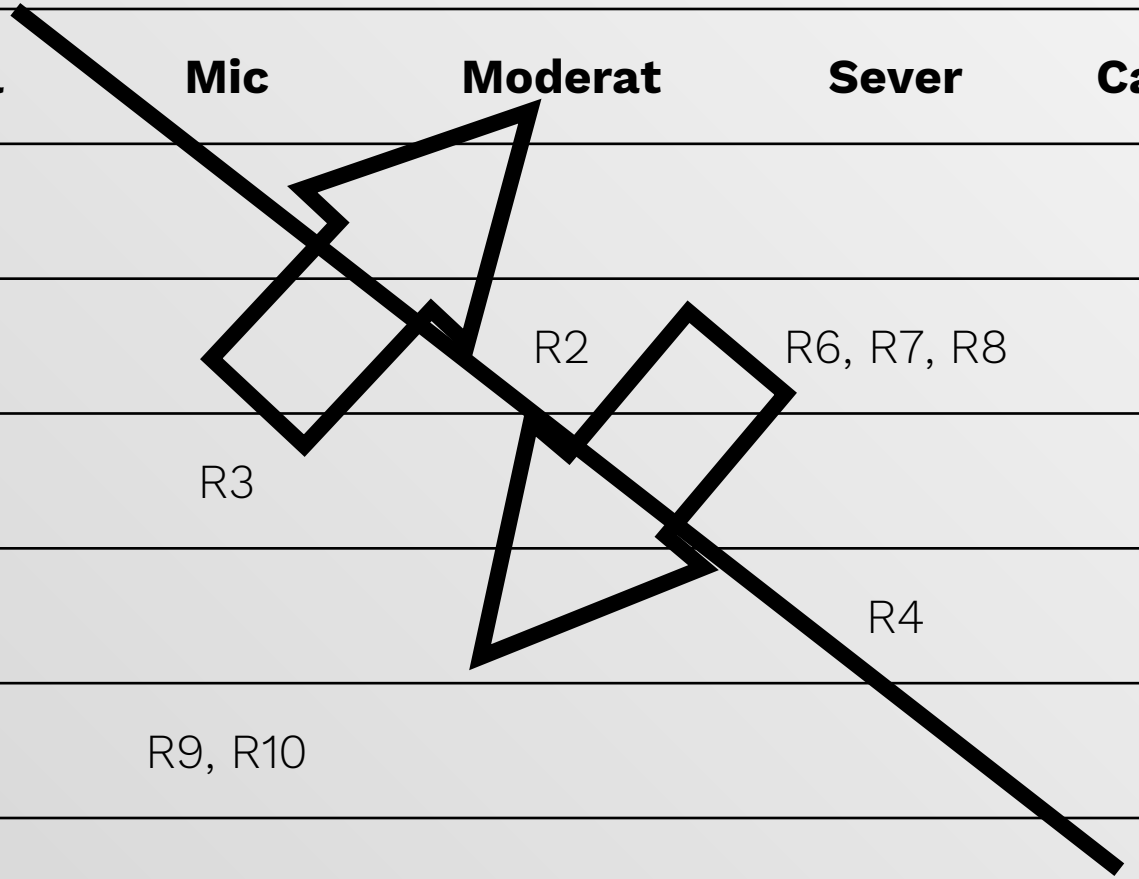
Matricea de risc

	Neglijabil	Mic	Moderat	Sever	Catastrofic
Foarte înalt	R1				R5
Înalt			R2	R6, R7, R8	
Moderat		R3			
Mic				R4	
Foarte mic		R9, R10			

Matricea de risc

Decidem unde vrem să tragem linia

	Neglijabil	Mic	Moderat	Sever	Catastrofic
Foarte înalt	R1				R5
Înalt			R2	R6, R7, R8	
Moderat		R3			
Mic				R4	
Foarte mic		R9, R10			



Matricea de risc

Roșu necesită tratament special (sau renunță la proiect)

Portocaliu Necesită monitorizare

Verde stardarul unui proiect

	Neglijabil	Mic	Moderat	Sever	Catastrofic
Foarte înalt	R1				R5
Înalt			R2	R6, R7, R8	
Moderat		R3			
Mic				R4	
Foarte mic		R9, R10			

Evaluarea riscului

- Defineste clasele de probabilități și clase de clase de impact calitative sau numerice
- Exemple
  - Probabilitate foarte scăzută, scăzută, moderată, înaltă, foarte ridicată
  - Impactul neglijabil, scăzut, moderat, sever, catastrofal
  - Scorul de risc scăzut, mediu, ridicat sau numeric: **SCORE = P x I**

Risc construit social

Două probleme cu risc calitativ:

- **Modelatori**: suntem “analfabeți în riscuri” (credem că anumite lucruri sunt mai riscante, deși statisticile spun altfel)

- **Modele**: cine spune ce sunt probabilitățile? Cum calculăm expunerile de risc în mod obiectiv?

Exemple de riscuri: Cauzele morții

- Boală de inimă: 597,689
- Cancer: 574,743
- Bolile cronice ale căilor respiratorii inferioare: 138,080
- Accident vascular cerebral (boli cerebrovasculare): 129,476
- Accidente (răni neintenționate): 120,859
- Boala Alzheimer: 83,494
- Diabet: 69,071
- Nefrită, sindrom nefrotic și nefroză: 50,476
- Gripă și pneumonie: 50,097
- Auto-vătămare intenționată (sinucidere): 38.364

Sursă: http://www.cdc.gov/nchs/fastats/lcod.htm

Strategiile managementului de risc

Strategia managementului de risc

Scop găsirea unei rezolvări pentru riscurile inacceptabile și deciderea asupra unei strategii pentru riscurile rămase, în cazul în care acestea apar pe durata proiectului

Scenariul

	Neglijabil	Mic	Moderat	Sever	Catastrofic
Foarte înalt	R1				R5
Înalt			R2	R6, R7, R8	
Moderat		R3			
Mic				R4	
Foarte mic		R9, R10			

Strategii – Amenințări

- Evitare schimbarea planului pentru a evita amenințările (mărește timpul pentru a realiza cerințele)

- Transfer deplasarea rezultatului către un terț; transferă responsabilitatea, dar nu elimină riscul (ex. asigurările)

- Diminuare reduce probabilitatea sau impactul (ex. prototipurile)

Strategii – Oportunități

- Exploatare eliminarea incertitudinii (atribuirea oamenilor talentați asigură o calitate mai bună)

- Distribuire atribuirea responsabilității de exploatare către un terț (parteneriate)

- Sporire modificarea mărimii unei oportunități prin creșterea probabilității și/sau un impact pozitiv

Strategii – Obișnuințe

- Acept
  - **Pasiv**: lăsarea echipei să se ocupe de riscuri
  - **Activ**: furnizarea unui buffer (timp, bani etc.)

  - De ce?
    - Probabilitate sau impact mic
    - Mai simplu să te ocupi de risc, în cazul in care el apare, poți planifica un răspuns în avans

Planificarea unui răspuns

- Plan:
  - Strategiile pentru tratarea riscurilor trebuie să fie concrete
  - Declanșatoare (elemente utilizate pentru a monitoriza dacă a apărut un risc)
  - Persoane responsabile de monitorizarea riscului
  - Persoanele responsabile de aplicarea planurilor pentru evenimente neprevăzute

Registrul riscurilor

- Fiecare risc este caracterizat de:
    - ID, Titlu, Descriere, Categorie
    - Probabilitate, Impact, Scor
    - Cauză de bază
    - Interval de timp
    - Modalități de monitorizare
    - Status (activ, inactiv)

Controlul si monitorizarea riscurilor

Controlul și monitorizarea riscurilor

- Input:
  - registrul de risc
- Proces:
  - analizarea abaterilor de la planul nominal
  - identificarea cazurilor
  - evaluarea corecțiilor
  - modificarea planului curent
- Gândire:
  - riscurile planificate trebuie tratate ca mai sus
  - riscurile neplanificate necesită întregul proces

Concluzii

Și principalul risc al ... managementului de risc

Erori comune

- **În timpul fazei de planificare:**
  - **Neidentificarea valorii maxime a riscului**
    * renunțarea la un proiect prea riscant
  - **Nescrierea unui plan balansat**
    * mărimea și complexitatea trebuie să fie la un nivel potrivit
  - **Interpretarea greșită a efectelor ca fiind cauze**
    * îngrijirea față de un eveniment greșit, netratarea actualei probleme

Erori comune

- **În timpul fazei de monitorizare:**

  - **Homeostazia riscului:** tendința de a ne asuma mai multe riscuri pe măsură ce proiectul devine mai sigur

  - **Ancore și cadre:** tendința de a rămâne la ancore și cadre cu privirea la oportunități sau la nevoia de a schimba cursul

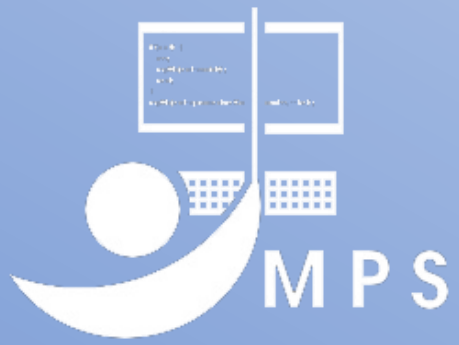
  - **Costuri de scufundare:** o ipoteza economică incorectă

  - **Disonanță cognitiva:** inconsecvențe neplăcute

Erori comune

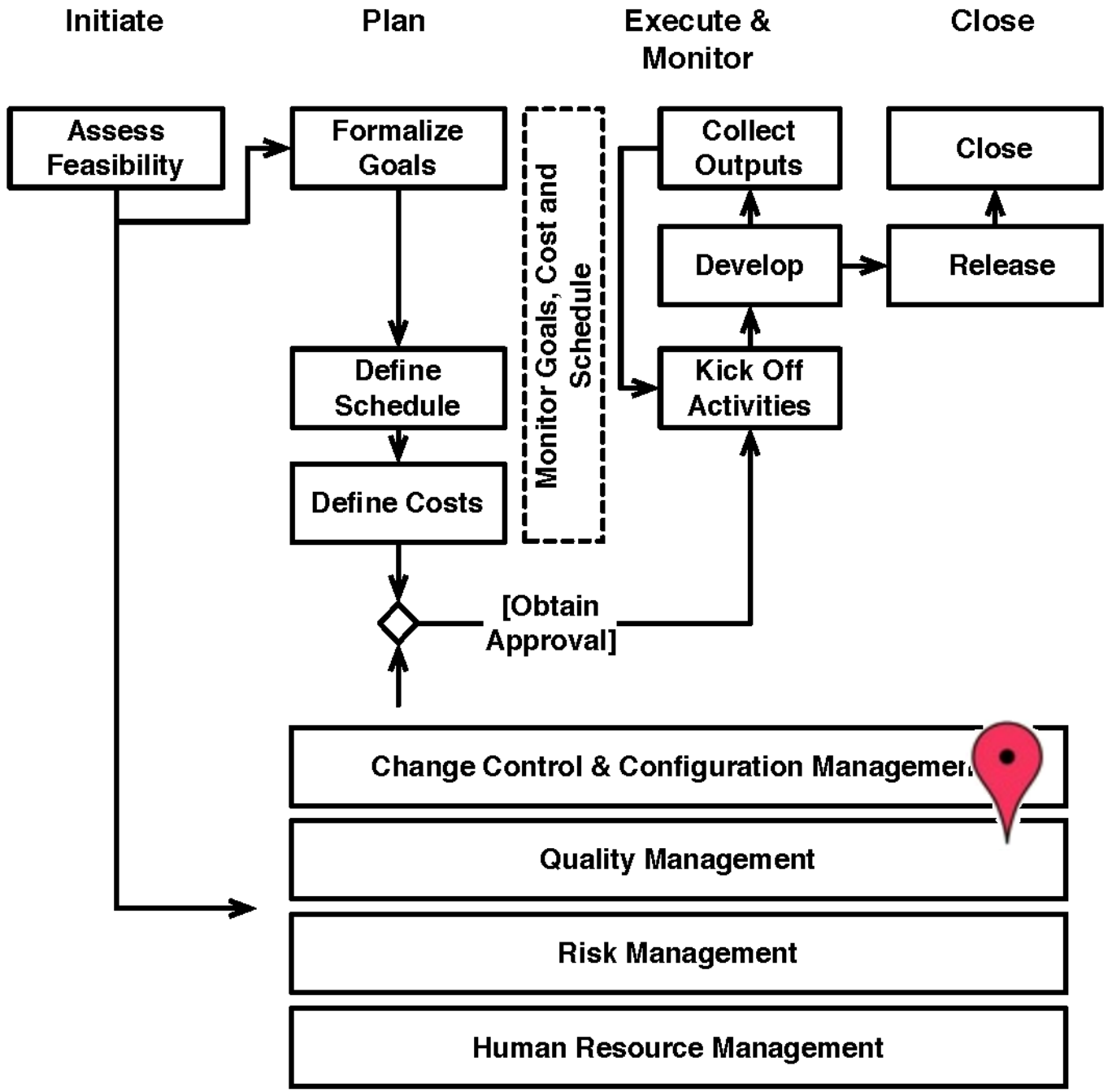
- **În timpul fazei de monitorizare:**
  - **Nu aplicați planurile de urgență** → utilizarea unor metode găsite în momentul apariției unui risc în loc de o strategie concepută în avans
  - **Nu implicați actori** → asigurați-vă că stakeholderii cunosc consecințele riscului, implicarea lor în rezolvarea lor
  - **Nu actualizați planul** → planul trebuie mereu adaptat la ultimele riscuri și condiții apărute

Managementul calității

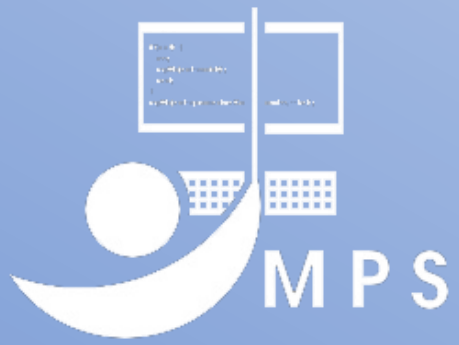


Obiective

- Înțelegerea imporatanței calității managementului în proiectele software
- Principalele tehnici pentru gestionarea calității în proiecte
- Pricipalele tehnici pentru gestionarea calității proiectelor livrabile
- Diferențele dintre testare software și managementul calității


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Asigurarea calității software



Asigurarea calității software

- Aplicarea planificată și sistematica a activităților pentru a asigura conformitatea ciclului de viata a proceselor și produselor la cerințele, standardele și procedurile cerute

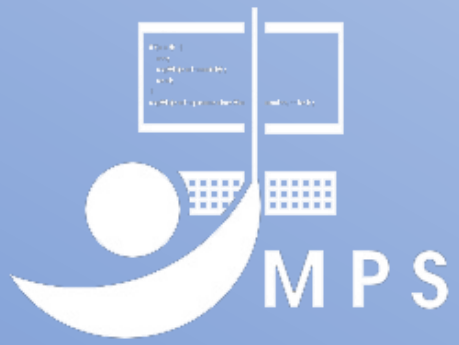
Observații

- Definiția se aplică atât proceselor cât și produselor
- Asigurarea calității este planificată și sistematică că multe alte activități
- Conformitatea este necesară, toate elemenetele caracterizând mediul software operațional

Procesul de asigurare a calității

- Planificarea calității – idenifica standardul și practicile relevante și modul în care le implementăm
- Asigurarea calității –pune accent pe asigurarea standardelor calității idenificate la pasul anterior
- Controlul calității –are rolul de a asigură că produsele respectă standardele calității identificate în etapă de planificare

Film



Planificare calitativă

- Scop: asigurarea îndeplinirii obiectivelor managementului calității într-un proiect
- Mijloace:

- Identificarea constrângerilor și obiectivelor de calitate în domeniul de aplicare
- Identificarea standardelor și procedurilor care trebuie aplicate - Identificarea tehnicilor care trebuie aplicate
- Alocarea resurselor (timp, oameni, buget) la activități de asigurare de calitate
- Roluri si responsabilități

- Output: document de planificare a asigurării calității

Observații

• Calitatea trebuie să fie echilibrată cu constrângerile celuilalt proiect (de exemplu, timp și costuri)

• Nu toate sistemele sunt la fel de critice: de exemplu, NASA distinge opt clase diferite de sisteme software

• Echipa de asigurare a calității trebuie să fie independentă de la echipa de dezvoltare

• Diferite "nivele" de independență:

- roluri diferite în proiect
- structuri diferite în cadrul organizației
- organizații independente

Asigurarea calității

- Obiectiv: asigurați-vă că proiectul aplică și urmează standarde de calitate
- Instrumentul principal: audituri de calitate
- Declanșator: timpul, etapele importante sau evenimentele critice din proiect (conform planului de calitate)
- Auditurile de calitate includ
  - Inspecții
  - Recenzii
  - Încercări
- Ieșire:
  - Principalele constatări și recomandări

Organizarea întâlnirilor de audit

• Au loc reuniuni de audit și revizuire pentru evaluarea statutului a unui produs sau a unui proiect

• Trei roluri "conflictuale" :

- Auditorii: analiza produselor și documentația proiectului
- Membrii proiectului, responsabili de furnizarea de clarificări și de explicarea alegerilor și stărilor proiectului
- Moderatorul, care asigură ordinea de zi și mediul de întâlnire rămâne productiv

Structura procesului de audit

- Definirea scopurilor și limitelor auditului
- Identificarea comitetului de audit (independență, competență, profesionalism)
- Distribuția întregului material relevant
- Pregătirea auditului de către auditori
- Întâlnire de audit
- Pregătirea raportului final

Semnele proiectelor problematice

- Potrivit datelor NASA, proiectele problematice includ:

- Schimbări frecvente ale etapelor importante

- Fluctuațiile inexplicabile ale personalului

- Întârzieri continue în livrarea de software

- Număr nerezonabil de rapoarte de neconformitate sau modificarea solicitărilor.

Controlul calității

- Scop: Asigurarea respectării standardelor de calitate propuse pentru produs în fază de planificare
- Metode folosite pentru asigurarea calității:
  - Inspecții
  - Analiză
  - Testare
- Factori declanșatori: milestone-uri sau evenimente critice (în funcție de planul de calitate)
- Rezultat: o listă cu rapoarte de neconformitate

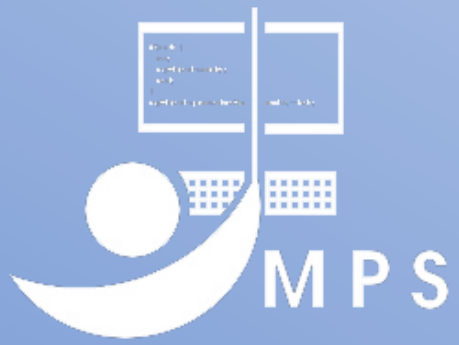
Controlul calității

- Controlul calității pentru sistemele software este greu deoarece:
  - Sistemele software se pot găsi în multe stări, deci testarea tuturor stărilor posibile este nepractică, sau chiar imposibilă
  - Mediul de operare este impredictibil
  - Discontinuitatea: orice mici schimbări în input-ul dat programului pot produce schimbări mari în output
  - Cerințele nefuncționale pot fi greu de stabilit
  - Automarea testelor poate fi foarte grea sau costisitoare (de exemplu testarea automată a unui GUI)
  - Sistemele din zilele noastre folosesc multe tehnologii diferite (HTML/CSS, Javascript, PHP, WebServer, OS)

Tehnici de control al calității pentru software

- Inspecții ale codului:
  - Sistemul este analizat de către o echipă independentă
- Analize:
  - Checkere statice care verifică folosirea corectă a diverselor construcții sintactice
  - Checkere dinamice care verifică diverse anomalii și situații suspecte executând cod instrumentat
  - Metrici ale codului sunt colectate pentru a măsură alte caracteristici de calitate (de exemplu procentul comentariilor în raport cu liniile de cod, acoperirea testelor unitare)
  - Verificarea formală (demonstrarea de teoreme, verificarea modelelor) arată proprietăți despre resprezentarea abstractă a sistemului
- Testarea:
  - Testele sunt derulate pe un sistem pentru a verifica comportamentul acestuia în diverse circumstanțe

Stabilirea unui program al metricilor



Stabilirea unui program de metrici

- Un **program de colectare a metricilor** apreciază cantitativ modul în care obiectivele proiectului sunt atinse
- Metricile procesului → măsoară diverse caracteristici ale proiectului
- Metricile produsului → măsoară diverse caracteristici ale unui produs al proiectului
- De obicei trendurile sunt mai importante decât numerele exacte
- Metricile sunt mai bune dacă sunt automatizate

Metricile unui produs - dimensiunea

- Metrici orientate pe dimensiune:
  - Numărul liniilor de cod (Source lines of code – SLOC)
  - Numărul claselor
- Metrici orientate pe funcționalitate:
  - Puncte funcționale (o “unitate de masură” care arată câtă funcționalitate oferă un sistem)
  - Puncte de obiecte (estimează efortul)
- Observații:
  - Metricile de dimensiune pot fi colectate automat
  - Numărătoarea făcută în cazul SLOC nu prea este consistentă
  - Metricile orientate pe funcționalitate necesită personal calificat care să le colecteze

Metricile unui produs - complexitatea

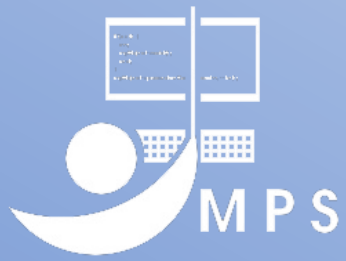
- Complexitatea ciclică
- Legăturile dintre obiecte
- Adâncimea în moștenire
- Fan-in, fan-out

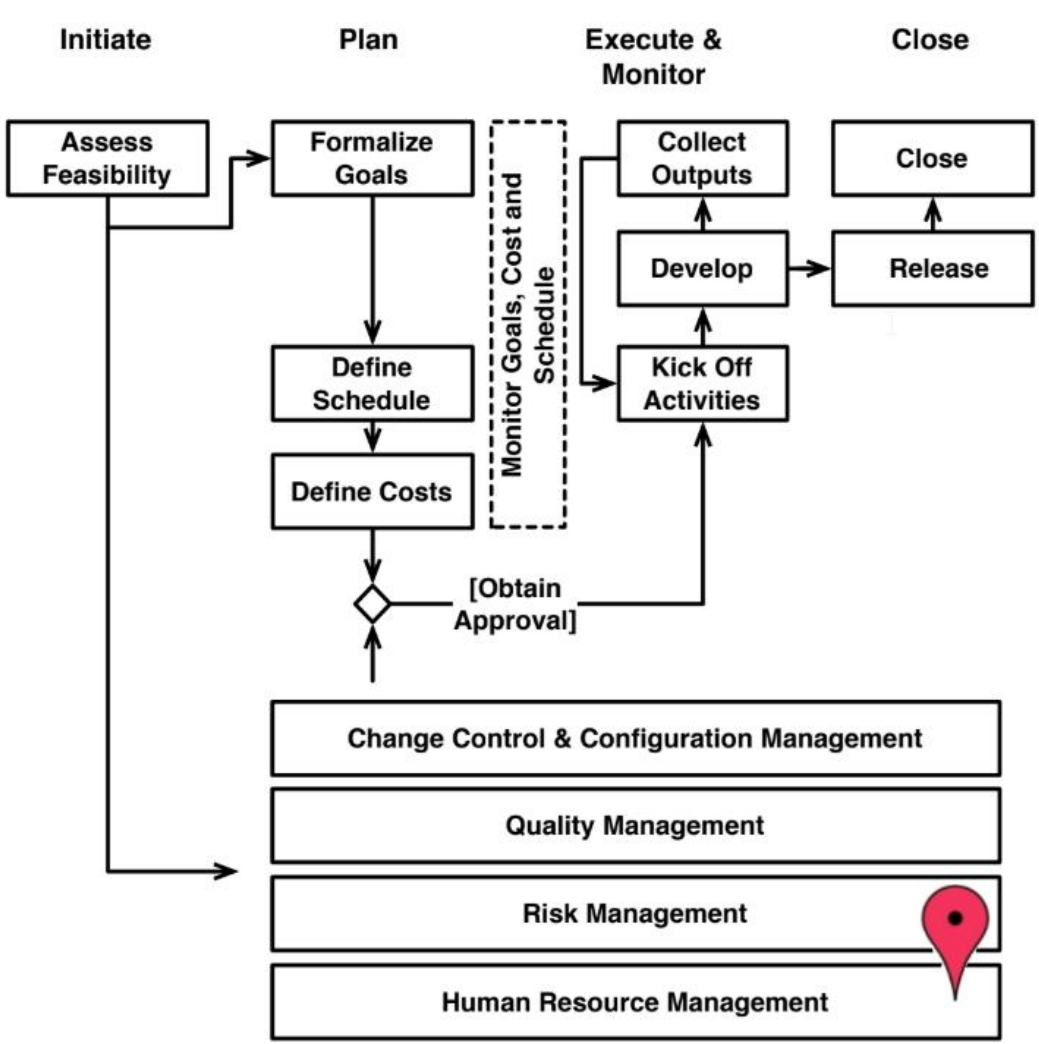
Metricile unui produs - metrici de calitate

- Raportul dintre numărul liniilor de comentarii și numărul liniilor de cod → indicativ al “mentenabilității” (maintainability) unui sistem
- Numărul de probleme care nu au fost încă rezolvate → arată dacă proiectul “converge”
- Densitatea erorilor: numărul de erori găsite în raport cu numărul de linii de cod -> ajută în a înțelege dacă în procesul de dezvoltare au apărut erori sistematice

Managementul oamenilor

Selectarea celor mai potriviți oameni și managementul acestora pentru lucrul individual și în echipă




Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Monitor Goals, Cost and Schedule
Collect Outputs
Close
Develop
Release
Define Schedule
Kick Off Activities
Define Costs
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

Procesul de selectare a Echipei de Proiect

Pas 1 - Definirea personalului necesar

Pas 2 - Alegerea persoanelor potrivite

Pas 3 - Organizarea staff-ului

Pas 4 - Punerea pe liber a personalului

Pas 5 - Profit

Definirea personalului necesar

- Scopul → identificarea skillset-ului necesar pentru realizarea proiectului
- Acțiunea → Traducerea și împărțirea proiectului în activități și competențe
- Exemple de criterii
  - Hard/soft skills
  - Experiența
  - Implicarea
  - Recomandări de la alți angajați

Skill Matrix

Activitate	Deprindere
Design	Creativitate Experiență în Photoshop
Baza de date	Cunoștințe bune de SQL
Front end development	Experiență în HTML&CSS Experiență în JavaScript
Back end development	Cunoștințe de PHP

Alcătuirea Echipei de Proiect

- Două abordări:
  - Internă
  - Externă
- Factori care influențează alegerea:
  - Disponibilitate
  - Încărcare curentă
  - Prioritatea proiectului
  - Politicile companiei

Alcătuirea Echipei de Proiect

- Intern
  - Tehnici:
    - Preemtiv: Angajații știu dinainte de startul proiectului că vor fi asignați pentru acesta;
    - Negociere: Staff-ul este “negociat” între managerii de proiect din cadrul companiei.
- Extern
  - Procesul:
    - Definirea și scoaterea la concurs a posturilor vacante;
    - Analizarea CV-urilor + organizarea unui interviu.
  - În colaborare cu departamentul de HR

Managementul oamenilor

Premise

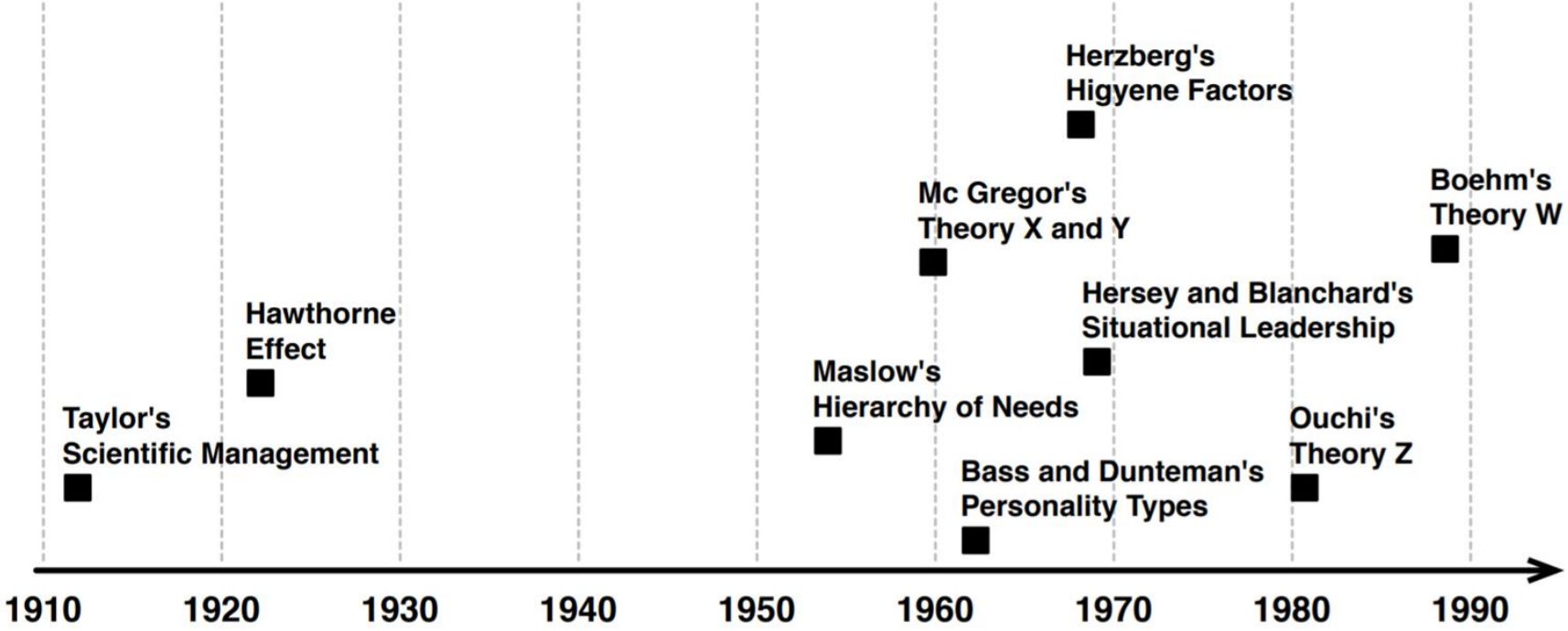
- Este dificil (dar important) de înțeles ce anume îi motivează pe oameni la locul de muncă
- Este foarte dificilă formularea unei teorii exacte, cu aplicabilitate generală

Studii principale în domeniu

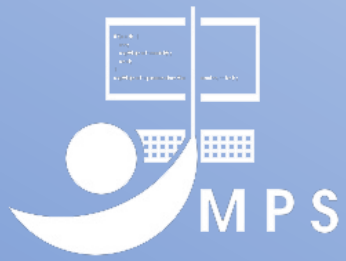
Categorii:

- Factori motivaționali
- Trăsături de personalitate și tipuri de interacțiune
- Strategii de management

Cronologie


Herzberg's
Higyene Factors
Mc Gregor's
Theory X and Y
Boehm's
Theory W
Hersey and Blanchard's
Situational Leadership
Hawthorne
Effect
Maslow's
Hierarchy of Needs
Taylor's
Scientific Management
Ouchi's
Theory Z
Bass and Dunteman's
Personality Types
1910
1920
1930
1940
1950
1960
1970
1980
1990

Factori motivaționali



Managementul științific (1911) - Taylor

- Obiective principale: îmbunătățirea **eficienței economice** și a **productivității muncii**
- Probleme expuse*:*
  - Muncitorii care trebuie să repete task-uri tind să lucreze în cel mai încet ritm care trece nepedepsit.
  - Nu te poți baza pe oameni pentru talent sau inteligență, toți muncitorii sunt motivați doar de bani.

Managementul științific (1911) - Frederick Taylor

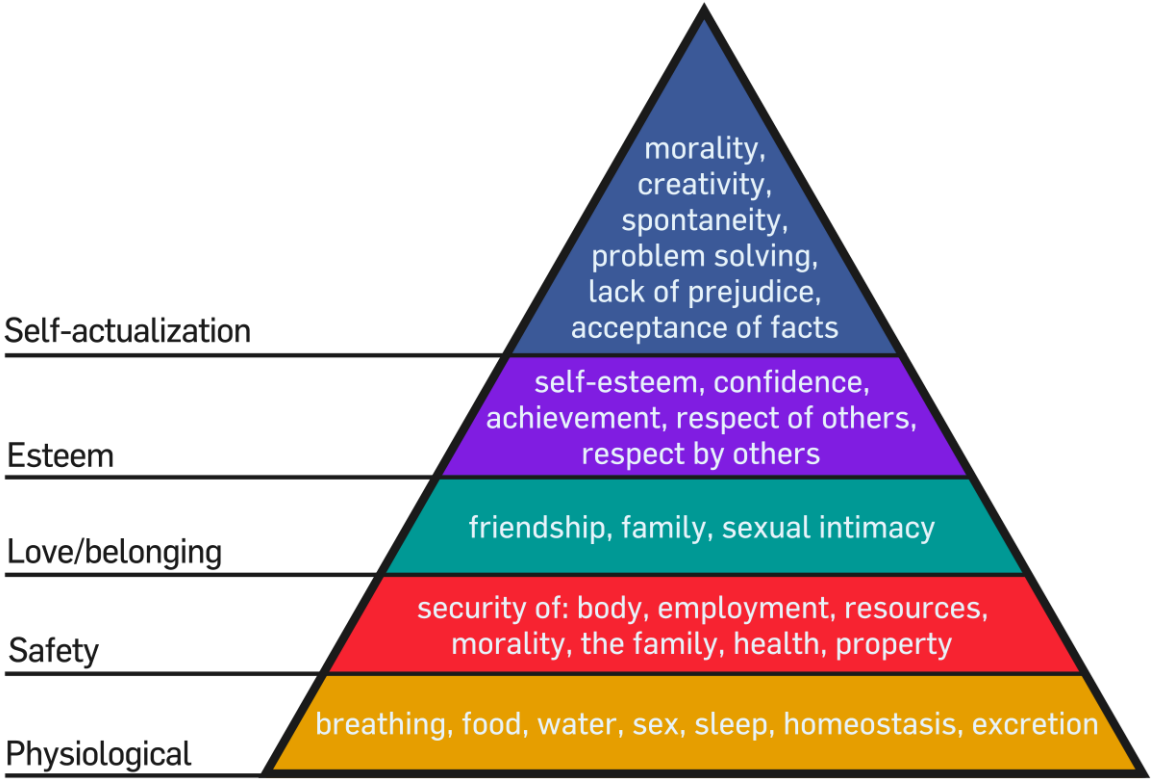
Soluții:

- Sistem de remunerare progresiv, în funcție de rezultate
- Selectarea angajaților în mod *științific* - instruire și perfecționare
- Pauzele = o necesitate pentru îmbunătățirea eficienței

Efectul Hawthorne (1924 - 1932) - Elton Mayo

- **Obiectiv inițial**: determinarea măsurii în care intensitatea luminii din fabrică are efect asupra productivității oamenilor
- **Rezultat**: Angajații și-au schimbat comportamentul (productivitatea a sporit) deoarece s-au simțit „sub lupă”, parte a unui experiment.
- **Concluzie**: Salariul și condițiile de lucru sunt factori mai puțin importanți decât recunoașterea importanței individului și apartenența, integrarea „în haită”

Piramida lui Maslow (1954)



Sursă foto: https://en.wikipedia.org/wiki/File:Maslow%27s_Hierarchy_of_Needs.svg

Piramida lui Maslow (1954)

- Ființa umană este motivată de o ierarhie de nevoi.
- Deficiency needs (D-needs) - urmare a lipsurilor:
  - Fiziologice
  - Siguranță
  - Dragoste / Sentimentul de apartenență
  - Stimă
- Growth needs (*being needs*: B-needs) - urmare a dorinței de creștere, împlinire
  - Dezvoltarea personală, autorealizarea
- Management bun → asigurarea ambelor tipuri de nevoi

Teoriile X și Y (1960) - McGregor

- **Teoria X** - Management *autocratic*, fondat pe idei precum:
  - Angajaților le displace munca, ar face orice să o evite
  - Angajații nu sunt ambițioși, nu doresc responsabilități
  - Ființele umane sunt lipsite de creativitate, au nevoie de instrucțiuni clare
  - Managerii trebuie să își controleze și să manipuleze angajații (principiul „biciul și zăhărelul”)
  - Satisfacerea angajatului → satisfacerea nevoilor fizice și de siguranță

Teoriile X și Y (1960) - McGregor

- **Teoria Y** - Management bazat pe *încredere*, fondat pe idei precum:
  - Munca este o activitate naturală, ca joaca și odihna.
  - Oamenii sunt în general foarte creativi.
  - Munca aduce împlinire omului.
  - Angajații devin responsabili atunci când se ține cont de părerea lor, sunt conștienți de sarcini.
  - Angajații sunt orientați către autorealizare și acceptă provocări.
  - Motivația vine din dorința de autorealizare și stimă de sine.

Teoriile X și Y (1960) - McGregor

Managerii aleg să folosească stilul de management corespunzător teoriei X sau teoriei Y în funcție de oamenii cu care au de a face.

„Cei mai mulți conducători acționează în concordanță cu teoria X, dar consideră că teoria Y este mai potrivită și reprezintă un ghid efectiv și eficient pentru acțiunea managerială.” - McGregor

Teoria Z - Ouchi (1981)

- Mix între stiluri de management americane și japoneze
- Principii de bază:
  - Pentru menținerea motivației, sunt necesare **obiective clare**.
  - **Motivația este esențială**.
  - **Chiar și oamenii motivați pot face greșeli**.
  - Task-urile trebuie să fie cât mai **standardizate** pentru o eficiență crescută.
  - **Obiectivele angajaților trebuie să se alinieze** cu nevoile companiei și cu condițiile de muncă.

Teoria W - Boehm (1988)

- Toate părțile interesate trebuie să aibă un câștig.
- Principii:
  - Înțelegerea a ce înseamnă conceptul de „câștig” pentru fiecare.
  - Crearea de așteptări realiste pentru fiecare proiect.
  - Identificarea clară a task-urilor bazată pe capacitățile fiecărui angajat.

Teoria factorilor duali a lui Herzberg (1968)

- KITA (Kick in the ass)
  - KITA fizic negativ
  - KITA psihologic negativ
  - KITA positiv

Teoria factorilor duali a lui Herzberg (1968)

- KITA (Kick in the ass)
  - KITA fizic negativ
  - KITA psihologic negativ
  - KITA positiv

NU FUNCȚIONEAZĂ!

Factorii duali

Factori MOTIVATORI (intrinseci, generatori de satisfacție)	Factori IGIENICI (extrinseci, generatori de insatisfacție)
Realizarea	Politicile companiei, administrarea
Recunoașterea	Supervizarea
Activitatea în sine	Relațiile interpersonale
Responsabilitatea	Condițiile de lucru & salariul
Avansarea	Statutul
Dezvoltarea profesională	Siguranța

!(SATISFACȚIE) ≠ INSATISFACȚIE

!(SATISFACȚIE) = LIPSA SATISFACȚIEI

!(INSATISFACȚIE) = LIPSA INSATISFACȚIEI

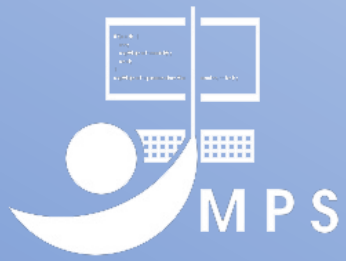
Teoria factorilor duali a lui Herzberg (1968)

Motivarea angajaților:

1. Eliminarea factorilor determinanți de insatisfacție (igienici)
2. Construirea factorilor determinanți de satisfacție (motivaționali)

Tipuri de interacțiune

Trăsături de personalitate



Trăsături de personalitate

- Oamenii sunt alcătuiți din trăsături de caractere diferite
- Echilibrul dintre aceste trăsături se modifică în funcție de circumstanțele personale și factorii externi
- Trăsăturile determină modul în care reacționăm la diverși stimuli
- Alegerea stimulului greșit poate da rezultate greșite

Bass and Dunteman (1963)

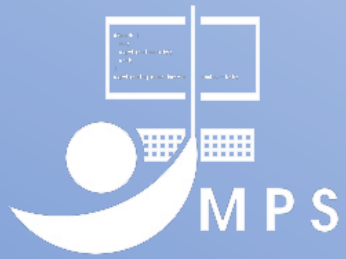
În mediul de lucru sunt definite trei tipuri de personalitate:

- **Task-oriented**
  - Plin de resurse, introvertit, competitiv, independent
  - Motivația de a lucra este lucrul însuși
- **Self-oriented**
  - Dezagreabil, agresiv, competitiv, introvertit, gelos, dogmatic
  - Câștig pe plan personal
- **Interaction-oriented**
  - Neagresiv, atent, fără nevoie de a se remarca sau de a câștiga
  - Motivația: acțiunile și rezultatele lucrului în echipă

Dr. David W Merrill and Roget Reid 1964 TRACOM™ Group 2004

- **Analitic**
  - Atent la detalii, înclină puțin spre risc, are nevoie de timp pentru a lua o decizie și vrea să aibă mereu dreptate
- **Amiabil**
  - Preferă lucrul în echipă și colaborările, nu ia decizii și nu îi place riscul, vrea să se simtă mereu în siguranță
- **Lider**
  - Intransigent, eficient, îi place riscul, vrea mereu rezultate vizibile
- **Creativ/expresiv**
  - Ambițios, entuziast, nu îi plac detaliile, îi place riscul

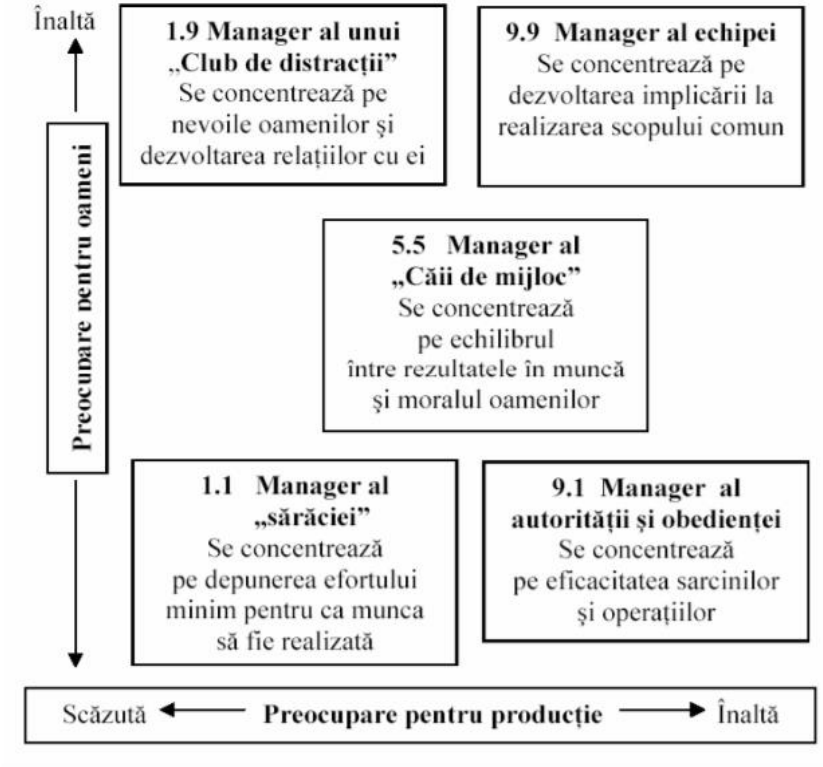
Strategii de management



Stiluri de management

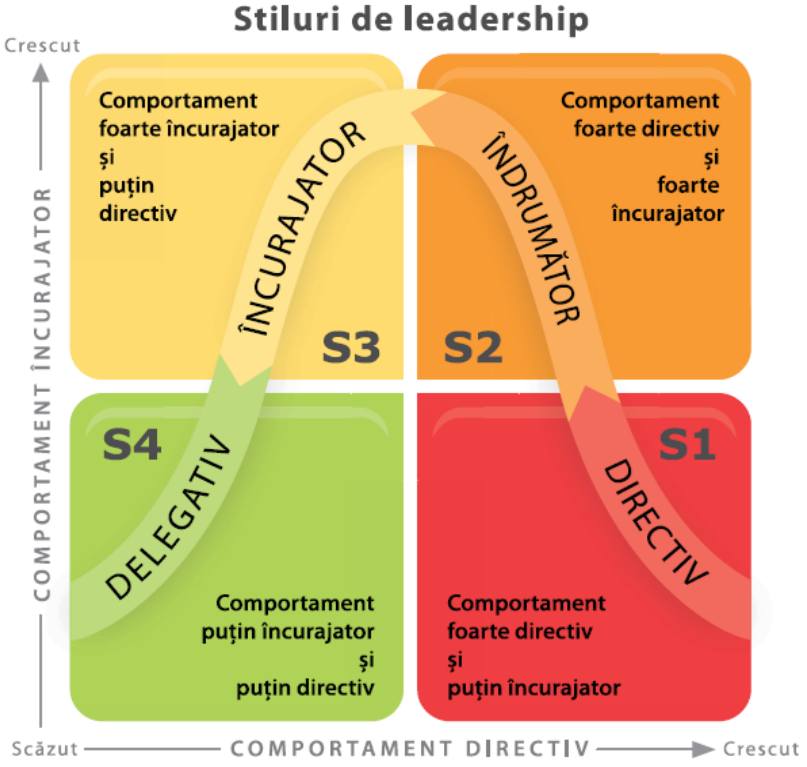
- **Autocratic**: managerul ia toate deciziile
- **Persuasiv**: managerul convinge subordonații să facă totul (tot autocratic)
- **Consultativ**: managerul implică echipa în procesul decizional, păstrând controlul asupra luării deciziilor
- **Democratic**: managerul permite echipei să ia parte în procesul decizional, alegând de comun acord
- **Laissez-faire**: managerul este mentor, acordând echipei libertate în luarea deciziilor

Modelul grilei manageriale
Robert Blake și Jane Mouton (1964)



Leadership situațional

Blanchard și Hersey (1969)



Structuri organizaționale

Ce influențează un proiect?

- Sistemul de management
  - Organizații bazate pe proiect
  - Organizații care nu sunt bazate pe proiect
- Cultura și stilul organizației
- Structura organizațională

Structura organizațională

- Necesară pentru a lucra eficient
- Nivelul de implicare al fiecărui participant depinde de modul de organizare
- Alegerea structurii adecvate este importantă.

Minim de informații

1.Cine este răspunzător de proiect?
2.Cum este împărțita informația intre participanții la proiect ?
3.Cum sunt remediate conflictele?

Tipuri de structuri organizatorice

- Ierarhică
- Matricială
- RACI
- Agile

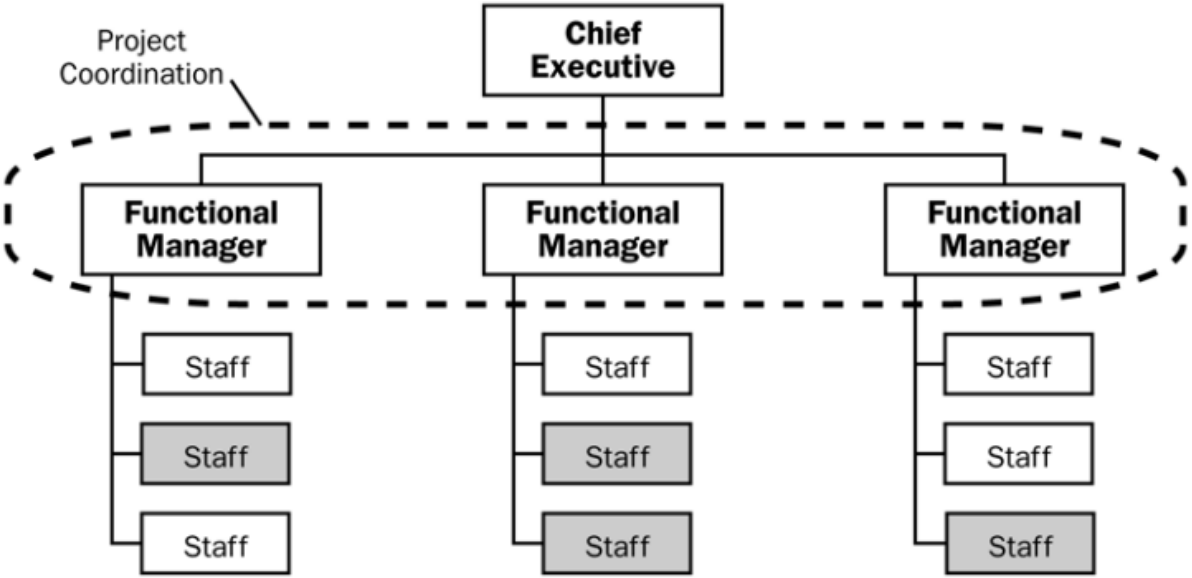
Exista 2 puncte de vedere:

- Structura organizației de care proiectul aparține
- Structura organizatorică aleasă pentru dezvoltarea proiectului

Structura ierarhică într-o organizație

- Fiecare participant are un superior clar stabilit
- Membrii personalului sunt împărțiți pe specializări
- Managerul de proiect are atât competente manageriale cât și tehnice pentru a administra proiectul
  - Variație: leadership-ul tehnic și administrativ sunt distincte.
- Deciziile sunt luate în vârful ierarhiei și propagate către celelalte departamente.

Structura ierarhică într-o organizație

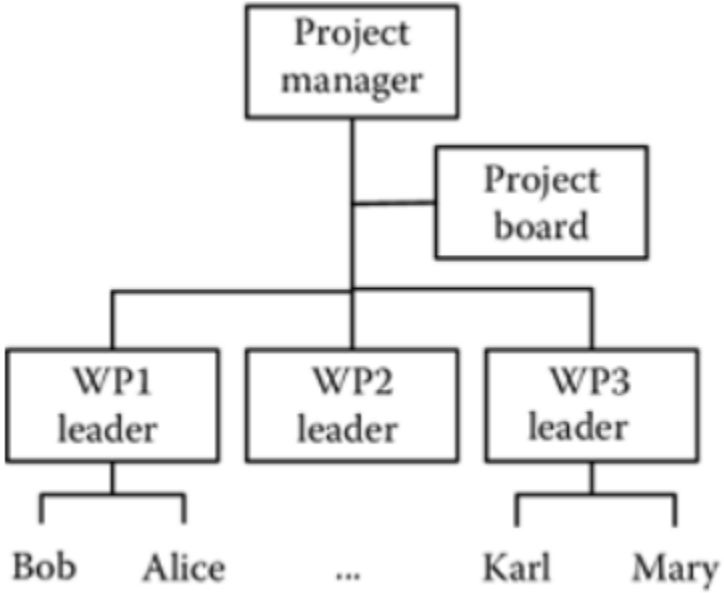


(Gray boxes represent staff engaged in project activities.)

Structura ierarhică într-un proiect

- PM are control asupra problemelor tehnice și manageriale
- Organizează munca în echipa de proiect.
- În proiectele mari se introduce un nivel intermediar: work-package leaders
- Comitet de management de proiect: compus din liderul fiecărui departament
  - Ajută la o mai bună comunicare
- Comitet de direcționare

Structura ierarhică într-un proiect



Structura divizională

- În cadrul organizațiilor mari
- Structura ierarhică nu mai face față
- Împărțire pe divizii
  - Concentrarea pe o activitate cu întreaga responsabilitate și autoritate
- Compartimentarea divizională se face pe baza produsului, ariei geografice, consumatorului sau, mai rar, procesului.

Structura divizională

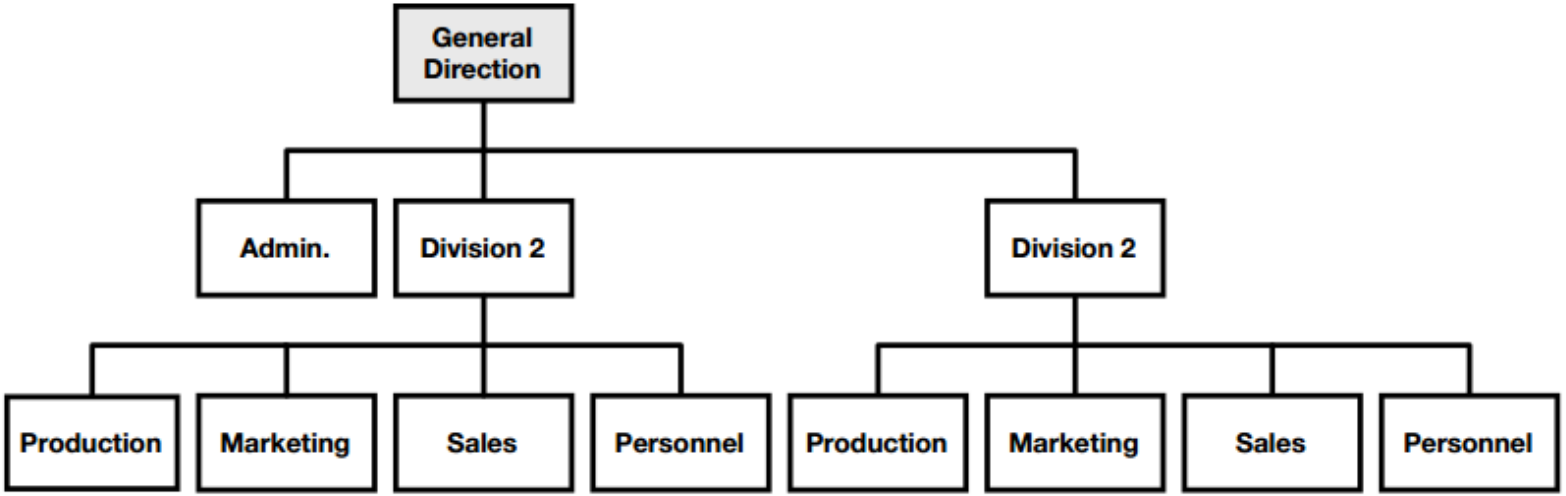
Avantaje

- concentrare asupra afacerii
- măsurare facilă a performanței
- focalizare asupra unei strategii a diviziei

Dezavantaje

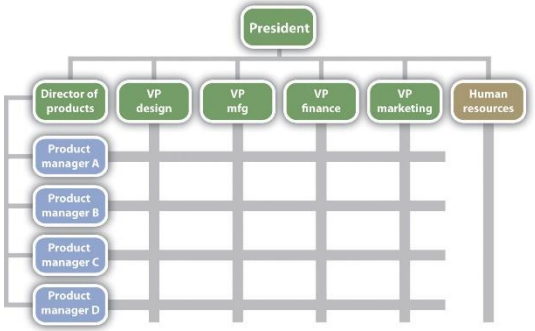
- conflicte între divizii
- costuri administrative ridicate
- complexitatea coordonării

Structura divizională



Structura matriceală într-o organizație

- Acest mod de organizare încearcă să găsească un echilibru între mai multe proiecte ce au nevoie de resurse funcționale comune.
- Funcționează cel mai bine în organizații cu multe proiecte mici ce se înscriu în tipare asemănătoare.
- În proiecte foarte mari în schimb, există posibilitatea de formare a unor grupuri, atunci când anumite necesități sunt acaparate de un anumit proiect, pierzându-se practice scopul acestei organizări.

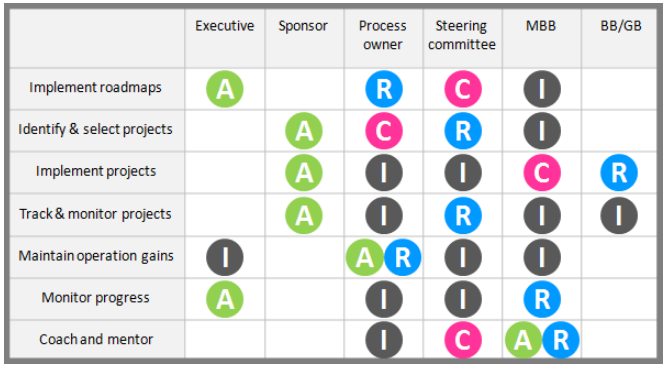


Structura matriceala într-un proiect

- Membrii sunt împărțiți în mai multe grupuri după funcția pe care o îndeplinesc
- Asemănător ca în cazul organizației, fiecare parte a proiectului va prelua după necesități membrii care să îndeplinească funcțiile necesare
- Este o soluție bună atunci când necesitățile bucăților din proiect sunt în continuă schimbare, drept pentru care resursele trebuiesc redistribuite
- Un dezavantaj provine din luarea de decizii și managementul indivizilor care vine din mai multe linii

Structura matrice RACI

- Această structură descrie responsabilitățile indivizilor sau rolurilor în cadrul unui proiect.
- Acestea sunt:
  - Responsabilitate
  - Răspundere
  - Consultare
  - Informare
- Specifică clar modul în care informațiile circulă, cât și modul în care deciziile sunt luate.



	Executive	Sponsor	Process owner	Steering committee	MBB	BB/GB
Implement roadmaps	A		R	C	I	
Identify & select projects		A	C	R	I	
Implement projects		A	I	I	C	R
Track & monitor projects		A	I	R	I	I
Maintain operation gains	I		A R	I	I	
Monitor progress	A		I	I	R	
Coach and mentor			I	C	A R	

Structura matrice RACI in proiect

- În cadrul unui proiect se face distincția între rol și individ:
  - Un individ poate avea mai multe roluri
  - Mai mulți indivizi pot avea același rol
- Fiecare task individual trebuie să aibă:
  - Cel puțin un rol R, dar cel mult un rol A
  - Este de preferat să nu existe sloturi goale într-o matrice RĂCI, dar în același timp să nu se exagereze rolurile (Consulted în loc de Informed de ex.)

Project Deliverable (or Activity)	Project Manager	Strategist	Designer	Front End Developer	Back End Developer
Design site map	C	R	A	I	I
Design wireframes	C	A	R	I	I
Create style guide	A	C	R	C	I
Code templates	A	I	C	R	C

Agile basics

- Ce este?
- Waterfall vs Agile
- Valorile și practicile Agile:
  - Indivizii și interacțiunile au întâietate asupra proceselor și instrumentelor
  - Software funcțional înaintea documentației complexe
  - Colaborarea cu clientul deasupra negocierilor contractuale
  - Răspunsul la schimbare înaintea urmăririi unui plan strict

© Scott Adams, Inc./Dist. by UFS, Inc.

Echipe Agile

- Echipe mici și dedicate
- Grup "cross-funcțional" de oameni
- Se preferă 5-9 oameni/echipă
- Rotație de roluri și funcții

Scrum

- Ce este?
- Caracteristici workflow Scrum:
  - Planificarea sprintului
  - Discuții interne zilnice
  - Demonstrație sprint
  - Retrospectivă sprint
- Roluri esențiale echipă Scrum:
  - ScrumMaster-ul
  - Clientul
  - Echipă

Pair-Programming

- Ce este?
- De ce este adoptată această tehnică?
- Beneficii:
  - Calitate
  - Integrare
  - Mentorat și învățare
  - Flexibilitate

Adantaje și dezavantaje echipe Agile

Avantaje

- Vizibilitate continuă
- Testare continuă
- Livrare rapidă
- Planificare efectivă pe termen scurt

Dezavantaje

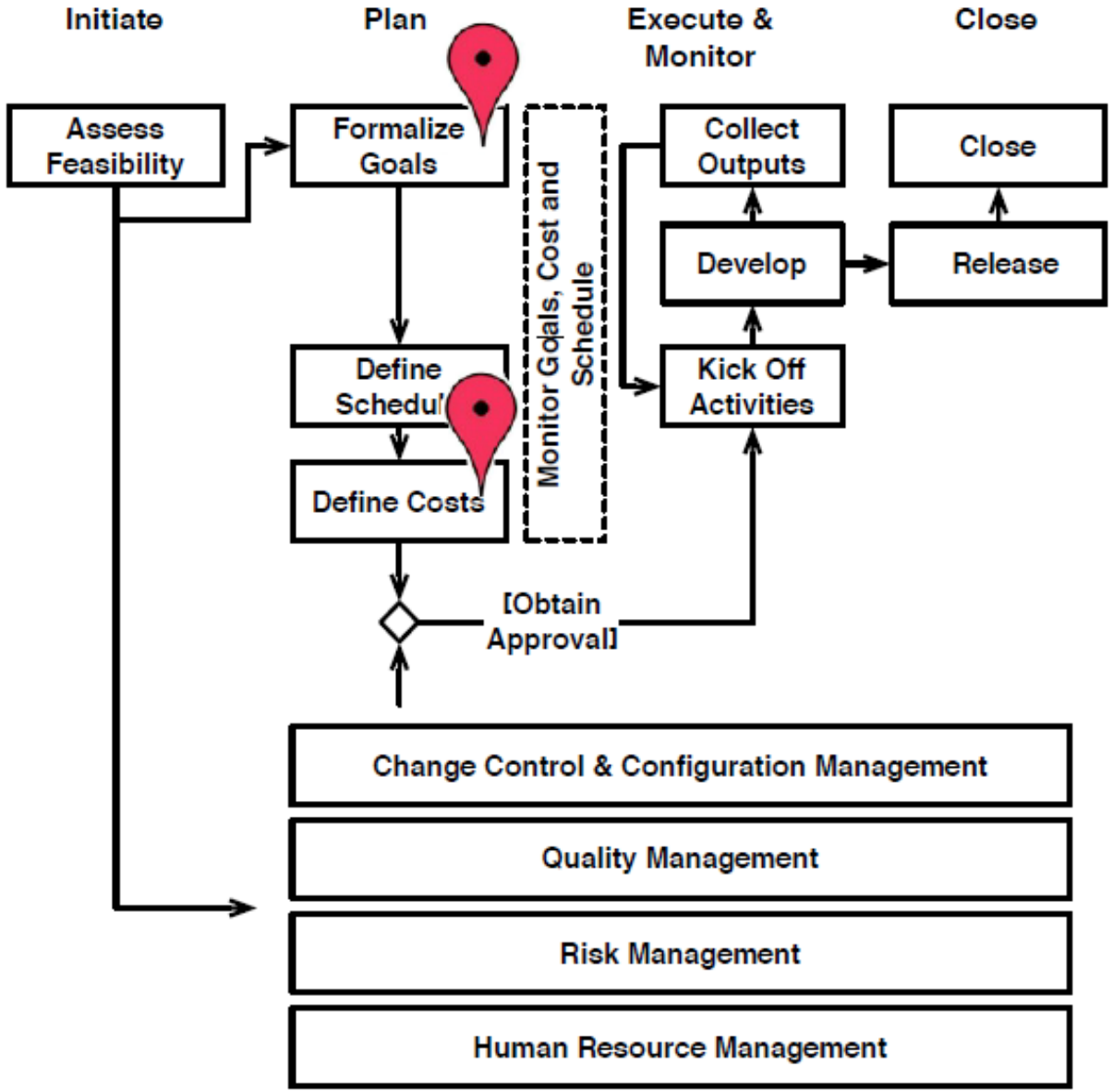
- Lipsă clarității poate duce la probleme
- Sunt necesare skilluri avansate
- Training insuficient
- Prea multă ambiție

Stabilirea prețurilor proiectului

Scop

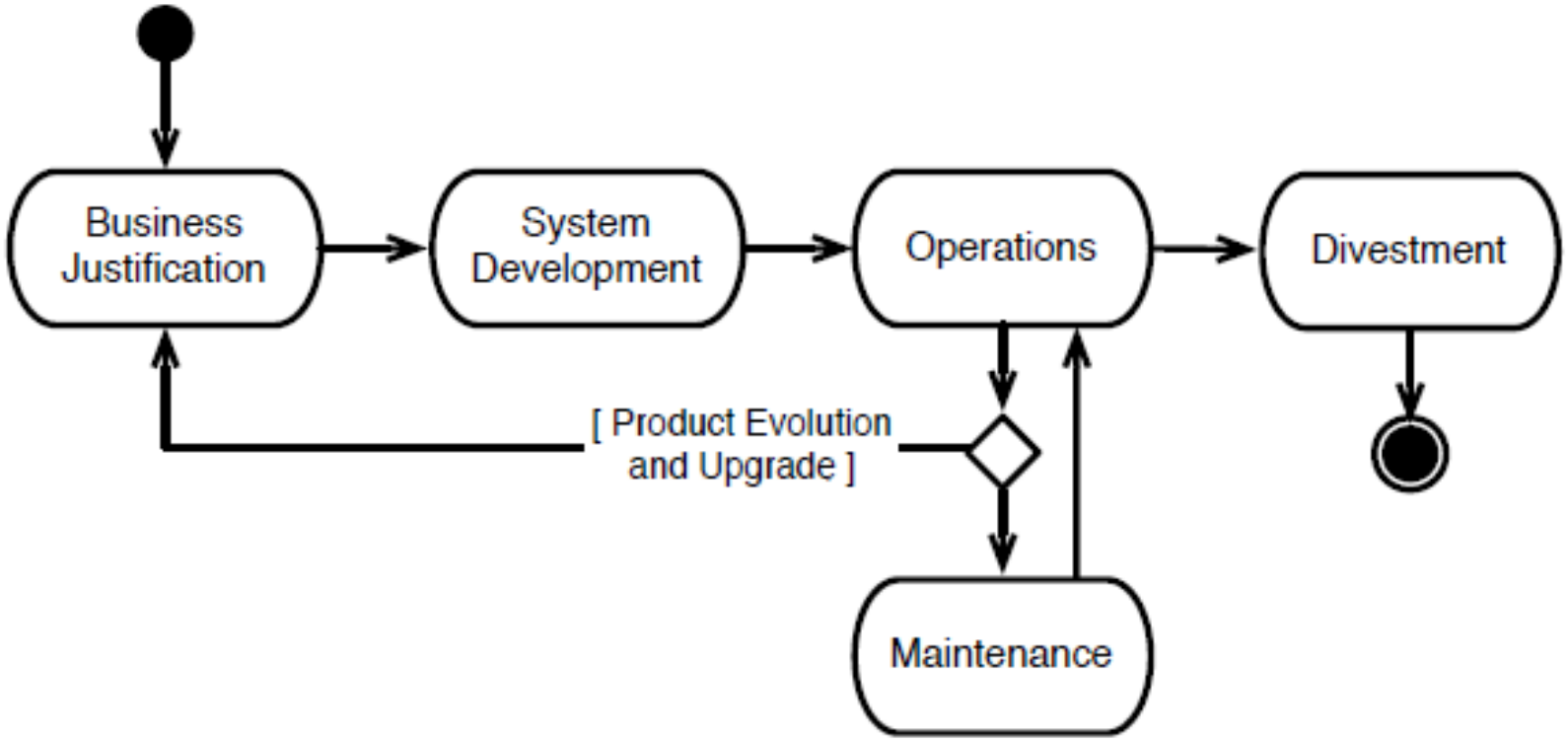
Cum determinăm costul rezultatelor proiectului.

Înțelegerea procesului de achiziție.


Initiate
Plan
Execute & Monitor
Close
Assess Feasibility
Formalize Goals
Define Schedul
Define Costs
Monitor Goals, Cost and Schedule
Collect Outputs
Develop
Kick Off Activities
Close
Release
[Obtain Approval]
Change Control & Configuration Management
Quality Management
Risk Management
Human Resource Management

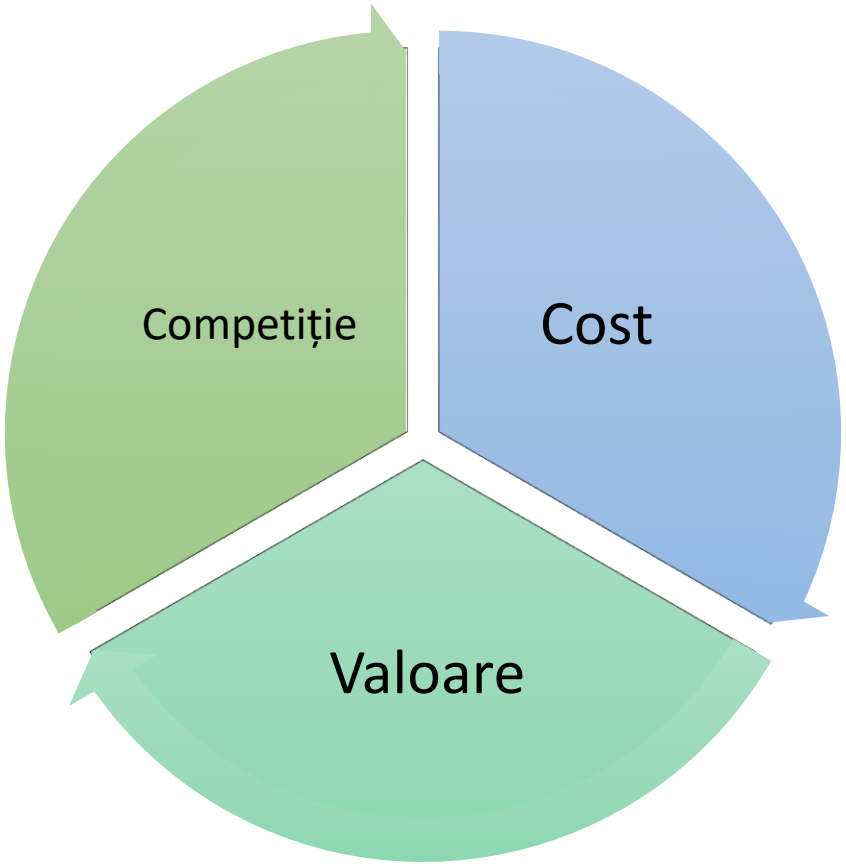
Durata de viață a unui proiect

Toate fazel



Modele de stabilire a prețului

Prețurile software-ului



Vânzare, licențiere, leasing

Deținerea codului sursă sau a executabilelor reprezintă un factor important în stabilirea prețului.

Software-ul este, de regulă, sub licență.

Leasing: folosirea temporară a produsului.

Software gratuit și Open Source

Caracteristici:

- Drepturi de folosire
- Datorii

Exploatare comercială:

- Instalare
- Pregătire
- Documentație
- Gadget-uri
- Donații

Sursă: https://tableplus.io/assets/images/open-source/open-source-community.png

Stabilirea prețului pentru proiect

Acorduri contractuale

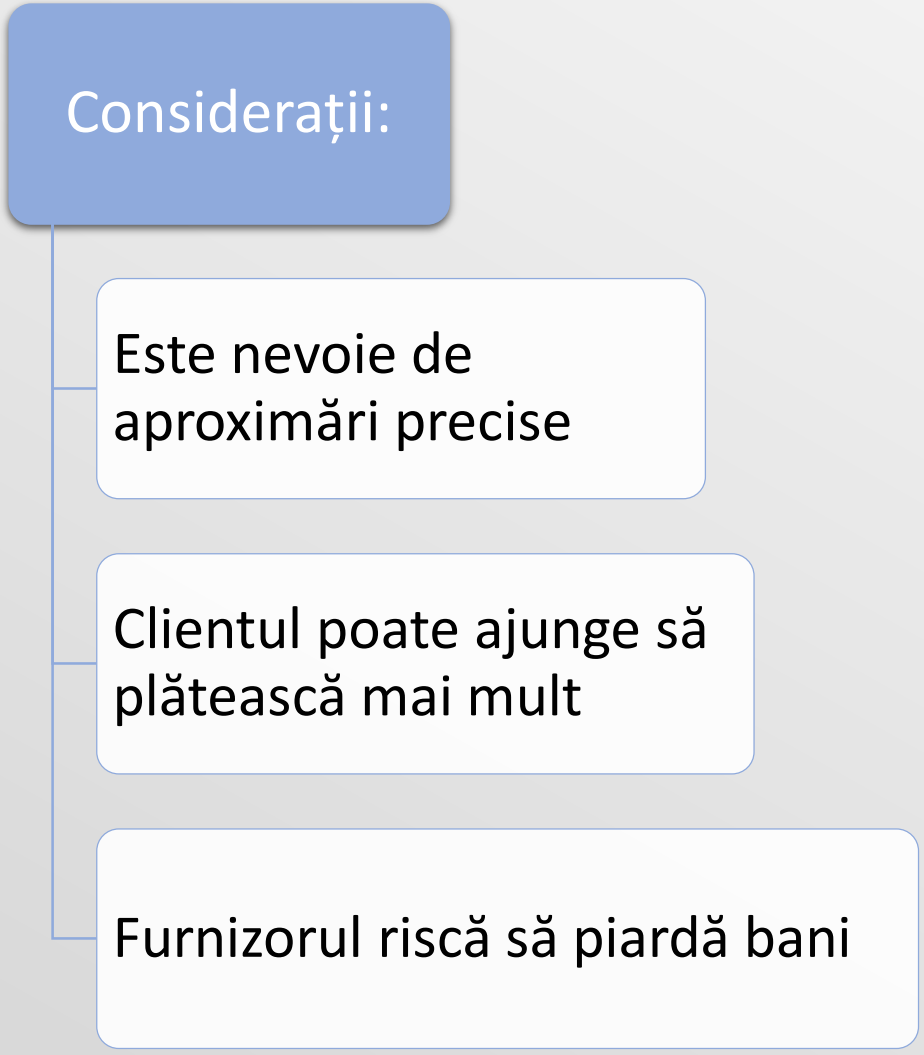
01

Clientul și contractorul au scopuri diferite pentru stabilirea prețului și planificarea plăților

02

Contractele reglementează aceste scopuri.

Contracte cu preț fix



Timp și materiale

- Furnizorul expune costurile către client și facturează conform costurilor efective.
- Considerații:
  - Se ține evidența timpului consumat
  - Risc de ineficiență

Reținere de fonduri

- Preț fix plătit către furnizor în schimbul unor servicii oferite mai târziu
- Considerații:
  - Potrivit dacă cererile nu sunt clare
  - Echivalent cu angajarea pe o perioadă determinată a forței de muncă
  - Plățile pot fi împărțite pe perioade fixe (lunar)

Cost Plus

Prețul de vânzare este format din prețul individual + o adiție specifică.

Tipuri:

- Contracte cu recompensă
- Contracte cu stimulent
- Contracte cu onorarii fixe

Considerații:

- Împarte riscul între client și furnizor
- Controlul costului suplimentar poate fi mai dificil

Acorduri contractuale și bugetul proiectului

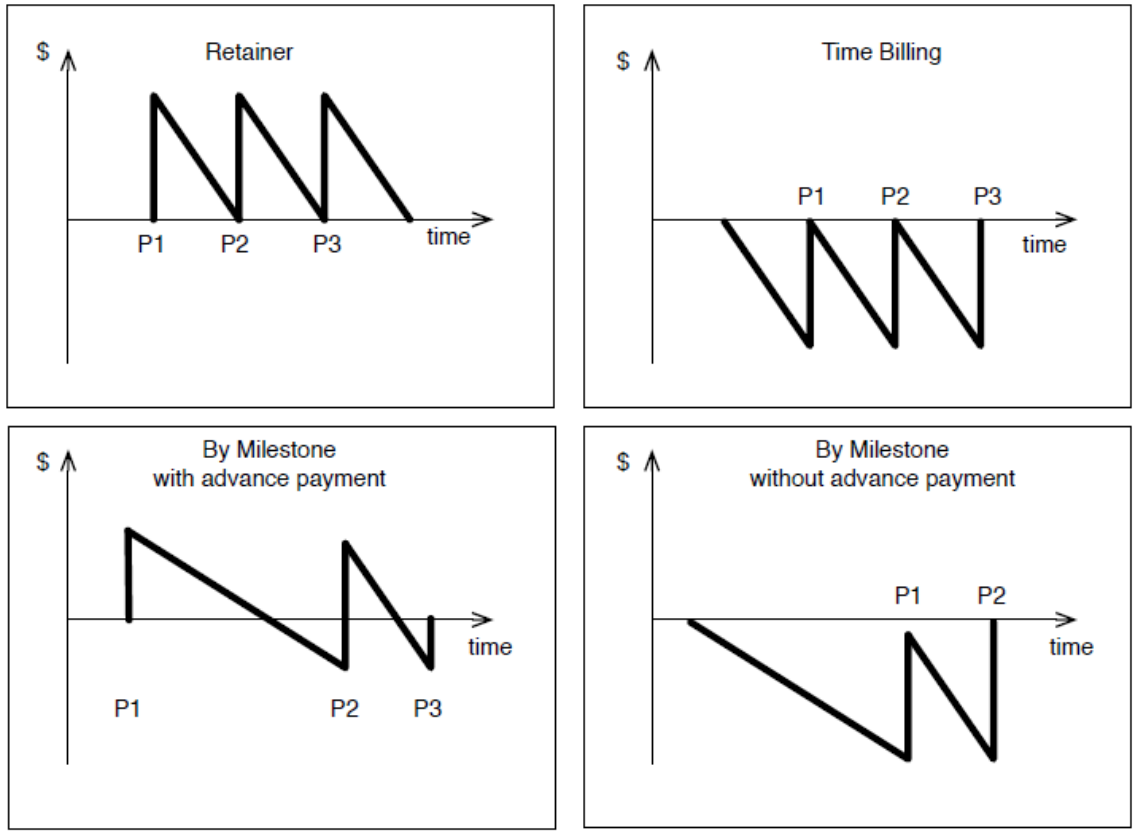
Intervalele de plată trebuie să fie reglementate pentru a echilibra riscul și implicarea

Din punct de vedere pur economic, costul de finalizare determină dacă are sens continuarea proiectului

Structuri de plată

Plățile și fluxul de bani

Nivelul de implicare al furnizorului depinde de structura plății



Achiziții și delegare

Procesul de achiziție

Multe proiecte au nevoie de servicii sau produse de la alți vendori

Trebuie alocat un timp suficient pentru achiziții și resurse suficiente pentru a monitoriza procesul.

Procesul de achiziție

Activități :

- Identificarea necesităților
- Identificarea și selectarea vendorilor (solicitare, selecție, recompensare)
- Administrarea îndeplinirii contractului
- Acceptarea produsului final

Invitație la licitare

Trebuie să cuprindă:

- Specificarea produselor și serviciilor oferite
- Constrângeri
- Modalități de propunere și criteriile de selectare ce vor fi folosite pentru îndeplinirea contractului

BOTSWANA
Qualifications Authority

INVITATION TO TENDER

Tender No BQA PR 48/18

Botswana Qualifications Authority (BQA) invites interested companies to tender for the **SUPPLY, DELIVERY AND INSTALLATION OF OFFICE FURNITURE.**

REQUIRED PPADB CODES:

- Code: 213 General Manufacturers/Producers
  - Sub Code: 06 Furniture

Sursă: https://www.bqa.org.bw/sites/default/files/news-image/screen_shot_2018-09-03_at_10.35.59_am.png

Solicitări

Ce poate fi făcut depinde de constrângerile legale și organizaționale.

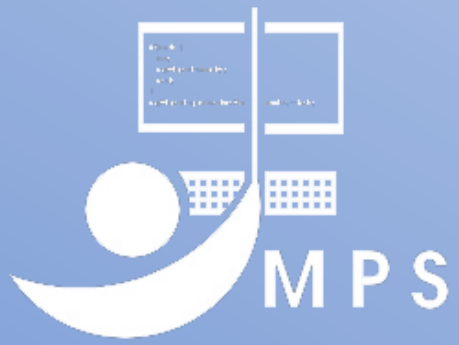
Invitația la licitație este făcută publică prin mai multe modalități

Motivație: Gestionarea compromisului dintre preț, calitate și timp

Considerente temporale ale licitațiilor deschise

- Trebuie să se țină cont de durata următoarelor activități:
  - Pregătirea invitației pentru licitație
  - Luarea la cunoștință a ofertei de către potențialii clienți
  - Evaluarea propunerilor
  - Acordarea contractului
  - Livrarea produselor sau serviciilor

Procese pentru Dezvoltarea Software



Procese pentru dezvoltarea software
Motivație

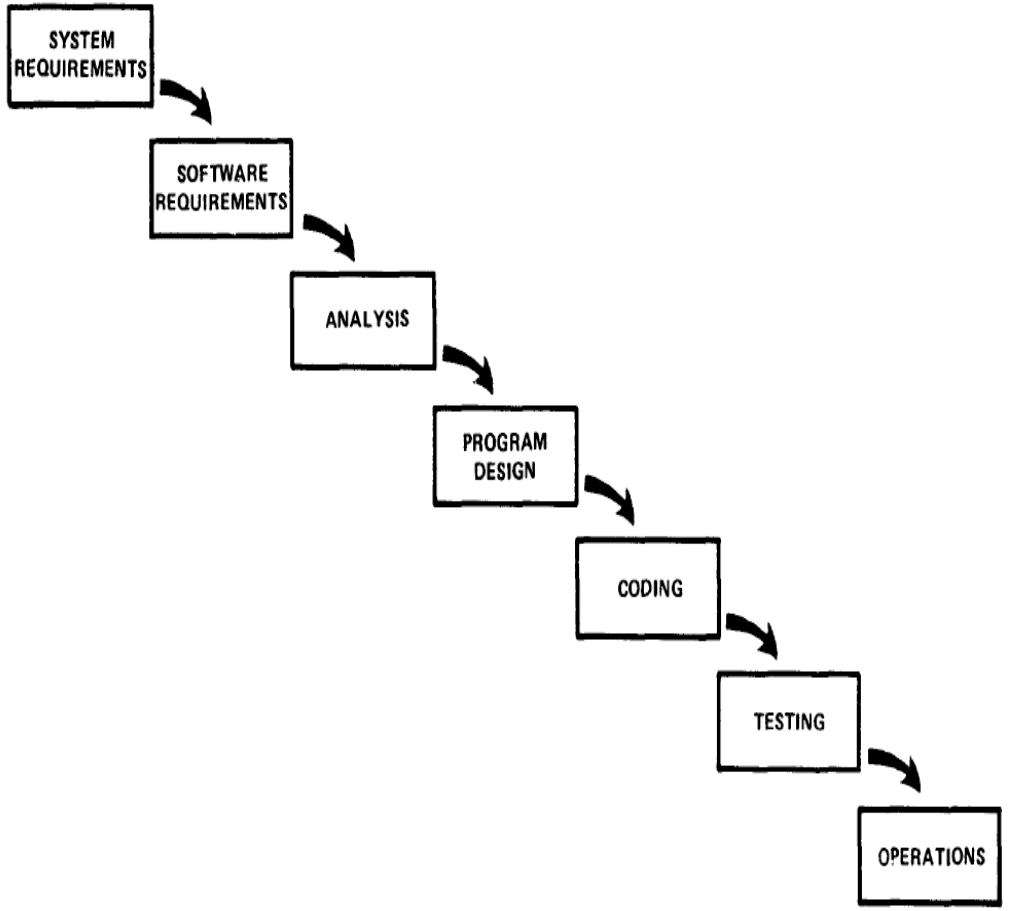
- Noi procese de dezvoltare software au fost introduse pentru a aduce o serie de îmbunătățiri la modelul clasic de dezvoltare
- **Procese de Dezvoltare Software** → model structurat și progresiv de la o idee la aplicația finală
- Îmbunătățiri:
  - Mai multă flexibilitate în procesul de dezvoltare
  - Satisfacția utilizatorului (aplicațiile sunt mai appropriate de cerințele clientului)
  - Eficientă
  - Timp și costuri
  - Calitate

Modele de dezvoltare

- Waterfall
- Prototype
- Incremental
- Spiral
- V-Model
- Cleanroom
- RUP
- XP
- SCRUM
- RAD
- ASAP
- MSF
- DFDM

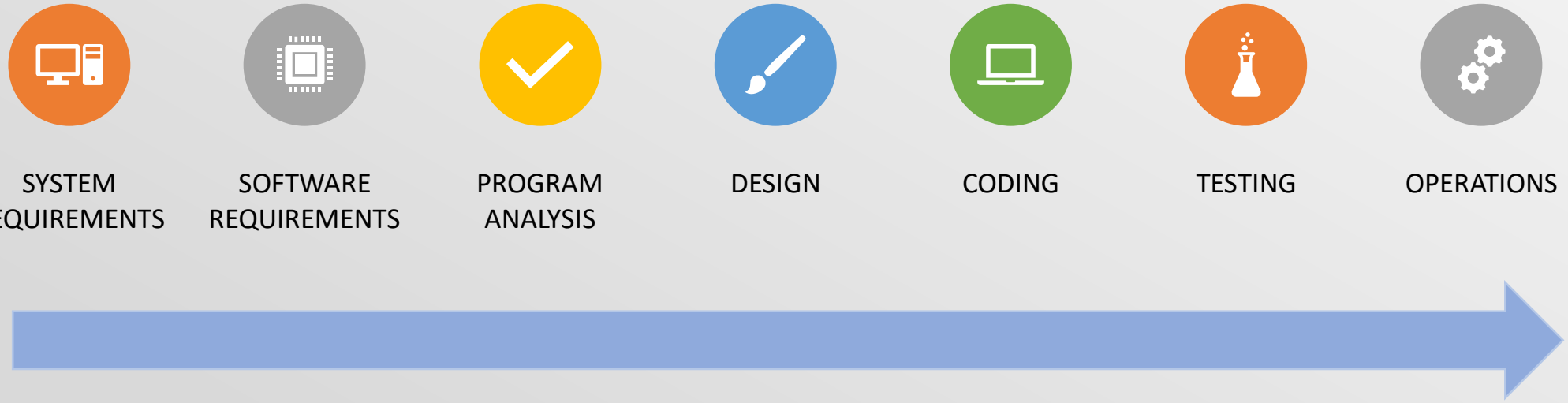
Modelul Waterfall

- Modelul Waterfall reprezintă o abordare relativ secvențială a procesului de dezvoltare software.
- Flow-ul modelului Waterfall decurge în principal într-o singură direcție, etapele unui proiect fiind abordate secvențial.



Etapele modelului Waterfall

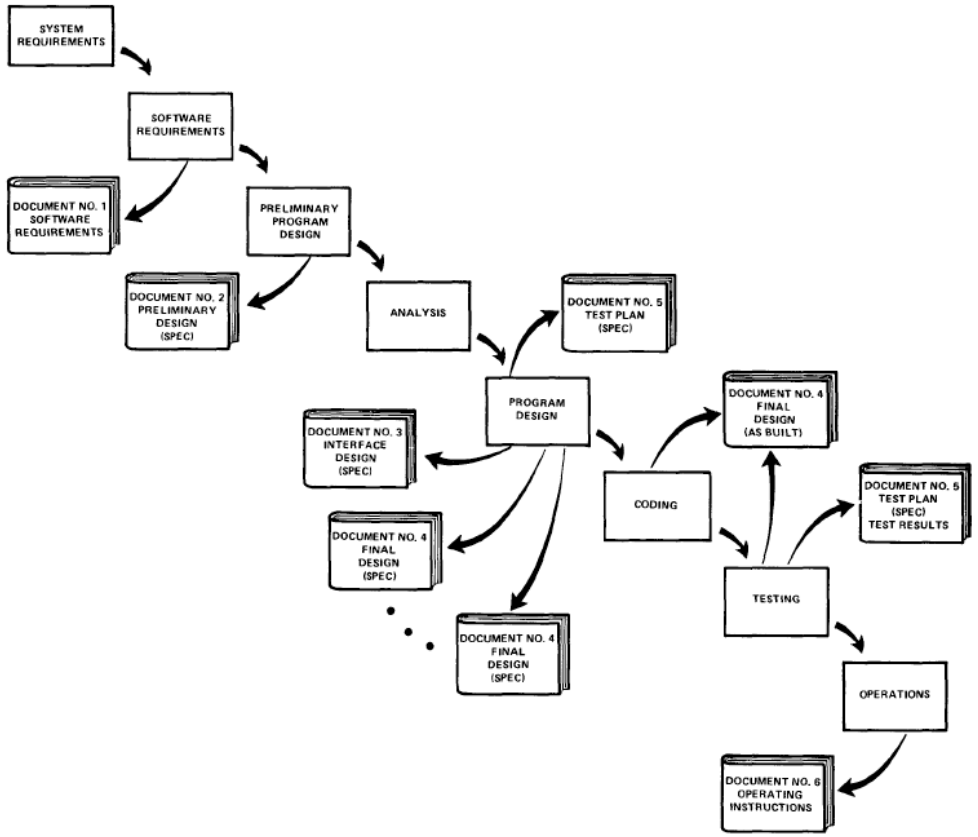
Etapele modelului Waterfall



Etapele Modelului Waterfall

- System and Software Requirements: sunt documentate toate specificațiile hardware și software ale aplicației → Product Requirements Document(PRD) / Software Requirements Document(SRD) (include toate cerințele aplicației)
- Analysis: din această etapă rezultă modele, scheme ale aplicației
- Program Design: arhitectură software-ului
- Coding:
  - Etapă de development propriu-zisă;
  - De asemenea, codul trece prin prima etapă de testare(unit-testing);
  - Software Integration: integrarea tuturor modulelor și serviciilor într-o singură aplicație funcțională.
- Testing: etapă de testare sistematică a aplicației
- Operations: Include instalarea, migrarea, suportul și mententanta aplicației.

Etapele Modelului Waterfall detaliat

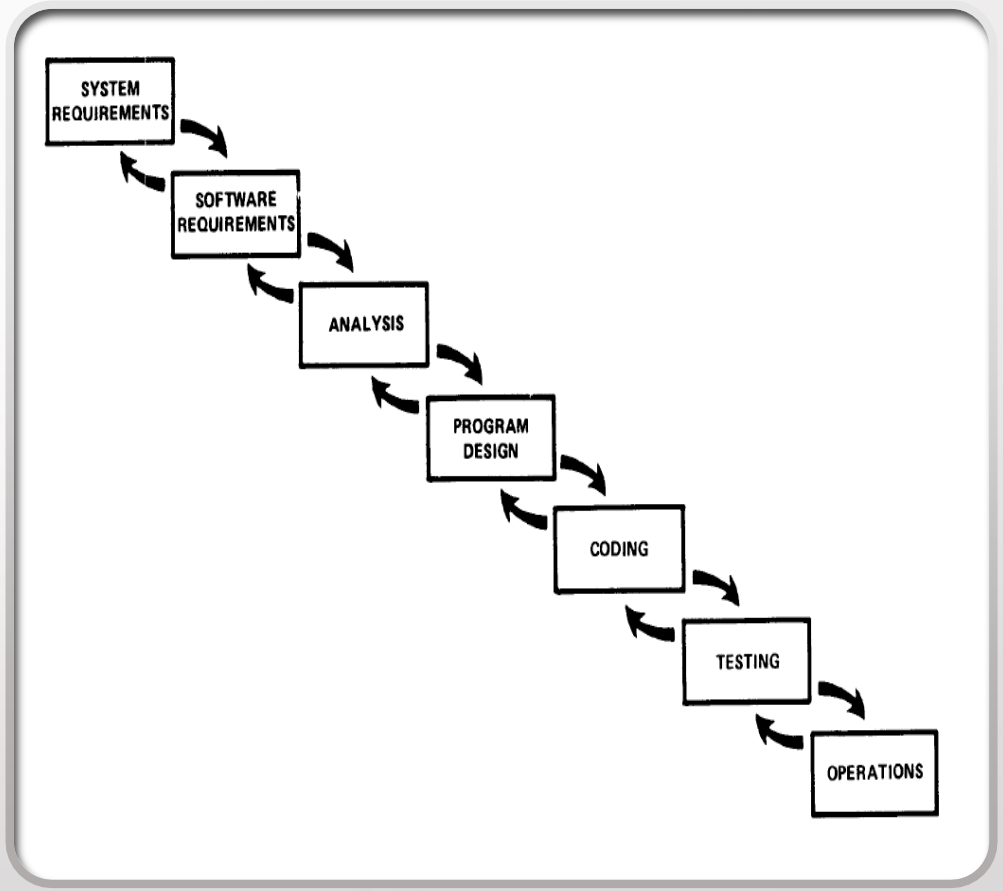


Dezavantaje metoda Waterfall

- Partiționare ineficiența a proiectului în stagii distincte
- Flow-ul sevential al modelului Waterfall face dificilă schimbarea pe parcurs a cerințelor utilizator → interacțiune cu clientul redusă
- Acest model este recomandat atunci când:
  - Cerințele sunt stabilite clar de la început și nu vor mai fi schimbate pe parcursul dezvoltării
  - “Freeze” the requirement document

Waterfall – Modelul final al lui Winston W. Royce

- S-a constatat faptul că este necesar și recomandat un feedback între etape, care să meargă în ambele sensuri
  - Feedback de la testing la design(în etapă de testare se pot identifică erori de design)
  - Feedback de la design la requirements(probelemele de design ar putea necesită modificarea cerințelor)
- De asemenea, se urmărește implicarea clientului cât mai mult în procesul de dezvoltare al proiectului.
- Se recomandă cât mai minuțioasă a documentației(“doing the job twice if possible”

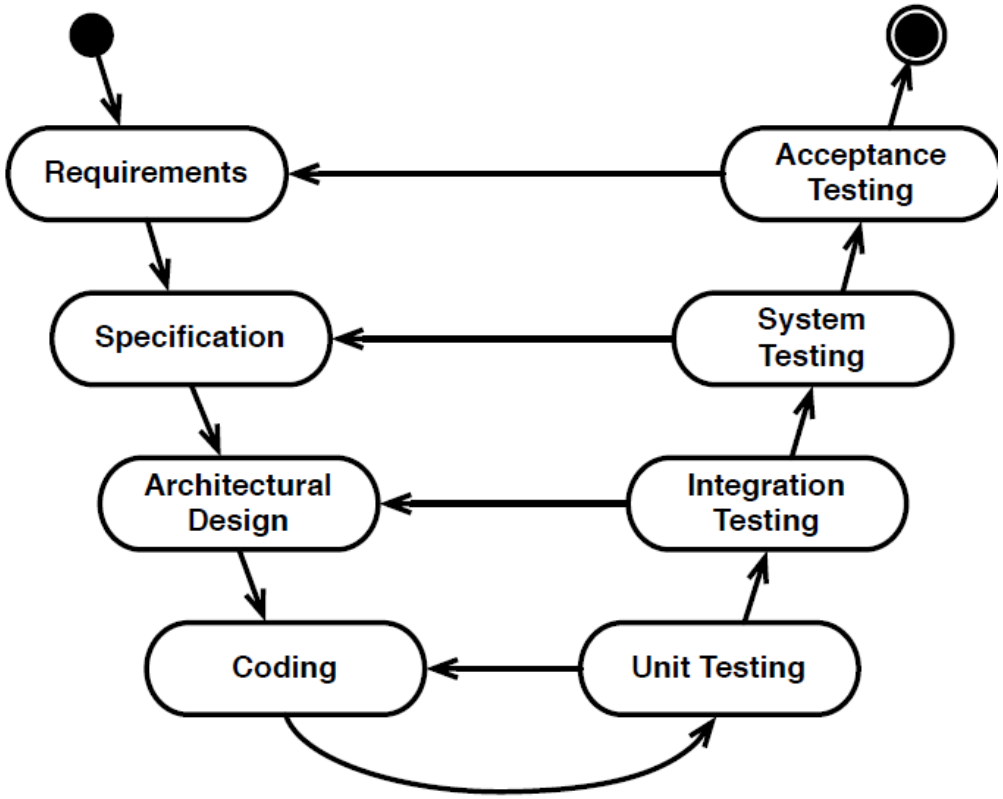


Variații ale modelului Waterfall

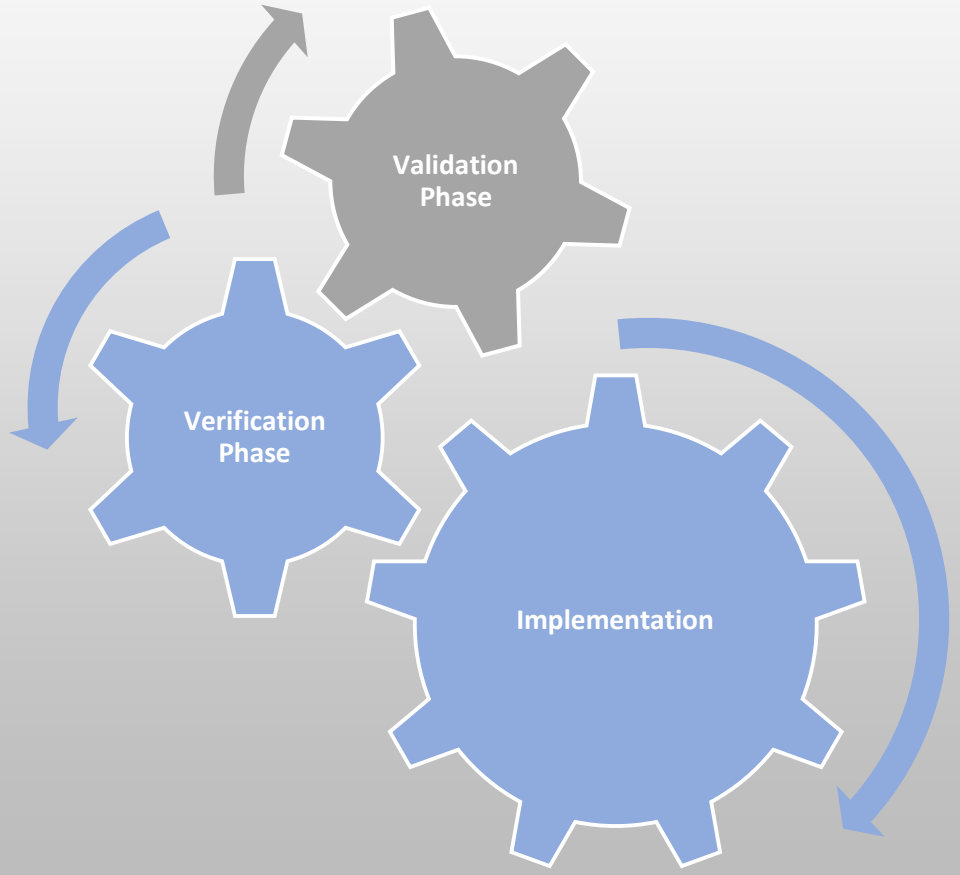
- Sashimi Waterfall: activitățile se pot suprapune.
- Waterfall cu sub-proiecte: implementarea în parallel a componentelor.
- Waterfall cu risk-reduction: se realizează o analiză inițială a riscurilor posibile, astfel încât să se diminueze impactul problemelor care pot apărea în fazele ulterioare ale procesului de dezvoltare.

V-Cycle

- Considerat o extensie a modelului Waterfall → accent pe testare
- În loc să se urmeze traseul clasic al Waterfall(etapele se succed în jos), în modelul V-Cycle etapele procesului de dezvoltare se succed în sus după etapă de coding, de aici rezultând formă în V a modelului.
- Modelul în V demonstrează relațiile dintre fiecare fază a ciclului de dezvoltare și fază asociată de testare.

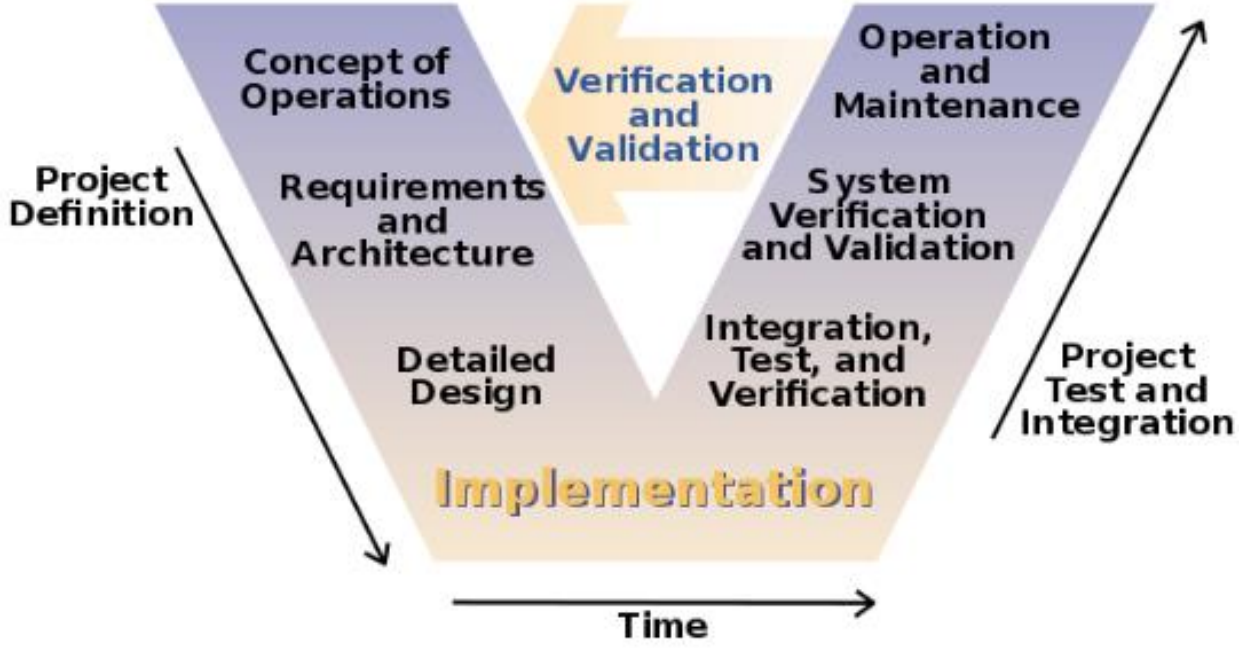


V-Cycle



V-Cycle

În acest model, fiecare faza a etapei de verificare are un corespondent în etapa de validare (testare).



V-Cycle – Verification Phases

Requirement Analysis: sunt colectate cerințele sistemului analizând cerințele utilizatorului → *User Requirements Document(URD)*

System Design: se analizează tehnicile prin care se pot implementa cerințele utilizatorului așa cum sunt descrise în URD → *Software Specification Document (*blueprint pentru etapa de dezvoltare)

Arhitecture Design*(high-level design):* design-ul arhitecturii hardware și software

Module Design*(low-level design):* se “sparge” aplicația în module mai mici, iar fiecare modul este explicat în detaliu. Se realizează o logică detaliată (în pseudocod) a functionalității modulului.

V-Cycle – Validation Phases

Unit Testing: sunt dezvoltate în etapa de *Module Design*.

Scop: eliminarea bug-urilor la nivel de modul.

Integration Testing: sunt dezvoltate în etapa de *Arhitectural Design*

Scop: verificarea comunicării dintre module după ce acestea au fost integrate.

System Testing: sunt dezvoltate în etapa de *System Design*, de către o echipă din partea clientului.

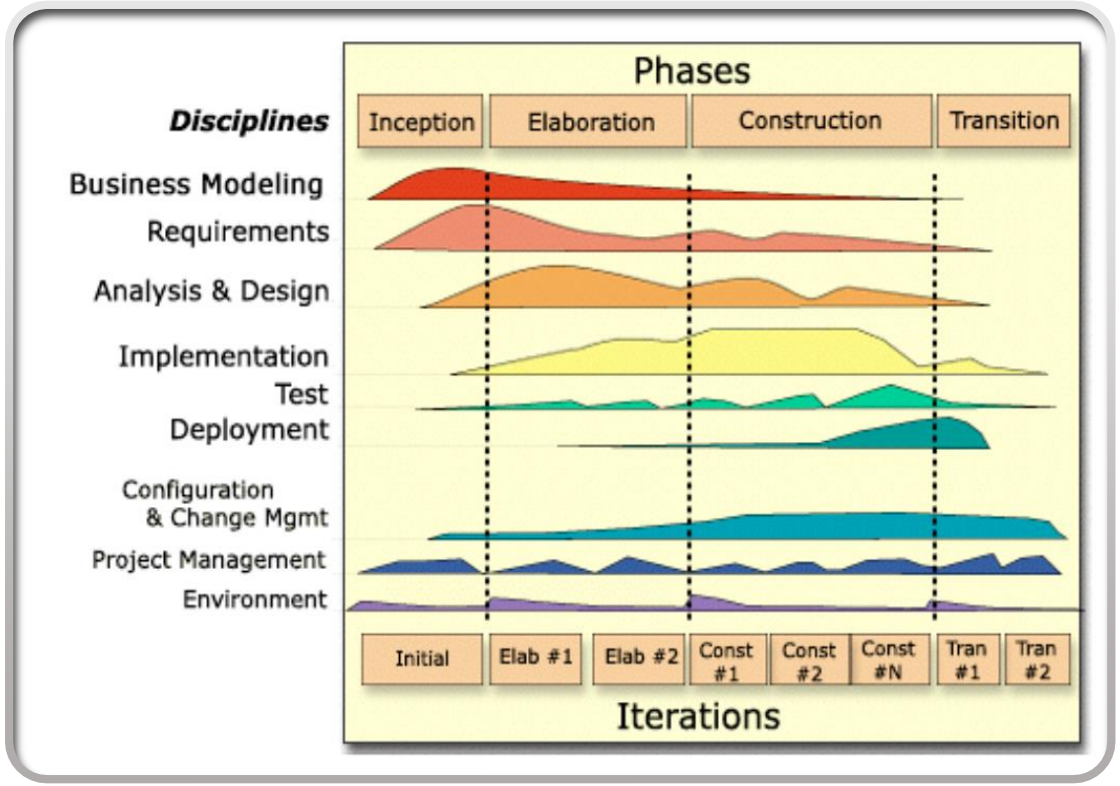
Scop: se verifică dacă au fost îndeplinite cerințele aplicatiei. Se testează întreaga aplicație.

User Acceptance Testing: dezvoltate în etapa de *Requirement Analysis,* de către utilizatorii bussiness.

Scop: se testează aplicația într-un mediu realistic, asemănător celui de producție; se verifică dacă aplicația îndeplineste cerințele user-ului și dacă este gata de dat în folosință în mediul de producție.

Rational Unified Process(RUP)

- Procesul de dezvoltare este împărțit în 2 dimensiuni:
  - **Fazele**, organizate în iterații, corespund unor niveluri diferite de maturitate
  - **Workflows**, concentrate pe un anumit aspect al dezvoltării
- Se suprapun și sunt caracterizate de niveluri diferite de intensitate



Rational Unified Process – Best Practices

- Există 6 practici principale ce defines principiile RUP:
  - Dezvoltare software în mod iterativ
  - Managementul cerințelor(inclusive evaluarea alternativelor)
  - Utilizarea de arhitecturi component-based(componente robuste)
  - Reprezentare simplă și neambiguă a modelului software
  - Verificarea calității software-ului
  - Schimbări de control ale software-ului (atât pentru calitate cât și pentru management)

Open Unified Process (OpenUP)

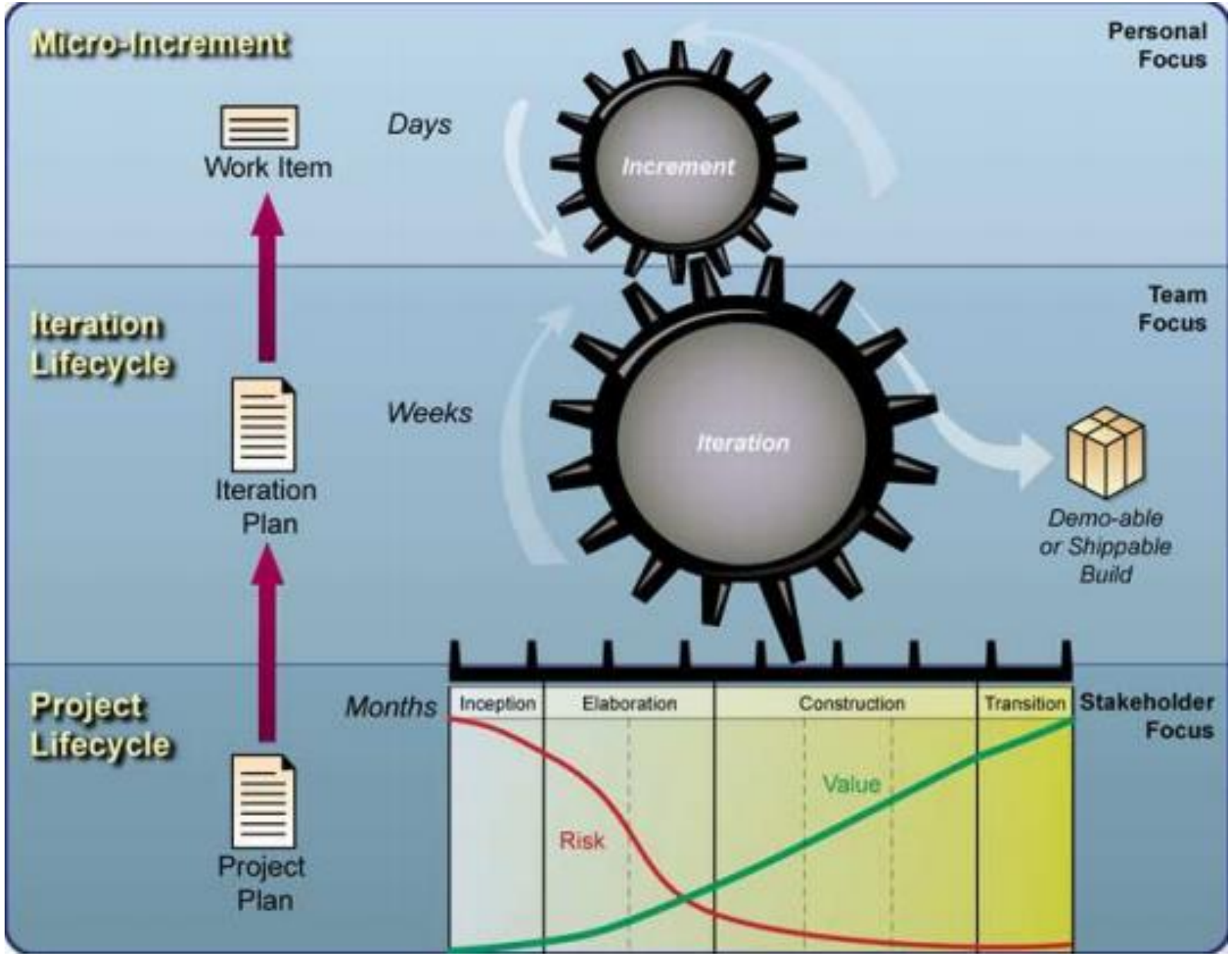
- Evoluție a Rational Unified Process (RUP)
- Păstrează caracteristicile esențiale ale RUP
  - Dezvoltare iterativă
  - Cazuri de utilizare
  - Scenarii
  - Evaluarea riscurilor
- Marea majoritate a părților opționale ale RUP au fost excluse

Open Unified Process (OpenUP)

- Conține componente RUP ce au fost unite
- Rezultă un proces mult mai simplu care respecta principiile RUP
- Dedicat echipelor mici (3-6 oameni) care doresc o abordare agile si iterativă => 3-6 luni de dezvoltare

OpenUp vs Agile

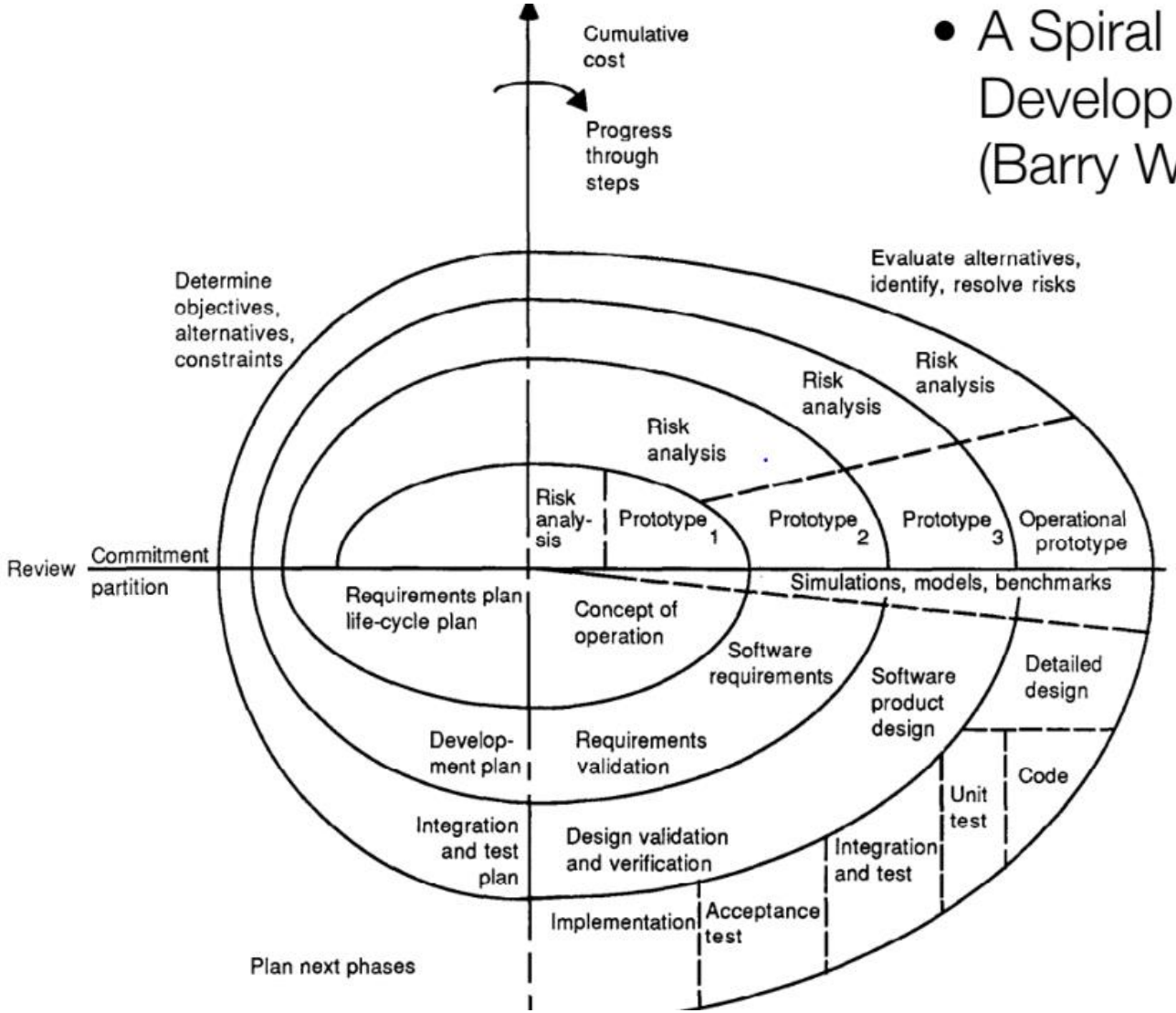
OpenUP	AGILE
Colaborare pentru stabilirea obiectivelor și pentru a înțelege demersul proiectului	Interacțiuni între membrii pentru a stabili modul de desfășurare si tehnologiile folosite
Cântărirea priorităților pentru a maximiza profitul părților implicate	Colaborare cu clienții/ negociere contract
Focusarea pe arhitectură in stagiile incipiente pentru a minimiza riscurile si a organiza procesul de dezvoltare	Dezvoltarea softului având o documentație completă
Se lucrează la produs pentru a obține feedback continuu și a îmbunătății	Aplicarea unor schimbări în timp ec se urmează un plan


Micro-Increment
Personal Focus
Work Item
Days
Increment
Iteration Lifecycle
Team Focus
Weeks
Iteration
Iteration Plan
Demo-able or Shippable Build
Project Lifecycle
Months
Inception
Elaboration
Construction
Transition
Stakeholder Focus
Value
Risk
Project Plan

Spiral

- Poate fi considerat o altă vedere a modelului în cascadă
- Softul este dezvoltat în cicluri → metodă iterativă
- Fiecare ciclu reprezintă o fază
- Fiecare ciclu este ales în funcție de ce e nevoie → nu există o ordine prestabilită
- Prima fază constituie o evaluare a riscurilor și alternativelor

- A Spiral Model of Software Development and Enhancement (Barry W. Boehm)



Spiral

Avantaje

- Bazată pe analiză de risc și alternative
- Preia diferite practici de dezvoltare software
- Evoluția softului (mentenanța reprezintă un ciclu din spirală)

Dezavantaje

- Respectarea cerințelor de soft din contract
- Dificultăți în a face estimări la început
- Flexibilitate scăzută

Activităţile unui ciclu de producţie

- Planificare
  - Stabilirea obiectivelor, alternativelor de rezolvare şi a restricţiilor pentru ciclul curent
- Analiza riscurilor
  - Analizează alternativele de rezolvare şi restricţiile din (1)
  - Identifică factorii de risc
  - Decizia GO/NO GO (continuă/renunţă)
- Inginerie - începutul unui ciclu nou
  - Dezvoltarea produsului pe următorul nivel de detaliere
  - Se pot folosi
    - Modelul clasic
    - Prototipizarea - pentru clarificarea unor cerinţe
- Evaluarea clientului

Prototiparea

- După ce sunt stabilite cerințele, un prototip al aplicației este dezvoltat (de exemplu GUI-ul) și este evaluat de client
- Se poate focusa pe aplicație sau pe o funcție specifică
- Poate fi de 3 tipuri
  - Throw away
  - Evoluționar
  - Incremental

Tipuri de prototipare

- **Throw away**
  - Presupune a fi folosit doar pentru a testa unele idei după care se renunță la el când se dorește dezvoltarea sistemului
- **Evoluționar**
  - Este dezvoltat și modificat până când devine sistem funcțional
- **Incremental**
  - Este dezvoltat si modificat in etape mici, dupa fiecare etapă fiind primit un feedback

Avantaje și dezavantaje – Model Spiral

Avantaje

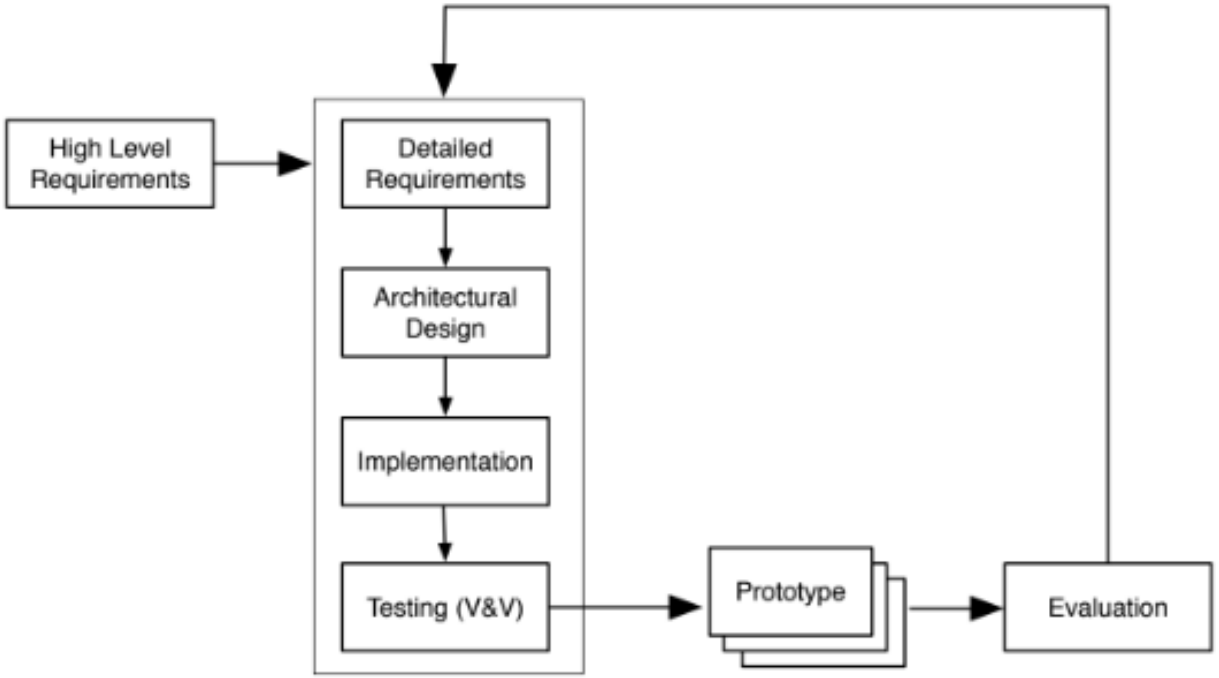
- Comunicare client – dezvoltator → clientul poate oferi feedback mai ușor
- Se pot testa diferite prototipuri si poate fi ales cel mai bun
- Documentație redusă datorita testării practice a prototipurilor
- Componentele mai greu de implementat pot fi dezvoltate la început, astfel testându-se dacă este o abordare potrivită

Dezavantaje

- Este nevoie ca unele componente să fie reimplementate
- Nu se cunosc foarte clar etapele procesului de implementare
- Abordarea evoluționară conduce la arhitecturi slabe (sunt construite cu informații incomplete)

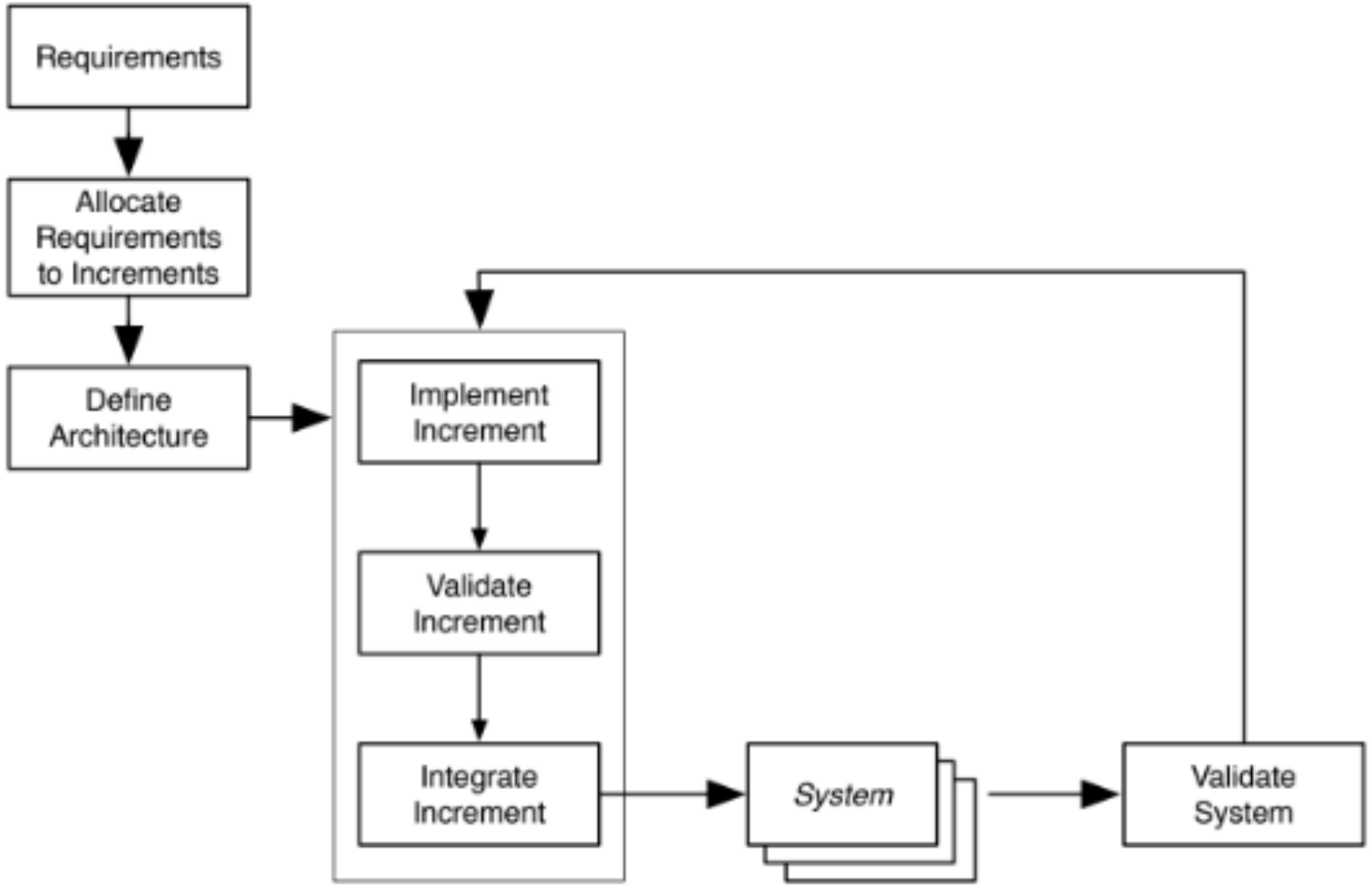
Prototipare

- Aplicabilitate
  - Sisteme de dimensiune mică sau medie
  - Părți ale unor sisteme mai mari
  - Sisteme care nu vor opera un timp indelungat



Modelul incremental

- Dezvoltarea și livrarea sunt împărțite în părți (incrementuri)
- Fiecare increment livrează o parte din funcționalități
- La început sunt livrate incrementuri cu cerințe mai importante
- Cerințele pentru incrementuri ulterioare pot evolua
- Cerințele nu pot fi modificate daca se lucrează la un increment


Requirements
Allocate Requirements to Increments
Define Architecture
Implement Increment
Validate Increment
Integrate Increment
System
Validate System

Avantaje și dezavantaje – Model incremental

Avantaje

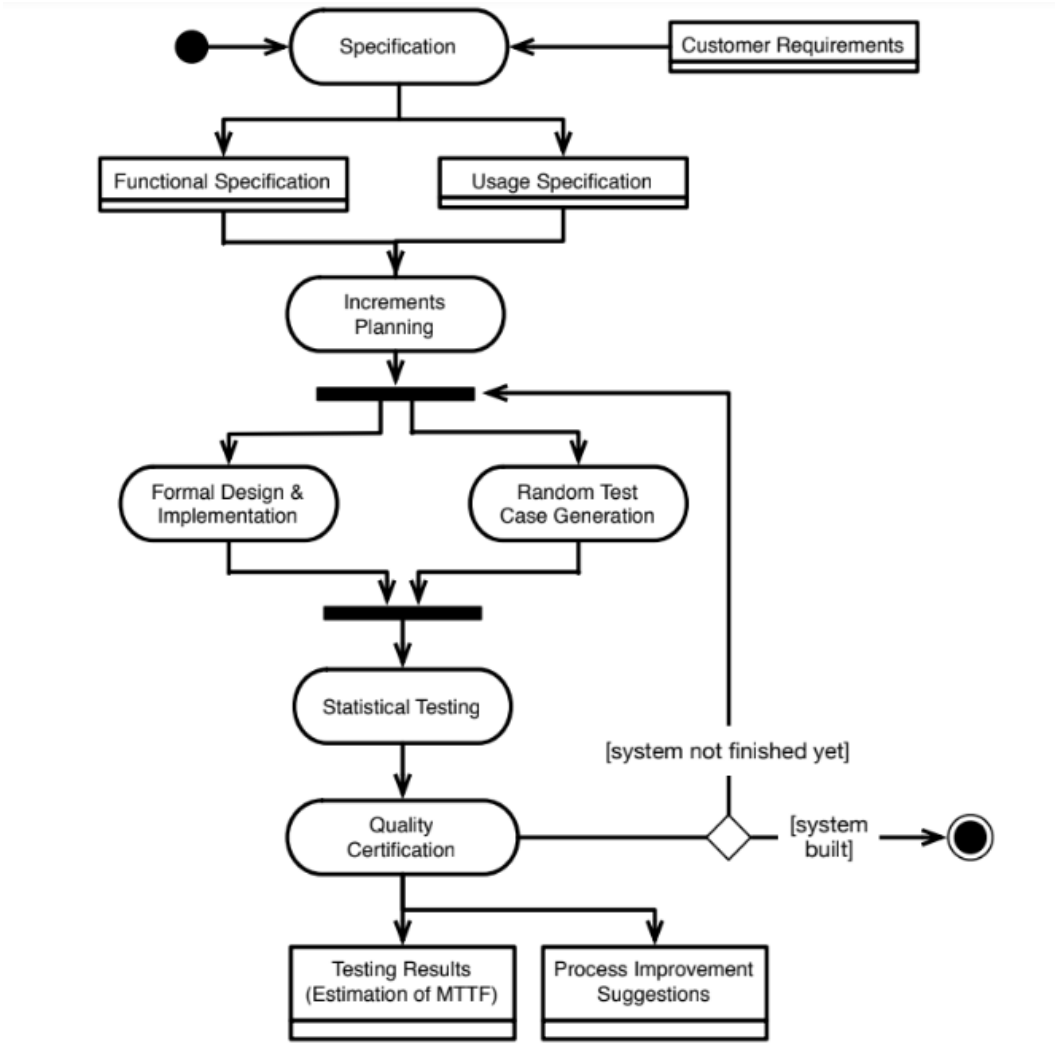
- Se obține un sistem funcțional mai rapid deoarece la icneput sunt livrate cerințele critice
- Incrementurile inițiale servesc drept prototip ajutând pentru elaborarea viitoarelor cerințe
- Risc scăzut de eșec al proiectului
- Cerințele cu prioritatea cea mai amre au parte de mai multă atenție (testare)

Dezavantaje

- Arhitecturile de sistem tind să se degradeze
- Unele incrementuri pot conduce la reimplementări semnificativ

Clean Room Software Engineering

- Proces pentru software fără defecte
- Bazat pe metode formale și teste statistice
- Rapid și corect
- Tip incremental
- Orientat pe prevenirea defectelor, nu pe rezolvarea lor


Specification
Customer Requirements
Functional Specification
Usage Specification
Increments Planning
Formal Design & Implementation
Random Test Case Generation
Statistical Testing
[system not finished yet]
Quality Certification
[system built]
Testing Results (Estimation of MTTF)
Process Improvement Suggestions

Open source development process

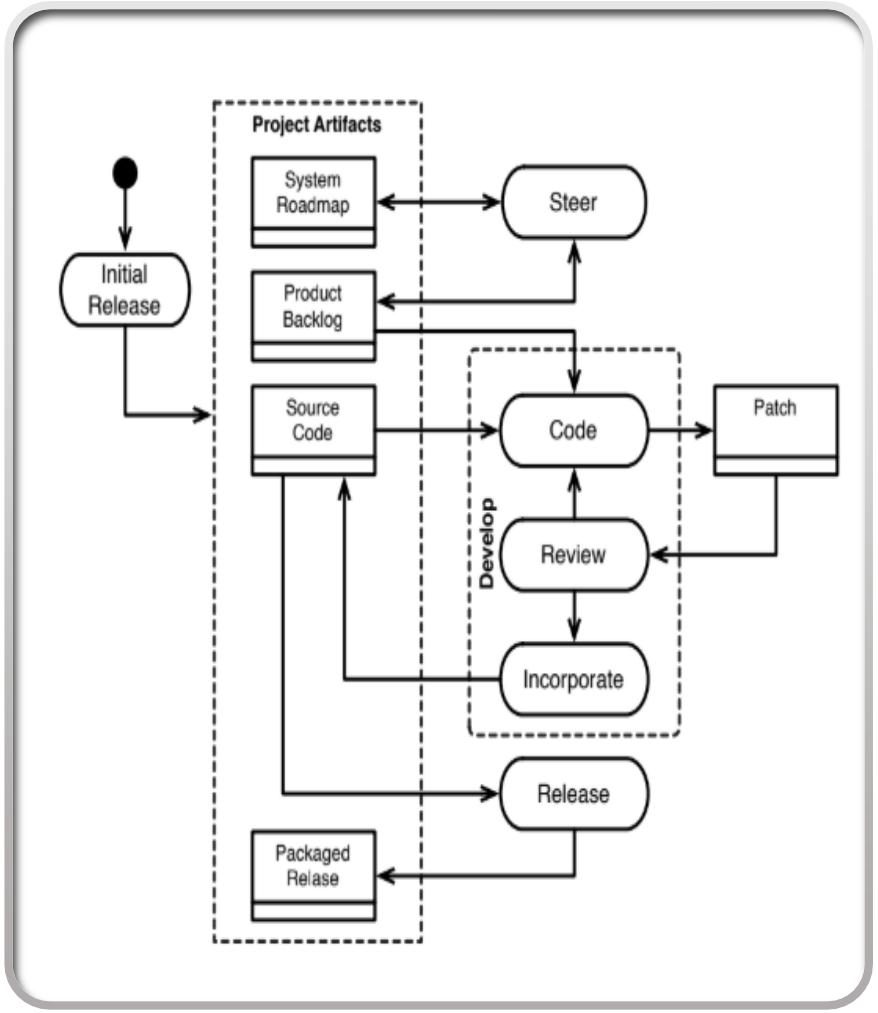
- Proiectul începe de la cod existent, uneori fiind prezente cantități considerabile de cod
- Comunitate de dezvoltatori din diferite zone cu diferite experiențe
- Provocări diverse
  - Implicare în diferite roluri
  - Control al calității
  - Lansare dicil de programat

Open source development process

- O hartă definește evoluția procesului de dezvoltare
- Dezvoltare pe cod neterminat/ cu erori, lansate ca patchuri
- Calitate asigurată de re-evaluări ale codului
- Există două modele
  - Funcționalități fixe (livrat cand e gata)
  - Livrare la date fixe (cu funcționalitățile implementate până la momentul respectiv)

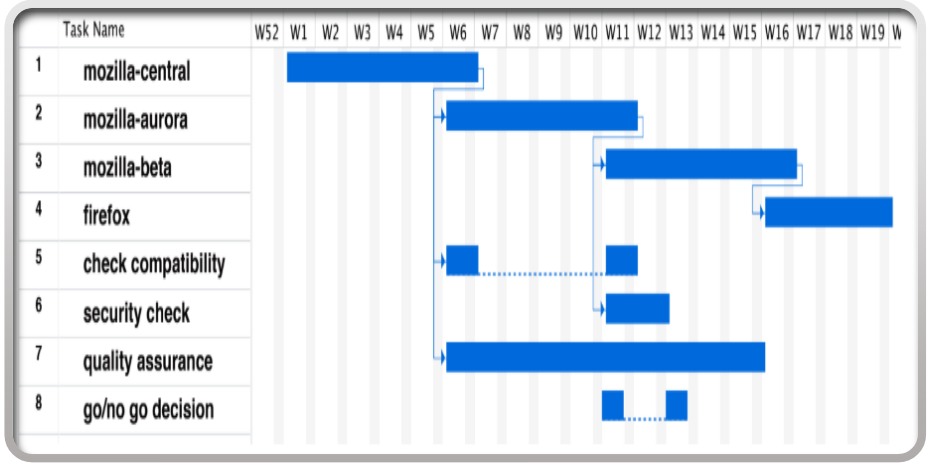
Open Source Development Process

- Primul pas în crearea și dezvoltarea open-source este alegerea unei platforme și definitivarea acestei alegeri.
- Ca pas următor , avem aducerea tool-urilor necesare pentru developer(precum IDEs , licențe și tot ce ține de un mediu de dezvoltare propice).
- Urmează pasul dezvoltării software propriu-zise , unde se face develop , se testează , se încorporează servicii și se integrează alte aplicații cu cea curentă. Există două modele:
  - funcționalități care sunt livrate când sunt terminate(dă o oarecare libertate)
  - cu deadline și livrare la o data fixă(se apropie de modelul AGILE)
- Se fac și reviews care conduc mai departe la patch-uri , o versiune oarecum mai bună decât cea curentă.
- Succesul este constituit de cât mai mulți contribuitori, care aduc funcționalități noi.



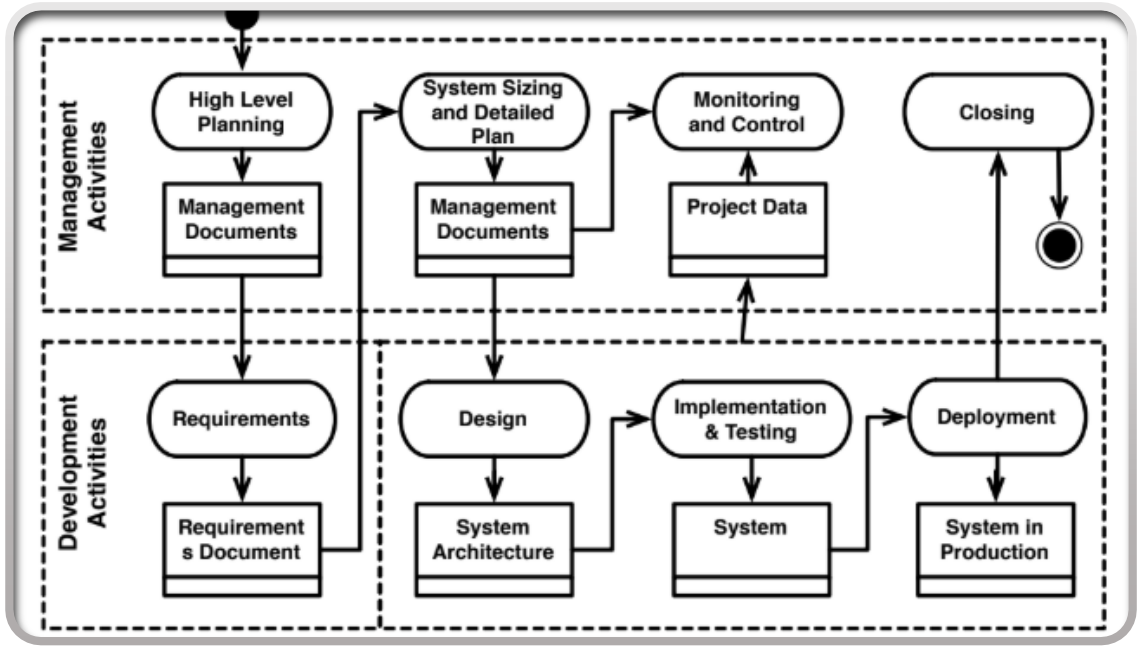
Firefox Release Schedule

- Putem observă în decurs de un an release-urile pentru firefox , dar și dependințele între acestea.
- Astfel firefox-central a fost lansat primul, după care se verifică funcționalitățile.
- Se verifică compatibilitatea cu alte aplicații , calitatea produsului , dar și versiunea livrată că urmare a acesteia, fiind vorba de aurora.
- În pasul următor se verifică din nou factorii de la pasul anterior pentru versiunea curentă . După cum putem observă, mozilla a intrat după mai multe release-uri în beta, fiind o versiune înaintea ultimelor release-uri
- Astfel, ultima versiune și cea stabile se poate consideră firefox, care derivă din versiunea beta.

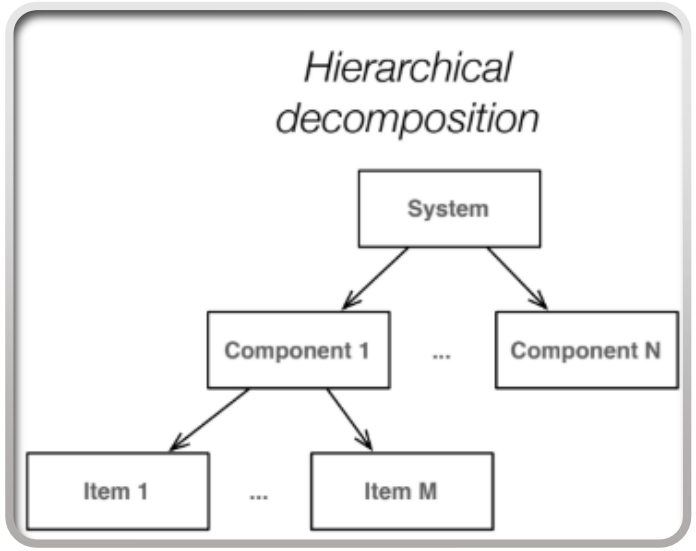


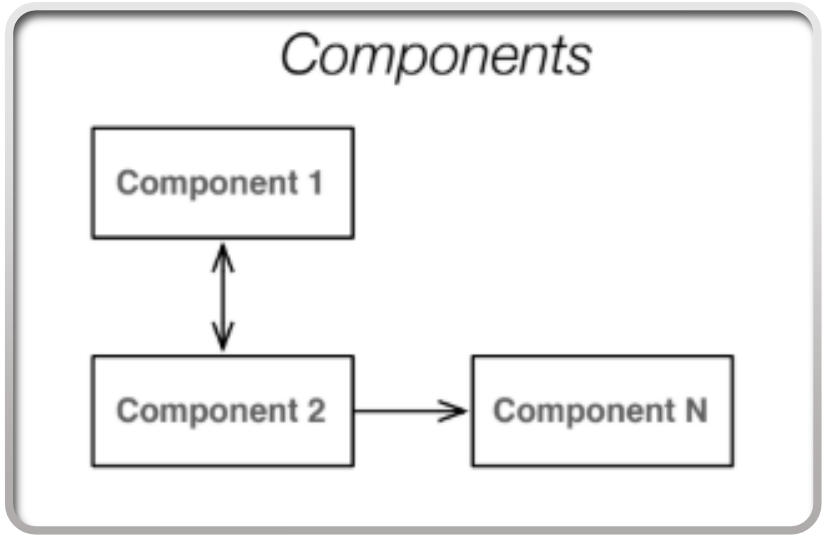
Additional Concerns

- Apar și anumite îngrijorări datorită modului mai greu de gestionare a aplicațiilor open-source între management și activitățile de dezvoltare în șine.
- Trebuie monitorizat controlul și flow-ul aplicației , design-ul trebuie să corespundă cu planul în detaliu generate de management.
- Arhitectura sistemului trebuie să fie în concordanță cu documentul generat prin activitățile de management.



Organizarea și descompunerea sistemului





- Sistemele complexe sunt structurate în mai multe componente elementare.
- Putem avea structură de arbore
- Structură ierarhică, în care avem dependințe între componente sub formă de fii și părinte (putem avea oricâte componente). Este un arbore cu o lista de fii.
- Putem avea sisteme cu dependințe liniare între componente

Strategii de organizare

Top-Down

- Se abordează aplicația în componente mari, din care se deduc componentele mai mici, adică fii unei componente ce acoperă o funcționalitate mai mare.
- Frunzele arborelui de componente și dependențe ar trebui să fie componente elementare, sau chiar funcții, fiind la cel mai jos nivel al arborelui.

Bottom-Up

- Abordarea bottom-up presupune dezvoltarea de componente mici, după care, mai departe acestea trebuie integrate între ele pentru a obține componentele mai mari de care avem nevoie, funcționalități integrate cu alte funcționalități, independente sau nu între ele.
- Se bazează pe obținerea rădăcinii din Frunze. Se pornește de jos în ierarhie și se ajunge la părinți.

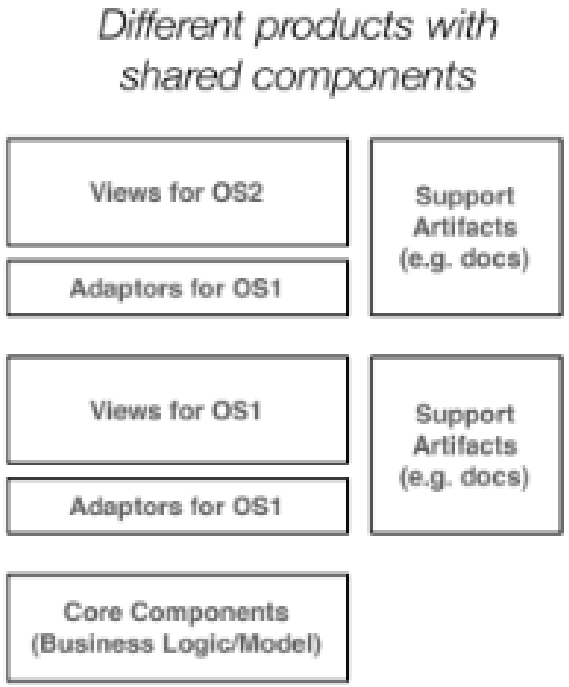
Familii de produse

Probleme

- Sisteme de operare
- Diferență dintre platforme
- Device-uri(probleme ce se leagă de inerfata cum ar fi setări de bază(capacitate , putere))
- Tehnologiile disponibile

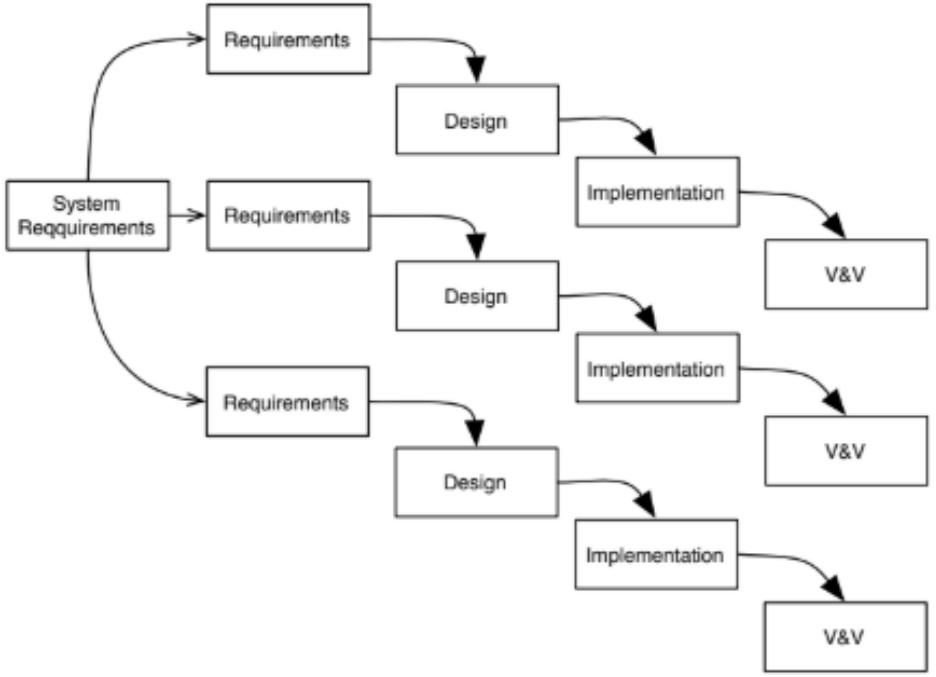
Abordări

- Folosirea unei aplicații cu o singură arhitectură
- Componente adaptive (design care să permită să rescrii componente ușor , integrarea să fie soft)
- Produse și aplicații diferite cu componente commune
- Dezvoltare independența cu interfețe comune sau API-uri.
- API-ul este separate de user interface și poate fi considerate o componența de server ce nu cauzează probleme dacă este izolată într-un mod rațional.



Consecinte

- Pentru sistemele complexe dezvoltarea de software trebuie făcută într-un mod rațional în așa fel încât:
  - Design-ul aplicației trebuie să fie făcut astfel încât componentele să se integreze ușor între ele(integrare soft), componentele să poată fi rescrise ușor.
  - Dezvoltarea software e compusă din procese care rulează în paralel
  - Dependențele între componente au de cele mai multe ori o arie ierarhică , ceea ce da senzația de recursive între activitățile dezvoltării software.
  - Aplicația trebuie încadrată în cererile de sistem



Exercițiu

Vrem să dezvoltăm o aplicație web. Această este definită că o bibliotecă online, cu toate funcționalitățile necesare acestui subiect.

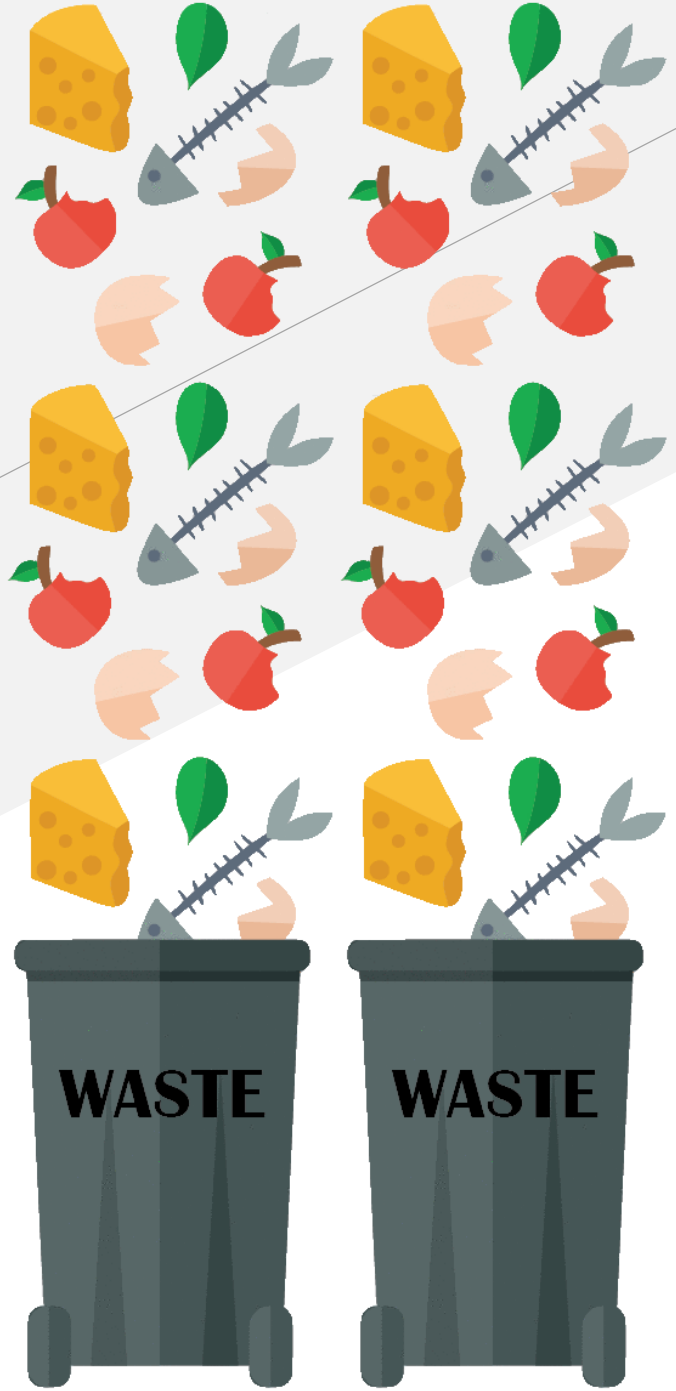
- Ce activități avem ?
- Ce procese avem ?
- Ce produse avem ?
- Cum folosim API-ul?
- Ce fel de componente avem?
- Ce dependențe există între acestea?
- Cum integrăm componentele?

Agile Software Development

Anghel Bogdan-Ștefan
Carpen Diana-Maria
Țuicu Daniel-George

Outline

- Despre risipă
- Introducere în Metodologia Agile
- Procese de dezvoltare Agile
  - DSGM
  - eXtreme Programming
  - Scrum



Scurtă discuție despre risipă

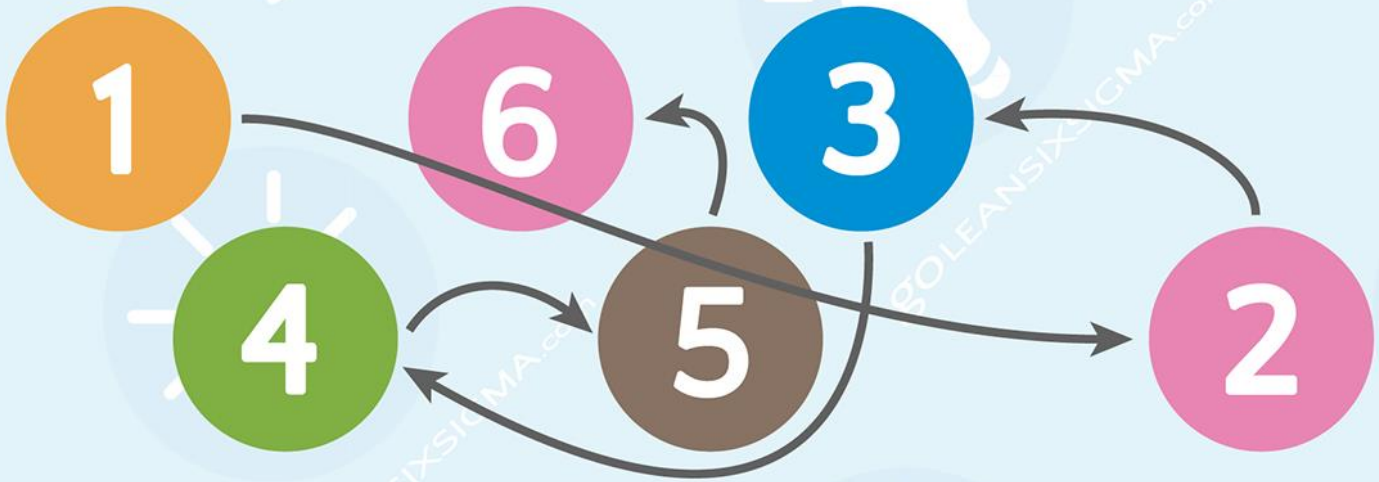
LEAN

Ce este LEAN?

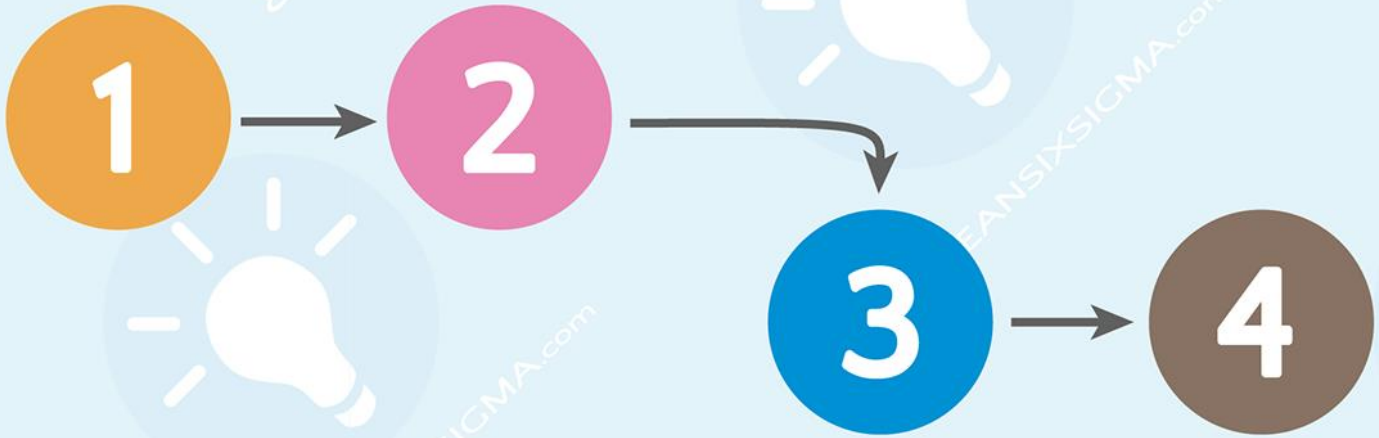
- Lean este o abordare sistematică prin care se identifică și se elimină risipa.
- Risipa reprezintă activitățile care nu produc valoare
- Valoare = ceea ce clientul este dispus să plătească

Lean: Process Before & After

PROCESS BEFORE LEAN:



PROCESS AFTER LEAN:



goLEANSIXSIGMA.com

© Copyright 2016 GoLeanSixSigma.com. All Rights Reserved.

FOURPRINCIPLES

THE LEAN MANAGEMENT EXPERTS

Get Lean in 90 Seconds

video by Quality Schnallity Inc.

LEAN

Tipuri de risipă în LEAN

- MURI - suprasolicitare

Muri

- MURA - dezechilibru, neregulat, inegalitate

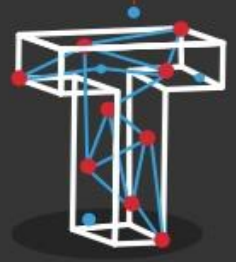
Mura

- MUDA - activități care nu aduc valoare

Muda

TRANSPORTATION

Refers to the relocation of finished or semi-finished products liable to deterioration, loss, longer lead times and additional workloads.



INVENTORIES

Refers to intermediate storage and to inventories made during the process. These do not contribute to the conversion of the product and generate additional costs that do not create added value.

MOTION

Refers to the pointless relocation of human resources and equipment during the manufacturing process that can have an impact in terms of pointless deterioration, wear and safety.

WAITING

Refers to the waiting time for an activity or task to be undertaken when the products or services are not in the process of transport, manufacture or conversion.

OVER-PROCESSING

Refers to pointless activities and tasks and/or those that do not create added value for the client, and to practical tools (software, procedures, work instructions) that are more specific, more complex or more costly than necessary.

OVER-PRODUCTION

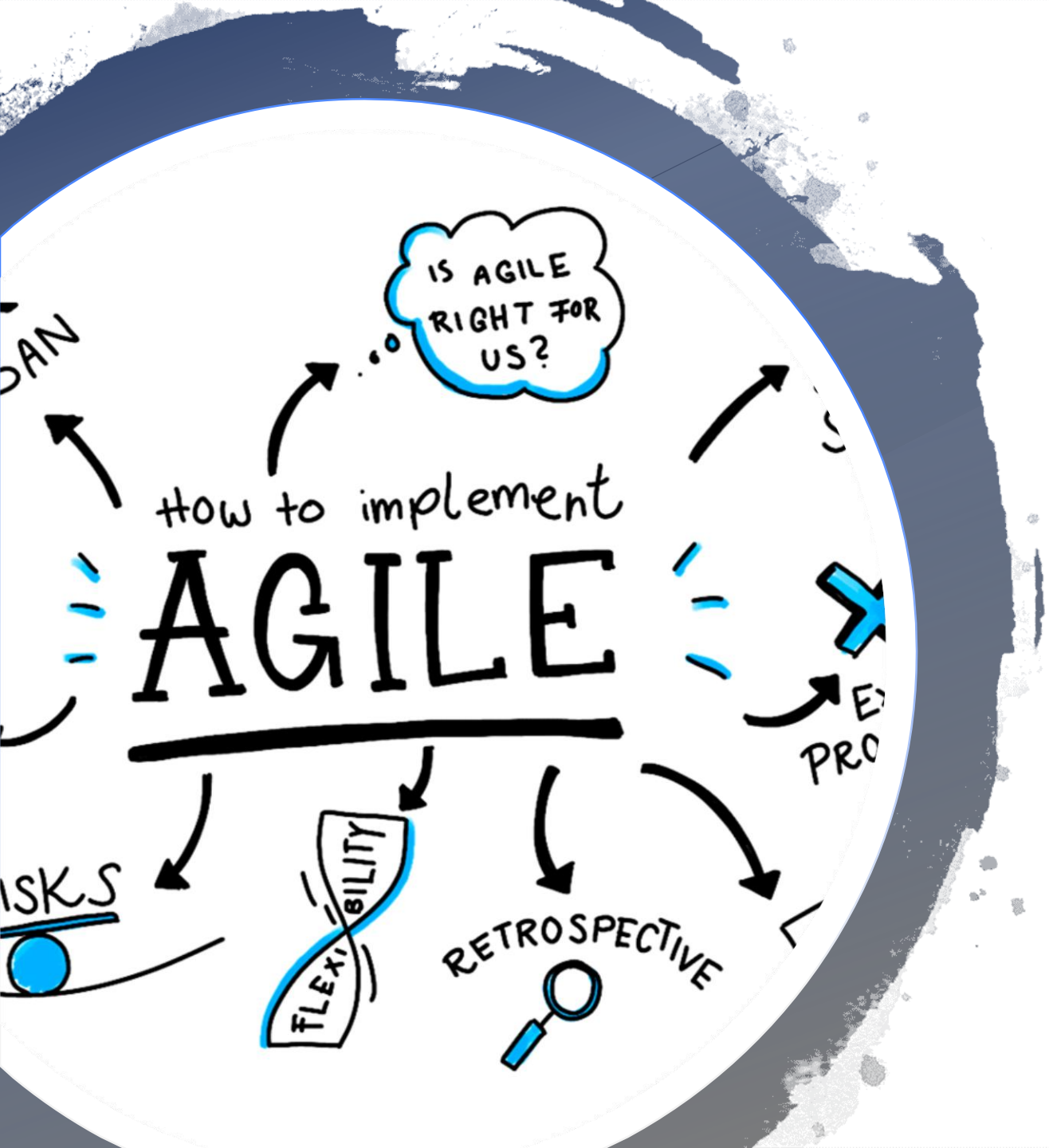
Refers to the overproduction of manufactured products in relation to client demand / orders. In the field of IT Operations, there is also the issue of underproduction.

DEFECTS

Every time defective and rejected finished and semi-finished products occur, this means additional costs to rework the products, destroy defective products, reprogram production etc.

Cauzele risipei în dezvoltarea software

- Software-ul nu poate fi măsurat (nu este material/palpabil)
- Flexibilitate în adăugarea de noi funcționalități
- Complexitate
- Tehnici de dezvoltare noi, care nu sunt perfecționate



INTRODUCERE ÎN

Metodologia Agile

De ce Agile?

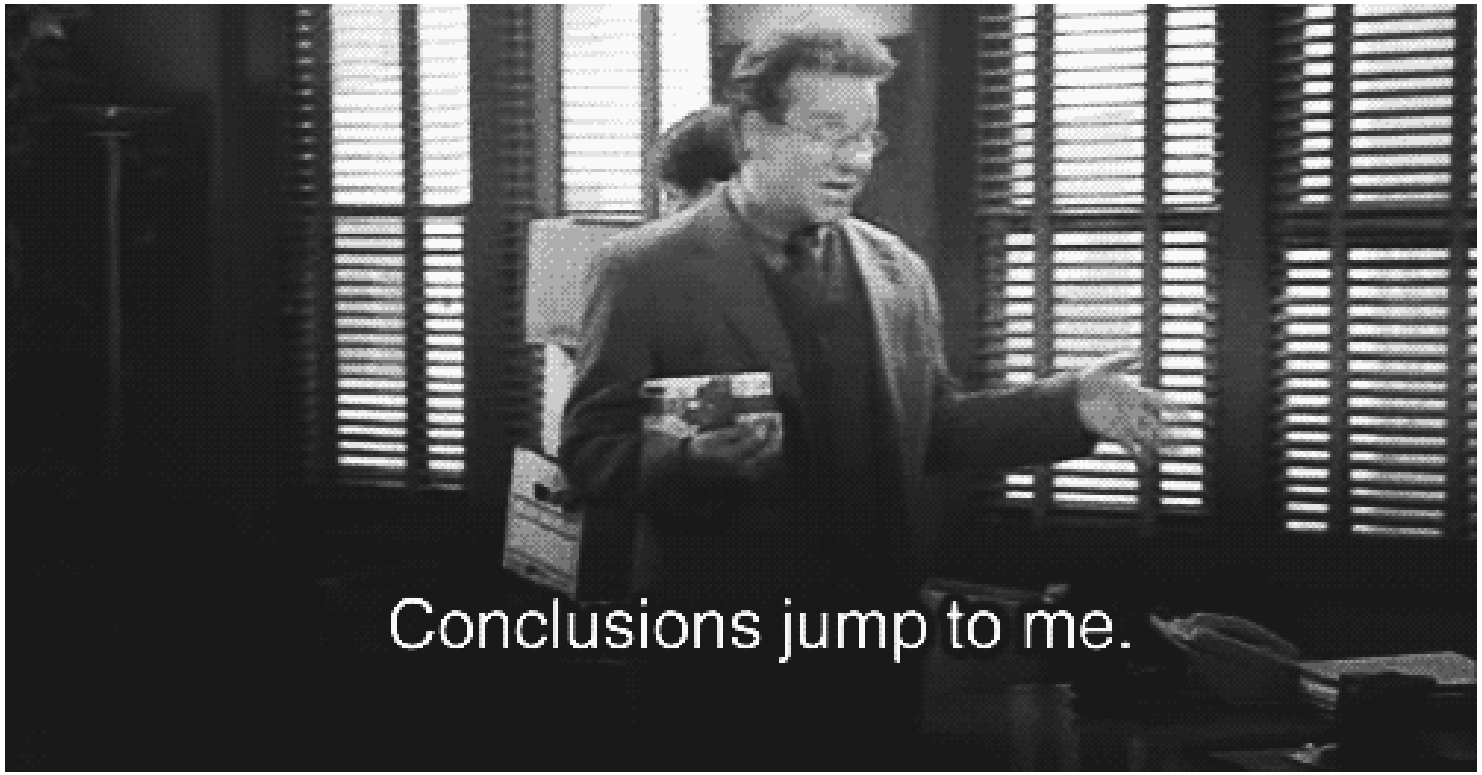
- **Agile Software Development:** un grup de metodologii de software development bazate pe următoarele principii:
  - Un proces de management care încurejează inspectarea și adaptarea frecventă
  - Lucru în echipă și self organization
  - Livrarea rapidă de software de calitate
  - O abordarea care unește cerințele clienților cu rezultatul software și cu obiectivele companiei

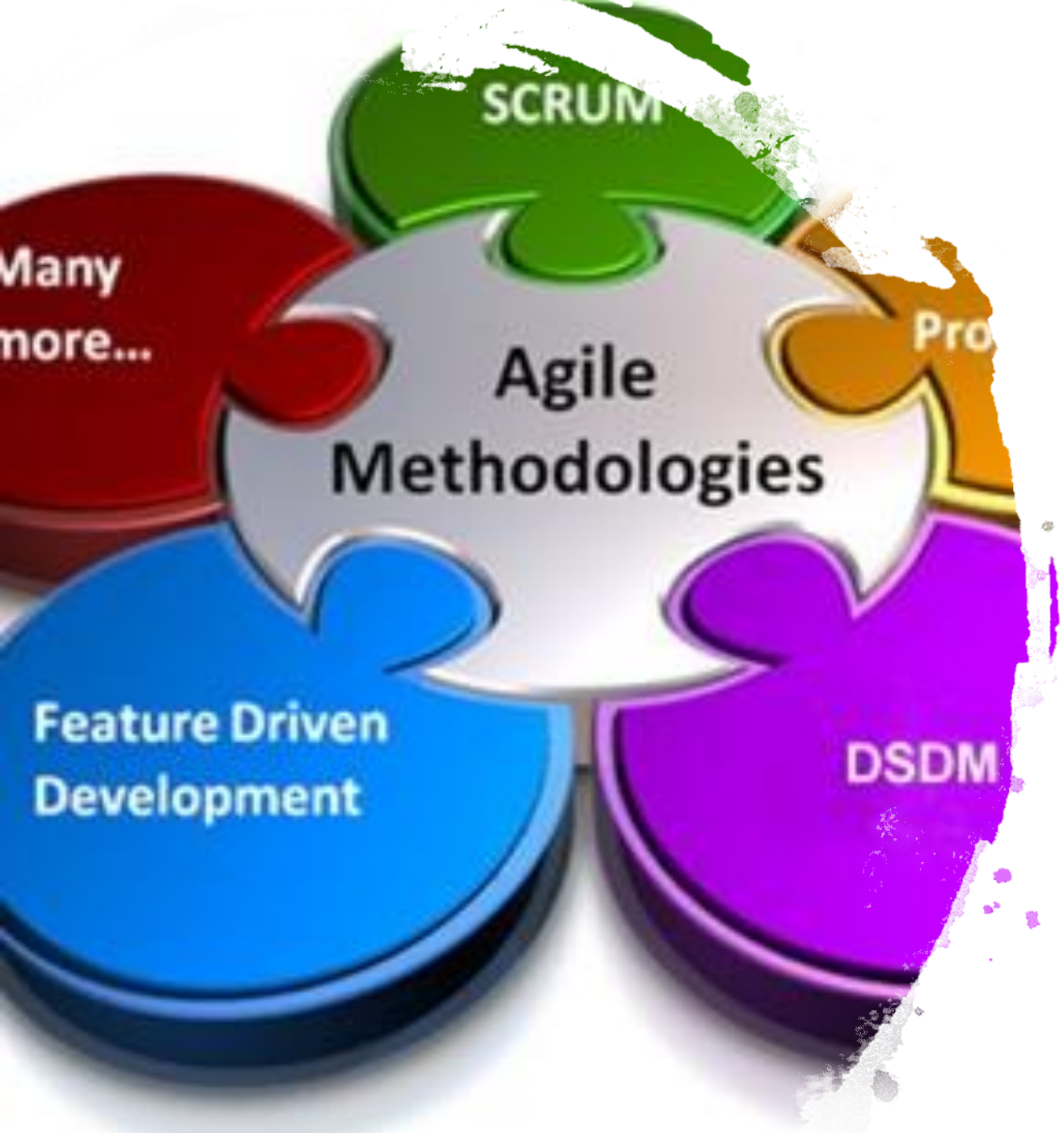
Ce consecințe sunt

- Clienți safisfăcuți
- Software funcțional livrat rapid
- Software-ul funcțional este modul de a măsura progresul
- Schimbări pe parcursul dezvoltării = no problem
- Simplitate

1 2 3 4 5 6 7 8 9 10
A
B
C
D
E
F
G
H
I
J
XTH 73
XAP 49
XJU 80
VCH

Concluzii Agile vs Waterfall





Procese de dezvoltare Agile

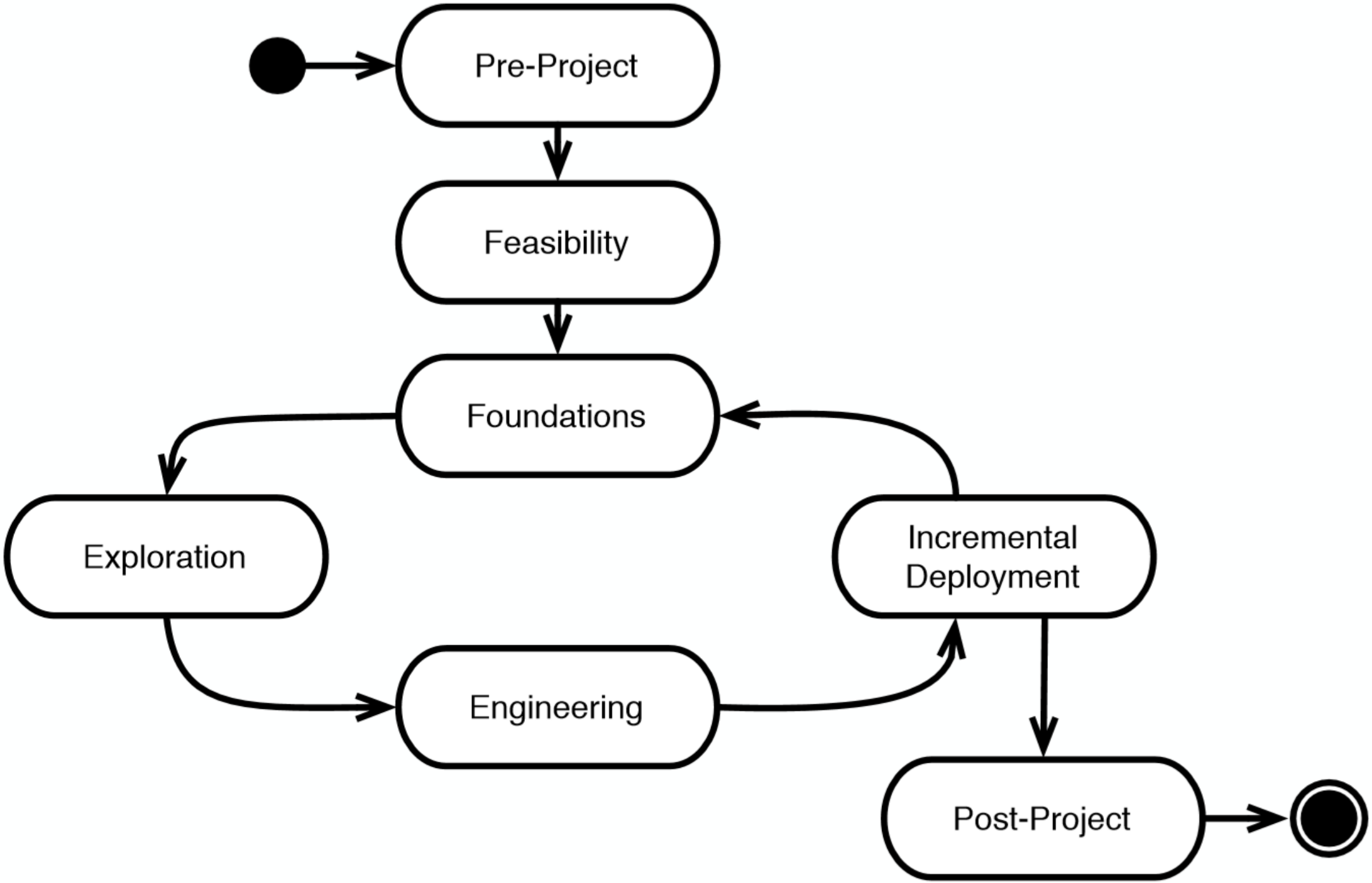
Dynamic System Development Method

- Metodologie Agile
- Se axează pe întregul ciclu de viață al proiectului
- Abordare iterativă, incrementală
- Bazată pe RAD (Rapid Application Development) 1950

Dynamic System Development Method

Practici utilizate

- Facilitated Workshops
- Modelling and Iterative Development
- MoSCoW Prioritisation
- Time boxing


Pre-Project
Feasibility
Foundations
Exploration
Incremental
Deployment
Engineering
Post-Project

Extreme Programming

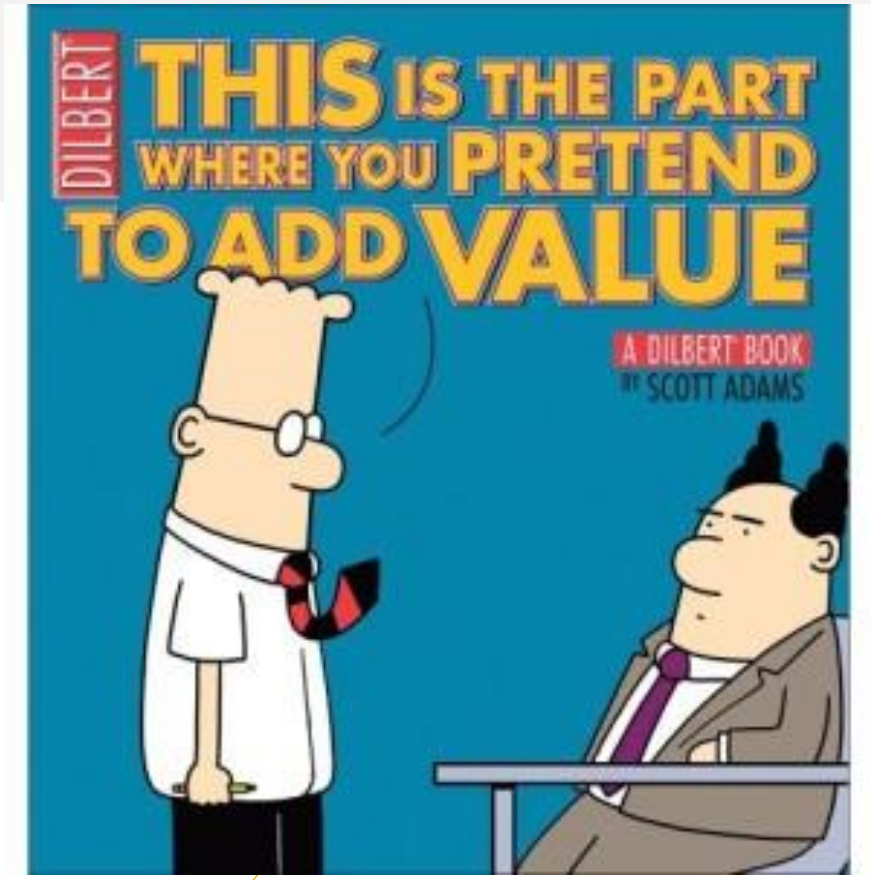
Extreme Programming

Definiție

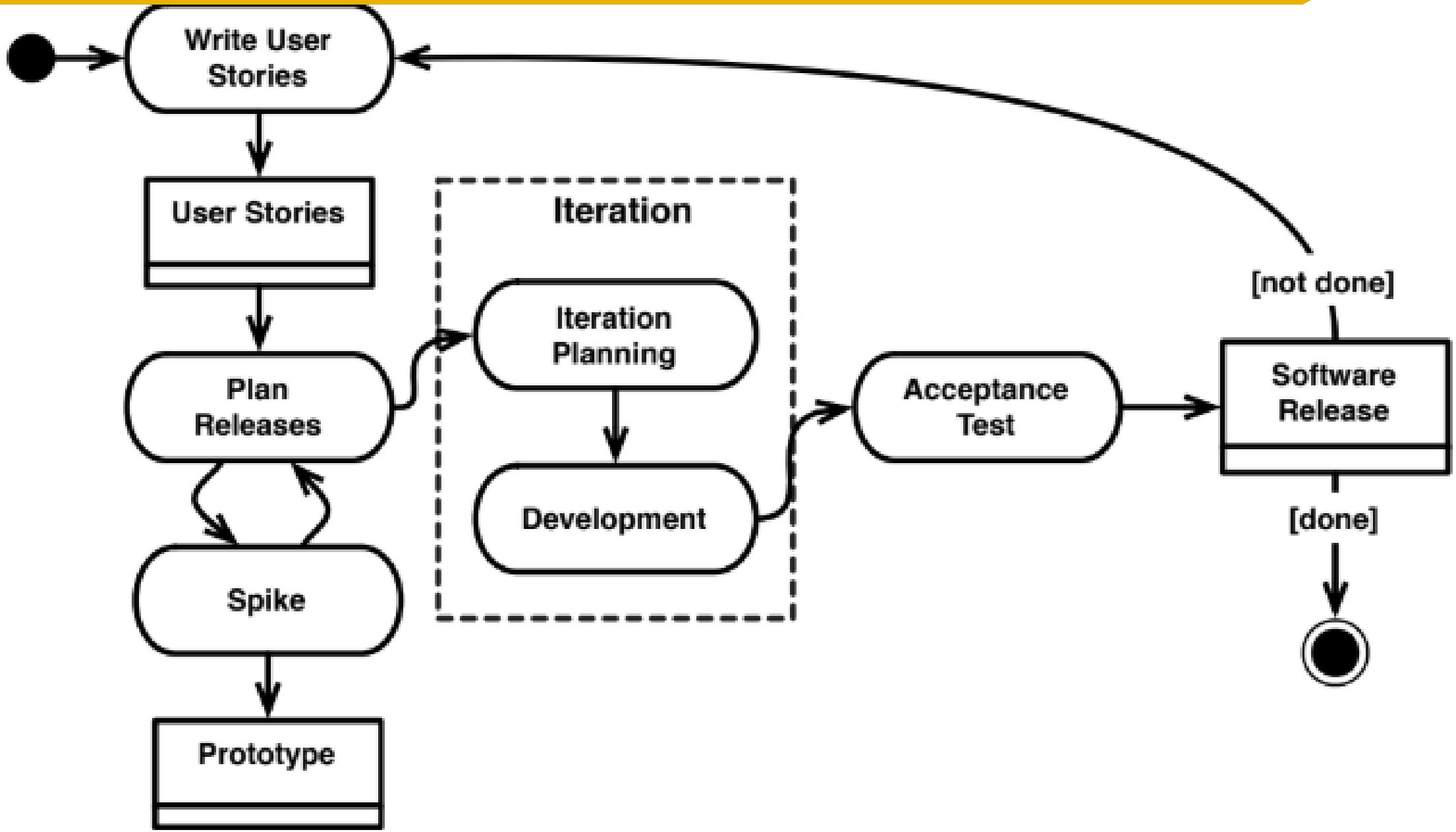
- XP este un Framework de dezvoltare software agile care are ca scop realizarea de software de calitate și îmbunătățirea mediului în care lucrează echipa de dezvoltare

Valori

- Comunicare
- Feedback
- Simplitate
- Curaj
- Respect



Procesul XP


Write User Stories
User Stories
Plan Releases
Spike
Prototype
Iteration
Iteration Planning
Development
Acceptance Test
[not done]
Software Release
[done]

Planificarea XP

- User stories
- Planificarea termenelor creează programul de lucru
- Modificări mici și frecvente
- Proiectul este împărțit în iterații
- Fiecare iterație începe cu planificarea iterației

XP Management

- Asigurarea unui mediu de lucru optim
- Ritmul potrivit de dezvoltare
- Discuții zilnice (daily standups)
- Măsurarea progresului
- Realocarea sarcinilor
- Change the rules

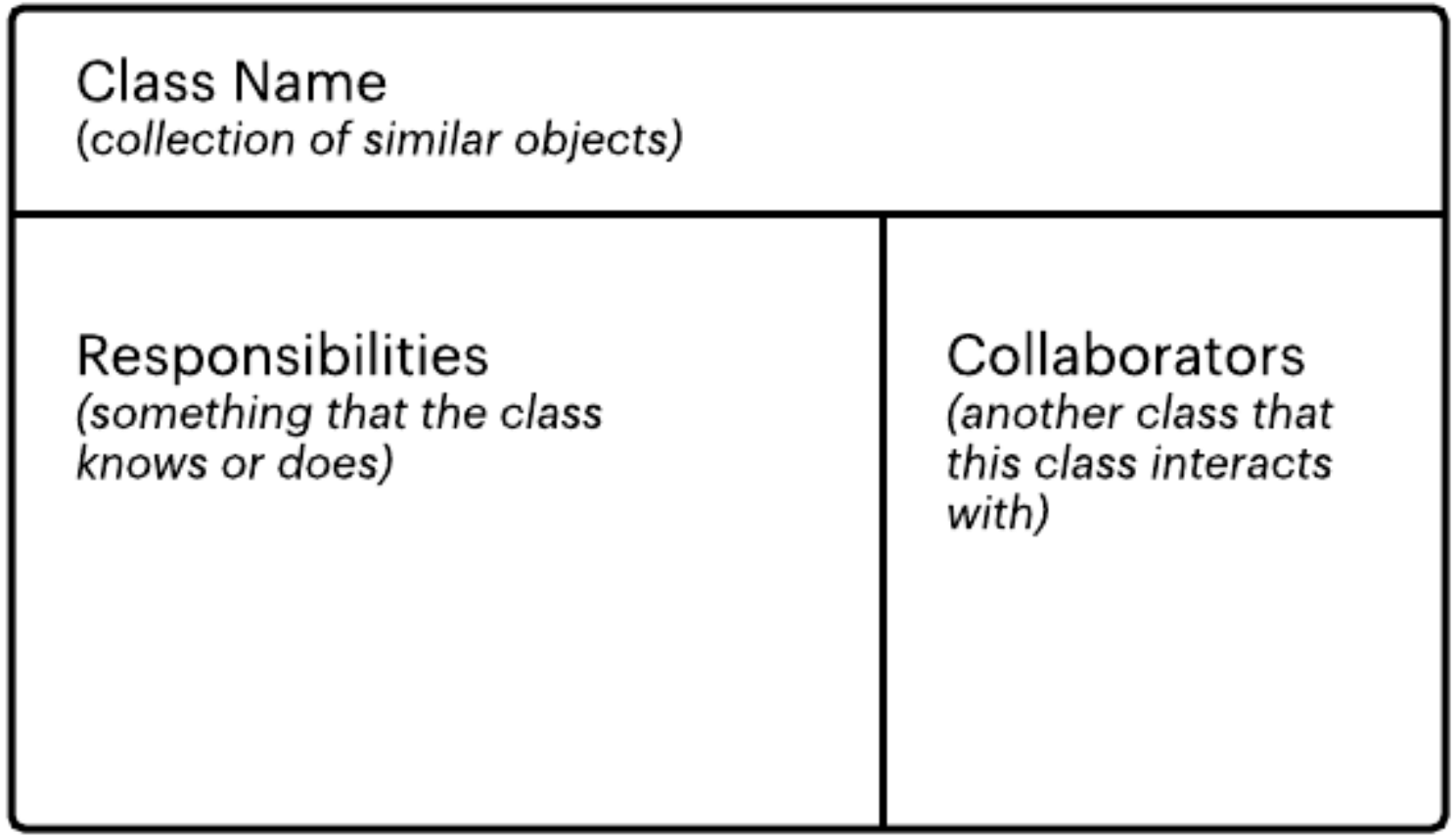
XP Design

- Simplitate
- Sistemul de metafore
- Soluții Spike
- Nicio funcționalitate prea devreme
- Refactor

XP Design

continuare

- Utilizare cartonașe CRC (class-responsibility-collaboration)



XP Coding

- Clientul este disponibil oricând
- Standarde de coding
- Unit Testing first
- Tot codul scris -» Pair programming
- Integrarea se face des și doar de o echipa la un moment dat
- Integrare pe un computer dedicat
- Collective ownership

XP Testing

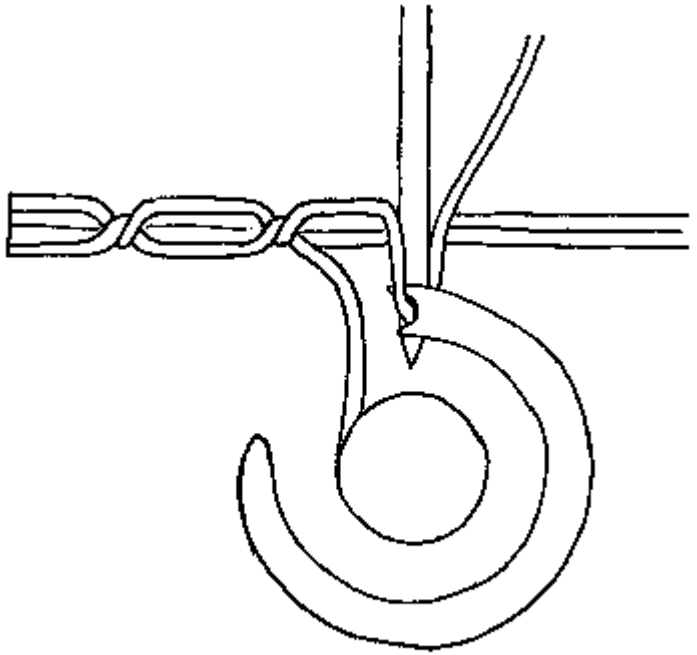
- Tot codul este testat
- Tot codul trebuie sa treacă toate testele
- Teste create la găsirea unui bug
- Teste de acceptare rulate des; scor publicat

Pair Programming

- Doi programatori lucrează la același calculator
- Câteva considerente:
  - Rezultă aproximativ 15% mai puțin cod decât 2 programatori solo
  - Code review continuu => design mai bun, mai puține erori
  - Fiecare învață de la celălalt (technical skills++)

Simple design that works

- Implementarea celor mai simple features care funcționează
- Trec toate testele
- Nu există cod duplicat
- Cât mai puține clase și metode



Refactorizarea

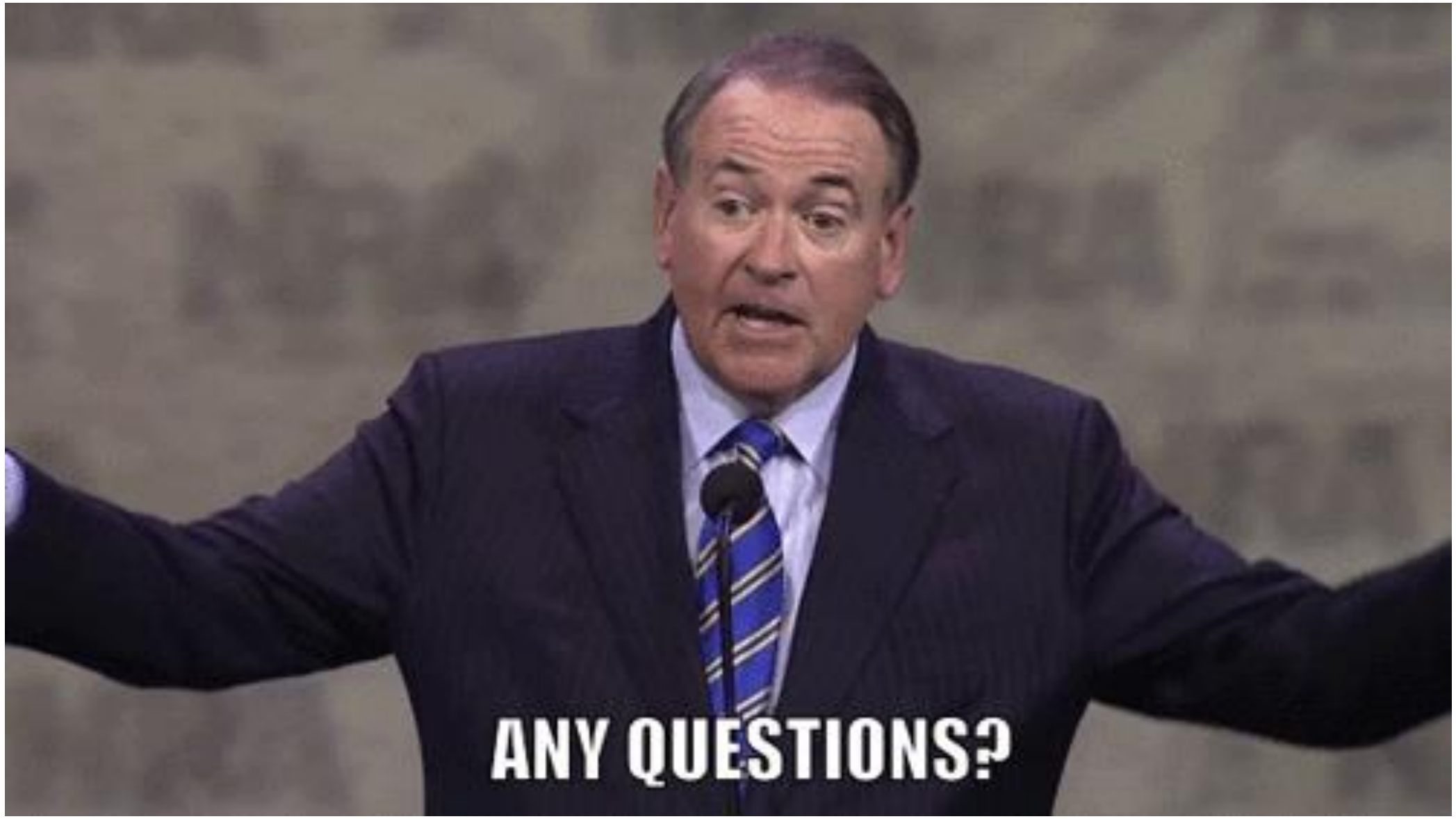
- Sofware design devine o activitate zilnică pentru toată lumea
- Îmbunătățire continuă a calității codului
- Consecințe:
  - Dezvoltare software rapidă
  - Codul poate fi schimbat cu ușurință

De ce XP?

- Light-weight: disciplină fără birocrație
- În condiții de stres, oamenii aleg de obicei calea mai ușoară
- XP are beneficii atât pe lungă durată cât și pe scurtă durată
- Codul servește drept documentație
- XP is fun

Critici XP

- Prea multe modificări = costuri ridicate
- Generează stres
- Cerințele se modifică foarte des => poate rezulta in muncă ineficientă (staring over)
- Poate fi foarte dificil de implementat din cauza lipsei de documentație si structură oferite de la început


ANY QUESTIONS?

THANK
YOU

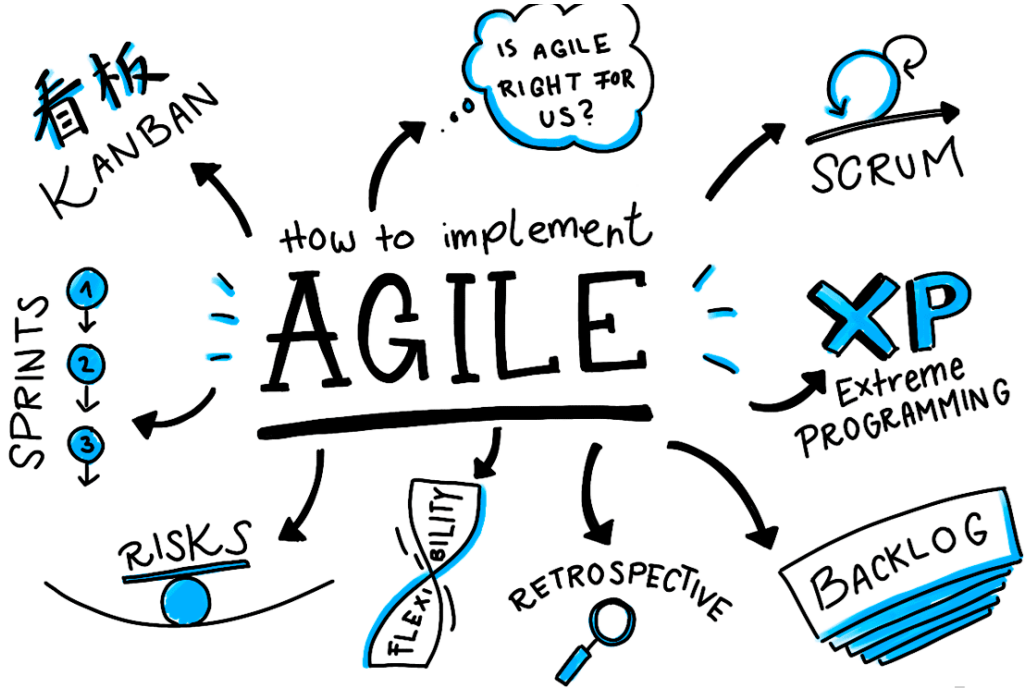
Cursul 13

Bouleanu Oana-Bianca
Preda Andreea-Cristina
Grupa 343C5

Cursul 13

Metodologia Scrum

Recap. de data trecuta..



Ce este Agile?

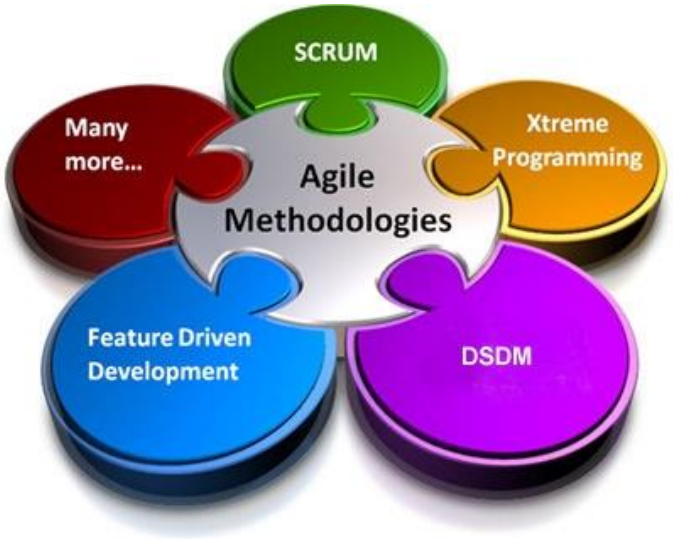
Metodologie de management a proiectelor ce incearca sa micsoreze riscurile de dezvoltare si timpul de executie prin implementarea proiectelor in forma foarte flexibila si interactiva.

Metodologii care deriveaza din Agile

AGILE exista in multe feluri:

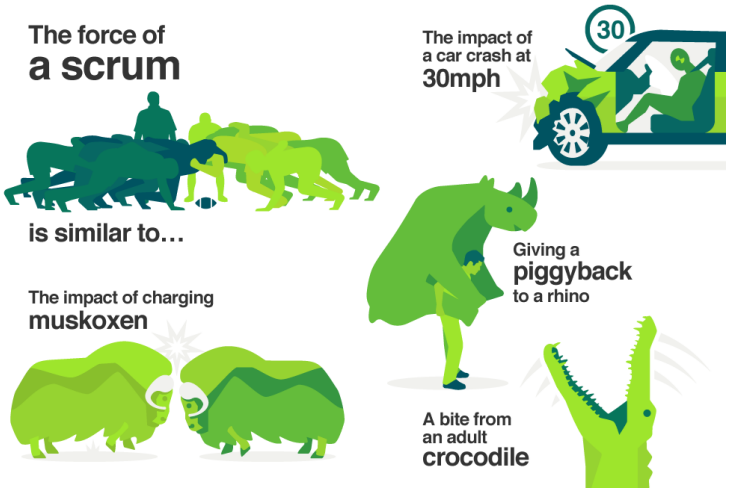
- XP
- SCRUM
- DSDM
- Crystal
- Feature Driven
- Etc.

Toate acestea folosesc principiile de baza ale filozofiei AGILE, dar o implementeaza in moduri diferile.



Abordarea scrum

O abordare de echipa in care toti membrii se indreapta, impreuna, catre acelasi scop, stabilind ritmul si adaptandu-se schimbarilor, similar a ceea ce se intampla la rugby.



Principii scrum

1. Instabilitate: scopuri extensive si directii strategice generale
2. Echipele de proiect se organizeaza singure
3. Faze de dezvoltare suprapuse
4. Multi-invatare astfel incat echipa poate invata atat din surse interne, cat si externe si sa se adapteze repede la conditii si medii schimbatoare
5. Control subtil: directionarea proiectului fara a interveni prea mult (ex: selectarea oamenilor potriviti pentru fiecare rol, crearea unui mediu deschis, tolerarea si anticiparea greselilor)
6. Transfer de invatare organizational prin asigurarea transferului si reutilizarii cunostintelor acumulate in urma unui proiect in cadrul altui proiect

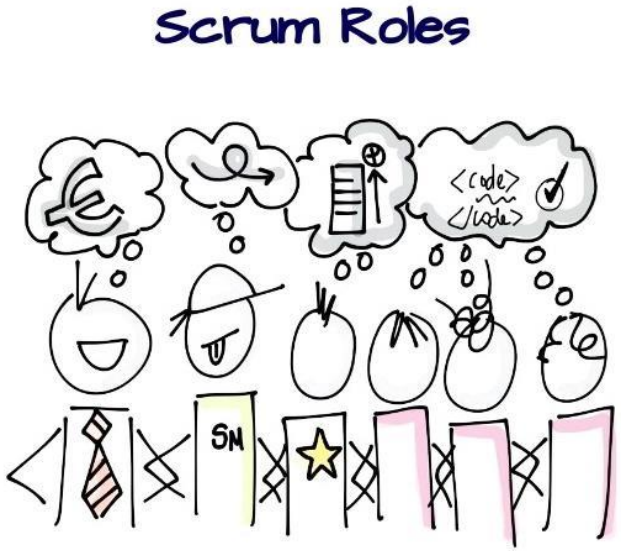
Scrum a fost folosit de catre:

- Microsoft
- Yahoo
- Google
- Electronic Arts
- IBM
- Lockheed Martin
- Philips
- Turner Broadcasting
- Oce
- Siemens
- Nokia
- Capital One
- Nielsen Media
- First American Real Estate
- BMC Software
- Ipswitch
- John Deere
- Lexis Nexis
- Intuit
- Sabre
- Salesforce.com
- Time Warner
- BBC

Scrum a fost folosit pentru:

- Software comercial
- Aplicatii in-house
- Aplicatii la comanda
- Proiecte cu pret fix
- Aplicatii financiare
- Aplicatii certificate ISO 9001
- Sisteme inglobate (embedded)
- Sisteme cu cerinte de disponibilitate 99.999%, 24x7
- Programul Joint Strike Fighter
- Dezvoltarea de jocuri video
- Sisteme aprobate de FDA, life-critical
- Software de control al satelitilor
- Site-uri web
- Software pentru dispozitive mobile
- Telefoane mobile
- Aplicatii de switching in retele
- Aplicatii ISV (vanzatori independenti de software)
- Unele din cele mai mari aplicatii utilizate

Rolurile SCRUM



Stakeholders, Scrum Master, Product Owner, Development Team

Rolurile scrum intr-un singur slide

- Scrum Master
- Facilitator, se asigura ca Scrum este aplicat si ca echipa poate opera si lucra
- Protejeaza echipa de interferente externe

- Detinatorul produsului
- Defineste caracteristicile produsului, decide prioritatile, accepta sau refuza rezultatele activitatii

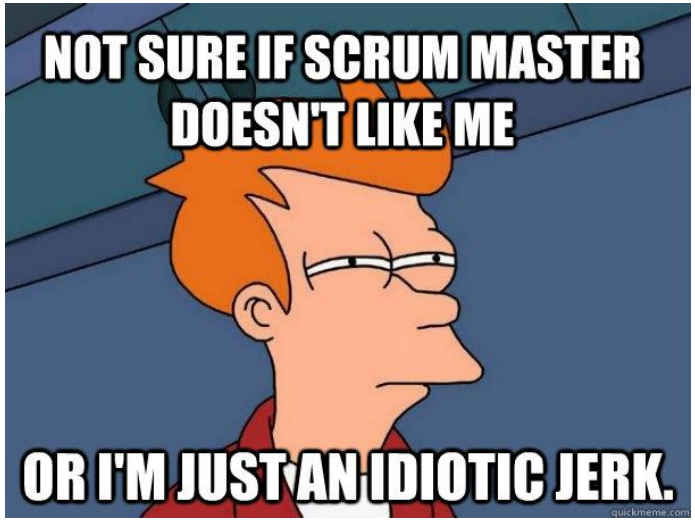
- Echipa
- Se organizeaza singura, ideal lucreaza full-time, restransa (5-9 membri)

Scrum Master

- Reprezinta managementul pentru proiect
- Responsabil de implementarea valorilor si practicilor Scrum-ului
- Elimina impedimentele
- Se asigura ca echipa este complet functionala si productiva
- Asigura cooperarea apropiata intre toate roulrile si functiile
- Protejeaza echipa de interferentele externe

Detinatorul proiectului

- Defineste proprietatile proiectului
- Decide data lansarii si continutul
- Responsabil de profitabilitatea produsului (ROI)
- Prioritizeaza caracteristici in functie de cerintele pietii
- Ajusteaza functiile si prioritatile la fiecare iteratie, in functie de necesitati
- Accepta sau refuza rezultatele muncii

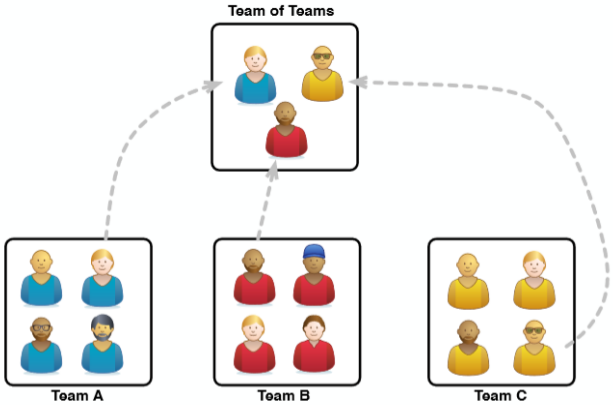


Echipa

- In mod normal, formata din 5-9 membri
- Inter-functionala
- Programatori, testeri, designeri de experienta de utilizator etc.
- Membri ar trebui sa lucreze full-time
- Pot exista exceptii (ex: administratorul de baza de date)
- Se autoorganizeaza
- Apartenenta la echipa se poate schimba numai intre cicluri de dezvoltare

Echipa de echipe

- Pentru proiecte mari se pot organiza echipe de echipe



Scrum in 100 de cuvinte

Scrum e un proces agil care ne permite sa ne concentram pe livrarea a ceea ce e mai valoros pentru o afacere in timpul cel mai scurt.

Ne permite ca in mod rapid si repetat sa evaluam software care intr-adevar functioneaza (odata la doua saptamanai sau lunar).

Clientul seteaza prioritatile. Echipele se organizeaza sigure pentru a determina cel mai bun mod de a livra functionalitatile cele mai prioritare.

Odata la doua saptamani sau cel mult lunar oricine poate vedea un software functional si sa decida sa il lanseze asa cum e sau sa continue sa il imbunatateasca in nou sprint.

Joke Time

Knock knock.
Who's there?
Done.
Done who?
Depends on who you ask.

"Knock knock."
Who's there?
"Carrie."
Carrie who?
"Carrie over to next sprint."

Knock knock.
Who's there?
Processes and tools.
Processes and tools who?
See, I told you he was ~~Agile!~~

A Pig and a Chicken are walking down the road.

The Chicken says: "Hey Pig, I was thinking we should open a restaurant!"

Pig replies: "Hm, maybe, what would we call it?"

The Chicken responds: "How about 'ham-n-eggs'?"

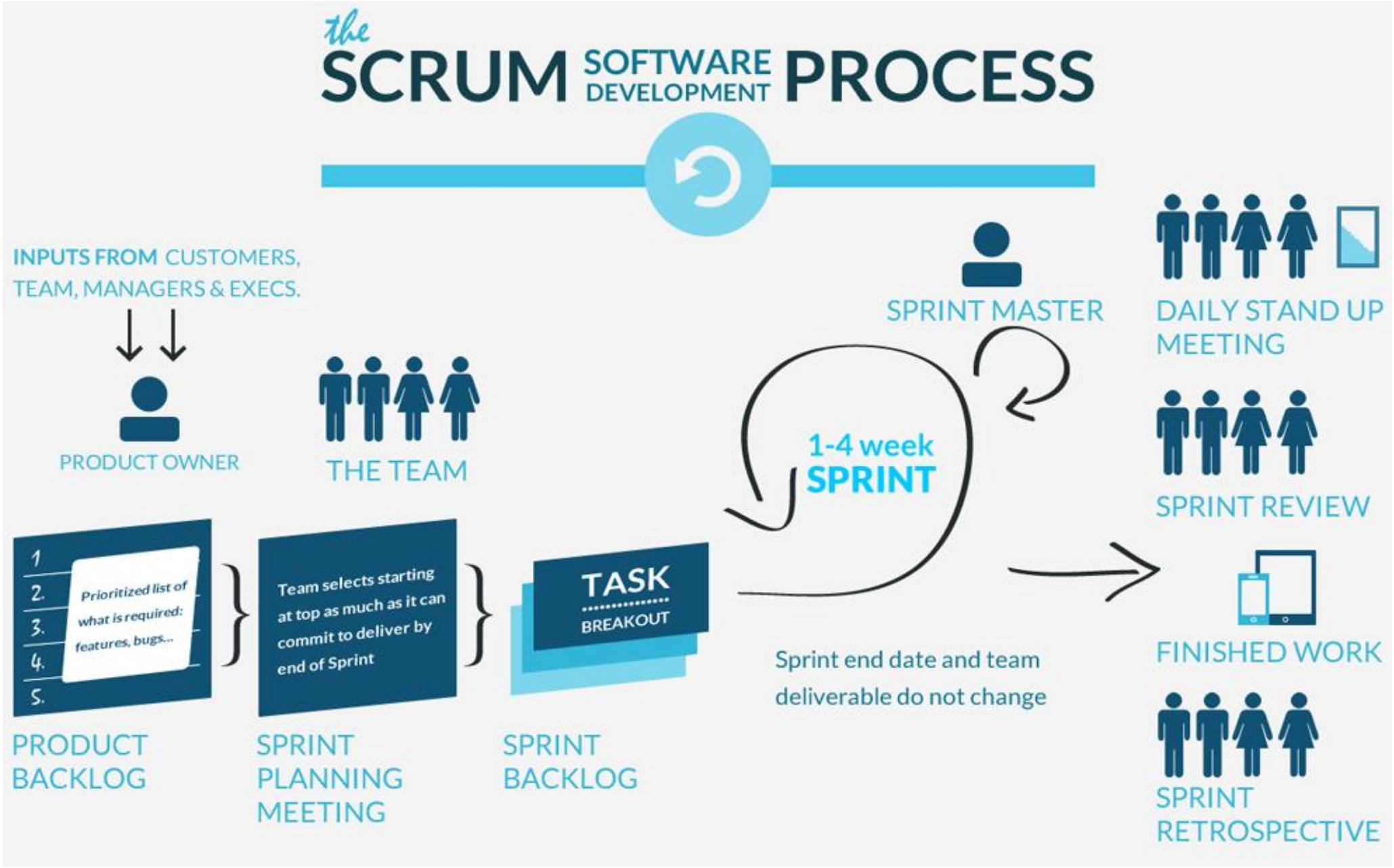
The Pig thinks for a moment and says: "No thanks. I'd be committed, but you'd only be involved."

Data viitoare..

- Procesul Scrum
- Produsul Backlog
- Planificarea iteratiilor
- Rularea si urmarirea iteratiilor
- Gestionarea proceselor Scrum

AGILE SOFTWARE DEVELOPMENT

Besea Georgiana
Draghici Ruxandra
342C5


the
SCRUM SOFTWARE DEVELOPMENT PROCESS
INPUTS FROM CUSTOMERS, TEAM, MANAGERS & EXECS.
PRODUCT OWNER
THE TEAM
SPRINT MASTER
DAILY STAND UP MEETING
1-4 week SPRINT
SPRINT REVIEW
1
2.
3.
4.
5.
Prioritized list of what is required: features, bugs...
Team selects starting at top as much as it can commit to deliver by end of Sprint
TASK
BREAKOUT
Sprint end date and team deliverable do not change
FINISHED WORK
PRODUCT BACKLOG
SPRINT PLANNING MEETING
SPRINT BACKLOG
SPRINT RETROSPECTIVE

Product backlog

- Lista ordonata ce contine tot ce este necesar produsului
- Compus din **User stories**
- Dinamic (produs adecvat, competitiv si util)
- Verificat de **Product Owner** (continut, disponibilitate, indeplinirea cererii)

User story

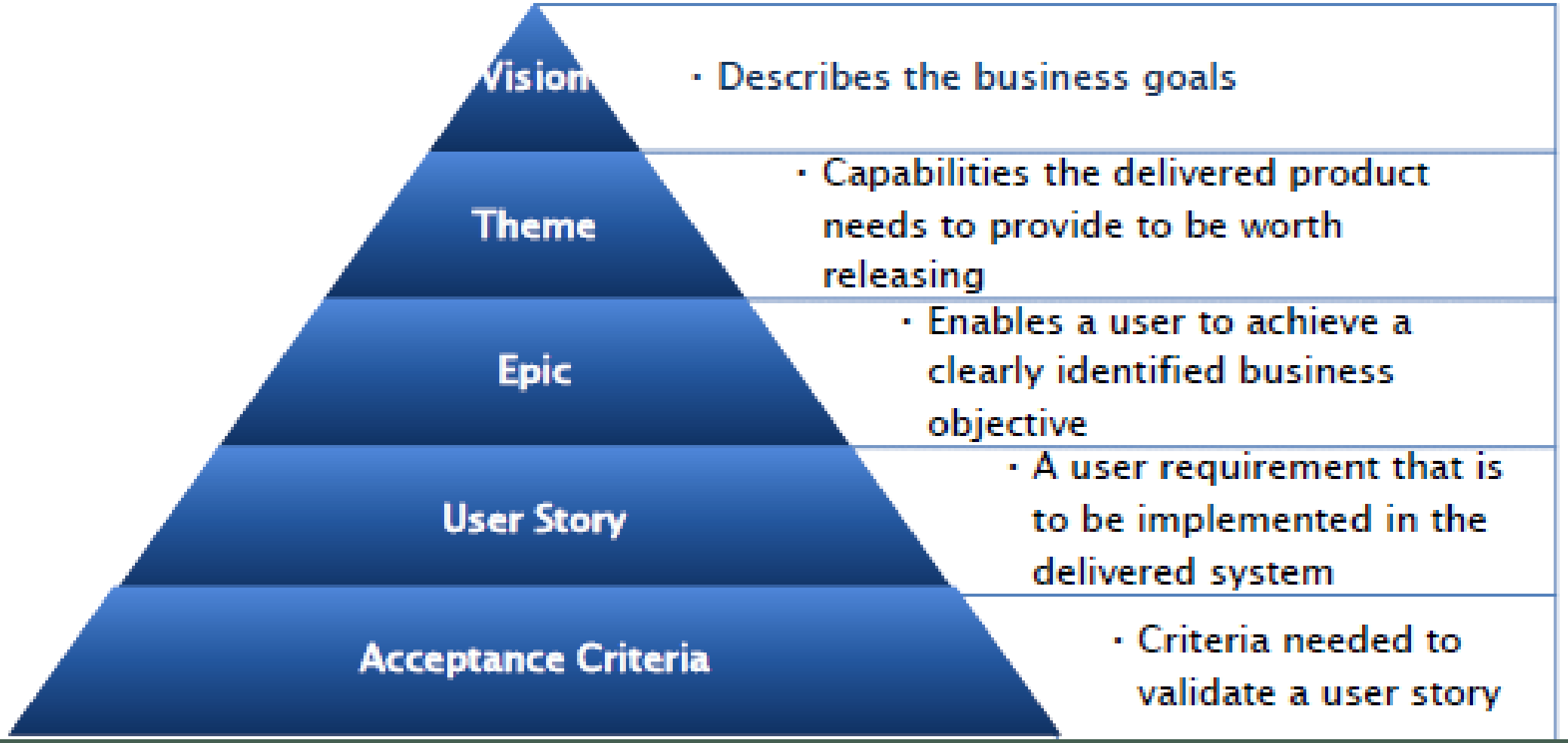
Format din:

- Rolul utilizatorului
- Descriere feature dorit
- Motiv

"As a **user,** I want **to upload photos** so that **I can share photos with others**."

Formula user story:

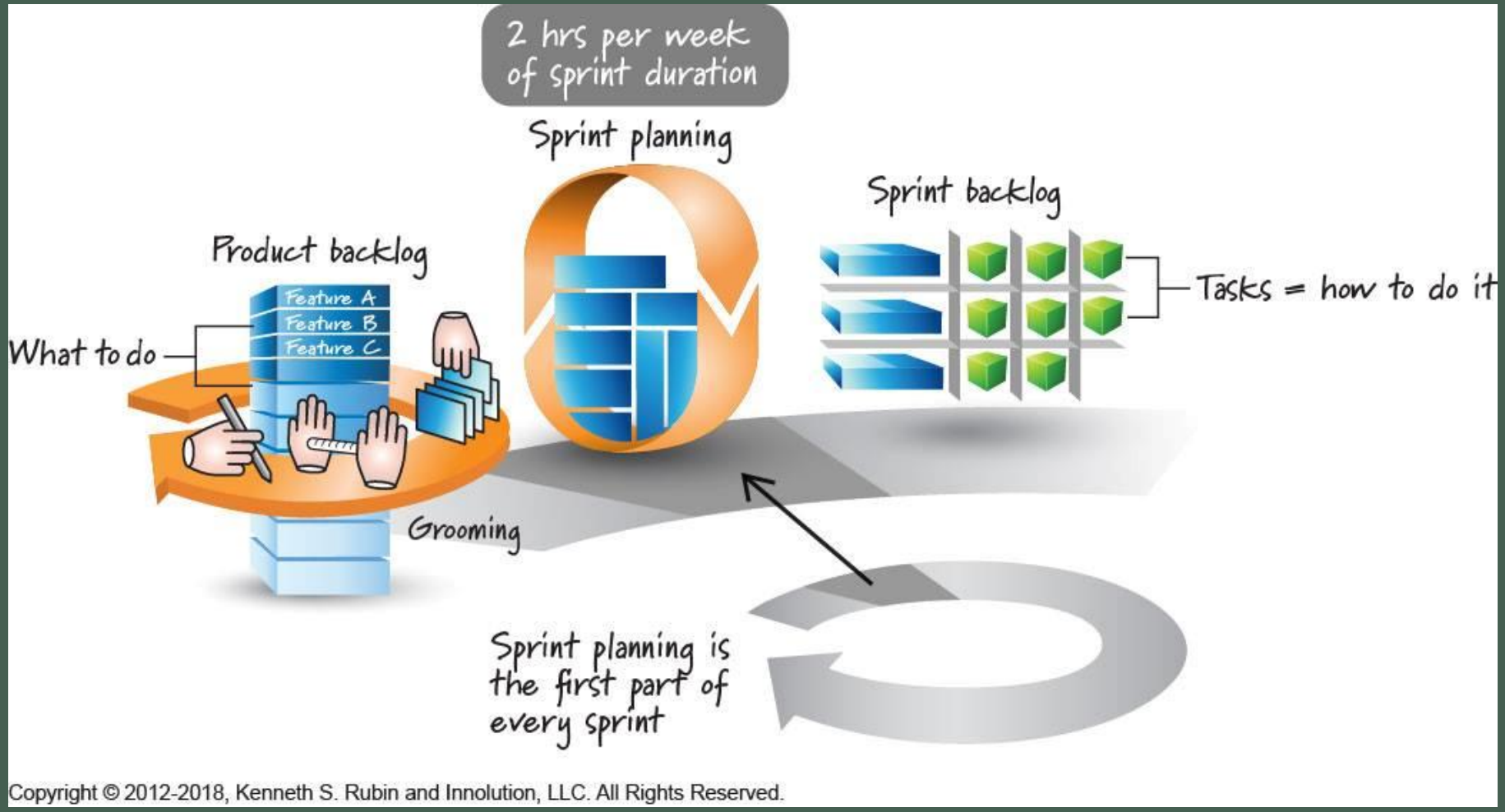
Card + Conversation + Confirmation (Creare User Story + Dezbatere pe tema user story-ului + Obiective atinse)


Vision
Describes the business goals
Theme
Capabilities the delivered product needs to provide to be worth releasing
Epic
Enables a user to achieve a clearly identified business objective
User Story
A user requirement that is to be implemented in the delivered system
Acceptance Criteria
Criteria needed to validate a user story

Pro user stories

- Prioritizare
- Colaborare
- Libertatea alegerii modului de dezvoltare
- Intelegerea scopului indiferent de pozitie

Sprint planning



Copyright © 2012-2018, Kenneth S. Rubin and Innolution, LLC. All Rights Reserved.

Participanti si roluri

Product Owner:

- scopul sprintului
- definirea user story-urilor in detaliu
- prioritizarea listei de USER
- descrierea rezultatului ce se doreste a fi obtinut pentru US si pentru sprint

Scrum Master:

- asigura desfasurarea in bune conditii a planningului
- verifica disponibilitatea si capacitatea de lucru a membrilor echipei

Dev Team:

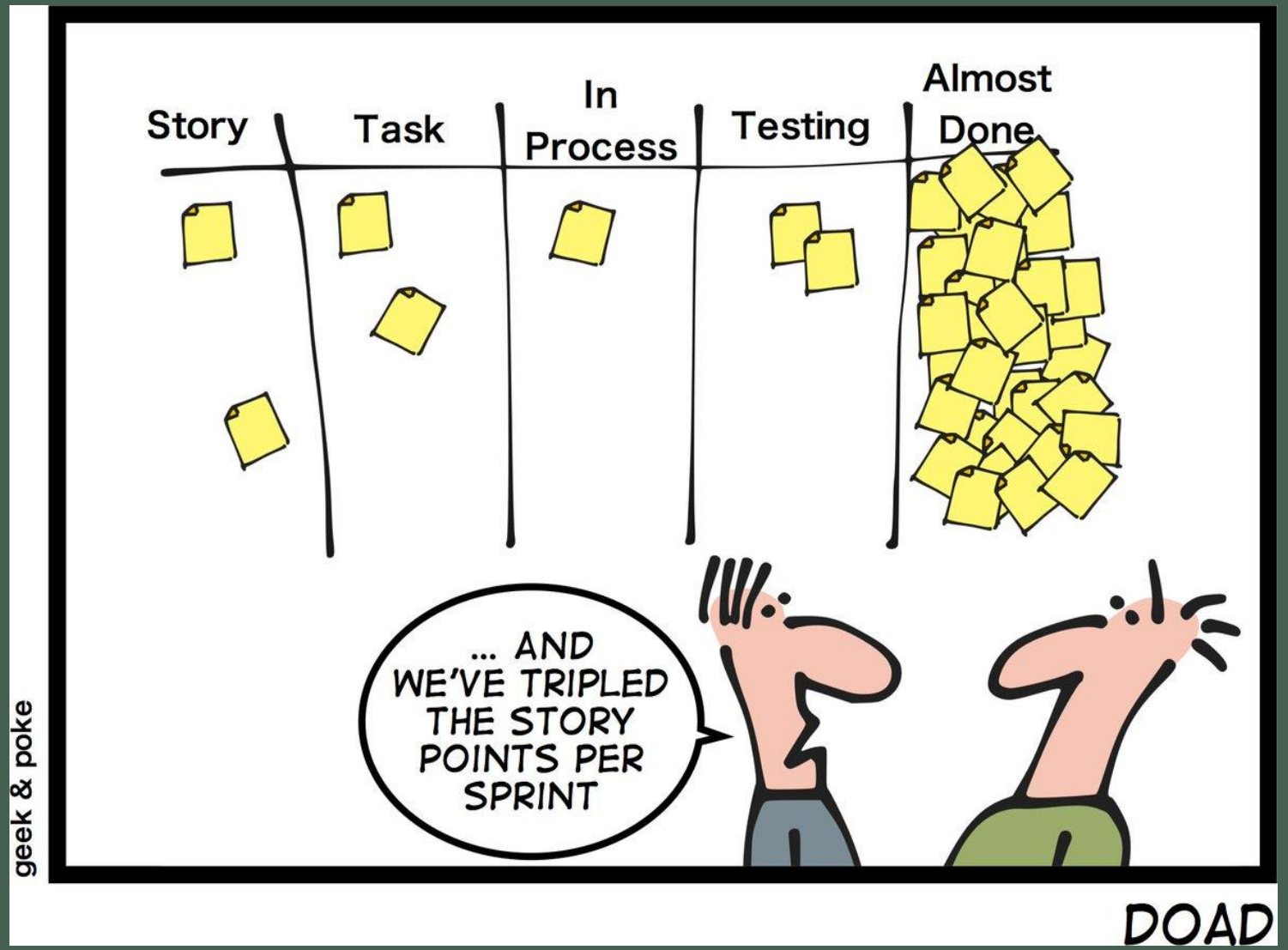
- adresare intrebari pentru clarificarea ambiguitatilor
- isi prezinta disponibilitatea in urmatorul sprint
- participa la estimarea fiecarui US

Estimarea

Metoda de masurare: Story points

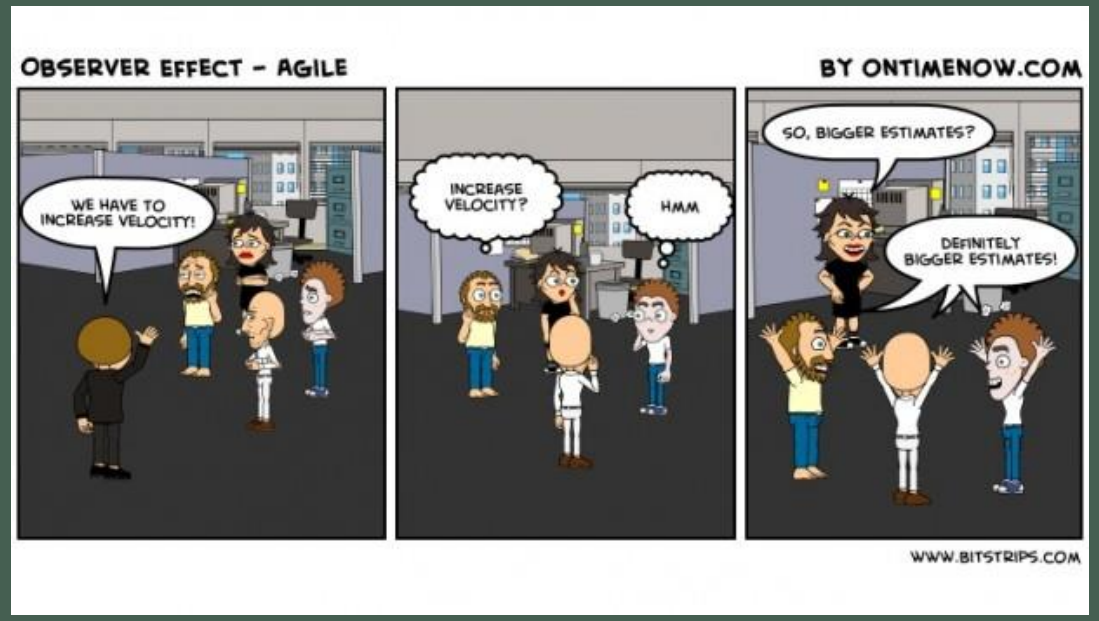
Metode de estimare a sprintului:

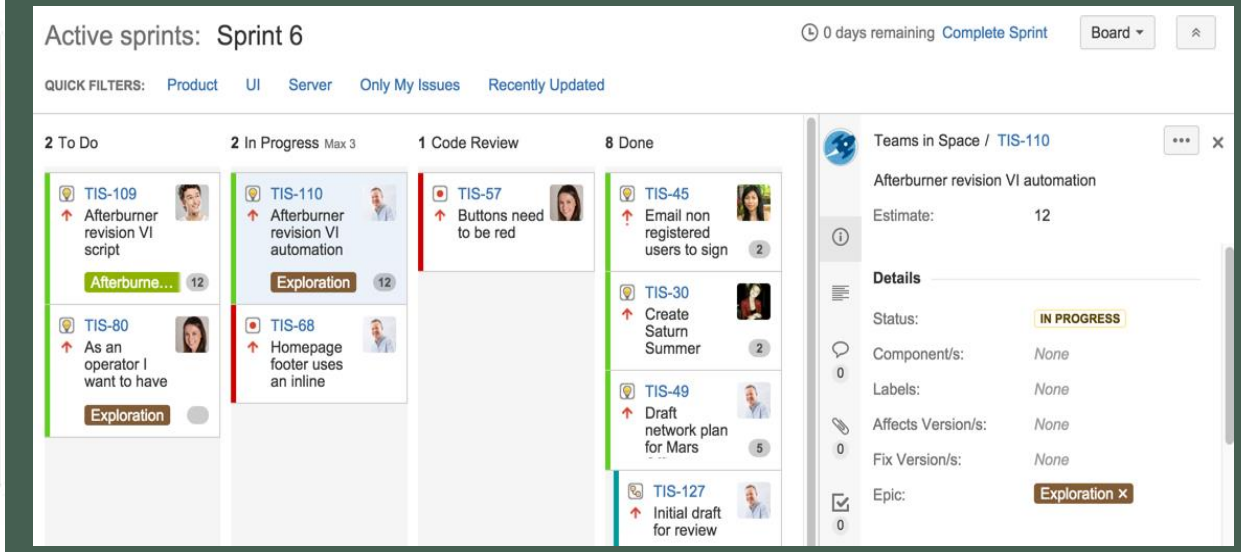
White Elephant Sizing Game https://www.youtube.com/watch?v=L94URgiPSGA



Scrum Tracking

- Masuratori: viteza, eficienta, efort.
- Metoda: Aplicatii de software development tracking – Jira, Trello.



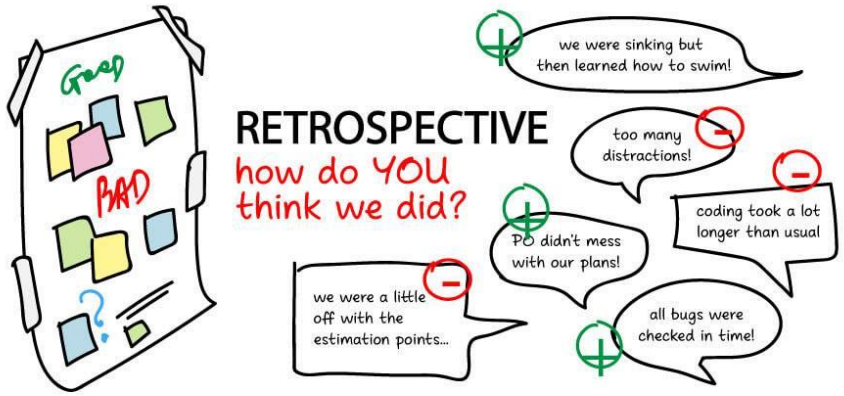


Daily Scrum

- Intalniri de 15 minute
- Fiecare participant raspunde la intrebarile:
  - Ce ai facut ieri?
  - Ce vei face azi?
  - Probleme intampinate – discutate in intalnirea de Problem Solving
- Abordare corecta: https://www.youtube.com/watch?v=H02BITXpcto

Sprint review

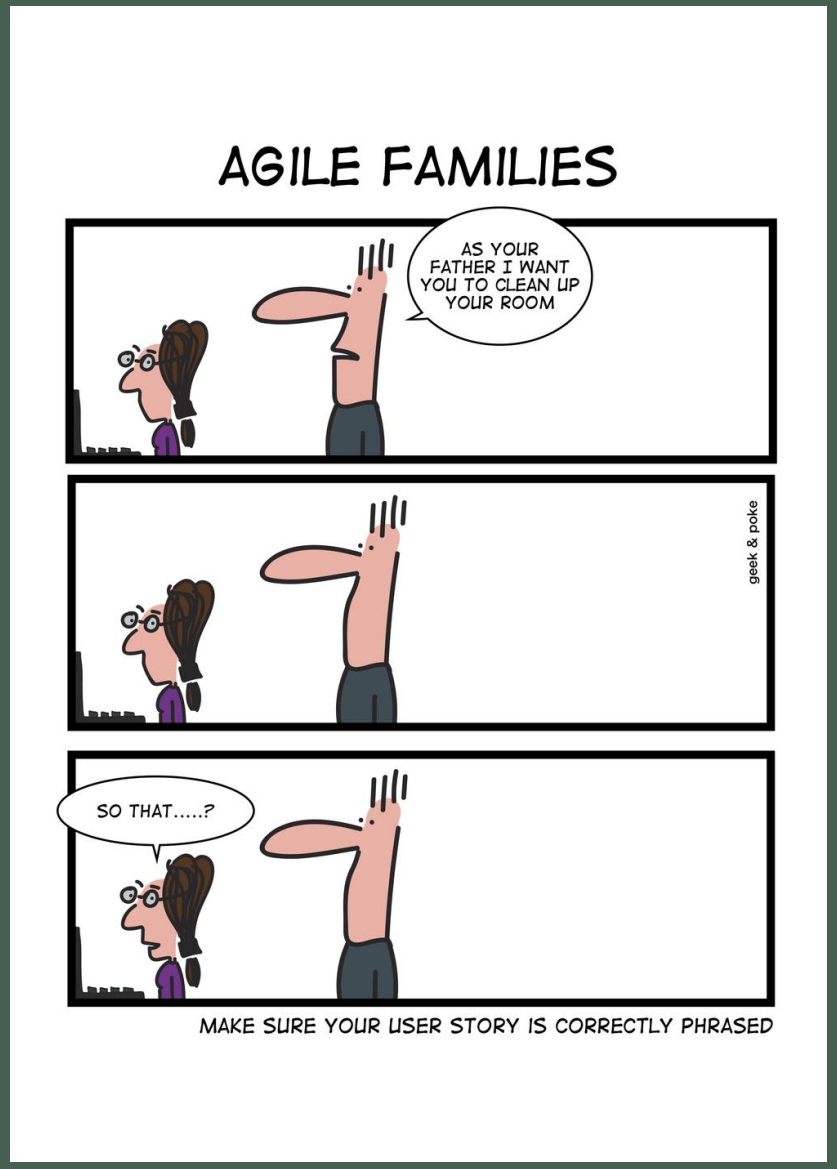
- Prezentarea produsului obtinut = **demo**
- **Cine** prezinta? – Scrum Master
- **Cui**? – Product owneri
- **Metode**:
  - inregistrari pentru fiecare feature/task
  - Lansarea ultimei versiuni a software-ului
  - Mock up uri



Sprint retrospective

- Ce a mers bine?
- Ce a mers prost?
- Ce putem schimba?
- Au avut effect schimbarile din ultimul sprint retrospective?

- Metode: Board stickers cu idei/ discutie libera/ jocuri



As a presenter of this topic I want to finish the presentation here so that all of us can have a break